THÈSE

En vue de l'obtention du

DOCTORAT DE L'UNIVERSITÉ DE TOULOUSE

Délivré par : *l'Institut Supérieur de l'Aéronautique et de l'Espace (ISAE)*

Présentée et soutenue le *13/01/2022* **par :**

Juan José MONTERO-JIMENEZ

Knowledge reuse to enhance predictive maintenance systems architecture

---

JURY

ERIC BONJOUR	Professor at University of Lorraine	Rapporteur
FRANCES BRAZIER	Professor at TU DELFT	Rapporteur
ANABEL FRAGA-VAZQUEZ	Associate Professor at Universidad Carlos III	Membre du Jury
ELISE VAREILLES	Professor at ISAE-SUPAERO	Membre du Jury
BERNARD GRABOT	Professor at ENIT	Directeur de Thèse
ROB VINGERHOEDS	Professor at ISAE-SUPAERO	Directeur de Thèse

École doctorale et spécialité :
EDSYS : Génie Industriel 4200046
Unité de Recherche :
ISAE-ONERA CSDV - Commande des Systèmes et Dynamique du Vol
Directeur(s) de Thèse :
Rob VINGERHOEDS et *Bernard GRABOT*
Rapporteurs :
Eric BONJOUR et *Frances BRAZIER*

Acknowledgements

First and foremost, I would like to pass my heartfelt thanks to my supervisors Prof. Rob Vingerhoeds and Prof. Bernard Grabot, theirs door were always open to answer my doubts and provide me with guidance, advise, and support in the realisation of my thesis.

I would like to express my thanks to Tecnológico de Costa Rica, Institut Français d'Amérique Centrale, Campus France, ISAE-SUPAERO, and ENIT for sponsoring and supporting my PhD studies in France.

I would like to express my deepest thanks to my wife Estefanny Guillén and my daughter Edith Montero for being that unconditional support that kept me sane along this wonderful process full of ups and downs. It would have been impossible to achieve the success without both of you! I would also like to express my deepest thanks to my parents, sister, and all other the members of my family who were always supporting me at the distance while doing this thesis.

I would like to express my special thanks to Dr.Sebastien Schwartz who was a great teammate, we developed many things together while having a nice work environment. I would also like to express my special thanks to M.Eng Carlos Piedra, M.Eng Sebastian Mata, who helped me check the spanish version of my thesis summary, and along with the group of students from TEC, helped in the creation of the case base for my research. My special thanks to Augusto Miyagawa, Hugo Muñoz, Johanna Mazouzi, Okjin Lee, Tran-vu Nong, and Wendi Ding, students or interns in my research project who helped me in different tasks in the creations of the case-based reasoning system. Also special thanks to Corinne Boisrobert for checking the french version of my thesis summary.

I would like to thank the friends I found in France, Carlos Vaca, Didier Allieres, Stephanny Bardon, Corinne and Phillipe Boisrobert, Franco Peschiera, Vatsal Pant, Jasdeep Singh, the "TICOS en TOULOUSE" and the friends from the Master and DISC department. All of you helped to make wonderful my experience in France and made feel that I was not far from home!

Intentionally left blank

Contents

Intentionally left blank

List of Figures

Intentionally left blank

List of Tables

Table of acronyms

AI *Artificial Intelligence*

BFO *Basic Formal Ontology*

CBM *Condition-Based Maintenance*

CBR *Case-Based Reasoning*

CCO *Common Core Ontologies*

DSS *Decision Support Systems*

EN *European Standard*

FMECA Failure Mode, Effects and Critical Analysis

FPM Failure Propagation Model

GUI Graphical User Interface

HPC High-Pressure Compressor

HPT High-Pressure Turbine

IEC International Electrotechnical Commission

INCOSE International Council on Systems Engineering

ISO International Organization for Standardization

LPC Low-Pressure Compressor

LPT Low-Pressure Turbine

LSTM Long-Short Term Memory

NASA National Aeronautics and Space Administration

NF *Norme Française*

NN Neural Network

OMSSA *Ontology for Maintenance Strategy Selection and Assessment*

OPMAD *Ontology for Predictive Maintenance Architecture and Design*

OWL2 *Web modelling language Version 2*

PhD *Doctor of Philosophy*

PHM *Prognostics and Health Management*

PdM *Predictive Maintenance*

RBR *Rule-Based Reasoning*

RDF *Resource Description Framework*

RNN *Recurrent Neural Network*

SDK *Software Development Kit*

SOM *Self Organizing Map*

SVM *Support Vector Machine*

SysML *Systems Modelling Language*

W3C *World Wide Web Consortium*

CHAPTER 1

Introduction

"A daring beginning is halfway to winning."

Heinrich Heine

Content

1.1 The journey begins

After some years working in the industry in maintenance and subsequently in a research project specifically on predictive maintenance during the M.Sc. in Aerospace Engineering program at ISAE-SUPAERO, my passion and curiosity for the topic drove me to make the decision to continue my research in predictive maintenance and undertake a three-year program to opt for the degree of Doctor of Philosophy (PhD). It was a big challenge at a personal and professional level. Close to the end of the experimentation, one of my thesis advisors shared with me the above-mentioned quote from Heinrich Heine and it made me remember the big bet I made when I decided to start a PhD. The research work consolidated in the current manuscript testifies to the bet I made was fully worth it.

This chapter aims at introducing the context and the motivations for this research which is developed around two main topics: predictive maintenance and systems architecture. On one hand, predictive maintenance is seen as a strategy in the maintenance field which is carried out by specialized systems. On the other hand, systems architecture is part of a concept design stage of complex systems. The principles of systems architecture are applied to enhance the design of predictive maintenance systems.

1.2 Maintenance

Maintenance is one of the most important activities during the life cycle of assets to ensure their functionality. It is possible to illustrate the term with basic examples, like a person changing a wheel of a car, or with the most complex ones which could be the use of high technology techniques to predict faults in a critical component of a system. Defining the word maintenance is not simple. The European standard NF EN 13306 X60-319 [Eur09] gives a complete definition that addresses the different aspects of maintenance; one can understand maintenance as "all of technical, administrative and management activities throughout the life cycle of an asset in order to maintain or restore it to the state where it can perform the intended functions".

Moreover, maintenance is strongly related to other important aspects in industry, such as risks in safety, costs, and image and in the end profits for any company. An example of this can be the delay or cancellation of a flight due to technical problems. The customers are ready to take their flight, and a little time before the departure, they receive a notification that the flight has been delayed due to a malfunctioning of the engine and it may last three hours to be evaluated. At this moment, all the customers receive a meal ticket in compensation for the flight's delay which means an unexpected cost. The customers wait three hours and then the airline informs that the fault is worse than expected, it affects directly the safety of the flight, and finally it has been totally cancelled. The customers will demand reimbursement or even higher compensation. The customers can eventually look for another airline to fly to their destination. It is possible that next time these customers will choose a different airline and will share their bad experience with other people, directly affecting the image or prestige of the company.

Considering these kinds of circumstances, proper maintenance should be applied in industry, at the right moment, not too early as it engages high costs and not too late as it increases the risk of failures. There are different strategies that will define the right moment to trigger maintenance actions depending on the risk of an eventual failure.

Maintenance strategies can have different classifications. One of the best-known taxonomies divides maintenance into three main strategies: corrective maintenance, preventive maintenance, and predictive maintenance. A good combination of the three is what makes a system reliable, decreasing the maintenance costs. Some components of a maintainable system do not represent a major problem when they fail and a proactive strategy will not be justified for its implementation; these components will be run until failure following a corrective maintenance strategy. For some other components, an unexpected failure is not acceptable and for them, it is normal to find preventive maintenance strategies that will trigger maintenance actions based on operational intervals. For some critical components, specialized systems can be implemented to monitor their condition and trigger maintenance actions accurately when needed, following a predictive maintenance strategy. A brief introduction to these maintenance strategies is provided hereafter.

1.2.1 Corrective Maintenance

This is the oldest type of maintenance; it basically means to replace or repair a failed component of a system in order to restore it to its functional state [Vin+95]. Cavemen already replaced the broken stones of their axes and used tools to restore them. This type of maintenance is inherent to every system, there is no perfect system that can last forever and unfortunately, even for systems that use high technology in maintenance, it remains impossible to avoid unforeseen failures.

Despite that corrective maintenance is a classic strategy, it still is an important topic of research. For example, [FZ15] aims at analyzing the impact of failures in aircraft components in the maintenance and support costs. A good description of what activities are involved in corrective maintenance is provided.

These activities are: “malfunctioning location, malfunctioning isolation, replace, re-install, adjustment, verification and fix the damaged parts”.

Another recent corrective maintenance study [Wan+14] aims to provide corrective maintenance procedures for engineering equipment, using management based on Failure Modes, Effects and Critical Analysis (FMECA) and representing failure mechanisms using a failure propagation model (FPM). The authors propose a depth-based fault diagnosis and a failure risk metric; a binary decision tree can be built for failures ascertainment for corrective maintenance.

It is important not to confuse a corrective maintenance strategy with corrective maintenance due to an unexpected failure [Hod+21]. When corrective maintenance strategies are assigned to a specific component or maintainable system, a complete analysis has been carried out concluding that the most efficient manner to trigger maintenance actions is when the failure occurs. This kind of failure does not represent a significant impact on the maintainable system. In contrast, corrective maintenance due to an unexpected failure often catches maintenance staff unprepared and it is often the most detrimental in terms of safety and costs. For these cases then preventive and predictive maintenance strategies offer an alternative at increasing the reliability and safety by triggering maintenance actions before a failure happens.

1.2.2 Preventive Maintenance

Preventive maintenance is carried out at predetermined intervals or according to prescribed criteria and intends to reduce the probability of failure or the degradation of the functioning of a maintainable system [Leg+17]. Preventive maintenance can also be seen as a set of activities aimed at improving the reliability and availability of a system by triggering maintenance actions before a failure occurs, for instance, based on operational intervals [MU11]. Traditional activities related to preventive maintenance are inspection, cleaning, lubrication, adjustment, alignment, and/or replacement of sub-components that wear out.

The aim of this strategy is to prevent failures to occur. Maintenance actions are defined with corresponding intervals or thresholds (e.g., after a certain time, cycles, rounds, distance). Preventive maintenance implies planned shutdowns which normally are shorter than the unplanned shutdowns which are present in the corrective maintenance. The maintenance action will be triggered when the component reaches the predetermined operational interval threshold even if the component in appearance is in operational condition. Recent trends in preventive maintenance are focused on the optimization of intervals between maintenance actions.

1.2.3 Predictive Maintenance

Predictive maintenance (PdM) proposes an alternative strategy to preventive maintenance. Sometimes preventive maintenance actions are triggered too early so the component is still in good operational conditions and could have worked a longer period. Or too late so that an unexpected failure occurs. Consequently, the maintenance action impacts negatively the maintenance costs, even more for a modern industry where the products and equipment are more and more complex, and thus, require higher reliability and better quality. “Eventually preventive maintenance has become a major expense of many industrial companies” [JLB06]. Then, predictive maintenance was born as a solution to define the right maintenance threshold, to monitor the condition of the components, and trigger maintenance action accurately when needed, not too early so that it increases the maintenance costs of a company but not too late so that a failure occurs.

Predictive maintenance is strongly related to Condition-Based Maintenance (CBM) and Prognostics and Health Management (PHM). The three terms address diagnostics and prognostics for maintenance

purposes up to the point that the terms are used as if they were synonyms. Predictive maintenance and Condition-Based Maintenance seem to appear simultaneously around the 1940s. Both terms refer to the maintenance strategy that aims at anticipating failures of machines based on their condition. However, it is not before the early 1990s that this strategy has taken on importance thanks to the implementation of monitoring systems and computational tools able to carry on with the diagnostics tasks. Prognosis was always an inaccurate discipline and even when it is mentioned in predictive maintenance and CBM it remained an unsolved problem. In the early 2000's PHM discipline emerged aiming to cover the gap in prognosis. For the last 20 years, the research in diagnosis and prognosis for maintenance has earned a lot of attention from academy and industry, thanks to the continuous increments in computational power and given the benefits of their implementation. Over the last decade, different contributions have been made under the different terms (predictive maintenance, CBM and PHM) and refer to the same field of research.

1.3 Systems concept stage and the system architecture process

According to the INCOSE Handbook [INC15], the life cycle of a system can be divided into six generic stages (see Figure 1.1). The system life cycle starts with the concept stage which aims at exploring possible solutions to meet the initial stakeholders' needs. The concept stage is followed by the development stage in which a detailed design of the system components and their interfaces takes place. After the detailed design, the production stage begins; it is dedicated to the system implementation, meaning the fabrication, coding, creation of the different components and their integration in the whole system. Verification and validation of the system are part of the production stage. Once the system has been validated, it is delivered to the client for its utilization stage. In the utilization stage, the system performs the function for which it was created. The maintenance stage goes in parallel with the utilization stage during the system operation. This maintenance stage aims at keeping the system at its optimal operation. At the end of the system life cycle, the disposal stage aims at managing properly the system residuals when it is taken out from the operation.

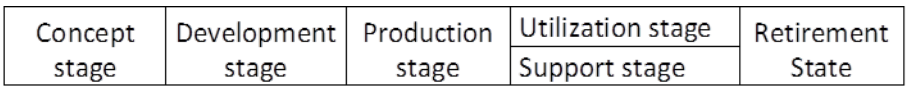


Figure 1.1: Generic stages in a system life cycle [INC15]

The concept stage starts by gathering all needs and desires from the stakeholders and formalizing them into stakeholder requirements. These requirements are then used for the creation of the system architecture that will serve as a basis for the detailed design and implementation of the system. The development of the architecture can be divided into three levels [Roq18]; [INC15]: functional architecture, logical architecture and physical architecture. The functional architecture describes how the different sub-functions of a system interact to fulfil a specific objective but does not provide any detail about the components that fulfil each sub-function. The logical architecture provides as much detail as possible of the architecture components and their interfaces but not engaging to any specific technology, meaning that the logical architecture remains generic. The physical architecture provides the details of the technologies that will fulfil each logical architecture component. The union of these three architecture levels can be called as the system architecture and can be then defined as the "fundamental concepts or properties of a system in its environment embodied in its elements, relationships, and in the principles of its design and evolution" [ISO11]. In simple words, architecture is how a set of elements, which could be physical or informational, are organized to fulfil a specific objective.

Creativity plays a vital role in the architecture process, especially when identifying and selecting suitable components to fulfil the logical architecture. This creativity can be non-structured or structured [CCS15].

On one hand, non-structured creativity is based on unconscious processing, an immediate vision of the solution for a problem. However, this creativity process narrows down the solution space and does not always lead to the "best" possible solution. On the other hand, structured creativity follows a method to explore the design space considering the initial function or functions to be fulfilled. An example of structured creativity methods is the components recombination. It supposes that there is more than one option per component available. Different concepts of the system (possible solutions) are obtained by making as many combinations of the possible options of each component. Some of the combinations might not be feasible, some of them might be traditional solutions that are already known, but some others can be innovative solutions that have not been imagined before and at the same time can represent a better solution to fulfil the system requirements.

A conscious exploration of the solution space is necessary to carry out structured creativity for the architecture process. All possible options for each component must be known so that innovative solutions can be determined. The solution space exploration can be a complex and long-lasting task, especially when there exist several options to fulfil the logical components. When applying structured creativity, the number of possible architecture solutions is directly linked to the options to fulfill each logical component. Figure 1.2 presents how the solution space evolves during the architecture process when performing structured creativity to identify innovative solutions. When the possible architectures are known, a trade-off analysis starts to narrow down the solution space by eliminating all non-feasible options and by assessing the feasible ones considering performance indicators and constraints identified in the initial requirements. In the end, the goal is to have one selected architecture from which the detailed design begins. There could be also the case that more than one architecture is selected to continue to the detailed design, but the number remains limited for resources, time, and budget limitations.

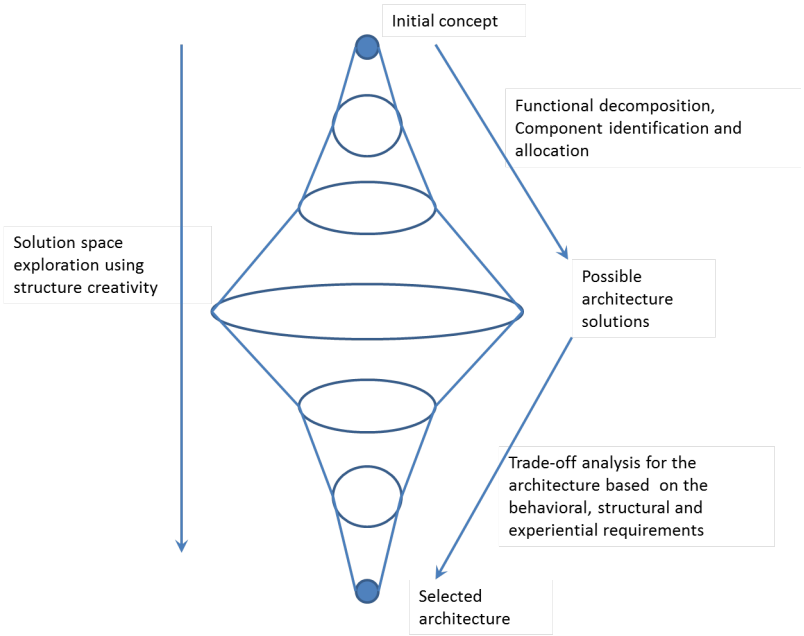


Figure 1.2: Solution space exploration using structured creativity. Inspired by [CCS15]

1.4 Research statements

Predictive maintenance is carried out by specialized systems to perform diagnostics and prognostics tasks and determine the right moment to trigger maintenance actions. The design and development of such

specialized systems is still based on trial and error. There exist several generic architectures in norms and standards, such as for example [MIM01], but the design of a new system starts long ago in the concept stage. A generic architecture does not provide a link to the initial needs and desires for a new predictive maintenance systems and often, these needs and desires are not met with a generic architecture. Besides, a generic architecture does not provide any guidelines to select the suitable technologies that can fulfil the generic components it proposes [MV19].

Predictive maintenance addresses diagnostics and prognostics tasks. But not every predictive maintenance system covers the same functions, for example a new predictive maintenance system may be intended to perform diagnostics in different components of a system and several diagnostics modules of the same type will be needed and no prognostics modules would be included. Generic architectures do not provide a systematic approach to design systems that need only a subset of their proposed components or when several components of the same type are needed.

There exists an important number of options to fulfil the diagnostics and prognostics functions in a predictive maintenance system, and there is no guidelines to help the architect select the suitable models, techniques, or algorithms that can carry out these functions. The exploration of the solution space to determine the suitable components can then be complex and long-lasting. This thesis aims at facilitating the architecture process of predictive maintenance systems by enabling a more efficient way to explore the solution space and propose the most suitable components for the system architecture.

Before deepening in the design of predictive maintenance systems, it is important to understand the research field of predictive maintenance itself and the different available options to perform diagnostics and prognostics. The following research questions are proposed to guide the state-of-the-art study in this topic:

1. What are the current trends in diagnostics and prognostics in predictive maintenance?
2. What kind of models, techniques or methods are used to address diagnosis and prognosis in predictive maintenance?
3. What are the main challenges facing predictive maintenance in diagnostics and prognostics?

After the state-of-the-art study, the research questions are refined according to the study findings and the initial motivation of this research related to the design of predictive maintenance systems (see Chapter 2).

1.5 Organization of the thesis

The following manuscript is organized in eight different chapters. The purpose of this section is to explain the structure of the message along with the different chapters. The structure of the thesis is summarized in Figure 1.3.

Chapter 2 addresses the state-of-the-art of predictive maintenance based on the initial research questions introduced in this chapter. This chapter is composed of a published article that includes the current trends in models for diagnostics and prognostics in the maintenance field. The chapter ends by refining the research questions that motivated the rest of the research.

Chapter 3 proposes a systems engineering approach to predictive maintenance design, specifically in the concept stage from the gathering of the initial needs and desires from the stakeholders until the proposal of a logical architecture. The logical architecture remains generic as no specific technology has been chosen to fulfil the architecture components. The chapter is composed of a published conference article and it ends by

presenting the predictive maintenance component selection as the main challenge that will be addressed in the following chapters. A Decision Support System (DSS) that combines ontologies and Case-Based reasoning is proposed as a possible solution to address the component selection in the systematic approach.

Chapter 4 explains one of the building blocks of the Decision Support System: ontologies. Complementary research has been performed specifically on ontologies that have turned out in a journal article, accepted for publication at the time this manuscript was written. The theoretical background of ontologies is introduced, highlighting their importance in the research community due to their capabilities to formally model domain vocabulary and perform reasoning with it. The chapter explains the development of the ontology that is later used in the DSS for component selection.

Chapter 5 explains the second building block of the DSS: Case-Based Reasoning (CBR). The principles of the CBR paradigm are introduced including the phases of the CBR cycle. In a first attempt, the DSS is focused in the retrieval phase of CBR. An explanation of the implementation of the CBR retrieval engine using open source code is provided.

Chapter 6 explains the general framework of the integration of the building blocks of the DSS and how it fits in the systematic approach to design predictive maintenance systems. The chapter is composed of a published conference article which is the continuation of the article presented in Chapter 2. Cross-validation is performed to demonstrate the DSS capabilities.

Chapter 7 is intended to further validate the DSS. A case study is selected to develop the complete approach proposed in the current research and based on the DSS suggestions one example is implemented. The results of this implementation are explained and discussed.

Chapter 8 concludes the thesis by summarizing the lessons learned, enumerating the limitations encountered during the research, and listing the perspectives of future work.

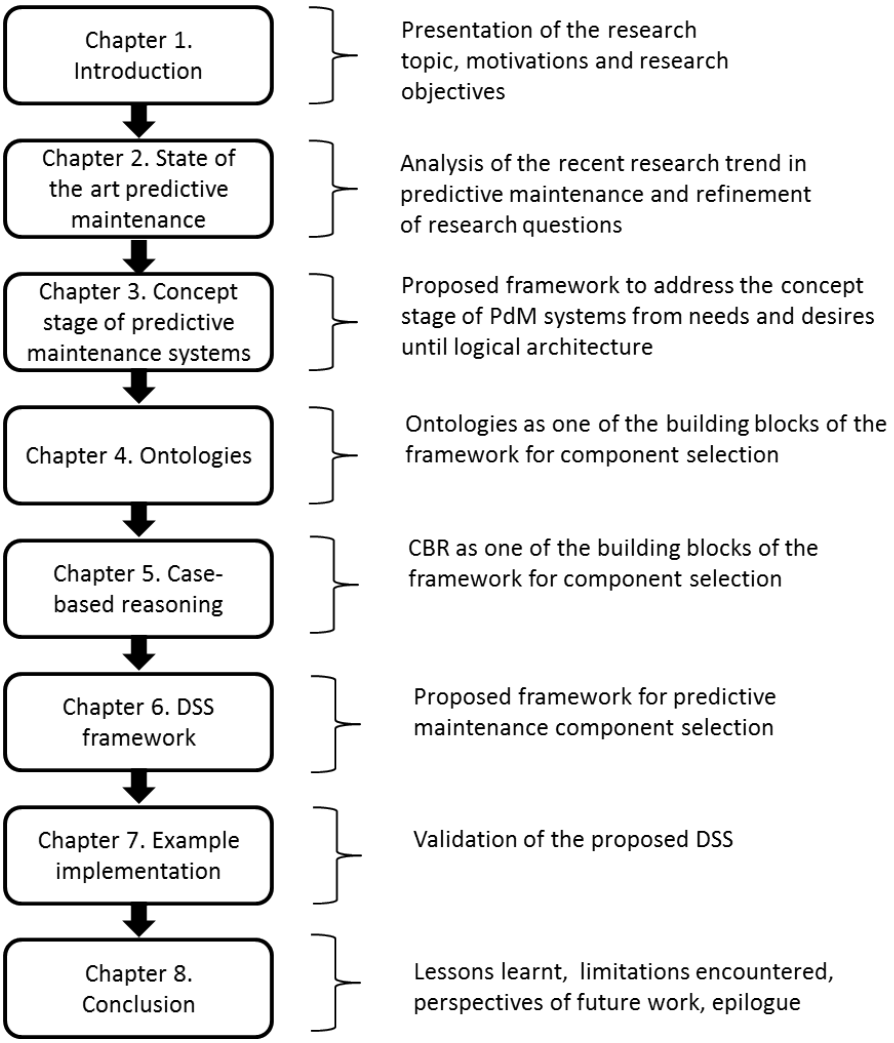


Figure 1.3: Thesis organization

CHAPTER 2

Towards multi-model approaches to predictive maintenance: A systematic literature survey on diagnostics and prognostics

"L'homme ne peut découvrir de nouveaux océans tant qu'il n'a pas le courage de perdre de vue la côte."
"Man cannot discover new oceans unless he has the courage to lose sight of the shore."

André Gide

Content

2.1 Exploring the topic and defining the state-of-the-art

Predictive maintenance has been one of the motivations of the current research. It is an important topic in which several academic laboratories and private enterprises have focused sharply because of its potential benefits. This chapter aims at determining the state-of-the-art in predictive maintenance, defining the techniques and models that can be used for diagnostics and prognostics of maintainable systems. The state-of-the-art allows understanding the current challenges that predictive maintenance faces. These challenges are the source of knowledge to refine the research questions that the rest of the thesis attempts to answer.

The following state-of-the-art study is based on the following research questions:

1. What are the current trends in diagnostics and prognostics for predictive maintenance?
2. What kinds of models, techniques or methods are used to address diagnosis and prognosis in predictive maintenance?
3. What are the current challenges facing predictive maintenance in diagnostics and prognostics?

2.2 Towards multi-model approaches to predictive maintenance: A systematic literature survey on diagnostics and prognostics (Article 1)

Journal of Manufacturing Systems 56 (2020) 539–557

Contents lists available at ScienceDirect

Journal of Manufacturing Systems

journal homepage: www.elsevier.com/locate/jmansys

Towards multi-model approaches to predictive maintenance: A systematic literature survey on diagnostics and prognostics

Juan José Montero Jimenez[a,b,*], Sébastien Schwartz[a,c], Rob Vingerhoeds[a], Bernard Grabot[d], Michel Salaün[a]

[a] ISAE-SUPAERO, Université de Toulouse, 10 Avenue Edouard Belin, 31400, Toulouse, France
[b] TEC-Tecnológico de Costa Rica, Calle 15, Avenida 14, 1 km Sur de la Basílica de los Ángeles, Provincia de Cartago, Cartago, 30101, Costa Rica
[c] Capgemini DEMS, R&D Dpt., Aeropark, 3 Chemin de Laporte, 31100, Toulouse, France
[d] ENIT- INP Toulouse, 47, avenue d'Azereix, BP 1629, 65016, Tarbes, France

ARTICLE INFO

Keywords:
Predictive maintenance
Systematic literature review
Diagnostics
Prognostics
Single-model approaches
Multi-model approaches

ABSTRACT

The use of a modern technological system requires a good engineering approach, optimized operations, and proper maintenance in order to keep the system in an optimal state. Predictive maintenance focuses on the organization of maintenance actions according to the actual health state of the system, aiming at giving a precise indication of when a maintenance intervention will be necessary. Predictive maintenance is normally implemented by means of specialized computational systems that incorporate one of several models to fulfil diagnostics and prognostics tasks. As complexity of technological systems increases over time, single-model approaches hardly fulfil all functions and objectives for predictive maintenance systems. It is increasingly common to find research studies that combine different models in multi-model approaches to overcome complexity of predictive maintenance tasks, considering the advantages and disadvantages of each single model and trying to combine the best of them. These multi-model approaches have not been extensively addressed by previous review studies on predictive maintenance. Besides, many of the possible combinations for multi-model approaches remain unexplored in predictive maintenance applications; this offers a vast field of opportunities when architecting new predictive maintenance systems. This systematic survey aims at presenting the current trends in diagnostics and prognostics giving special attention to multi-model approaches and summarizing the current challenges and research opportunities.

1. Introduction

The use of a modern multi-technological system requires a good engineering approach, optimized operations, and proper maintenance in order to keep the system in an optimal state of operation. Predictive maintenance focuses on the organization of maintenance actions according to the actual health state of the system, aiming at giving a more precise indication of when a maintenance intervention will be necessary. This is performed by using specialized models and techniques that make possible to perform diagnostics and prognostics over the multi-technological system health state.

Predictive maintenance research has a lot of attention in industry and academy due to its potential benefits in terms of reliability, safety and maintenance costs among many other benefits. As explained by [1], predictive maintenance might reduce maintenance costs by 25 %–35 %, eliminate breakdowns by 70 %–75 %, reduce breakdown time by 35 %–45 %, and increase production from 25 %–35 %. These percentages do not consider important aspects such as system safety and company image.

This article aims at performing a systematic literature review on predictive maintenance, the state of the art on the models used for diagnostics and prognostics, the current challenges, and new potential opportunities of research. Fast expanding trends such as Industry 4.0 boost the use of predictive maintenance, and the interest on the topic remains increasing. Recent reviews mainly focus on a limited scope: prognostics and data-driven models, as for example [2–4]. This motivates an update of the reviews as every year hundreds of publications related to the topic are published.

The methodology to perform the literature review is based on [5] and concerns a systematic literature review methodology that aims at summarizing the existing work of a specific topic. The systematic literature review helps to carry out the literature review process in a

* Corresponding author at: 10 Avenue Edouard Belin, 31400, Toulouse, France.
E-mail addresses: juan.montero@itcr.ac.cr, juan-jose.montero-jimenez@isae-supaero.fr (J.J. Montero Jimenez).

https://doi.org/10.1016/j.jmsy.2020.07.008
Received 16 March 2020; Received in revised form 26 May 2020; Accepted 10 July 2020

structured manner so to obtain a better overview of the subject under study. The systematic literature review protocol includes four main parts: research questions definition, search strategy, study selection, and data synthesis. The research questions are:

- RQ1: What are the current trends in diagnostics and prognostics for predictive maintenance?
- RQ2: What kinds of models, techniques or methods are used to address diagnosis and prognosis in predictive maintenance?
- RQ3: What are the current challenges facing predictive maintenance in diagnostics and prognostics?

The search was divided into two steps; the first one was aimed to check the previous literature reviews on predictive maintenance so to understand the evolution of the topic over the last years. This first search step also helped to identify the model types used for diagnostics and prognostics: knowledge-based, data-driven, physics –based and multi-model approaches. The second search step is based on trial searches using various combinations of search terms derived from the research questions so to identify the main trends in the different models for diagnostics and prognostics. Special attention is given to multi-model approaches, as these models present a promising opportunity to overcome current challenges in predictive maintenance applications. For example, more than one model could be used to address different sources of heterogeneous data, complementary models could be used to reduce uncertainty and improve accuracy on diagnostics and prognostics (see Sections 6 and 7). For both search steps, four sources were consulted: IEEE Xplore, ScienceDirect, Springer and Web of Science. The selection of studies was performed accordingly to the research questions. An assessment on each publication was performed considering the clearness of research objectives, the explanation of proposed model results and the case studies completeness. Appendix A offers further explanation of the structured literature review process followed for this survey.

The rest of the paper is organized as follow: Section 2 introduces predictive maintenance. Section 3 shows some statistics from the systematic literature review. Section 4 shows the findings of the first search step on previews reviews of the topic. Section 5 explains the main single-model approaches identified in the current literature review. Section 6 addresses the multi-model approaches. Section 7 summarizes the identified challenges for predictive maintenance as potential opportunities for future research. Section 8 concludes the current systematic literature review.

2. Predictive maintenance

Within the maintenance strategies to trigger maintenance actions, three terms are commonly applied: corrective maintenance, preventive maintenance and predictive maintenance. Corrective maintenance triggers the maintenance actions once the failure of a component or system has occurred. Preventive maintenance uses intervals of time such as cycles, kilometres, flights, etc. to determine the right moment to trigger the maintenance actions. As explained by [6], the existence of faults is frequently unknown in preventive maintenance. This may lead to replacing components with still remaining useful life, which may be costly. Predictive maintenance can be presented as a maintenance strategy aiming at defining the accurate moment to trigger actual maintenance actions [7]. Too early interventions could represent a waste of resources by changing components with an important Remaining Useful Life (RUL), too late interventions could lead to catastrophic failures. As strategy, predictive maintenance is complementary to corrective and preventive maintenance. Predictive maintenance finds its bases in using specialized techniques and tools to identify the existence of faults on the technical systems and forecast their remaining useful life. A combination of the three mentioned strategies is needed to reach an efficient maintenance management [7].

Predictive maintenance has the goal of improving maintenance activities, performance, safety and reliability [8]. It is a vast topic with two main scopes diagnostic and prognostic. Diagnostics aims at detecting faults, determining their root cause and determining the current health state of the system to prevent unexpected failures. Prognostics are dedicated to predictions of future states of the system and the remaining useful life. Diagnostics and Prognostics can be performed on-line or off-line. In online applications, data is gathered, processed and analysed in real time to generate alarms or trigger maintenance or adjustment action while the system is running. Off-line applications focus on gathering all operational information to be analysed later (off-line) by the maintenance team. They are not constrained by online real-time limitations [9].

Predictive maintenance is not a new topic. Some studies like [10] state that predictive maintenance already existed in the 1940′s and during the current systematic literature review, publications from the 1970′s were easily found [11]. However, the last 25 years show a growth of interest of the topic year by year. Two extensions of predictive maintenance are found in literature: Condition-Based Maintenance (CBM) and Prognostics and Health Management (PHM). According to [10], CBM was also introduced in the 1940′s while PHM is the most recent term, introduced in the early 2000′s [12]. These terms frequently substitute predictive maintenance in literature and there is no consistency on how these terms are used or how they fit together in

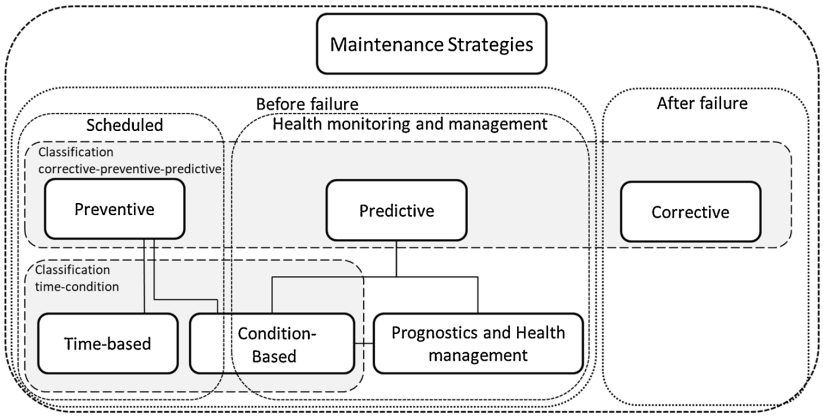


Fig. 1. An overview of Maintenance strategies.

the maintenance field. Over the last years, different contributions are made under different terms and refer to the same field of research. It has an impact on the current review. As result, the three terms were considered for the current review: predictive maintenance, CBM, and PHM. Fig. 1 shows an overview of the maintenance strategies. Predictive maintenance is traditionally grouped along with preventive and corrective maintenance [10,13]. CBM is traditionally shown as a counterpart to time-based maintenance [14]. Besides these traditional classifications, Fig. 1 shows a simple taxonomy which initially classifies the strategies between two categories with regards the maintenance actions triggering: before or after a failure occurs. For the strategies that trigger maintenance actions before failure there is sub-division between strategies with fixed schedule for maintenance actions and strategies that use health monitoring to decide the precise moment to trigger the maintenance actions. This last group includes the chosen strategies for this literature review. It is important to point out that Fig. 1 is not a hierarchical diagram. The lines connecting the different strategies only represent the existence of important commonalities among the connected ones. The clarification of potential confusion in the use of these terms is out of the scope of the current review.

Predictive maintenance is normally implemented through specialized systems which collect data or information from the technical system for diagnostics or prognostics purposes. Norms and standards like OSA-CBM [15,16] offer a list of the traditional functional blocks of these predictive maintenance systems. Fig. 2 shows a functional decomposition of a predictive maintenance system for remaining useful life (RUL) estimation on one machine component subjected to a single failure mode. It shows the traditional functional blocks: collect data (F1), pre-process data (F2), detect and identify faults (F3), assess degradation (F4), compute RUL (F5) and make report (F6). These functional blocks may be present or not in a predictive maintenance system, they could be duplicated or modified depending on the system architecture which relays on the technical system complexity, the requirements for predictive maintenance system and the available knowledge, data and/or information [17]. According to the scope of this literature review, the models used for the functional blocks F3, F4 and F5 are addressed. It is important to point out that one or more models can be used to fulfil one single functional block, see also Section 6.

3. Survey process and some statistical results

The systematic literature review shows that predictive maintenance is gaining importance in the research community, especially over the last 25 years. To illustrate this, Fig. 3 shows the number of publications mentioning the terms “predictive maintenance”, “condition based maintenance and “prognostics and health management” over the last 25 years in one of the consulted search sources (ScienceDirect). The tendency on IEEE Xplore, Springer and Web of Science is the same. The topic has a high importance in the research community; hundreds of articles are published every year with new contributions. It is important to mention that not all the papers mentioning the terms of interest are directly related to the scope of the current survey. The articles related to maintenance management practices, maintenance policies, maintenance schedule optimization, are examples of topics discarted in the scrutinity process for this survey as they were out of scope.

During the first search step 23 survey articles were consulted (see Appendix B) to identify the models used for predictive maintenance and the evolution on the trends over the years. The taxonomies used to classify the different models for diagnostics and prognostics in predictive maintenance show slight variations on the terms from one study to another. Two main approaches can be extracted: single-model approaches and multi-model approaches. For single-model approaches there are three model types; for this survey these model types will be named knowledge-based models, data-driven models and physics-based models. Multi-model approaches combine at least two models from the three mentioned models types. Multi-model approaches may have different configurations and sometimes are called hybrid models; however, not all multi-model approaches should be referred to as hybrid (see Section 6).

The identified groups were the basis for the second search step. Following the mentioned taxonomy, Table 1 shows the distribution of consulted articles from 2015 to 2019. The survey also considered studies from previous years; however, due to the scrutiny process recent articles with similar scope and case study substituted older articles. The table shows that recent papers have been consulted to illustrate each category. Further explanation of model types, their current challenges and research opportunities are discussed in Sections 4,5 and 6. Data driven models are divided into three categories to be consistent with the taxonomy shown in Section 5. The three categories are: statistical models, stochastics models and machine learning models.

An important aspect is the distribution of the mentioned models between diagnostics and prognostics as main task for the consulted studies. In the end, out of the consulted articles in the second search step, 48.9 % were dedicated to diagnostics while 51.1 % to prognostics so that it is possible to say that they have equal share. Fig. 4 shows the distribution of the consulted studies between single-model approaches and multi-model approaches for diagnostics (left part of Fig. 4) and prognostics (right part of Fig. 4), for all the consulted articles in the second search step. It is important to point out that diagnostics and prognostics are not always exclusive to each other. To perform prognostics is normal to have a previous diagnostic step to determine the current health state of the technical system to estimate future behaviours of the technical system. The main contribution presented by each consulted study in the second research step was considered to classify the scope between diagnostics and prognostics. For both, diagnostics and prognostics, single-model approaches are more presented than multi-model approaches. For diagnostics, knowledge-based models have a higher importance than for prognostics. Consulted studies on physics-based models were dedicated almost exclusively to prognostics. Data-driven models have the main part of consulted studies for diagnostics and prognostics.

4. Findings on the first search step

The first search step was dedicated to previous reviews on predictive maintenance and it helped to study the commonalities among

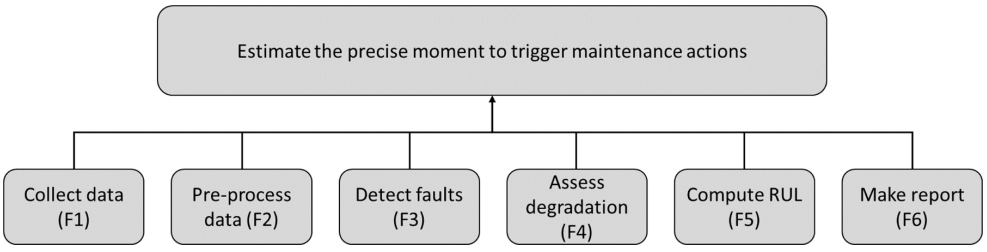


Fig. 2. Functional decomposition for an example of predictive maintenance system, modified from [17].

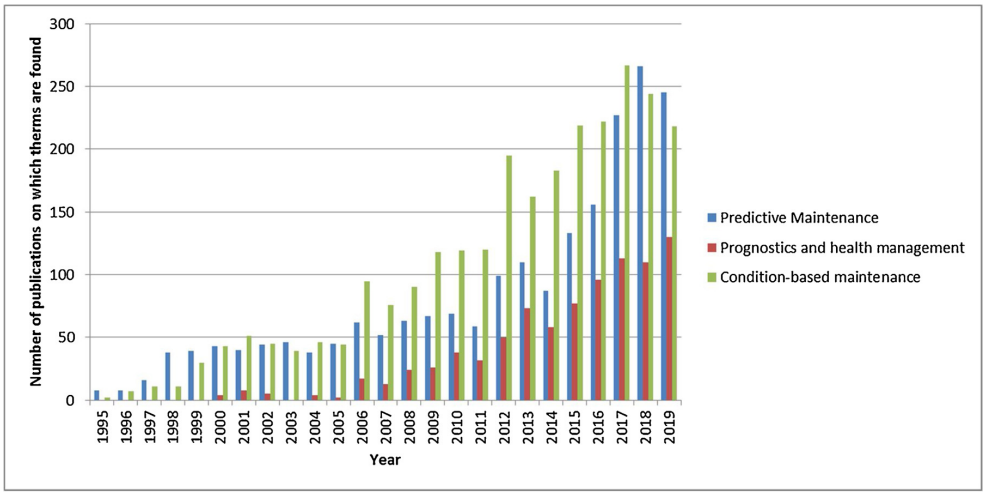


Fig. 3. Number of publications over the last 25 years related to prognostics and diagnostics in maintenance using three search terms in ScienceDirect.

Table 1
Distribution of publications per model from 2015 to 2019 in the systematic literature review on Predictive maintenance.

Approach	Model type	Models	2015	2016	2017	2018	2019
Single-model approaches	Knowledge-based models	Rule-based, Case-based, and fuzzy models	[18–20]	[21]	–	[22–24]	[25–28]
	Data-driven models	Statistical, stochastic and machine learning models	[29–33]	[34–37]	[38–49],	[50–62,7,63–75]	[76–81]
	Physics-based models	Laws of physics governing the degradation of the system.	–	[82]	[83,84]	[85]	[76,78,86]
Multi-model approaches	Different configurations	Combination of two or more models.	–	[87]	[88]	[69]	[76,78,89–94]

the terms “predictive maintenance”, “condition-based maintenance” (CBM) and “prognostics and health management” (PHM). The use of these terms is not homogeneous in literature. Sometimes CBM and PHM are presented as if they were synonyms to predictive maintenance [95,96], while other studies shown CBM and PHM as extensions or subdivisions of predictive maintenance [6,10,97–99]. Opposite statements are also found clustering predictive maintenance as sub-part of CBM [100]. The three terms were developed by different research communities and today many contributions concern similar maintenance activities done under different names. None of the consulted reviews presents an alignment of the three terms in the same study.

Trying to align these terms, this survey adopts predictive maintenance as the first term to refer the maintenance strategy. CBM is suggested as an extended version of predictive maintenance where alarms are added to warn when the system has overpassed predetermined thresholds. CBM has been used as preferred term to describe diagnostics tasks in norms [99,101] and in some referential books such as [102]. Likewise, PHM is suggested as an extension of CBM as an answer to the need to improve on predictability and life cycle management of the assets [100,102,103]. Fig. 5 shows a summary of the evolution on the use of the terms considering the consulted reviews. When performing literature research, it is then worthy to consider the three terms.

Besides a general notion of the terminology, the first search step of the current survey allowed the study of evolution of the research trends on the topic. Out of the 22 consulted reviews, the first one dates from

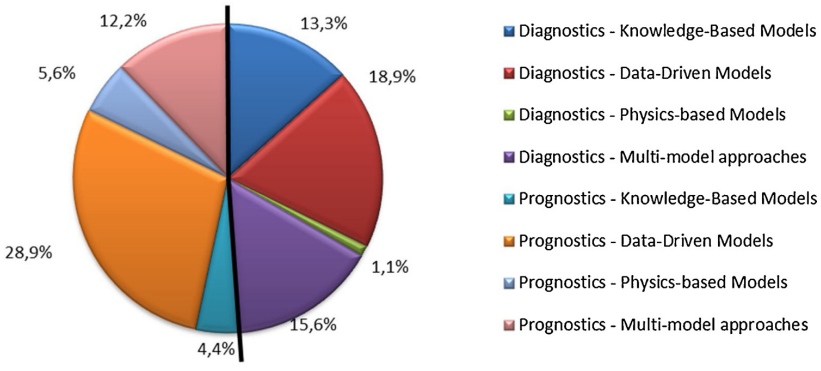


Fig. 4. Studies distribution for diagnostics and prognostics considering the consulted papers for the second search step.

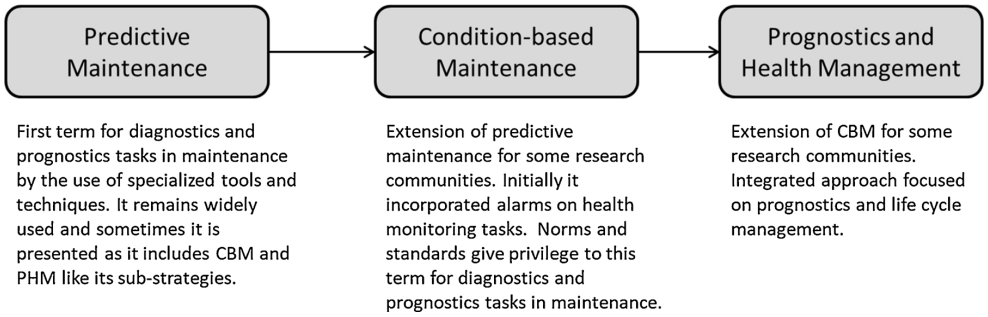


Fig. 5. Relationship among Predictive Maintenance, CBM and PHM.

2006 and is frequently cited by others [104]. This first survey dedicates a different section for diagnostics a prognostics, summarizing the most important techniques for each approach (only single-model approaches). Other survey articles have more specific subtopics that where addressed: RUL estimation through vibration data analysis on bearings and gears [105]; prognostics with a bound scope on data-driven or physics based models [106]; or on bibliometric indicators for predictive maintenance [107]. More general overviews on predictive maintenance models were covered by conference papers [97,108]. These conference articles have a limited scope to only few models and examples.

The most complete reviews are [2] and [3], from 2017 and 2018 respectively. These two reviews gave privilege to the term PHM to address the topic and offer an overview from the data collection to the decision making process (all functional blocks mentioned in Section 2); however, their main scope is on predictability and remaining useful life (RUL) estimation. They both respect the taxonomy of four model types (with slight differences in the naming): knowledge-based, data-driven, physics-based and hybrid (partially addressed). These two articles update and extend the work done by previous survey studies on prognostics and RUL estimation [109–111]. The latest consulted survey is from 2019 and is exclusively dedicated to data-driven methods for prognostics tasks, especially those related to machine learning and deep learning [4]. A summary of the previous reviews could be found in Appendix 2.

The consulted reviews on predictive maintenance are mainly focusing on single-model approaches, data-driven models and prognostics. The second search step covers these gaps giving special attention to the use of multi-model approaches in both diagnostics and prognostics. The following Sections (5, 6 and 7) cover the findings of the second search step based on a complementary point of view to recent reviews to cover the mentioned gaps.

5. Single-model approaches for predictive maintenance

This section briefly introduces to single-model approaches used for diagnostics or prognostics, their strengths and weaknesses and how recent studies already implement complementary models to fulfil the intended tasks in predictive maintenance systems and overcome complexity. It is a complementary point of view to recent and complete review that have extensively covered single-model approaches. The models in this section are divided into three model types: knowledge-based models, data-driven models and physics-based model.

5.1. Knowledge-based models

Knowledge-based models build upon experiences. Experience can be represented by rules, facts or cases that have been gathered over the years of operation and maintenance of the technical system [9,112,113]. Experience can be used to identify faults, describe the degradation and forecast a potential failure of components or systems. These rules, facts or cases, can be used in computational intelligence techniques to automate the inference on diagnostics and prognostics for maintenance purposes. It was the state of the art of maintenance in the early 1990's. Publications such as [9,114,115] describe how knowledge based models were used to perform diagnostics in technical systems. Knowledge based models remain an important field of research for maintenance purposes and three main topics were identified in the systematic literature review: rule-based models, case-based models and fuzzy knowledge-based models.

Rule-based models are knowledge-based models in which the knowledge is represented by rules in the format "IF-THEN", allowing to perform an inference supposed to simulate a simplified reasoning mechanisms of human experts [112]. Rule-based systems consist of a knowledge base gathering all the rules, a fact base and an inference engine. The inference using rules is an iterative process. Initial "facts" are used as inputs. The inference engine compares these inputs with the set of rules contained in the knowledge base and produces conclusions as outputs. The inference engine uses these conclusions as new facts to be compared again with the set of rules so that new conclusions are obtained. This process is repeated depending on the inference engine design until the reasoning process comes to an end. Fig. 6 shows a simplified generic model of a rule-based system for diagnostics and prognostics.

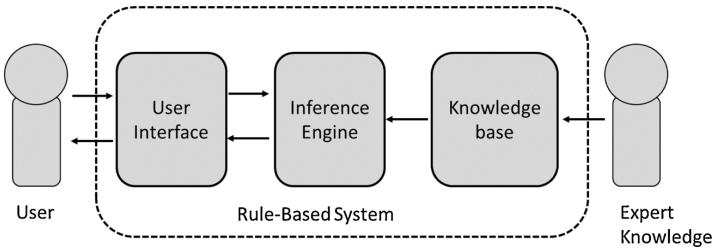


Fig. 6. Basic RBS inspired on [18].

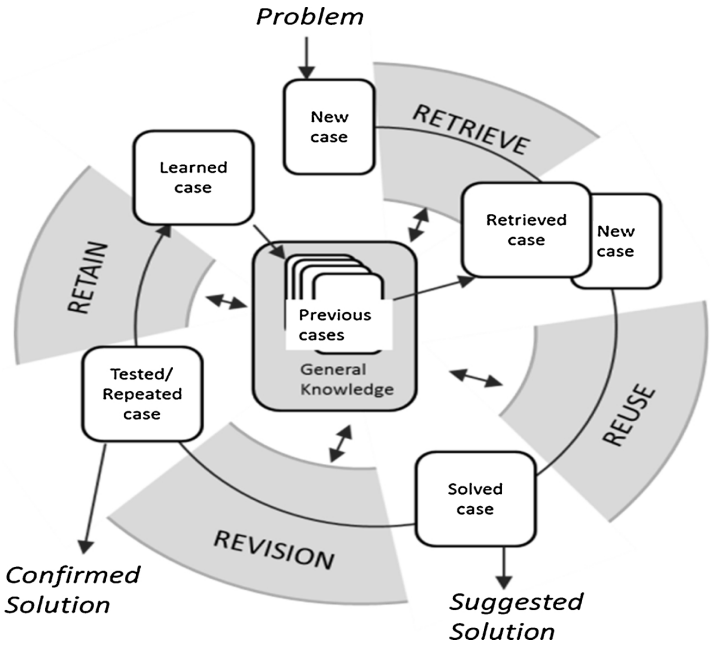


Fig. 7. The Case-based reasoning cycle [116].

Case-based models are knowledge-based models whose knowledge representation is through cases, obtained from previously experienced, concrete problem situations [116]. Cases are normally represented by a paired knowledge, like for example (problem, solution), in a case base. When facing a new problem, the most similar case is retrieved from the case base. Once a similar case has been identified, its "solution" is reused to adapt the solution for the new. There is a revision to confirm if the suggested solution solves the new problem. If the solution is confirmed, the new case can be retained as learnt knowledge in the case base. Fig. 7 shows the standard case-based reasoning cycle. Unlike rule-based reasoning, case-based reasoning can be used when the relations between facts cannot be declared explicitly [9]. A case is described by a set of attributes that could be numeric data and/or text-based data. Finding the relevant attributes to describe the cases is a difficult task when developing case-based systems.

Fuzzy knowledge-based models use basically the same format of rules IF-THEN as rule-based systems but the statements use intentionally fuzzy logic [110]. Unlike Boolean logic in which a proposition can only be true or false, in fuzzy logic there are intermediate values to describe the level of truth or falsehood of a statement [117]. Fuzzy logic is strongly related with human perceptions. Symbolic linguistic terms such as 'hot', 'cold', 'small', 'large', etc., are frequently used. This characteristic makes fuzzy logic an important tool for uncertainty management. Fuzzy logic can be also used in case-based reasoning and other data-driven models.

Knowledge-based models find limitations for prognostics as it is very difficult to obtain accurate knowledge for predictability purposes from experience. The identified examples in the current literature review are more related to diagnostic tasks and those which are intended for prognostics also include complementary models to estimate remaining useful life. Another drawback of knowledge-based models is the limited access to experts or knowledge sources to build the systems. Current trends in knowledge-based models use data mining techniques to extract the required knowledge from databases. [118,119] are examples for rules extraction while [22,120] aim at extracting cases from databases. One strong point of knowledge-based models forms the explicative results they offer [101]. It is possible to explain each reasoning step these models perform, it makes easier to justify their implementation against authority regulations for safety-critical systems, such as aircraft or nuclear power plants. Table 2 shows a summary of the identified applications of knowledge-based models in the current systematic literature review.

5.2. Data-driven models

Data-driven models have gained a lot of importance in recent years thanks to the improved availability of computational power and the production of large amounts of data coming every day from technical systems. Modern technical systems include an important number of operational parameters constantly measured and recorded. The resulting high volume of data can be used explicitly or implicitly for many purposes, including maintenance. Information obtained from data can be used to study the degradation of components, the current health state of the system or its remaining useful life. Fig. 8 shows an example of jet-engines degradation assessment based on trend analysis of measured data [7]. For the current survey, data-driven models are classified in three groups: statistical models, stochastic models and machine learning models.

One of the main challenges for data-driven models is the management of uncertainty coming from data [3,104,108,127]. Probability theory plays a vital role in data-driven models, as it is the most common way to manage uncertainty. Other studies use Dempster–Shafer models [31,40] (evidence theory), fuzzy logic [128] or possibility theory to manage data uncertainty.

5.2.1. Statistical models

Statistical models aim at analysing the behaviour of random variables based on recorded data. For predictive maintenance, statistical models are used to determine the current degradation and the expected remaining life of the technical systems. This is performed by comparing their current behaviour of measured random variables against known behaviours represented by series of data. Normalization and data

Table 2
Summary of identified applications for knowledge-based models in this systematic literature review.

Model	References	Tasks	Case Studies	Complementary models
Rule-based models	[18,19,21,26,118,121,122]	Fault diagnostic, root cause analysis, RUL estimation (along with complementary techniques)	Power circuit breakers, overhead cranes, mining excavators, lubricant pipe abrasion, high voltage circuits, xenon lamps, oil pipe lines	Markov models, Grey theory, Bayesian model, data mining models.
Case-based models	[20,9,120,123,124]	Fault diagnostic, RUL estimation (limited number of applications)	Induction motors, power equipment, railway systems, simulated jet-engines data	Rule-based systems, data mining models.
Fuzzy knowledge-based models	[23,24,125,126]	Fault identification, uncertainty management	Grinding wheels, oil pumps, rolling bearings, simulated jet-engines data.	Data mining models.

cleaning are common preliminary tasks performed on data series to obtain the distribution function before the trend analysis. This prevents from outliers, constants, binary or any other variable that is not useful for degradation analysis.

For degradation analysis, the trend analysis of random variables is vital. The random variables must show a correlation with operational time or any other non-random variables that describe the lifecycle of the technical system. This correlation will show the evolution of degradation along the life cycle. For instance [129], used correlation to select the variables to describe the degradation on jet engines data. Covariance evaluations are frequently performed when the degradation is described by several variables [30]. Statistical models are also used for prognostics. Regression analysis will help to determine the relationship between the random variables and the life cycle of the technical systems so that a computation of future behaviours is possible.

Besides regression analysis there are two other statistical approaches that stand out: Autoregressive models and Bayesian models. Autoregressive-moving average models (ARMA) are statistical models for which a future value of a random variable is assumed to be a linear function of past observations and random errors [110]. ARMA models and their variants [50,130–133] are used to forecast future values of data series. Autoregressive-models have the advantage of simplicity in their computation. However, as they rely on statistical degradation trends, their accuracy could be affected when assessing new degradation trends where no previous information was available [3].

Bayesian models are those which apply Bayesian theorem [108], a statistical inference method to estimate conditional probability. It computes the probability of an hypothesis based on the prior (initial) probabilities of events that are related to the hypothesis [134]. Finding these prior probabilities poses the main problem for Bayesian theorem application. For predictive maintenance purposes, Bayesian models can be applied when data including anticipated failures with their corresponding symptoms and life expectancy is available [96,108]. Bayesian models play an important role on data-driven models for predictive maintenance, specially combined with other data-driven models to manage uncertainty [36,53,54,135]. Table 3 presents a summary of the applications of these two statistical approaches identified in this literature review.

Statistical models offer an important number of potential solutions to fulfil diagnostic and prognostic tasks. The main drawbacks of statistical models concern the need of enough previous data to build a reliable model and uncertainty management. For predictive maintenance systems, statistical models are often implemented in multi-model approaches.

5.2.2. Stochastic models

Stochastic models are probability models aiming at the study of the evolution of random variables over time [134]. The building blocks of stochastic models are stochastic processes. In the literature review, three main stochastic processes were identified for diagnostics and prognostics: Gaussian processes, Markov processes and Levy processes.

- A *Gaussian process* is a collection of random variables or any finite variable number of which have a joint Gaussian distribution [137]. Gaussian processes can be used for non-linear regression [138]. This property has motivated the use of Gaussian processes for diagnostics and prognostics in the maintenance field. According to [139] Gaussian processes are flexible models to work with small or large-dimensional datasets for prognostics purposes. However, it requires a high computational power to perform the predictive tasks.
- *Markov chains* are part of a bigger family of stochastic tools called Markov processes [140]. Markov chains suppose that given a process in its present state, the future depends on the present state independently of the past of the process. According to [110] the main shortcomings of Markov models for predictive maintenance are: 1) the need of large volume of data for training, 2) the impossibility to

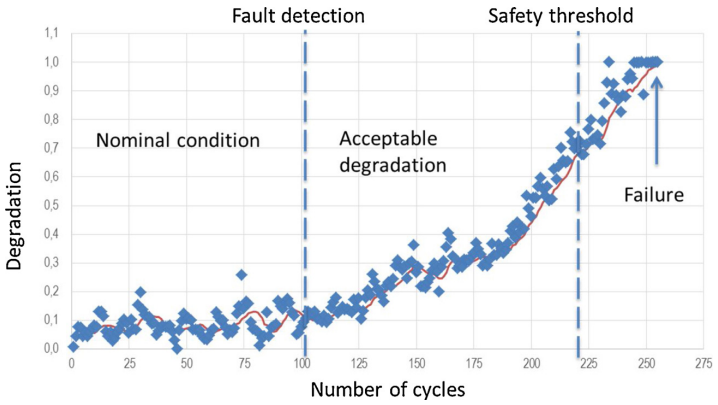


Fig. 8. An example of degradation analysis based on series of data [7].

model different degradation stages, and 3) the impossibility to model unanticipated failures or faults. As these models cannot be used to model different degradation stages, they are not suitable for reparable components that have been partially restored. It should be noted that the model complexity increases when the degradation does not follow an exponential trend [3].

- Lévy *processes* are stochastic processes within the family of Markov processes [141]. These processes represent the motion of random variables whose displacements are independent and stationary within time intervals of the same length [142]. Weiner processes, Gamma processes and Poisson processes belong to the category of Lévy processes used for predictive maintenance. Extensive reviews in Lévy processes for predictive maintenance can be found in [59,60]. Lévy share the general limitations of Markov processes. In prognostics, Lévy processes are bound to monotonic degradation processes [3,110].

Table 4 presents a summary of the stochastic models addressed in this literature review. It can be seen that this type of models is more suitable for degradation modelling and RUL estimation because of their regression capabilities. These models have many drawbacks in common, such as for example high computational power requirements, advanced mathematical knowledge to be implemented and uncertainty management. Complementary techniques or models are often used along stochastic models.

5.2.3. Machine learning models

Machine learning is a branch of artificial intelligence [96] that uses specialized learning algorithms to build models from data. These models are capable of dealing with and capturing complex relationships among data, difficult to obtain using physics-based, statistical or stochastic models. One key point of machine learning models is their learning process and depends on the application, goal and the available data for the system [146].

- *Supervised learning* is preferred when the expected outcomes of the model and data under study are known. Its training is an iterative process assessing the output error against the expected one. The training finishes when an "acceptable" level of error is reached.
- *Unsupervised learning* is used when no preliminary outcomes are known. No error levels are measured to assess or to end the training process. These algorithms use other criteria to end the training process, such the number of training iterations or the progress of a convergence indicator over time [147]. Clustering is an example of tasks performed by unsupervised learning algorithms.
- *Reinforcement learning* aims to train a model by experience instead of examples [148]. The model "interacts" with an environment and receives a "reward" depending on the interaction. This reward is linked to a performance indicator that the learning algorithm tries to optimize. The final outcomes of the learning are not known.

Within the identified machine learning models for predictive maintenance applications, artificial neural networks are computational models inspired by biological neural networks in an attempt to mimic their unique processing capabilities [149]. They consist of elementary units called "neurons", usually represented graphically as nodes in a graph. Neurons are processing units which receive several inputs and produce one or multiple outputs that may be the input for other neurons. A neuron's output is equal to the weighted sum of its inputs values by means of an activation function. The learning process of the neural network aims at choosing and adjusting the weights of the neurons' inputs. Neurons are organized into layers. These layers can be organized in different configurations (architectures). For predictive maintenance the most used configurations are multi-layer perceptron neural networks, recurrent neural networks (including long-short term memory neural networks), convolutional neural networks, self-organizing maps and support vector machines with different variants. An extensive explanation of these models can be found in the reviews [3] and [4].

The learning process used to train the neural-network will depend on its architecture and the available data, information or knowledge for training. As [99] suggests, artificial neural networks do not need in-depth knowledge of dysfunctions of the technical system which makes artificial neural networks a strong tool to get implicit knowledge from the data.

Table 5 summarizes some applications of machine learning models identified in this literature review. Opposite to other predictive maintenance models, machine learning approaches might not include all the functional blocks F3, F4 and F5 mentioned in Section 2 (see Fig. 2). Some neural network applications with several internal layers of neurons (Deep Learning) aim at letting the algorithm learn from raw data to obtain directly the desired outcome, whether it is diagnostic or prognostic. Even when good results are already obtained, a comprehensive explanation of the trained algorithm behaviour is even more difficult to be justified against regulations on safety-critical systems, such as aircraft or nuclear power plants. Once the model has been trained, it is difficult to explain how it works, what is reasoning behind in the model. Explaining the reasoning inside a trained machine learning model is a promising opportunity of research for the coming years. Even when publications are found in this area (e.g. [150]), they do not cover predictive maintenance applications.

Table 3
Summary of identified applications of statistical models in predictive maintenance.

Model	References	Tasks	Case Studies	Complementary models
Regression analysis	[34,38,39,76,129,136,75]	RUL estimation, fault diagnostics, health assessment	Power transformer, electric cooling fan, simulated jet engine data, air compressor, aluminium plates, lithium-ion batteries, bearings	Other statistical models, physics-based models
ARMA	[50,131–133]	RUL estimation	Bearings, aircraft generators, structural damage, semiconductors switches	Other statistical models, Neural Networks
Bayesian models	[36,41,53,135,33]	Stochastic degradation parameter identification and update. Fault diagnostics	Laser machines, sensors, high power circuits, circuit boards.	Stochastic models, other statistical models, Monte-Carlo simulation.

Table 4
Summary of identified studies applying stochastic models for predictive maintenance.

Model	References	Task	Case Study	Complementary models
Gaussian Process	[77,143]	Fault diagnostics, RUL prediction	Wind Turbines, slow speed bearings	For signal processing
Markov Chains and Hidden Markov chains	[36,52,56,57,122,144]	Produce stationary distribution for RUL computation, Degradation simulation	Milling machines, simulated jet-engine data, semiconductor manufacturer machine, continuous stirred tank reactor, asphalt roads	Bayesian model, genetic algorithm, belief rule-based model, Monte Carlo.
Levy process	[43,58,145]	RUL computation, Degradation Modelling	Simulated jet-engine data, high-power white LEDs, automotive engine cranking	Proportional hazard model, combination of different stochastic models

Table 5
Summary of identified applications for machine learning models.

Model	Reference	Task	Learning process	Case Study	Complementary models
MLP	[44,45,63,64]	Fault identification	Supervised	Wind turbine gear boxes, combustion engines, nuclear power plants, rotary machines	Signal processing models, radial basis function network, multiple NN
RNN	[148,46,47,66,74]	Fault diagnostic, RUL estimation	Supervised, reinforcement	Gear boxes, jet engines, mill fans, rolling bearings	Different NN
CNN	[67,73,79,80]	Fault diagnostics, RUL estimation	Supervised	Jet engines, gear boxes, bearings.	Statistical models, Extreme learning machine, autoencoder
SOM	[2,32,68,94,127],	Degradation modelling, fault detection	Unsupervised	Jet Engines, Cyber-physical systems, railway point machines	Statistical models
SVM, SVR	[34,37,70–72,72,81,151]	Health assessment, fault detection, prognostics	Supervised, Unsupervised	Battery cells, metal-mechanics equipment, electrical equipment, chemical industry, chillers, Tennessee Eastman process, industrial simulated data	Statistical models, RNN (LSTM)

NN: Neural Network. MLP: Multi-layer Perceptron. RNN: Recurrent Neural Network. CNN: Convolutional Neural Network. SOM: Self-Organizing Maps. SVM: Support Vector Machines. SVR: Support Vector Regression. LSTM: Long-short memory Neural Network.

5.3. Physics-based models

Physics-based models use the laws of physics to assess the degradation of components. They demand high skills on mathematics and physics of the phenomena for the application. This kind of mathematical model remains an important topic of research with interest for many disciplines. With an accurate model of the physical behaviour of a system it is possible to perform accurate simulations to study the degradation behaviour on a specific component or a system. Within the identified studies of physics-based models there are fatigue and crack propagation models for mechanical and structural components [102,152]. With the computational power rising over the last decades, the use of finite element methods has increased for damage propagation and failure prediction, some identified examples are [83] on a rotor cage and [85] on solenoid valves. Other physics based model have been used to study the tube erosion of boiler head exchangers [86], clogging prognostics on filters of fluids [82], degradation evaluation of industrial robots [84] and remaining useful life estimation on lithium-ion batteries [76]. Physics-based models offer a possibility to study and assess degradation by means of computational simulations. However, many physics phenomena cannot yet be accurately described. The outcomes of a physics-based model will be as good as the "accuracy" or "completeness" of the model. The operational context of a technical system affects its performance. External influence such as temperature, pressure or any other environmental conditions might drastically change the expected operational parameters and the actual behaviour. Incorporating external influence data is a challenge already mentioned by other studies such as [108,153]. These may be solved by adding complementary models (potentially other physics-based model).

6. Multi-model approaches for predictive maintenance

Single-model approaches hardly address all the diagnostics and prognostics tasks of complex systems; the consulted studies with single-model approaches often proposed complementary models to overcome the weak points of some models. It is increasingly common to find research studies that combine different models in multi-model approaches to overcome complexity of predictive maintenance tasks. Increasing complexity includes the number potential faults and failure modes of the technical system, the type and number of information and/or data sources obtained from it and the number of diagnostics and prognostics tasks that are targeted, all these apart from the design complexity of the selected model. Most of the consulted studies had limited case studies with only few failure modes (sometimes only one) which poses a challenge to extrapolate single-model approaches to real complex systems applications [98,103]. Identified studies that had more complex case studies usually applied multiple models to fulfil the predictive maintenance system tasks.

However, even when the consulted studies in this section usually had simple and limited case studies, multiple models were often involved. As explained in Section 2, predictive maintenance systems include different functional blocks depending on their initial requirements and the complexity of the available knowledge, data and/or information for the implementation. A multi-model approach is often implemented to fulfil all functional blocks for the predictive tasks (except for some deep learning approaches, see Section 5.3).

A related term to multi-model approaches is "hybrid model". Hybrid models are usually presented as the fourth classification of model types along with knowledge-based, data-driven and physics-based. However, there exist many multi-model approaches which cannot be named as hybrid. The definition of a hybrid model evolves over the different consulted publications. After a careful analysis this literature review suggests that hybrid models are part of multi-model approaches in which two or more models are combined to fulfil one single functional block (F3, F4 or F5 in Fig. 2, see Section 2) of the predictive maintenance systems and there is mutual cooperation among the combined

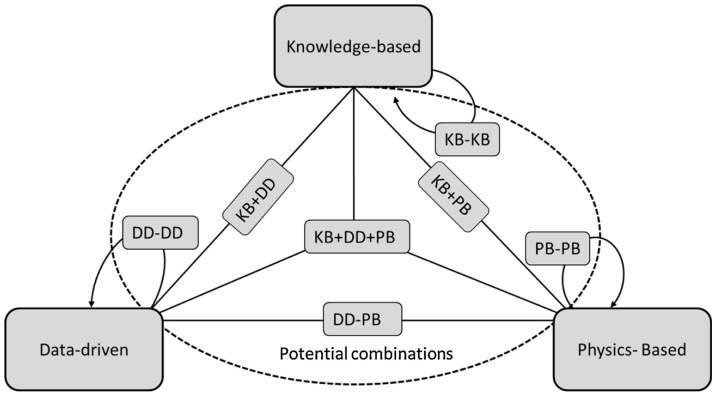


Fig. 9. Potential combinations for multi-model approaches.

models to obtain their outputs.

An introduction to the notion of different types of multi-model approaches (under the name of hybrid models) can be found in [154]. The authors classified them into 5 groups: knowledge-based models combined with data-driven models, knowledge-based models combined with physics-based models, combination of multiple data-driven models, data-driven models combined with physics-based models, and a combination of one models of each type. However, multi-model approaches can also include two more categories not mentioned in their proposed hybrid model taxonomy: combination of multiple knowledge-based models and combination of multiple physics based models. Fig. 9 presents a diagram of the potential combinations of multi-model approaches for predictive maintenance purposes.

6.1. Configurations for multi-model approaches

Before presenting the findings on the different model combinations and expand on the potential opportunities for future research in multi-model approaches, it is important to explain how they could be combined from a systems architecture point of view. There are many possible combinations, however, there are three basic configurations on top of which more complex architectures can be built: models working in series, models working in parallel, a model working as a subpart of another model (embedded model), see Fig. 10. These configurations explain the flow of information, data or knowledge through the predictive maintenance system. When architecting new predictive maintenance systems, it is important to consider the potential configurations in order to find the "best" solutions to fulfil the requirements for the system.

Two models are *in series* when the output of a first model is the input for a second one. The functional blocks presented in Section 2 present intuitively a configuration in series where a single model is used to fulfil each functional block. For example [147] presents a series configuration of SOM along with a statistics model using probability density function to address the functional blocks of degradation modelling and fault detection. Nevertheless, as complexity in the information or data increases, two complementary models could be used in series to fulfil a single functional block. Multi-model approaches using a series configuration are not usually referred to as hybrid, even when combined models are used to fulfil one single functional block; there is no mutual cooperation among the models to obtain the outputs.

Two models are *in parallel* when they process their input simultaneously and their outputs are combined in a single one. It is important to point out that the input could be the same for both models working in parallel or they could have different but related inputs. For example, given a technical system, one model could address text-based data (from operational or maintenance logs), while another could address measured data from sensors. Paper [2] gives a good example of a multi-model approach in parallel, with a data-driven model to address all the data coming from the technical system along with a physics-based model for RUL computation for bogie components. Two parallel models fulfilling one single functional block are usually referred as a hybrid model as there is mutual cooperation between the models to obtain the final result.

For the *embedded* configuration a model is incorporated as a subpart of another one. [155] presents for example a neuro-fuzzy model including a hidden Markov model as part of its internal functioning. Actually, neuro-fuzzy models could be seen as an example of a model embedding another one. They implement a fuzzy inference system within a neural-network architecture. Some identified applications of neuro-fuzzy models in the current survey are degradation prognostics on bearings [90] and fault diagnostics on railways track circuits [156]. Paper [94] combines a Kalman filter embedded an online sequential extreme learning machine (OS-ELM) for remaining useful life

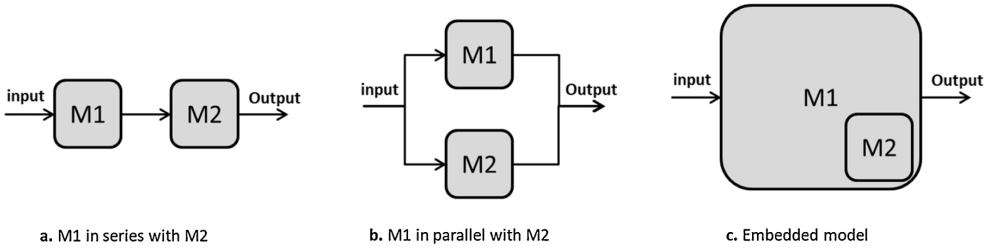


Fig. 10. Generic basic configurations for multi-model approaches.

estimation, and refer to it as KFOS-ELM. Another example for embedded models could be a case-based reasoning system embedding a rule-based system for one or more tasks of the case-based reasoning cycle as proposed by [157]. Multiple models in embedded configuration are usually referred as a hybrid model.

The configurations shown in Fig. 10 could include more than two models and could be combined among them. This means that one can propose for example a set of many parallel models (more than two) followed by another model to combine their result, which is what paper [69] proposes for fault diagnostics on aircraft turbojet engines. They present nine data-driven models in parallel and in the end, the outputs where assessed and combined with a knowledge based model. Having a clear idea of the basic configurations and their potential combinations allows expending the creativity process at architecting new predictive maintenance systems.

6.2. Combinations for multi-model approaches

This section summarizes the survey findings on the different combinations for multi-model approaches presented in Fig. 10. The explanation of the identified examples for each combination includes the architecture configuration used to combine the different models, and which functional blocks of predictive maintenance system they are covering.

6.2.1. Multiples knowledge-based models

This type of multi-model approaches has not been widely used in recent research on predictive maintenance applications. A multi-model approach using only knowledge-based models keeps the same challenges as for single-model approaches and besides, it has an additional difficulty component at designing the combination of multiple models. Nevertheless, the combination of multiple knowledge-based models allows addressing complex diagnostics tasks while explaining the reasoning. The authors of [158] present a case-based system combined with a rule-based system for problem diagnostic in IT services, the rule-based system is embedded in reuse phase of the case-based system. A comprehensive review of case-based, reasoning systems combined with other knowledge-based models can be seen in [157].

6.2.2. Multiple data-driven models

This approach combines several data-driven models to perform either diagnostics or prognostics. Neural networks are the most used data-driven models to build this kind of multi-model approaches because of the current trends in machine learning and deep-learning research that aim at incorporating more autonomous and intelligent systems. However, as mention in Section 5.2.3 these models still face problems as their results remain non-explicative [101]. Within the examples found in this literature review [159], presents a multi-layer perceptron and a radial basis function neural network in a parallel model to estimate the remaining useful life from input sensors on simulated jet-engines data [160]. performs fault prognostic with a mixture of Gaussian hidden Markov model (stochastic model) to evaluate the health index and fixed size least squares support vector regression (statistical model) for remaining useful life estimation on the same jet-engines simulated data. A more recent article [91] suggests a hybrid deep learning neural network for RUL estimation on the simulated jet-engines data. The authors propose a parallel analysis of the input data by a convolutional neural network and a long-short term neural network. The fusion of the results is done by three layers of neurons with different activation functions. Some of the presented examples show that the combination of data-driven models gives more accurate results compared to single-model approaches for the same tasks.

6.2.3. Multiple physics-based models

Multiple physics-based models can be used to increase the accuracy of a more general model. The precision usually brought by the laws of physics embedded in a mathematical model may indeed allow improving the accuracy of diagnostics and prognostics estimations. Most of the identified applications of multiple physics-based model approaches present a series configuration. Initially, a model is used to assess the health state of the technical system (diagnostics), then, another model for remaining useful life estimation (prognostics) [161] mixes physics based models (crack and fatigue models) for helicopter gear prognostics [162] applied a physic-based model to predict the machine condition (i.e. diagnostic), complemented by Forman crack growth remaining useful life estimation in a series configuration [163]. presents an example of multiple physics-based models in parallel configuration. The multiple Kalman filter models are used for single fault detection in jet-engines using temperature, pressure and rotation speed as parameters. The use of multiple physical based models is not widespread. Their implementation requires high skills in mathematics, physics and a large knowledge of the technical system under study, making their implementation difficult. However, they offer a large set of opportunities to obtain accurate and explicative results in the predictive maintenance domain. Several commercial solutions for finite-element analysis include multi-physics models working in parallel to improve the accuracy of the technical system model. Such solutions are widely used for structures and fluid dynamics modelling and simulations [83,85].

6.2.4. Knowledge-based models with data-driven models

This multi-model approach has allowed taking advantage of the strong points of both model types. Knowledge-based models could incorporate valuable information from human experts to complement the results of data-driven models for diagnostics or prognostics tasks. For example [87], presents a combination of a fuzzy knowledge-based model and Markov chain for degradation prognostics in aero-engines. The already mentioned neuro-fuzzy systems are other examples of fuzzy logic combined with a data-driven model [90,156,69]. presents a rule-based system to summarize and combine the diagnostics coming from multiple neural networks assessing the same aero- engines data base. Knowledge-based combined with data-driven models offer a vast field of opportunities to innovate at architecting new predictive maintenance systems, allowing analyzing more complex and heterogeneous data coming not only from sensors but also from declared data obtained from the technical system operators or extracted from large databases [118,119] by means of data-mining techniques. This declared data, normally assessed by knowledge base systems may reduce the uncertainty in data-driven models. One example of this is presented in [2] on a train suspension case study.

6.2.5. Knowledge-based models with physics-based models

These models use the experts' knowledge to improve the accuracy of physics-based models. The number of studies related to this approach is limited for predictive maintenance applications as this combination gathers the main drawbacks from both model types: difficulty to gather the experts' semantic knowledge and high mathematics complexity to develop physics-based models. However, a strong point of this model combination is the high explicative results they may offer. Within the identified studies [164], combines a fuzzy knowledge-based system with a physics based model for different prognostics tasks on mechanical parts. Potential unexplored applications could include hybrid parallel models using knowledge-based systems to address the declared knowledge by the technical system user, combined with a physics-based model to model its degradation. Another application option could be this multi-model approach to incorporate external influence data to predictive maintenance models. External factors affect directly the performance of technical systems and so their degradation. It is a challenging unexplored field that could offer many interesting solutions in the maintenance field.

6.2.6. Data-driven models with physics-based models

This multi-model approach is the most common in recent research because of the increasing popularity of data-driven models and their complementarity with physics-based models for degradation modelling. Within the possible combinations of data-driven and physics-based models, three main combinations were identified for predictive maintenance applications:

- Statistic models with physics-based models such as [165] uses a physics-based model to build a health index that is later analysed by a support vector-machine to estimate the health state of the system, fitted with an exponential regression. A similarity-based approach is finally used to compute the remaining useful life.
- Stochastic models with physics-based models, such as [145] that uses a stochastic process (Wiener process) combined with a data analysis method (Principal Component Analysis) to model the deterioration of the components that is fitted by an exponential physical degradation, and to estimate the remaining useful life on a case study [92]. presents a more recent example of this type of combination. It uses a physics-based model along with hidden Markov chains and particle filter model for RUL computation of railway tracks.
- Neural network models with physics-based models, such as [166] that use a multi-layer network to generate the system health state, then a physical degradation model of exponential type is used to evaluate the remaining useful life. [167] presents a regression vector machine (which is already a combination of two data-driven models) along with a physics based model to predict the remaining useful life of aluminium plates under fatigue stresses. This is not precisely predictive maintenance but the predictability approach can be extended to other mechanical components.

6.2.7. Knowledge- based models with data-driven models and physics-based models

This approach combines at least one of each model type. It benefits from the strengths of every model type, the explicability from knowledge-based models, and physics-based models and the ability to analyse past data to gather additional important information. As an illustration [168], combines a physics based models with a support-vector machine to obtain an analytical health index of rolling bearings, the results combination is performed by a fuzzy rule-based system [169]. proposes the combination of the three model types to address the diagnostics and prognostics of rolling bearing, it presented the models to address different but complementary input data. The number of studies in predictive maintenance combining the three model types remains limited, based on the literature findings. As [2,154] state, this combination could be extremely difficult. The implementation of a multi-model approach combining knowledge-based, data-driven and physics-based models not only represents the individual complexity of each model type but also includes the complexity at architecting the whole system and fusing the outputs of each model.

6.3. Some general observations on multi-model approaches

The development of multi-model approaches of any type of the mentioned combinations face particular challenges. Besides the difficulty to develop the individual models composing the multi-model approach, their combination poses additional challenges. The lack of systematic approach for designing predictive maintenance systems is an important challenge [108,170]. However, multi-model approaches present a vast field of opportunities in research for the coming years. The number of unexplored alternative combinations remains huge. These multi-model approaches may help incorporating external influence data, semantic knowledge from experts, and the laws of physics governing the degradation of components or systems to manage uncertainty and improve the accuracy in diagnostics and prognostics. Also, combining different models may give the opportunity to extrapolate diagnostics and prognostics approaches (today focused on single failure mode applications) to complex systems that include many components with many failure modes.

7. Summary of the identified challenges and research opportunities

This systematic literature review allowed the study of the different models, the different approaches to implement them, their benefits, drawbacks and challenges for diagnostics and prognostics in predictive maintenance. This section summarizes the most relevant challenges for diagnostics and prognostics identified in the literature review.

- *The extrapolation of existing solutions to complex system applications, including multiple components, and their associated faults* [98,103]. Most of the identified applications were focused on a single component with a limited number of faults. However, real-life applications are frequently complex systems composed of many components and many faults associated to each component and to the system itself. Multi-model approaches offer a potential solution to overcome complexity in predictive maintenance systems. As [171] states, complexity can be reduced by functional decomposition and later each function can be addressed individually. As explained in Section 2, a predictive maintenance system could have several functional blocks for each component and/or failure mode in a complex system. It is necessary at least one model to fulfil each functional block for each component and failure mode. However, the implementation of these models is not trivial and adds another complexity factor for the model combination.
- *The lack of a systematic approach to design and develop predictive maintenance systems* [108,170]. There exist standards, norms and generic architectures to develop new predictive maintenance systems, such as OSA-CBM [15]. However, they only focus on the basic functional components of the system and do not cover important aspects regarding performance indicators or context constraints of the technical system. Besides, they do not offer yet a consistent explanation on which models to use depending on the initial needs of the predictive maintenance system. The lack of a systematic approach limits the implementation of predictive maintenance systems on real scale industrial applications. When developing a new predictive maintenance system the number of potential models to solve the problem is too high. For engineers the simple fact of choosing the right model or a reduced set of models remains a challenging task. It turns out to be very difficult to perform an objective selection of models as there are not enough comparative studies of the use of different models on the same tasks for predictive maintenance systems. None of the consulted publications in this survey gives extensive explanations for the selection of the proposed method and the architecture methodologies to create a concept of the system varies from one study to another. Besides, many studies do not present detailed design parameters for their proposed models, or the case study data is not available. All these aspects make it difficult to reproduce results and even more difficult to retrieve models from previous studies for use in new predictive maintenance systems. There are no clear guidelines for selecting the right model or models for a specific task given the operational modes and available data to perform diagnostics and prognostics.
- *The fusion of large and different sources of condition monitoring data* [3,153]. This challenge is related to the extrapolation of current models in predictive maintenance to complex technical systems. Technical systems may have different types of data sources, for example sensor measurements, maintenance logs, operational logs, design documents, etc. Important knowledge could be gathered from all these sources to implement new predictive maintenance systems. However, the heterogeneity of these data sources makes

knowledge modelling and fusion a difficult task for predictive maintenance purposes. Today, the main part of studies uses time series to perform diagnostics and prognostics and important information coming from text-based data is frequently ignored [17]. Text-based data is difficult to analyse when it is not in a structured form. Current trends in maintenance aim at analysing natural text log to extract information that can be used to improve maintenance tasks. Different models are needed to address text-based data while others address measured data from sensors. Multi-model approaches can be used to fuse heterogeneous data sources.

- *The incorporation of external influence data* [108,153]. Systems operation may differ depending on their operational context. Changes in the operational context may affect directly the performance of the technical system and consequently the readings on the health monitoring. It may trigger false alarms suggesting fault existence, or it may prevent existing fault identification. This could be addressed by complementary models able to incorporate the external influence for predictive maintenance purposes.
- *Uncertainty management* [3,104,108,127]. Uncertainty affects directly the accuracy of the diagnostics and prognostics. It can be due to the collected data or to imperfections of the model used for the analysis. It may affect the trustworthiness of the results. Uncertainty management is vital for critical systems subject to authorities' regulations. This is the case for critical systems like nuclear power plants and aircrafts on which the regulations are restrictive to keep safety standards and avoid catastrophic events. Probability theory, Dempster–Shafer theory and fuzzy logic have been the most common techniques used to manage uncertainty observed in the systematic literature review. Multi-model approaches may be a solution to address uncertainty in complex systems.

8. Conclusion

This systematic literature review performed on predictive maintenance shows that its importance in research has been increasing dramatically over the last 25 years. The search was performed initially using three related terms: "Predictive Maintenance", "Condition Based Maintenance" and "Prognostics and Health Management". Different contributions were found under the different terms but referring to the same activities. Considering all the terms helped to have a wider overview on the current trends used for diagnostics and prognostics in the maintenance field. The survey allowed to answer research question 1 by identifying two main approaches for model implementation: single model approach and multi-model approach. The current trends lead towards the use of multi-model approaches as one single model is not able to cover all necessary functional blocks in a predictive maintenance system.

Deepening in both approaches, the survey allowed to answer research question 2. The identified models for single-model approaches can be clustered in knowledge-based models, data-driven models and physics-based models. A brief explanation of the most single models used in recent research was presented in Section 5. It is a complementary point of view to other recent reviews that already covered single model approaches. Most of the consulted papers already used complementary models but they are not presented as multi-model approaches. For multi-model approaches, seven different combinations considering the model type were identified: knowledge-based models combined with data-driven models, knowledge-based models combined with physics-based models, data-driven models combined with physics-based models, combination of multiple data-driven models, combination of multiple knowledge-based models, combination of multiple physics-based models, and the combination of one model from each model type. Some of these combinations have not been widely explored in predictive maintenance applications. Besides, three basic configurations are presented to perform the combination of models: in series, in parallel and embedded. Out of these basic configurations more complex architectures can be conceived. Depending on the configuration and the task to fulfil, multi-model approaches can be named hybrid models; however, not all multi-model approaches are hybrid models. There must be mutual cooperation among the models to be a true hybrid model.

To answer the research question 3, the identified challenges are the extrapolation of current solutions on diagnostics and prognostics to complex systems, the lack of a systematic approach for predictive maintenance system design, fusion of different types of data sources, incorporation of external influence data and uncertainty management. These challenges open a branch of opportunities for future research in the topic.

Declaration of Competing Interest

Juan José Montero Jiménez, Sébastien Schwartz, Rob Vingerhoeds, Bernard Grabot and Michel Salaün declare that they do not have conflict of interests.

Appendix A. Systematic literature review process

The methodology used in this paper concerns a systematic literature review methodology that aims at summarizing the existing work of a specific topic [5]. Systematic literature reviews help to carry out the literature review process in a structured manner to ensure impartial results and thus a better overview of the subject under study [5]. stresses the importance of a well-structured protocol to carry out the systematic literature review. This protocol spans from the planning of the review until its reporting. For the work presented here, the protocol consists of four steps: research questions definition, search strategy, study selection, and data synthesis. The protocol is summarized in Fig. A1.

Research questions

The systematic literature review starts by defining the Research Questions (RQ) that drive the review process to define the state of the art of a specific topic and identify the opportunities for future research, setting the boundaries for the search. As the goal of the survey is an update of models or techniques used for diagnostics and prognostics for predictive maintenance, the research questions were chosen as follow:

- RQ1: What are the current trends in diagnostics and prognostics for predictive maintenance?
- RQ2: What kinds of models, techniques or methods are used to address diagnosis and prognosis in predictive maintenance?
- RQ3: What are the current challenges facing predictive maintenance in diagnostics and prognostics?

Search strategy

In the search strategy phase, the search terms and resources are selected, as well as the time lapse to be covered by the search. For predictive maintenance, the strategy divides the search into two steps, and both of them considered the last 25 years as time lapse. Older papers were consulted

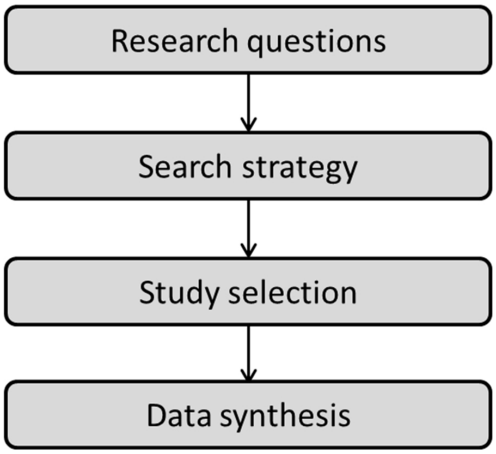


Fig. A1. Systematic literature review protocol [5].

to identify when exactly some of the terms started to be used. Four search sources were selected: IEEE Xplore, ScienceDirect, Springer and Web of Science.

The first search step focuses on existing literature reviews on predictive maintenance to check if any of them answer to the proposed research questions. To do so, the following search terms pattern have been used: ("Predictive" OR "Prognosis" OR "Diagnosis" OR "Prognostics" OR "Diagnostics") AND ("Maintenance" OR "Condition-based" OR "Health Management") AND ("Survey" OR "Review" OR "Benchmark"). This leads to search "Predictive Maintenance Survey" or "Diagnostics Maintenance Benchmark" for example. Since many terms used for predictive maintenance are also present in medical research, a refinement on the search is done by discarding all papers and journals on human medicine and diseases. The search on previous literature reviews topics helped to study the evolution of the topic over the last 14 year since the first review was published in 2006 [104]. It was possible to identify some research opportunities that were used to perform the second research step.

The second search step is based on trial searches using terms derived from the identified models in the first search step. The search terms have been refined for this search step. The following terms pattern have been used: ("Predictive" OR "Prognostics" OR "Diagnostics" OR) AND ("Maintenance" OR "Condition-based" OR "Health monitoring") AND ("Data-driven" OR "Physics-based" OR "Knowledge-based" OR "Hybrid" OR "Multi-model). The terms "diagnosis" and "prognosis" were not used in this second search step as there were no differences when using "diagnostics" and "prognostics" in the first search step.

Study selection

The search considers publications from four different search sources. The types of publications could be from journals articles, conference proceedings, workshops, symposiums, bulletins or book chapters. For the scrutiny of relevant publications, a first analysis through the titles is performed. The search was limited to publications in the English language. If a publication appeared in more than one search list, it was considered only once. All publications out of the scope of the research questions were discarded such as publications regarding maintenance scheduling optimization, maintenance management, corrective maintenance, scheduled maintenance, signal processing, and requirements elicitation for maintenance systems. Some publications turned out to be extensions of previous published works. In such cases, only the most recent and complete were considered. The references of these identified publications were consulted to identify relevant studies missed in the search process. In the end, 187 relevant publications were found, from which 23 are previous reviews and 164 correspond to the second search step.

After the scrutiny, a quality assessment of the publications took place to consider only the most relevant publications. For the first research step, all identified reviews were kept. For the second research step, four characteristics were assessed for the quality assessment: the clearness of research objectives, the explanation of proposed model to fulfil a specific task, the case study explanation, and the comparison to other models or approaches.

The consulted studies were ranked with points. For each study, each characteristic is ranked with one out three possible values: 0, 0.5 and 1. The max score for a publication is 4 points. For this review, publications with a score higher than 3 points were kept. This process narrowed down the number of publications from 187 to 158; from which 23 are previous reviews and 135 correspond to the second search step. An important reason to discard certain papers was the absence of sufficient explanation of the case study and/ comparisons with other existing models to fulfil the same tasks.

The current paper includes 175 references, 158 are from the structured literature review and 17 of them are for theoretical background of some models. These background references were not identified in the systematic literature review.

Appendix B. Summary of previous reviews

Year	Author	REF.	Single model approach			Multi-model approach
			Physics-based	Data-driven	Knowledge based	
2006	Jardine et al.	[104]	Physics of failure	Statistical models, AI models (FFNN, CCNN)	Experts systems, Fuzzy Logic	No

2006	Kothamasu et al.	[6]	N/A	Statistics and Stochastic models. Bayesian models and Markov models	Rule-based systems and Fuzzy logic	No
2007	Vachtsevanos et al.	[102]	FEM, Physics of failure	ANN, stochastic and statistics	Expert systems	Yes
2009	Dragomir et al.	[109]	Physics of failure	AI techniques, ANN, NFL, Bayesian, Markov models	N/A	
2009	Liu et al.	[113]	N/A	HMM, ANN,	Experts systems, Fuzzy Logic	No
2010	Peng et al.	[172]	First principle modeling.	ANN, state space model, hazard rate, proportional hazard rate, gray model	Experts systems, Fuzzy Logic	Yes
2011	Sikorska et al.	[110]	Physics of failure	Statistical models, stochastic models, ANN	Experts systems, Fuzzy Logic	Yes
2012	Prajapati et al.	[10]	N/A	Statistics, stochastic, artificial intelligence	Expert systems	Yes
2014	Okoh et all	[111]	Physics of failure	Statistics and Stochastic.	Experts systems	Yes
2014	Liao and Köttig	[154]	In hybrid models	In hybrid models	In hybrid models	Yes
2015	An et al.	[106]	Physics of failure	Neural Networks, Gaussian process regression	Out of scope	Yes
2015	Bailey et al.	[173]	Physics of failure	Statistical, Machine learning.	Out of scope	Yes
2015	Schmidt and Wang	[108]	Physics of failure	Stochastic, Statistic, ANN (Bayesian)	Experts systems	Yes
2016	Elattar et al.	[174]	Physics of failure	Probabilistic models, machine learning	Reliability-models	Yes
2016	Vanraj et al.	[175]	FEM	ANN with BPN, SOM	N/A	No
2017	Alaswad and Xiang	[98]	N/A	Markov, Gamma process, Gaussian, among others.	N/A	No
2017	Javed et al.	[96]	Physics of failure	Machine learning, ANN, Bayesian, MC, NFL, CBR.	Included in the data-driven models.	Yes
2017	Wang et al.	[105]	Physics of failure	Statistical models, machine learning	N/A	No
2017	Atamuradov et al.	[2]	Paris' Law, Forman Law, Others	ARIMA, Gaussian models, ANN, Bayesian Network	Experts systems, Fuzzy Logic	Yes
2018	Lei et al.	[3]	Physics of failure	Statistics, Stochastic, ANN, SVM/RVM	N/A	Yes
2018	Sakib and Wuest	[97]	N/A	MC Bayesian, MC, Machine learning, Monte Carlo.	N/A	No
2019	Zhang and Yang	[4]	N/A	ANN, DNN, Logistic regression, SVM, Random Forest, auto-encoder.	N/A	No

References

[1] Sullivan GP, Pugh R, Melendez AP, Hunt WD. Operations & maintenance best practices: a guide to achieving operational efficiency. U S Dep Energy, Fed Energy Manag Progr. 2010. https://doi.org/10.2172/1034595.

[2] Atamuradov V, Medjaher K, Dersin P, Lamoureux B, Zerhouni N. Prognostics and health management for maintenance practitioners - review, implementation and tools evaluation. Int J Progn Heal Manag 2017.

[3] Lei Y, Li N, Guo L, Li N, Yan T, et al. Machinery health prognostics: a systematic review from data acquisition to RUL prediction. Mech Syst Signal Process 2018;104:799–834. https://doi.org/10.1016/j.ymssp.2017.11.016.

[4] Zhang W, Yang D, Wang H. Data-driven methods for predictive maintenance of industrial equipment: a survey. IEEE Syst J 2019;13:2213–27. https://doi.org/10.1109/JSYST.2019.2905565.

[5] Kitchenham B, Charters S. Guidelines for performing systematic literature reviews in software engineering. Eng Rep EBSE-2007-01, Keele Univ Univ Durham 2007. https://doi.org/10.1145/1134285.1134500.

[6] Kothamasu R, Huang SH, Verduin WH, Kothamasu R, Huang SH, Verduin WH. System health monitoring and prognostics -a review of current paradigms and practices. Int J Adv Manuf Technol 2006;28:1012–24. https://doi.org/10.1007/978-1-84882-472-0_14.

[7] Montero Jimenez JJ, Vingerhoeds R. Enhancing operational fault diagnosis by assessing multiple operational modes. Proc. - Int. Conf. Model. Optim. Simul. MOSIM 2018. 2018. p. 237–44.

[8] Adams S, Malinowski M, Heddy G, Choo B, Beling PA. The WEAR methodology for prognostics and health management implementation in manufacturing. J Manuf Syst 2017;45:82–96. https://doi.org/10.1016/j.jmsy.2017.07.002.

[9] Vingerhoeds RA, Janssens P, Netten BD, Aznar Fernández-Montesinos M. Enhancing off-line and on-line condition monitoring and fault diagnosis. Control Eng Pract 1995;3:1515–28. https://doi.org/10.1016/0967-0661(95)00162-N.

[10] Prajapati A, Bechtel J, Ganesan S. Condition based maintenance: a survey. J Qual Maint Eng 2012;18:384–400. https://doi.org/10.1108/13552511211281552.

[11] Scott D, Westcott VC. Predictive maintenance by ferrography. Wear 1977;44:173–82. https://doi.org/10.1016/0043-1648(77)90094-1.

[12] Tinga T, Loendersloot R. Aligning PHM, SHM and CBM by understanding the physical system failure behaviour. Proc. Eur. Conf. Progn. Heal. Manag. Soc. 2014.

[13] Kothamasu R, Huang SH, Verduin WH. System health monitoring and prognostics - A review of current paradigms and practices. Handb Maint Manag Eng 2006;28:1012–24. https://doi.org/10.1007/978-1-84882-472-0_14.

[14] Albrice D, Branch M. A deterioration model for establishing an optimal mix of time-based maintenance (TbM) and condition-based maintenance (CbM) for the enclosure system. Fourth Build. Enclos. Sci. Technol. Conf. 2015.

[15] MIMOSA. Open system architecture for condition-based maintenance (OSA-CBM) Available on 2001Http://WwwMimosaOrg/Mimosa-Osa-Cbm/.

[16] Lebold M, Reichard K, Byington CS, Orsagh R. OSA-CBM architecture development with emphasis on XML implementations. Maint Reliab Conf 2002.

[17] Montero Jiménez JJ, Vingerhoeds R. A system engineering approach to predictive maintenance systems: from needs and desires to logical architecture. 5th IEEE Int. Symp. Syst. Eng. 2019, Edinburgh 2019. https://doi.org/10.1109/ISSE46696.2019.8984559.

[18] Hussain A, Lee SJ, Choi MS, Brikci F. An expert system for acoustic diagnosis of power circuit breakers and on-load tap changers. Expert Syst Appl 2015;42:9426–33. https://doi.org/10.1016/j.eswa.2015.07.079.

[19] Zhou A, Yu D, Zhang W. A research on intelligent fault diagnosis of wind turbines based on ontology and FMECA. Adv Eng Informatics 2015;29:115–25. https://doi.org/10.1016/j.aei.2014.10.001.

[20] Gang M, Linru J, Guchao X, Jianyong Z. A model of intelligent fault diagnosis of power equipment based on CBR. Math Problems Eng 2015.

[21] Chemweno P, Pintelon L, Jongers L, Muchiri P. I-RCAM: intelligent expert system for root cause analysis in maintenance decision making. 2016 IEEE Int. Conf. Progn. Heal. Manag. ICPHM 2016 2016. https://doi.org/10.1109/ICPHM.2016.7542830.

[22] Zhong Z, Xu T, Wang F, Tang T. Text case-based reasoning framework for fault diagnosis and predication by cloud computing. Math Probl Eng 2018:10. https://doi.org/10.1155/2018/9464971.

[23] Baban M, Baban CF, Moisi B. A fuzzy logic-based approach for predictive maintenance of grinding wheels of automated grinding lines. 2018 23rd Int. Conf. Methods Model. Autom. Robot. MMAR 2018 2018. https://doi.org/10.1109/MMAR.2018.8486144.

[24] Berredjem T, Benidir M. Bearing faults diagnosis using fuzzy expert system relying on an Improved Range Overlaps and Similarity method. Expert Syst Appl 2018;108:134–42. https://doi.org/10.1016/j.eswa.2018.04.025.

[25] Li-Qiang H, Chao-Feng H, Zhao-Quan C, Long W, Teng R. Track circuit fault predicition method based on grey theory and expert systems. J Vis Commun Image Represent 2019;58:37–45.

[26] Tang X, Xiao M, Liang Y, Zhu H, Li J. Online updating belief-rule-base using Bayesian estimation. Knowl-Based Syst 2019;171:93–105. https://doi.org/10.1016/j.knosys.2019.02.007.

[27] Boral S, Chaturvedi SK, Naikan VNA. A case-based reasoning system for fault detection and isolation: a case study on complex gearboxes. J Qual Maint Eng 2019;25:213–35. https://doi.org/10.1108/JQME-05-2018-0039.

[28] Vafaei N, Ribeiro RA, Camarinha-Matos LM. Fuzzy early warning systems for condition based maintenance. Comput Ind Eng 2019;128:736–46. https://doi.org/10.1016/j.cie.2018.12.056.

[29] Bagheri B, Siegel D, Zhao W, Lee J. A stochastic asset life prediction method for large fleet datasets in big data environment. ASME 2015 Int. Mech. Eng. Congr. Expo. Vol. 14 Emerg. Technol. Saf. Eng. Risk Anal. Mater. Genet. to Struct. 2015. https://doi.org/10.1115/IMECE2015-52458.

[30] Menon S, Jin X, Chow TWS, Pecht M. Evaluating covariance in prognostic and system health management applications. Mech Syst Signal Process 2015;58–59:206–17. https://doi.org/10.1016/j.ymssp.2014.10.012.

[31] Verbert K, De Schutter B, Babuška R. Reasoning under uncertainty for knowledge-based fault diagnosis: a comparative study. IFAC-Papers OnLine 2015;48:422–7. https://doi.org/10.1016/j.ifacol.2015.09.563.

[32] Jin W, Shi Z, Siegel D, Dersin P, Douziech C, Pugnaloni M, et al. Development and evaluation of health monitoring techniques for railway point machines. 2015 IEEE Conf. Progn. Heal. Manag. Enhancing Safety, Effic. Availability, Eff. Syst. Through PHAf Technol. Appl. PHM 2015 2015. https://doi.org/10.1109/ICPHM.2015.7245016.

[33] Dababneh A, Ozbolat IT. Predictive reliability and lifetime methodologies for circuit boards. Int J Ind Manuf Syst Eng 2015;37:141–8. https://doi.org/10.1016/

j.jmsy.2015.08.001.
[34] Berecibar M, Devriendt F, Dubarry M, Villarreal I, Omar N, Verbeke W, et al. Online state of health estimation on NMC cells based on predictive analytics. J Power Sources 2016. https://doi.org/10.1016/j.jpowsour.2016.04.109.
[35] Mosallam A, Medjaher K, Zerhouni N. Data-driven prognostic method based on Bayesian approaches for direct remaining useful life prediction. J Intell Manuf 2016;27:1037–48. https://doi.org/10.1007/s10845-014-0933-4.
[36] Zhang D, Bailey AD, Djurdjanovic D. Bayesian identification of hidden Markov models and their use for condition-based monitoring. IEEE Trans Reliab 2016. https://doi.org/10.1109/TR.2016.2570561.
[37] Xiao Y, Wang H, Xu W, Zhou J. Robust one-class SVM for fault detection. Chemometr Intell Lab Syst 2016;151:15–25. https://doi.org/10.1016/j.chemolab.2015.11.010.
[38] Lee W-J. Anomaly detection and severity prediction of air leakage in train braking pipes. Int J Progn Heal Manag 2017;8:12. https://doi.org/10.1007/978-3-319-60045-1_22.
[39] Barraza-Barraza D, Tercero-Gómez VG, Beruvides MG, Limón-Robles J. An adaptive ARX model to estimate the RUL of aluminum plates based on its crack growth. Mech Syst Signal Process 2017;82:519–36. https://doi.org/10.1016/j.ymssp.2016.05.041.
[40] Verbert K, Babuška R, De Schutter B. Bayesian and Dempster–Shafer reasoning for knowledge-based fault diagnosis–A comparative study. Eng Appl Artif Intell 2017;60:136–50. https://doi.org/10.1016/j.engappai.2017.01.011.
[41] Li G, Wang X, Yang A, Rong M, Yang K. Failure prognosis of high voltage circuit breakers with temporal latent Dirichlet allocation. Energies 2017;10:1913. https://doi.org/10.3390/en10111913.
[42] Chen Z, Li Y, Xia T, Pan E. Hidden Markov model with auto-correlated observations for remaining useful life prediction and optimal maintenance policy. Reliab Eng Syst Saf 2017;184:123–36. https://doi.org/10.1016/j.ress.2017.09.002.
[43] Yung KC, Sun B, Jiang X. Prognostics-based qualification of high-power white LEDs using Lévy process approach. Mech Syst Signal Process 2017;82:206–16. https://doi.org/10.1016/j.ymssp.2016.05.019.
[44] Singh K, Malik H, Sharma R. Condition monitoring of wind turbine gearbox using electrical signatures. 2017 Int. Conf. Microelectron. Devices, Circuits Syst. ICMDCS 2017 2017. https://doi.org/10.1109/ICMDCS.2017.8211718.
[45] Gajewski J, Vališ D. The determination of combustion engine condition and reliability using oil analysis by MLP and RBF neural networks. Tribol Int 2017;115:557–72. https://doi.org/10.1016/j.triboint.2017.06.032.
[46] Zhao R, Wang D, Yan R, Mao K, Shen F, Wang J. Machine health monitoring using local feature-based gated recurrent unit networks. IEEE Trans Ind Electron 2017;65:1539–48. https://doi.org/10.1109/TIE.2017.2733438.
[47] Dong D, Li XY, Sun FQ. Life prediction of jet engines based on LSTM-recurrent neural networks. 2017 Progn. Syst. Heal. Manag. Conf. PHM-Harbin 2017 - Proc. 2017. https://doi.org/10.1109/PHM.2017.8079264.
[48] Mathew J, Luo M, Khiang Pang C. Regression kernel for prognostics with support vector machines. 22nd IEEE Int. Conf. Emerg. Technol. Fact. Autom. 2017.
[49] Shuangshuang J, Zhenhuan W, Yudong F, Guoan Y. The remaining life prediction of the fan bearing based on genetic algorithm and multi-parameter support vector machine. 5th Int. Conf. Mech. Automot. Mater. Eng. 2017.
[50] Haque MS, Bin Shaheed MN, Choi S. RUL estimation of power semiconductor switch using evolutionary time series prediction. 2018 IEEE Transp. Electrif. Conf. Expo, ITEC 2018 2018. https://doi.org/10.1109/ITEC.2018.8450131.
[51] Wan A, Gu F, Chen J, Zheng L, Hall P, Ji Y, et al. Prognostics of gas turbine: a condition-based maintenance approach based on multi-environmental time similarity. Mech Syst Signal Process 2018;109:150–65. https://doi.org/10.1016/j.ymssp.2018.02.027.
[52] Hu Y-W, Zhang H-C, Liu S-J, Lu H-T. Sequential Monte Carlo method toward online RUL assessment with applications. Chin J Mech Eng 2018:31. https://doi.org/10.1186/s10033-018-0205-x.
[53] Tang D, Sheng W, Yu J. Dynamic condition-based maintenance policy for degrading systems described by a random-coefficient autoregressive model: A comparative study. Eksploat I Niezawodn - Maint Reliab 2018;20:590–601. https://doi.org/10.17531/ein.2018.4.10.
[54] Wang ZQ, Hu CH, Fan HD. Real-time remaining useful life prediction for a nonlinear degrading system in service: application to bearing data. IEEE/ASME Trans Mechatron 2018;23:211–22. https://doi.org/10.1109/TMECH.2017.2666199.
[55] Zhao S, Makis V, Chen S, Li Y. Evaluation of reliability function and mean residual life for degrading systems subject to condition monitoring and random failure. IEEE Trans Reliab 2018;67:13–25. https://doi.org/10.1109/TR.2017.2779322.
[56] Kinghorst J, Geramifard O, Luo M, Chan HL, Yong K, Folmer J, et al. Hidden Markov model-based predictive maintenance in semiconductor manufacturing: a genetic algorithm approach. IEEE Int. Conf. Autom. Sci. Eng. 2018. https://doi.org/10.1109/COASE.2017.8256274.
[57] Wu Z, Luo H, Yang Y, Lv P, Zhu X, Ji Y, et al. K-PdM: KPI-oriented machinery deterioration estimation framework for predictive maintenance using cluster-based hidden Markov model. IEEE Access 2018;6:41676–87. https://doi.org/10.1109/ACCESS.2018.2859922.
[58] Man J, Zhou Q. Prediction of hard failures with stochastic degradation signals using Wiener process and proportional hazards model. Comput Ind Eng 2018;125:480–9. https://doi.org/10.1016/j.cie.2018.09.015.
[59] Zhang Z, Si X, Hu C, Lei Y. Degradation data analysis and remaining useful life estimation: a review on Wiener-process-based methods. Eur J Oper Res 2018;271:775–96. https://doi.org/10.1016/j.ejor.2018.02.033.
[60] Vališ D, Mazurkiewicz D. Application of selected Levy processes for degradation modelling of long range mine belt using real-time data. Arch Civ Mech Eng 2018;18:1430–40. https://doi.org/10.1016/j.acme.2018.05.006.
[61] Aye SA, Heyns PS. Prognostics of slow speed bearings using a composite integrated Gaussian process regression model. Int J Prod Res 2018;56:4860–73. https://doi.org/10.1080/00207543.2018.1470340.
[62] Kong D, Chen Y, Li N. Gaussian process regression for tool wear prediction. Mech Syst Signal Process 2018;104:556–74. https://doi.org/10.1016/j.ymssp.2017.11.021.
[63] Ayo-Imoru RM, Cilliers AC. Continuous machine learning for abnormality identification to aid condition-based maintenance in nuclear power plant. Ann Nucl Energy 2018;118:61–70. https://doi.org/10.1016/j.anucene.2018.04.002.
[64] Luwei KC, Yunusa-Kaltungo A, Sha'aban YA. Integrated fault detection framework for classifying rotating machine faults using frequency domain data fusion and artificial neural networks. Machines 2018:6.
[65] Luo H, Huang M, Zhou Z. Integration of Multi-Gaussian fitting and LSTM neural networks for health monitoring of an automotive suspension component. J Sound Vib 2018;428:87–103. https://doi.org/10.1016/j.jsv.2018.05.007.
[66] Hinchi AZ, Tkiouat M. Rolling element bearing remaining useful life estimation based on a convolutional long-short-Term memory network. Procedia Comput Sci 2018;127:123–32. https://doi.org/10.1016/j.procs.2018.01.106.
[67] Li X, Ding Q, Sun JQ. Remaining useful life estimation in prognostics using deep convolution neural networks. Reliab Eng Syst Saf 2018;172:1–11. https://doi.org/10.1016/j.ress.2017.11.021.
[68] Von Birgelen A, Buratti D, Mager J, Niggemann O. Self-organizing maps for anomaly localization and predictive maintenance in cyber-physical production systems. Procedia CIRP 2018;72:480–5. https://doi.org/10.1016/j.procir.2018.03.150.
[69] Nyulászi L, Andoga R, Butka P, Főző L, Kovacs R, Moravec T. Fault detection and isolation of an aircraft turbojet engine using a multi-sensor network and multiple model approach. Acta Polytech Hungarica 2018;15:189–209.
[70] Li S, Fang H, Shi B. Multi-step-ahead prediction with long short term memory networks and support vector regression. Chinese Control Conf. 2018. https://doi.org/10.23919/ChiCC.2018.8484066.
[71] Laib dit Leksir Y, Mansour M, Moussaoui A. Localization of thermal anomalies in electrical equipment using Infrared Thermography and support vector machine. Infrared Phys Technol 2018;89:120–8. https://doi.org/10.1016/j.infrared.2017.12.015.
[72] Onel M, Kieslich CA, Guzman YA, Pistikopoulos EN. Simultaneous fault detection and identification in continuous processes via nonlinear support vector machine based feature selection. Comput Aided Chem Eng 2018;44:2077–82. https://doi.org/10.1016/B978-0-444-64241-7.50341-4.
[73] Ren L, Sun Y, Cui J, Zhang L. Bearing remaining useful life prediction based on deep autoencoder and deep neural networks. J Manuf Syst 2018;48:71–7. https://doi.org/10.1016/j.jmsy.2018.04.008.
[74] Zhang J, Wang P, Yan R, Gao RX. Long short-term memory for machine remaining life prediction. J Manuf Syst 2018;48. https://doi.org/10.1016/j.jmsy.2018.05.011. 78–76.
[75] Xu M, Jin X, Kamarthi S, Noor-E-Alam M. A failure-dependency modeling and state discretization approach for condition-based maintenance optimization of multi-component systems. J Manuf Syst 2018;47:141–52. https://doi.org/10.1016/j.jmsy.2018.04.018.
[76] Downey A, Lui Y-H, Hu C, Laflamme S, Hu S. Physics-based prognostics of lithium-ion battery using non-linear least squares with dynamic bounds. Reliab Eng Syst Saf 2019;182:1–12.
[77] Li Y, Liu S, Shu L. Wind turbine fault diagnosis based on Gaussian process classifiers applied to operational data. Renew Energy 2019;134:357–66. https://doi.org/10.1016/j.renene.2018.10.088.
[78] M.Mehdi Hassani.N S, Xiaoning J, Jun N. Physics-based Gaussian process for the health monitoring for a rolling bearing. Acta Astronatica 2019;154:133–9.
[79] Huang W, Cheng J, Yang Y, Guo G. An improved deep convolutional neural network with multi-scale information for bearing fault diagnosis. Neurocomputing 2019;359:77–92. https://doi.org/10.1016/j.neucom.2019.05.052.
[80] Chen Z, Gryllias K, Li W. Mechanical fault diagnosis using convolutional neural networks and extreme learning machine. Mech Syst Signal Process 2019:133. https://doi.org/10.1016/j.ymssp.2019.106272.
[81] Chaoqun D, Chao D, Ning L. Reliability assessment for CNC equipment based on degradation data. Int J Adv Manuf Technol 2019;100:421–34.
[82] Eker OF, Camci F, Jennions IK. Physics-based prognostic modelling of filter clogging phenomena. Mech Syst Signal Process 2016;75:395–412. https://doi.org/10.1016/j.ymssp.2015.12.011.
[83] Climente-Alarcon V, Nair D, Sundaria R, Antonino-Daviu JA, Arkkio A. Combined model for simulating the effect of transients on a damaged rotor cage. IEEE Trans Ind Appl 2017. https://doi.org/10.1109/TIA.2017.2691001.
[84] Qiao G, Weiss BA. Quick health assessment for industrial robot health degradation and the supporting advanced sensing development. J Manuf Syst 2018;48:51–9. https://doi.org/10.1016/j.jmsy.2018.04.004.
[85] Li J, Xiao M, Liang Y, Tang X, Li C. Three-dimensional simulation and prediction of solenoid valve failure mechanism based on finite element model. IOP Conf. Ser. Earth Environ. Sci. 2018. https://doi.org/10.1088/1755-1315/108/2/022035.
[86] Cholette ME, Yu H, Borghesani P, Ma L, Geoff K. Degradation modeling and condition-based maintenance of boiler heat exchangers using gamma processes. Reliab Eng Syst Saf 2019;183:184–96.
[87] Liao WZ, Li D. An improved prediction model for equipment performance degradation based on Fuzzy-Markov Chain. IEEE Int. Conf. Ind. Eng. Eng. Manag. 2016. https://doi.org/10.1109/IEEM.2015.7385597.
[88] Chang Y, Fang H, Zhang Y. A new hybrid method for the prediction of the remaining useful life of a lithium-ion battery. Appl Energy 2017;206:1564–78. https://doi.org/10.1016/j.apenergy.2017.09.106.

[89] Hanachi H, Liu J, Kim IY, Mechefske CK. Hybrid sequential fault estimation for multi-mode diagnosis of gas turbine engines. Mech Syst Signal Process 2019;115:225–68. https://doi.org/10.1016/j.ymssp.2018.05.054.

[90] Jiang Y, Zhu H, Ding C, Pfeiffer O. A novel ensemble fuzzy model for degradation prognostics of rolling element bearings. J Intell Fuzzy Syst 2019;37:4449–55. https://doi.org/10.3233/JIFS-179277.

[91] Al-Dulaimi A, Zabihi S, Asif A, Mohammadi A. A multimodal and hybrid deep neural network model for remaining useful life estimation. Comput Ind 2019;108:186–96. https://doi.org/10.1016/j.compind.2019.02.004.

[92] Chiachío J, Chiachío M, Prescott D, Andrews J. A knowledge-based prognostics framework for railway track geometry degradation. Reliab Eng Syst Saf 2019;181:127–41. https://doi.org/10.1016/j.ress.2018.07.004.

[93] Che C, Wang H, Fu Q, Ni X. Combining multiple deep learning algorithms for prognostic and health management of aircraft. Aerosp Sci Technol 2019:94. https://doi.org/10.1016/j.ast.2019.105423.

[94] Lu F, Wu J, Huang J, Qiu X. Aircraft engine degradation prognostics based on logistic regression and novel OS-ELM algorithm. Aerosp Sci Technol 2019;84:661–71. https://doi.org/10.1016/j.ast.2018.09.044.

[95] Ferreiro S, Konde E, Fernández S, Prado A. Industry 4.0 : predictive intelligent maintenance for production equipment. Eur. Conf. Progn. Heal. Manag. Soc. 2016.

[96] Javed K, Gouriveau R, Zerhouni N. State of the art and taxonomy of prognostics approaches, trends of prognostics applications and open issues towards maturity at different technology readiness levels. Mech Syst Signal Process 2017;94:214–36. https://doi.org/10.1016/j.ymssp.2017.01.050.

[97] Sakib N, Wuest T. Challenges and opportunities of condition-based predictive maintenance: a review. 6th CIRP Glob. Web Conf. "Envisaging Futur. Manuf. Des. Technol. Syst. Innov. era," Elseiver B.V. 2018:267–72.

[98] Alaswad S, Xiang Y. A review on condition-based maintenance optimization models for stochastically deteriorating system. Reliab Eng Syst Saf 2017;157:54–63. https://doi.org/10.1016/j.ress.2016.08.009.

[99] International Organization for Standardization (ISO). ISO 13379-1:2012 - Condition monitoring and diagnostics of machines – Data interpretation and diagnostics techniques – Part 1: general guidelines. 2012.

[100] Gouriveau R, Medjaher K, Zerhouni N. From prognostics and health systems management to predictive maintenance 1: monitoring and prognostics. 2016. https://doi.org/10.1002/9781119371052.

[101] Vogl GW, Weiss Ba, Donmez MA. Standards for prognostics and health management (PHM) techniques within manufacturing operations. Annu Conf Progn Heal Manag Soc 2014.

[102] Vachtsevanos G, Lewis F, Roemer M, Hess A, Wu B. Intelligent fault diagnosis and prognosis for engineering systems. 2007. https://doi.org/10.1002/9780470117842.

[103] Zerhouni N, Atamuradov V, Medjaher K, Dersin P, Lamoureux B. Prognostics and health management for maintenance practitioners-review, implementation and tools evaluation. Artic Int J Progn Heal Manag 2017;8:31. https://doi.org/10.1016/j.euprot.2015.07.015.

[104] Jardine AKS, Lin D, Banjevic D. A review on machinery diagnostics and prognostics implementing condition-based maintenance. Mech Syst Signal Process 2006;20:1483–510. https://doi.org/10.1016/j.ymssp.2005.09.012.

[105] Wang D, Tsui KL, Miao Q. Prognostics and health management: a review of vibration based bearing and gear health indicators. IEEE Access 2017;6:665–76. https://doi.org/10.1109/ACCESS.2017.2774261.

[106] An D, Kim NH, Choi JH. Practical options for selecting data-driven or physics-based prognostics algorithms with reviews. Reliab Eng Syst Saf 2015;133:223–36. https://doi.org/10.1016/j.ress.2014.09.014.

[107] Noman MA, Nasr ESA, Al-Shayea A, Kaid H. Overview of predictive condition based maintenance research using bibliometric indicators. J King Saud Univ - Eng Sci 2018. https://doi.org/10.1016/j.jksues.2018.02.003.

[108] Schmidt B, Wang L. Predictive maintenance: literature review and future trends. Conf Proc 25th Int Conf Flex Autom Intell Manuf 2015;1:232–9.

[109] Dragomir OE, Gouriveau R, Dragomir F, Minca E, Zerhouni N. Review of prognostic problem in condition-based maintenance. Eur. Control Conf. (ECC'09) 2009. https://doi.org/10.1128/JB.00591-09.

[110] Sikorska JZ, Hodkiewicz M, Ma L. Prognostic modelling options for remaining useful life estimation by industry. Mech Syst Signal Process 2011;25:1803–36. https://doi.org/10.1016/j.ymssp.2010.11.018.

[111] Okoh C, Roy R, Mehnen J, Redding L. Overview of remaining useful life prediction techniques in through-life engineering services. Procedia CIRP 2014;16:158–63. https://doi.org/10.1016/j.procir.2014.02.006.

[112] Boullart L. A gentle introduction to artificial intelligence. In: Boullart L, Krijgsman A, Vingerhoeds RA, editors. Appl. Artif. Intell. Process Control. Pergamon Press; 1992. p. 5–40.

[113] Liu H, Yu J, Zhang P, Li X. A review on fault prognostics in integrated health management. ICEMI 2009 - Proc. 9th Int. Conf. Electron. Meas. Instruments 2009. https://doi.org/10.1109/ICEMI.2009.5274082.

[114] Majstorovic VD, Milacic VR. Expert systems for maintenance in the CIM concept. Comput Ind 1990;15:83–93.

[115] Freyermuth B. Knowledge based incipient fault diagnosis of industrial robots. IFAC Proc Vol 1991;24:369–75.

[116] Agnar A, Plaza E. Case-Based reasoning: Foundational issues, methodological variations, and system approaches. AI Commun 1994;7:39–59. https://doi.org/10.3233/AIC-1994-7104.

[117] Vepa R. Introduction to fuzzy logic and fussy sets. In: Boullart L, Krijgsm A, Vingerhoeds RA, editors. Appl. Artif. Intell. Process Control. 1991. p. 146–63.

[118] Ruiz PP, Foguem BK, Grabot B. Generating knowledge in maintenance from Experience Feedback. Knowl-Based Syst 2014;68:4–20. https://doi.org/10.1016/j.knosys.2014.02.002.

[119] Grabot B. Rule mining in maintenance: analysing large knowledge bases. Comput Ind Eng 2018;139:105501https://doi.org/10.1016/j.cie.2018.11.011.

[120] Ramasso E. Investigating computational geometry for failure prognostics. Int J Progn Heal Manag 2014;5:18.

[121] Kumar P, Srivastava RK. An expert system for predictive maintenance of mining excavators and its various forms in open cast mining. 2012 1st Int. Conf. Recent Adv. Inf. Technol. RAIT-2012 2012. https://doi.org/10.1109/RAIT.2012.6194607.

[122] Zhou ZJ, Hu CH, Xu DL, Chen MY, Zhou DH. A model for real-time failure prognosis based on hidden Markov model and belief rule base. Eur J Oper Res 2010;207:269–83. https://doi.org/10.1016/j.ejor.2010.03.032.

[123] Yang BS, Jeong SK, Oh YM, Tan ACC. Case-based reasoning system with Petri nets for induction motor fault diagnosis. Expert Syst Appl 2004;27:301–11. https://doi.org/10.1016/j.eswa.2004.02.004.

[124] Li S, Lv C, Guo Z, Wang M. Health condition-based maintenance decision intelligent reasoning method. Proc. 2012 Int. Conf. Qual. Reliab. Risk, Maintenance, Saf. Eng. ICQR2MSE 2012 2012. https://doi.org/10.1109/ICQR2MSE.2012.6246263.

[125] Phillips P, Diston D. A knowledge driven approach to aerospace condition monitoring. Knowl-Based Syst 2011;24:915–27. https://doi.org/10.1016/j.knosys.2011.04.008.

[126] Kothamasu R, Huang SH. Adaptive Mamdani fuzzy model for condition-based maintenance. Fuzzy Sets Syst 2007;158:2715–33. https://doi.org/10.1016/j.fss.2007.07.004.

[127] Sankararaman S, Daigle MJ, Goebel K. Uncertainty quantification in remaining useful life prediction using first-order reliability methods. IEEE Trans Reliab 2014;63:603–19. https://doi.org/10.1109/TR.2014.2313801.

[128] Moutinho MN. Fuzzy diagnostic systems of rotating machineries, some Eletronorte's applications. 2009 15th Int. Conf. Intell. Syst. Appl. to Power Syst. ISAP' 09 2009. https://doi.org/10.1109/ISAP.2009.5352882.

[129] Coble JB, Hines JW. Applying the general path model to estimation of remaining useful life. Int J Progn Heal Manag 2011;2:13.

[130] Du X, Zhou Y, Dong S. Residual life prediction from statistical features and a GARCH modeling approach for aircraft generators. Proc Inst Mech Eng Part G J Aerosp Eng 2014;228:137–46. https://doi.org/10.1177/0954410012472838.

[131] Zhan Y, Mechefske CK. Robust detection of gearbox deterioration using compromised autoregressive modeling and Kolmogorov-Smirnov test statistic. Part II: experiment and application. Mech Syst Signal Process 2007;21:1983–2011. https://doi.org/10.1016/j.ymssp.2006.11.006.

[132] Zhan Y, Mechefske CK. Robust detection of gearbox deterioration using compromised autoregressive modeling and Kolmogorov-Smirnov test statistic-Part I: compromised autoregressive modeling with the aid of hypothesis tests and simulation analysis. Mech Syst Signal Process 2007;21:1953–82. https://doi.org/10.1016/j.ymssp.2006.11.005.

[133] Cheng C, Yu L, Chen L. Structural nonlinear damage detection based on ARMA-GARCH model. Appl Mech Mater 2012;204–208:2891–6. https://doi.org/10.4028/www.scientific.net/AMM.204-208.2891.

[134] Ghahramani S. Fundamentals of probability: with stochastic processes. Third edition 2015. https://doi.org/10.1201/b19602.

[135] Pourbabaee B, Meskin N, Khorasani K. Multiple-model based sensor fault diagnosis using hybrid Kalman filter approach for nonlinear gas turbine engines. 2013 Am Control Conf 2013. https://doi.org/10.1109/TCST.2015.2480003.

[136] Hu C, Youn BD, Wang P, Taek Yoon J. Ensemble of data-driven prognostic algorithms for robust prediction of remaining useful life. Reliab Eng Syst Saf 2012;103:120–35. https://doi.org/10.1016/j.ress.2012.03.008.

[137] Rasmussen CE, Williams CKI. Gaussian process for machine learning. the MIT Press; 2006.

[138] Rasmussen CE. Gaussian processes in machine learning. In: Bousquet O, von Luxburg U, Rätsch G, editors. Adv. Lect. Mach. Learn. Springer; 2003. p. 63–71.

[139] Kan MS, Tan ACC, Mathew J. A review on prognostic techniques for non-stationary and non-linear rotating systems. Mech Syst Signal Process 2015;62–63:1–20. https://doi.org/10.1016/j.ymssp.2015.02.016.

[140] Markov A. Extension of the limit theorems of probability theory to a sum of variables connected in a chain. Append. B R. Howard. Dyn. Probabilistic Syst. Vol. 1 Markov Chain. John Wiley and Sons; 1971. https://doi.org/citeulike-article-id:911035.

[141] Protter PE. Stochastic integration and differential equations. Second edition Springer; 2005.

[142] Rudnicki R, Tyran-Kamińska M. Piecewise deterministic processes in biological models. Springer; 2017.

[143] Aye SA, Heyns PS. An integrated Gaussian process regression for prediction of remaining useful life of slow speed bearings based on acoustic emission. Mech Syst Signal Process 2017;84:485–98. https://doi.org/10.1016/j.ymssp.2016.07.039.

[144] Kobayashi K, Kaito K, Lethanh N. A statistical deterioration forecasting method using hidden Markov model for infrastructure management. Transp Res Part B Methodol 2012;46:544–61. https://doi.org/10.1016/j.trb.2011.11.008.

[145] Le Son K, Fouladirad M, Barros A, Levrat E, Iung B. Remaining useful life estimation based on stochastic deterioration models: a comparative study. Reliab Eng Syst Saf 2013;112:165–75. https://doi.org/10.1016/j.ress.2012.11.022.

[146] Russel S, Norvig P. Artificial intelligence—A modern approach. 3rd edition 2012. https://doi.org/10.1017/S0269888900007724.

[147] Schwartz S, Montero Jiménez JJ, Salaün M, Vingerhoeds R. A fault mode identification methodology based on self-organizing map. Neural Comput Appl 2020:1–19.

[148] Koprinkova-Hristova P. Reinforcement learning for predictive maintenance of

industrial plants. Inf Technol Control 2014;11:21–8. https://doi.org/10.2478/itc-2013-0004.
[149] Wells G. An introduction to neural networks. In: Boullart L, Krijgsm A, Vingerhoeds RA, editors. Appl. Artif. Intell. Process Control. 1992. p. 164–200.
[150] Hailesilassie. Rule extraction algorithm for deep neural networks: a review. Int J Comput Sci Inf Secur 2016;14:371–81.
[151] Zhao Y, Wang S, Xiao F. Pattern recognition-based chillers fault detection method using Support Vector Data Description (SVDD). Appl Energy 2013;112:1041–8. https://doi.org/10.1016/j.apenergy.2012.12.043.
[152] Nasution FP, Sævik S, Gjøsteen JKØ. Fatigue analysis of copper conductor for offshore wind turbines by experimental and FE method. Energy Procedia 2012;24:271–80. https://doi.org/10.1016/j.egypro.2012.06.109.
[153] Si XS, Wang W, Hu CH, Zhou DH. Remaining useful life estimation - A review on the statistical data driven approaches. Eur J Oper Res 2011;213:1–14. https://doi.org/10.1016/j.ejor.2010.11.018.
[154] Liao L, Köttig F. Review of hybrid prognostics approaches for remaining useful life prediction of engineered systems, and an application to battery life prediction. IEEE Trans Reliab 2014;63:191–207. https://doi.org/10.1109/TR.2014.2299152.
[155] Soualhi A, Razik H, Clerc G, Doan DD. Prognosis of bearing failures using hidden markov models and the adaptive neuro-fuzzy inference system. IEEE Trans Ind Electron 2014;61:2864–74. https://doi.org/10.1109/TIE.2013.2274415.
[156] Chen J, Roberts C, Weston P. Fault detection and diagnosis for railway track circuits using neuro-fuzzy systems. Control Eng Pract 2008;16:585–96. https://doi.org/10.1016/j.conengprac.2007.06.007.
[157] Prentzas J, Hatzilygeroudis I. Combinations of case-based reasoning with other intelligent methods. CEUR Workshop Proc. 2008. https://doi.org/10.3233/his-2009-0096.
[158] Tung YH, Tseng SS, Weng JF, Lee TP, Liao AYH, Tsai WN. A rule-based CBR approach for expert finding and problem diagnosis. Expert Syst Appl 2010. https://doi.org/10.1016/j.eswa.2009.07.037.
[159] Peel L. Data driven prognostics using a Kalman filter ensemble of neural network models. 2008 Int. Conf. Progn. Heal. Manag. PHM 2008 2008. https://doi.org/10.1109/PHM.2008.4711423.
[160] Li X, Qian J, Wang GG. Fault prognostic based on hybrid method of state judgment and regression. Adv Mech Eng 2013. https://doi.org/10.1155/2013/149562.
[161] Kacprzynski GJ, Sarlashkar A, Roemer MJ, Hess A, Hardman W. Predicting remaining life by fusing the physics of failure modeling with diagnostics. JOM 2004;56:29–35. https://doi.org/10.1007/s11837-004-0029-2.
[162] Oppenheimer CH, Loparo KA. Physically based diagnosis and prognosis of cracked rotor shafts. Compon. Syst. Diagn., Progn. Heal. Manag. 2002;II. https://doi.org/10.1117/12.475502.
[163] Meskin N, Naderi E, Khorasani K. A multiple model-based approach for fault diagnosis of jet engines. IEEE Trans Control Syst Technol 2013;21:254–62. https://doi.org/10.1109/TCST.2011.2177981.
[164] Swanson DC. A general prognostic tracking algorithm for predictive maintenance. IEEE Aerosp. Conf. Proc. 2001. https://doi.org/10.1109/aero.2001.931317.
[165] Wang P, Youn BD, Hu C. A generic probabilistic framework for structural health prognostics and uncertainty management. Mech Syst Signal Process 2012;28:622–37. https://doi.org/10.1016/j.ymssp.2011.10.019.
[166] Riad AM, Elminir HK, Elattar HM. Evaluation of neural networks in the subject of prognostics as compared to linear regression model. Int J Eng Technol 2010;10:52–8.
[167] Neerukatti RK, Liu KC, Kovvali N, Chattopadhyay A. Fatigue life prediction using hybrid prognosis for structural health monitoring. J Aerosp Inf Syst 2014;11:211–32. https://doi.org/10.2514/1.I010094.
[168] Hong J, Miao X, Han L, Ma Y. Prognostics model for predicting aero-engine bearing grade-life. Proc. ASME Turbo Expo 2009. https://doi.org/10.1115/GT2009-59641.
[169] Orsagh RF, Sheldon J, Klenke CJ. Prognostics/diagnostics for gas turbine engine bearings. IEEE Aerosp. Conf. Proc. 2003. https://doi.org/10.1109/AERO.2003.1234152.
[170] Lee J, Wu F, Zhao W, Ghaffari M, Liao L, Siegel D. Prognostics and health management design for rotary machinery systems - reviews, methodology and applications. Mech Syst Signal Process 2014;42:314–34. https://doi.org/10.1016/j.ymssp.2013.06.004.
[171] Crawley E, Cameron B, Selva D. System architecture: strategy and product development for complex systems. Pearson Higher Education, Inc.; 2015.
[172] Peng Y, Dong M, Zuo MJ. Current status of machine prognostics in condition-based maintenance: a review. Int J Adv Manuf Technol 2010;50:297–313. https://doi.org/10.1007/s00170-009-2482-0.
[173] Bailey C, Sutharssan T, Yin C, Stoyanov S. Prognostic and health management for engineering systems: a review of the data-driven approach and algorithms. J Eng 2015;2015:215–22. https://doi.org/10.1049/joe.2014.0303.
[174] Elattar HM, Elminir HK, Riad AM. Prognostics: a literature review. Complex Intell Syst 2016;2:125–54. https://doi.org/10.1007/s40747-016-0019-3.
[175] Vanraj Goyal D, Saini A, Dhami SS, Pabla BS. Intelligent predictive maintenance of dynamic processing techniques-A review. Proc. - 2016 Int. Conf. Adv. Comput. Commun. Autom. ICACCA Systems Using Condition Monitoring and Signal 2016 2016. https://doi.org/10.1109/ICACCA.2016.7578870.

2.3 Lessons learnt

Article 1 of the current manuscript has shown an extensive study on the current trends of predictive maintenance. It allowed determining that multi-model approaches are increasingly used to address complex problems of diagnostics and prognostics. Multi-model approaches benefit from the strong points of different models allowing to overcome single-model approach limitations. The state-of-the-art analysis allowed to refine the research questions for the current thesis. These refined research questions are:

1. How to address the design of predictive maintenance systems?
2. How to suggest a suitable approach for a predictive maintenance system solution?
3. How to select a suitable model or combination of models given a new predictive maintenance problem to solve?
4. How can a designer benefit from the experience of existing systems to develop new predictive maintenance solutions?

These questions are inspired by the first two identified challenges of Article 1: the extrapolation of existing solutions to complex system applications, and the lack of a systematic approach to design and develop predictive maintenance systems. The following chapters explain the proposed framework of the current thesis that attempts to answer these questions.

CHAPTER 3

From needs and desires to a generic logical architecture for predictive maintenance systems

"Creativity is intelligence having fun."

Albert Einstein

Content

3.1 The creative process begins

The first refined question is about the design of new predictive maintenance systems. In the research statement presented in Chapter 1, it was explained that the design and development of predictive maintenance systems is still based on trial and error. Despite the existence of several generic architectures in norms and standards, such as for example [MIM01], the concept stage of such systems is not fully covered. There is a gap in the development of such systems from the gathering of needs and desires for the new system until the creation of the architecture that meets the initial needs and desires.

Creating a new system should always start by listening to those who are related or interested in the project, referred to as stakeholders. They provide all the needs and desires to be fulfilled by a new system. These needs and desires are the information source to establish the list of stakeholder requirements of the new system. The current research proposes a systems engineering approach to cover the concept stage of predictive maintenance systems. Special importance is given to the gathering of needs and desires and their translation into a formal set of stakeholder requirements. This provides the basis to build the system architecture for a new predictive maintenance system.

3.2 A systems engineering approach to predictive maintenance systems: from needs and desires to logical architecture (Article 2)

A Systems Engineering Approach to Predictive Maintenance Systems: from needs and desires to logical architecture.

Juan José Montero Jiménez
ISAE-SUPAERO, Toulouse, France
juan-jose.montero-jimenez@isae-supaero.fr
TEC, Tecnológico de Costa Rica
juan.montero@itcr.ac.cr

Rob Vingerhoeds
ISAE-SUPAERO
Université de Toulouse
Toulouse, France
rob.vingerhoeds@isae-supaero.fr

Abstract—**Predictive maintenance is an important field of research to determine the exact moment to trigger maintenance actions. Despite the potential benefits of predictive maintenance in terms of maintenance cost reduction and safety improvement, its implementation faces many shortcomings. One of the main shortcomings is the lack of a systematic approach to developing predictive maintenance systems. Existing generic architectures like OSA-CBM remain insufficient to address all requirements for new systems. A systems engineering approach starting from the needs and desires obtained from the stakeholders until a logical architecture is a potential solution. Specific analysis on the needs and desires is used to elicit, classify and prioritize the requirements for an easier transition to designing the systems architecture. The architecture process builds on the ARCADIA method.**

Keywords— Predictive Maintenance, Systems engineering, requirements elicitation, Architecture Process.

I. INTRODUCTION

Safe and efficient operation is of crucial importance for modern technical systems. Maintenance constitutes a vital discipline along the systems life-cycle to ensure their functionality. Within the maintenance strategies three types could be distinguished: corrective, preventive and predictive maintenance. In particular predictive maintenance is under a lot of attention at the moment and aims at analyzing relevant data to define the best possible moment to trigger maintenance actions [1]. Triggering too late may lead to failure occurrence, causing financial loses, sometimes image damage and may even lead to causalities and/or losses of human lives. Triggering too early may lead to replacing components that are not faulty through costly interventions.

Predictive maintenance proposes a solution by monitoring the health state of the technical system, identifying incipient faults and forecasting the moment of failure. Such predictive maintenance activities include several online and off-line tasks on the system information coming from different knowledge sources so to better understand the faults evolutions over time and trigger the corresponding action before failures occur. Current fast expanding trends such as machine learning, internet of things, Industry 4.0, big data, boost predictive maintenance implementation to reach safer, more reliable and more efficient technical systems.

Despite important benefits linked to use of predictive maintenance [2], its implementation faces many shortcomings. One of the root causes may concern to the absence of a systematic development approach fur such systems. Some generic architecture approaches to predictive maintenance, such as Open System Architecture for Condition-Based Maintenance (OSA-CBM™) [3] and the functional architecture presented in [4], propose different functional components of predictive maintenance systems. However, constraints can be identified on these solutions that may limit the usability to a smaller subset of applications or more complex scopes. Important missing parts concern for example performance indicators, potential structural constraints or partial imposed solutions, human factors, etc., hardly addressed in these generic architectures. The lack of performance indicators is pointed by [5] as one of main blocking points to validate new predictive maintenance systems.

The purpose of the current study is to propose a systematic development method for off-line predictive maintenance applications using a systems engineering approach covering from the earliest conceptual stages, to the concept identification and architecture proposal. The motivation is to have a wider overview on the needs and desires of the stakeholders related to a new predictive maintenance system, aiming to identify important information that may be missed by existing architectural approaches. It also includes a requirements prioritization process that aims at reducing ambiguity among the initial list of needs and desires, setting the boundaries of the new system scope. These preliminary steps help in the architecture process to determine the most suitable solution for a new predictive maintenance system.

The current study builds on existing concept design theories [6], [7], the notion of prioritization in taking into account requirements during the development process [8], and on the ARCADIA method [9]. These three building blocks allow for a structured development process, in this paper applied to predictive maintenance. The paper is organized as follows. Section 2 describes the predictive maintenance challenges as studied within the framework of this research. Section 3 resumes the theoretical framework of the approach of the architecture process. Section 4 presents the system engineering approach to predictive maintenance systems development at conceptual phases. Section 5 extends the architecture analysis using the case study. Section 6 draws conclusions and indicates some path for future research on this topic.

II. PREDICTIVE MAINTENANCE CONTEXT

Predictive maintenance is a maintenance strategy aiming at monitoring the health state of the system, detecting incipient faults and forecasting potential failure in the future to trigger the maintenance actions accurately when they are needed. As such it is a complementary strategy to corrective and preventive maintenance. A good

combination of the three techniques is vital to keep the system reliable [1]. Condition Based Maintenance (CBM) and Prognostics and Health Management (PHM) are strongly related to predictive maintenance. There exists a terminology disagreement between the definition of these terms on literature, for this study, CBM and PHM are considered as extensions of predictive maintenance.

Two main approaches can be identified ([10]–[13]): a diagnostics approach aims at determining the current health state of the system and the identification of the faults through the symptoms, and a prognostics approach aims at forecasting future failures and the remaining useful life of the technical systems.

For both approaches a vast set of techniques exist, suitable to specific cases of technical systems depending on the availability of knowledge on the concerned technical system and its complexity. One can see knowledge-based models, data-driven models and physics-based models [13]. In addition, as it is almost impossible to model any real-life application by using only one single technique [14], hybrid models combining several techniques are gaining attention. As each technique has its own characteristic advantages concerning the availability of knowledge, knowledge representation, learning capabilities, etc., a technique should be selected to best fit a given task. The different parts of predictive maintenance could therefore benefit from the best knowledge representation technique for each task [14].

As each technical system has its own application context, generic architectures approaches like OSA-CBM [3], which is based on the standard ISO-13374 [15], may not be suitable for the development of a new predictive maintenance system. Important missing information such as performance indicators, structural constraints and the operability by the users is not considered in these generic approaches. As [5] suggests, performance requirements are often blocking points to develop predictive maintenance systems. This lack of a systematic approach to develop new predictive maintenance systems motivates the current study.

III. SYSTEMS ENGINEERING APPROACH AT THE CONCEPTUAL PHASE

A system, as defined by the International Council of System Engineering (INCOSE), is “an integrated set of elements, subsystems, or assemblies that accomplish a defined objective” [16]. A systems engineering approach will give objectivity to problem solution by analyzing and understanding the desired behavior of the system, the interactions between the internal components to achieve the desired objective all along the system life-cycle, as well as the interfaces to the outside world.

One of the critical phases in any systems development is the concept design stage that focusses on the concept: understanding the implications of a system mission and core functionality, a business case together with requirements, their interconnections and dependencies specified in e.g. key performance indicators and trade-off indicators. Expressed (and potentially clarified) stakeholder needs and desires are translated into requirements. This stage results in a document describing a system mission, desired functionality, initial (qualified) requirements, and performance indicators (to determine in a later stage whether the desired functionality performance has been delivered). It also includes a description of a logical architecture of the system design and its subsystems (the upper-level architecture) that meets system requirements: a preliminary design of the product or service to develop [7].

The section presents a combination of three complementary methods that allow developers to be guided through the conceptual stage of a system.

A. Requirement elicitation and different type of requirement

John Gero suggested in [6] that three types of requirements exist (functional, behavioral, and structural), a proposal later extended to include a fourth type (experiential) [7] (see Fig. 1). Specifically: functional requirements state functions that a system must provide and are directly related to the mission of a system; behavioral requirements specify desired system behavior of a design with respect to its mission, together with key performance indicators with which this behavior can be determined; structural requirements define requirements for components/sub-systems of a system and their interdependencies, and experiential requirements define the desired impact of a system in the real world with real people [7]. Each of these categories has a unique contribution to the design and development process. For example, the functional requirements correspond in a first step to the system capabilities that the designer needs to address, etc. An important step in system development is therefore the understanding and translation of stakeholders’ needs and desires into functional, structural, behavioral, and experiential requirements.

As requirements elicitation method, this needs and desires analysis approach (FBSE analysis, for Functional, Behavioral, Structural and Experiential) helps to integrally address the requirements of the stakeholders for the predictive maintenance system. It could be performed iteratively and recursively at the different levels of the system as some components of the system are systems themselves that need a complete design process on which a systems engineering approach should be also applied.

B. Requirements prioritization

Starting from better organized requirements, split into functional, behavioral, structural and experiential, it may be beneficial to prioritize the obtained requirements, so to end up with a reduced complexity, thus increasing the chances on successful development. Prioritization reduces ambiguity on initial needs and desires and helps to define the system boundaries, as suggested by [8]. Different types of requirement prioritization techniques can be identified [17], of which nominal and ordinal techniques are used in the current study so that the requirements are properly classified and ranked. Nominal techniques classify the requirements into different categories, approach that can be complemented by ordinary scales to rank the importance of requirements within the categories.

Function	The purpose of the system	Why? For whom? Where?
Behavior	The way a system acts	How? When?
Structure	The components of a system and their relationships	What?
Experience	Feelings, emotions, perceptions associated with the system	With whom? By whom? With what effect?

Fig. 1. Functional, behavioral, structural and experiential requirements [7]

FBSE analysis as presented in the previous sub-section corresponds to a nominal prioritization. This helps development engineers to know with which requirements to work at any point in time during the development stage. Nevertheless, a complementary ordinal prioritization method is proposed to further reduce complexity. The current study builds upon the method proposed by [8].

The ordinal prioritization method divides the requirements into three categories: critical, important and desired. The method is used for ranking the requirements importance within their category. Critical requirements contain the three to seven absolute necessities for the system success [8]. Important requirements are not strictly necessary for success but they contribute to it. Finally desired requirements denote wishes. Requirements will have the format shown in Table I after prioritization.

C. Systems Architecture Process

Systems architecture is the "the embodiment of concept, the allocation of physical/informational function to elements of form, and the definition of relationship among the elements and with the surrounding context" [8]. Architecting is a creative process in which the architect searches for innovative solutions to a specific problem.

The architecture process in the current study builds on ARCADIA method [9]. This is a multi-layer Model Based Systems Engineering approach (see Fig. 2). Four layers are available to represent the system architecture.

The first layer is the operational analysis on which is defined what the users of system need to accomplish. This layer summarizes the stakeholders' requirements obtained from the previous sub-sections.

The second layer is the analysis of the system needs, what the system has to accomplish for the users. Functional analysis takes place at this layer. It starts by the definition of the primary function of the system to be fulfilled. The primary function should be as neutral as possible, it means without suggesting the final solution to the problem. This avoids narrowing down the solution space from the beginning of the project. This primary function is meant to be identified from the prioritized stakeholder requirements. Then, the primary function is decomposed into sub-functions. Decomposition is a common method used to manage the complexity of the primary function and explore the potential solutions for system architecture. Functional analysis includes functional exchanges which are the internal interactions among the sub-functions to fulfill the primary function. Depending on the system complexity, each sub-function of the system may represent a complete subsystem for which the whole system engineering approach should be performed. The consistent set of functions and the different interactions among them represents the functional architecture.

The third layer is the logical architecture, how the system will work to fulfill expectations. This layer helps to summarize the identified concept and its decomposition into the different functional levels. Having the functional architecture, each sub-function is allocated to logical components. Structured creativity could be used for allocating the functions to logical components [8]. The external and internal interfaces are identified as well as some system requirements. This set of logical components and the external and internal interfaces compose the logical architecture which starts to give technical meaning to the system. Further functional decomposition is performed at this layer depending on the system complexity. System requirements are identified along this architecture process.

The fourth and final layer is the Physical architecture, how the system will be developed and built. Here the rest of the system requirements are identified since the physical components to fulfill the logical architecture are selected.

TABLE I. REQUIRMENTS WRITING FORMAT

Critical requirements	Important requirements	Desired requirements
The system shall…	The system should…	The system might…

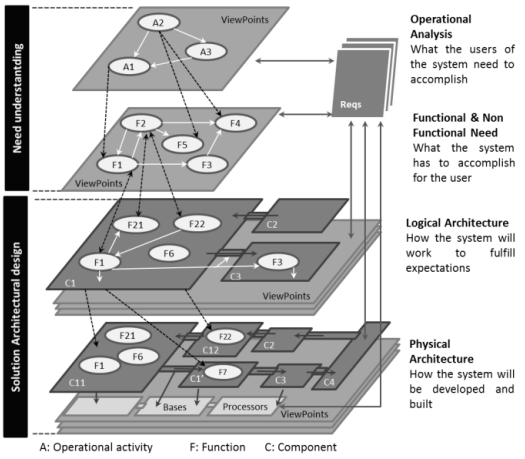


Fig. 2. Arcadia method layers summary, recreated from [9].

IV. SYSTEMS ENGINEERING APPROACH TO PREDICTIVE MAINTENANCE SYSTEMS

In this paragraph, the systems engineering approach to developing a predictive maintenance system is presented. The approach starts with a thorough analysis of the potential stakeholders and their needs and desires related to a predictive maintenance system. This analysis continues with an FBSE analysis so to classify the obtained requirements into functional, behavioral, structural and experiential. Then a requirements prioritization takes place, so to guide the developer on which requirements to address at which point in time. In the next phase, the ARCADIA method is being adopted so to work in a step-wise approach to the logical architecture.

The approach is illustrated with information from a case-study at the basis of the PHM'08 challenge [18]. In this challenge, engine condition monitoring on a 90,000 lb thrust class commercial jet aircraft engine was used. After every flight, performance engineers evaluate the evolution of engine critical parameters and derive from those analyzes to anticipate or to avoid incidents, to evaluate the effects of incidents or to provide a clear "no problem for the next few flights" indication. In the challenge the attention was at the data discovery, finding the engine problems present in the data sets. In the current study, we use the information on this particular example to illustrate the development process for predictive maintenance systems.

A. *Defining stakeholders'needs and desires*

Needs and desires consider the base of what stakeholders expect from the system under development. Many possibilities exist to elicit these needs and desires (e.g. interviews, observation, work process analysis …). In a first step, the stakeholders must be identified. Some potential stakeholders for a predictive maintenance system are summarized in Table II. Of course, depending on the technical system context some of these stakeholders may not be present or others not mentioned may be added to the list.

An initial list of needs and desires for predictive maintenance is gathered from a literature review [10]–[13], [18], [19] and is summarized in Table III. An ID is given for the traceability purpose of the needs and desires.

Let us note that not all the needs and desires are listed here. This list may be extended with specific needs and desires for a specific application. In the example of the jet engine, one may think of specific links to other tools or needs related to safety and reliability of the system.

B. *Eliciting and prioritizing requirements*

Next, the aforementioned FBSE analysis is performed on the initial list of needs and desires. The goal is to classify the needs and desires into functional, behavioral, structural and experiential requirements. This will have an important impact on the set-up and elaboration of the system architecture. Then, the mentioned ordinary scale prioritization technique is used to rank of importance of the requirements.

In the case of the jet engine, the requested tasks included to compute the remaining useful life of the jet engines, considering the measured data coming from the engines sensors. These initial tasks were included in the list of initial needs and desires. Table IV now presents the list of initial stakeholders' requirements after applying the FBSE analysis and the ordinary scale prioritization method on the needs and desires on Table III. The related ID's of the original needs and desires are indicated.

As can be seen in Table IV, at the beginning of the development more functional requirements than behavioral or structural requirements were listed. Behavioral and structural requirements tend to appear step-by step as the design process goes into more detailed levels. Translating behavioral needs and desires into requirements demands a clear set of performance indicators coming for example from the technical system, the stakeholder goals and authorities' regulations. A good starting point to determine the performance indicator of the predictive maintenance system is a Failure Modes, Effects and Criticality Analysis (FMECA) over the technical system [20]. This analysis will

TABLE II. POTENTIAL STAKEHOLDERS

Stakeholder	**Illustration jet engine**
Technical System owner	Airline, or rental company
Technical System user	Operating airline, MRO, …
Technical System manufacturer	Jet engine manufacturer
Predictive maintenance system developer(s)	Development company
Maintenance department or predictive maintenance system user	Operations departments at airlines, MRO's, …
Corresponding authorities	FAA, EASA, …

TABLE III. EXAMPLE OF NEEDS AND DESIRES LIST (NOT COMPLETE)

N°	**List of needs and desires for predictive maintenance:**
1	Improve safety of the systems compare to current condition
2	Reduce downtime of the system compare to current condition
3	Reduce maintenance costs compare to current condition
4	Identify what technique fits best to each predictive maintenance case.
5	Avoid unexpected breakdowns
6	Detect incipient faults
7	Identification of different types of faults
8	Determination of the current health state of the system
9	Failures Features selection and extraction
10	Enhance degradation determination accuracy
11	Remaining Useful Life (RUL) determination
12	Optimize maintenance schedules (time and cost performance indicators associated)
13	Decision making support in maintenance
14	Implementation of self-maintenance and self-adjustment
.	.
.	.
.	.
N	Additional need or desire

TABLE IV. EXAMPLE OF INITIAL LIST OF STAKEHOLDERS' REQUIREMENTS (NOT COMPLETE)

ID	Requirements	Traceability to Table III
F1	The system shall read the technical system data.	17-28
F2	The system should pre-process and "clean" the raw data.	17-20-23-24-25
F3	The system should detect incipient faults.	6-7-17
F4	The system should determine the current health state of the system.	5-7-8-17
F5	The system shall determine the remaining useful life.	11-17-29
F6	The system might suggest maintenance actions to the user.	13-17
F#	The system "additional functional requirement"	**
B1	The system should present a computational error less than "TBD"	27
B2	The system shall reduce the unexpected breakdown by "TBD"	2-3
B3	The system shall increase the system safety by "TBD"	1
B#	The system "additional behavioral"	**
S1	The system shall be compatible with the technical system (capable to read the technical system data format).	31
S2	The system shall have its own configuration management.	38
S3	The system should use specialized techniques and methods.	18
S#	The system might allow updates or upgrades.	**
E1	The system shall have a "friendly" interface with the user.	35
E#	The system "additional experiential requirement"	**

TBD: "To Be Defined".
(**): traceability to a potential additional need or desire.

lead to the critical failures for which the predictive maintenance system must be intended. Besides, historical maintenance, safety and financial reports or records are other sources of information to establish performance indicators.

It is important to point out that predictive maintenance can be seen as an improvement of other maintenance practices. This means that, building on previous maintenance strategy programs gives useful information. These prior maintenance programs may contain initial parameters to be improved from which the behavioral, structural and experiential requirements are established.

C. *Architecture process for the Predictive Maintenance System*

For the jet engine example, using the FBSE analysis, six functions have been identified and prioritized. These functions are part of a higher level function related to every predictive maintenance system: "Estimate the precise moment to trigger maintenance actions". Next the identified functional requirements are proposed as sub-functions of the primary function (see Fig. 3).

This functional decomposition suggests the first mapping between functions and components of the system, allocating one function to each component. For validation and verification purposes is advisable to allocate one function to one component of the system.

1) Functional architecture

The functional architecture is built on top of the functional decomposition (see Fig. 4). It includes the interactions between the different functions (functional exchanges). For the jet-engine example, the collected data from the technical system (F1) is taken by F2 to be preprocessed. Later, F3 assesses the data to detect failures; if a failure is detected F4 may request additional preprocessed data to determine the degradation over time. Likewise, if the prognostics function is to be part of the predictive maintenance system under development (F5), it may request additional pre-processed data to compute the remaining useful life. The output of F2 serves as input for F3, F4 and F5 and it may include a sub-function to store the pre-processed data. The outputs of F3, F4 and F5 are the inputs for F6 in charge of generating the reports and/or triggering the maintenance actions in automated systems. The "request data" functional exchanges from F3, F4 and F5, as well as the external exchanges of the system, are not shown on Fig. 4 for layout purposes.

2) External interfaces identification

For an off-line predictive maintenance system, the inputs come from technical system databases that have stored historical data on the system to be maintained. The outputs link the system to a user interface and once again to the technical system data bases to keep track of the diagnostics and prognostics assessments, as well as eventual intermediate results of the analysis. These input and outputs represent the external interfaces of the predictive maintenance system. Fig. 5 shows the model of the predictive maintenance system interacting with the external actors through the external interfaces. These external actors are the technical system data bases and the predictive maintenance user. Blue boxes in Fig. 5 represent the system and the external actors, the link between them represent the external interfaces. Green boxes represent the functions of the system and actors, continuous lines between these green boxes are the functional exchanges. Dashed lines represent the functional exchanges flow through the external interfaces

3) Logical Architecture

Having the functional architecture of Fig. 4 and the external interfaces of Fig. 5, a logical architecture is proposed. Here the functions are allocated to logical components. As for the external interfaces, the interaction between the logical components of the system is analyzed to determine the internal interfaces of the system. Functional exchanges from Fig. 4 are the starting point for this internal interface analysis. As mentioned in section 3, structured creativity could be used for defining the logical component for the system. Fig. 6 shows a generic logical architecture for the predictive maintenance system. It proposes one logical component for each function. This allows the verification of each functional requirement separately. Besides the internal components, this logical architecture also presents the internal interfaces and the data flow through them. Internal functional exchanges between functions have been hidden for layout purposes on Fig. 6 but are shown on Fig. 4.

At this stage, the proposed logical architecture remains generic. Logical components are to be replaced by suitable algorithms to complete the logical architecture (data-driven, knowledge-based, physical-based, hybrid…). An extensive review of the different algorithms that have been used to fulfill the different logical components used for the PHM'08 Challenge can be found in [19]. These publications have

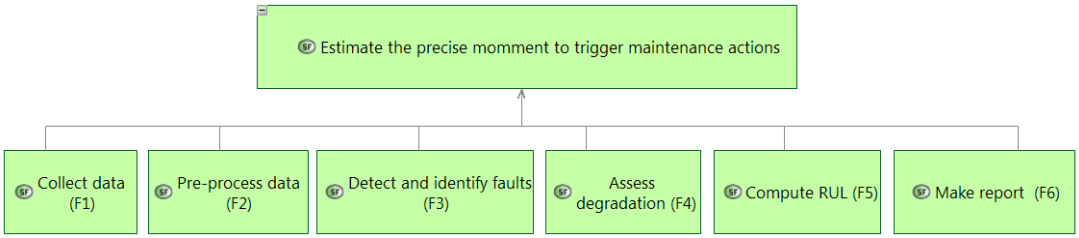


Fig. 3. Functional decomposition for the predictive maintenance system

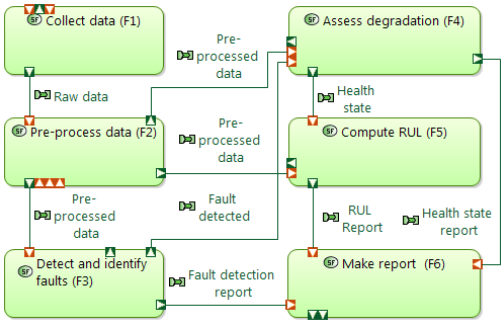


Fig. 4. Functional architecture for a predictive maintenance system concept

given special attention to functions F2, F3, F4 and F5 (see also Table V). It is important to point out that the suitable techniques rely on the available data sources and the predictive maintenance system scope. As can be seen in Table V, due to the nature of the original PHM'08 Challenge on which the jet engine example is based, data-driven techniques are more often found as potential solutions than other techniques. If the data sources include semantic knowledge (e.g. pilot feedback on witnessed engine behavior, or maintenance logs), one will see the set of suitable logical components evolve with other algorithms and potentially the architecture itself may evolve. When several solutions are identified to fulfill the functional components, trade-offs must be made; the architecture is assessed against performance indicators (the behavioral requirements). For the jet engine case study, the computational error on the remaining useful life was considered as the main performance indicator giving the best score to the statistical similarity based methods used by [21] at the time of the PHM'08 Challenge. These trade-off analyses as well as the physical architecture are out of the scope of this study.

V. Discussion

Generic architectures such as OSA-CBM [3] propose the functional blocks as starting points of a new predictive maintenance system. However, the new system may have a different scope that does not include all the proposed functional blocks or including new ones. Also, important information regarding the system performance, compatibility with existing systems and operability by the users may be missed when starting from a generic architecture. When a new predictive maintenance system is developed, it must be tailored to its context, considering all the needs and desires of relevant stakeholders. A generic architecture could be considered as a pre-established baseline and as the INCOSE Handbook [16] states; this could be a trap of tailoring that may bring the new system development to fail. Every new system has its own particular requirements.

Newer architecture approaches, like the one presented by [4], mention the importance of considering the stakeholders' requirements to develop a new predictive maintenance systems. However, they do not present a methodology to gather and interpret the initial list of needs and desires so that a consistent list of prioritized requirements could be obtained. Also, their scope remains on generic architectures which share the same tailoring problems mentioned in the previous paragraph.

The proposed approach in this study has as advantage over other existing generic architectures on the importance given to the conceptual phase. By performing a systems engineering approach, important information is obtained from the stakeholders' needs and desires. In terms of functional blocks (functional requirements), the logical architecture, obtained with the proposed methodology may be similar to the mentioned generic architectures. Nevertheless, the behavioral, structural and experiential requirements give a better overview of what the stakeholders expect from the system.

Behavioral requirements will help to assess the performance of the new predictive maintenance system. These requirements would facilitate later the validation process once the system is developed. Structural and experiential requirements will help to design the external interfaces of the system with other related systems or/and the system users.

Besides, by performing the requirements prioritization, the ambiguity among the requirements decrease, allowing a better understanding of what the system shall, should or might do. This information is vital to tailor the architecture for a new predictive maintenance system.

Finally the proposed approach employs the ARCADIA methodology to develop the logical architecture. Unlike other generic approaches that only show the architecture as functional blocks using a modeling language (UML/SysML), it offers a whole methodology organized by layers to develop the architecture for every new system. The information obtained from the FBSE analysis and the prioritization may be easily integrated in the architecture using ARCADIA.

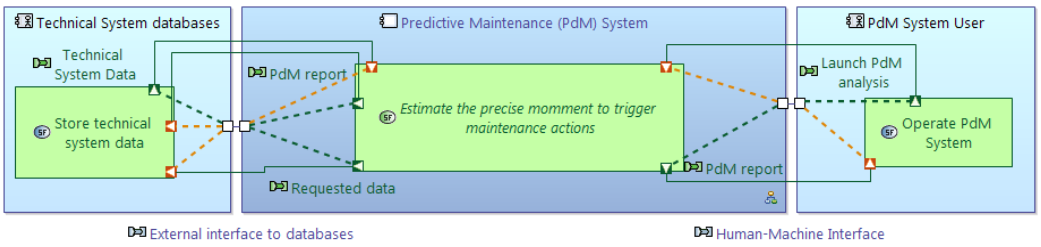


Fig. 5. External interfaces of the system

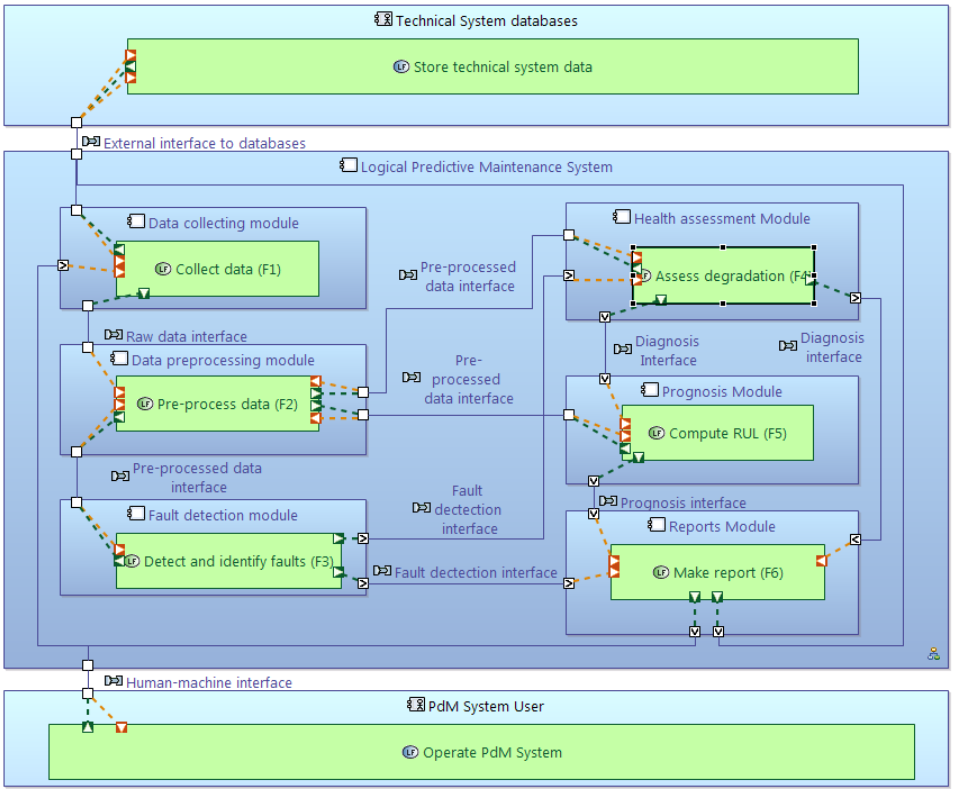


Fig. 6. Generic Logical Architecture for a Predictive maintenance off-line system.

TABLE V. SUMMARY OF POTENTIAL TECHNIQUES FOR LOGICAL COMPONENTS

Logical component	Type of techniques	Examples
Data Pre-processing	Statistical tests	Covariance Normalization
Fault identification	Clustering Classification	Support Vector Machine Neural Networks Kalman filter Bayesian Inference
Degradation Assessment	Regression Classification Stochastic Non-parametric	Statistical similarity Neural Networks Markov Chains
Remaining Useful life computation	Regression Classification Non-Parametric	Linear regression Neural Networks Markov Chains Case-based reasoning

VI. CONCLUSION AND PERSPECTIVES OF FUTURE WORK

Predictive maintenance is having a lot of attention because of its potential benefits in terms of safety improvement and maintenance cost reduction. The realization of such systems remains complicated, despite existing generic architectures and standards. The lack of a systematic approach to developing predictive maintenance systems remains one important shortcoming on.

A systems engineering approach to developing predictive maintenance systems is proposed to address the identified shortcoming. It starts by the analysis of the stakeholders' needs and desires, translating and classifying them into functional, behavioral, structural and experiential requirements. Then these requirements are prioritized with an ordinary scale method to rack the importance of each requirement for the predictive maintenance system. The identified and prioritized requirements are the basis for the architecture process.

The architecture development process begins with the functional analysis on the identified functional requirements. Functional decomposition is proposed for complexity management of the predictive maintenance system. The functional exchange between functions is studied to propose the functional architecture. The external actors and the interactions with the function are studied.

The logical architecture builds on top of the functional architecture. The functions are allocated to logical components and the external and internal interfaces are identified. The proposed logical architecture remains generic and it could evolve depending on the available data of technical system and the scope for the new predictive maintenance system.

Future research will be focused on the extension of the present approach on systems with heterogeneous types of available data and the methodologies to assess the different potential architectures to fulfill the predictive maintenance system.

ACKNOWLEDGEMENTS

The authors want to acknowledge the "Tecnológico de Costa Rica" for funding this research.

The authors want to thank Prof. Bernard Grabot from the "École Nationale d'Ingénieurs de Tarbes" for his valuables comments on previous versions of this manuscript.

REFERENCES

[1] J. J. Montero Jimenez and R. Vingerhoeds, "Enhancing operational fault diagnosis by assessing multiple operational modes," in *Proceedings - International Conference in Modelling, Optimization and Simulation MOSIM 2018*, 2018, pp. 237–244.

[2] G. P. Sullivan, R. Pugh, A. P. Melendez, and W. D. Hunt, "Operations & Maintenance Best Practices: A Guide to Achieving Operational Efficiency," *U. S. Departament of Energy, Federal energy management program.* 2010.

[3] MIMOSA, "Open System Architecture for Condition-Based Maintenance (OSA-CBM)," *http://www.mimosa.org/mimosa-osa-cbm/*, 2001. .

[4] R. Li, W. J. C. Verhagen, and R. Curran, "A functional architecture of prognostics and health management using a system engineering approach.," in *Proceedings of the European Conference of the PHM Society*, 2018, p. Vol 4 No 1.

[5] A. Saxena, I. Roychoudhury, and J. R. Celaya, "Requirements Specifications for Prognostics : An Overview," in *Proceedings of AIAA Infotech@Aerospace 2010*, 2010.

[6] J. S. Gero, "Design Prototypes: A knowledge Representation Schema for Design," *AI Mag.*, vol. 11, no. 4, pp. 26–36, 1990.

[7] F. Brazier, P. van Langen, S. Lukosh, and R. A. Vingerhoeds, "Design, Engineering and Governance of Complex Systems," in *Projects and People - Mastering success*, H. L. M. Bakker and J. P. Kleynen, Eds. NAP Foundation Press, 2018, pp. 34–59.

[8] E. Crawley, B. Cameron, and D. Selva, *System Architecture: Strategy and Product Development for Complex Systems*. Pearson Higher Education, Inc., 2015.

[9] P. Roques, *Systems Architecture Modeling with the Arcadia Method 1st Edition*. ISTE Press, 2018.

[10] N. Zerhouni, V. Atamuradov, K. Medjaher, P. Dersin, and B. Lamoureux, "Prognostics and Health Management for Maintenance Practitioners-Review, Implementation and Tools Evaluation," *Artic. Int. J. Progn. Heal. Manag.*, vol. 8, no. 60, p. 31, 2017.

[11] B. Schmidt and L. Wang, "Predictive Maintenance: Literature review and future trends," *Conf. Proc. 25th Int. Conf. Flex. Autom. Intell. Manuf.*, vol. 1, pp. 232–239, 2015.

[12] N. Sakib and T. Wuest, "Challenges and Opportunities of Condition-based Predictive Maintenance: a Review," in *6th CIRP Global Web Conference: "Envisaging the future manufacturing, design, technologies, and systems in innovation era,"* 2018, pp. 267–272.

[13] Y. Lei, N. Li, L. Guo, N. Li, T. Yan, and J. Lin, "Machinery health prognostics: A systematic review from data acquisition to RUL prediction," *Mech. Syst. Signal Process.*, vol. 104, pp. 799–834, 2018.

[14] R. A. Vingerhoeds, P. Janssens, B. D. Netten, and M. Aznar Fernández-Montesinos, "Enhancing off-line and on-line condition monitoring and fault diagnosis," *Control Eng. Pract.*, vol. 3, no. 11, pp. 1515–1528, 1995.

[15] International Organization for Standardization (ISO), *ISO 13374-1:2003 Condition monitoring and diagnostics of machines — Data processing, communication and presentation — Part 1: General guidelines*. 2003.

[16] INCOSE, *Systems Engineering Handbook. A guide for system life cycle processes and activities. Fourth Edition*. Wiley, 2015.

[17] P. Achimugu, A. Selamat, R. Ibrahim, and M. N. R. Mahrin, "A systematic literature review of software requirements prioritization research," *Inf. Softw. Technol.*, 2014.

[18] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *2008 International Conference on Prognostics and Health Management, PHM 2008*, 2008.

[19] E. . Ramasso and A. . Saxena, "Review and analysis of algorithmic approaches developed for prognostics on CMAPSS dataset," *PHM 2014 - Proc. Annu. Conf. Progn. Heal. Manag. Soc. 2014*, 2014.

[20] G. W. Vogl, B. a Weiss, and M. A. Donmez, "Standards for Prognostics and Health Management (PHM) Techniques within Manufacturing Operations," *Annu. Conf. Progn. Heal. Manag. Soc.*, 2014.

[21] T. Wang, J. Yu, D. Siegel, and J. Lee, "A similarity-based prognostics approach for remaining useful life estimation of engineered systems," in *2008 International Conference on Prognostics and Health Management, PHM 2008*, 2008.

3.3 Lessons learnt

Once the initial needs and desires for a new predictive maintenance system have been gathered, they need to be formalized into requirements. There exist several methods to elicit requirements. One of them is the Functional, Behavioral, Structural and Experiential analysis (FBSE) [Bra+18], which is an extension of the FBS analysis introduced by John Gero in [Ger90] and [GK07]. This method classifies the requirements into four main groups:

- Functional requirements: gather all intended functions of the system. These requirements derive from the purpose of the system, they answer the “why?" the system is being created.
- Behavioral requirements: are related to system performance. Behavioral requirements answer the “how?" and the “when?" of the development of the new system.
- The structural requirements: refer to the constraints imposed for the system development. Structural requirements are those related to “what" is fixed or expected previous to the concept design. For example, authorities’ constraints, connectivity with the existing system and available technologies.
- Experience requirements: are related to feelings, emotions and perceptions created by the system to the users. These are the most difficult requirements to elicit as some of them can be subjective and thus difficult to validate. Experiential requirements could be translated into more specific requirements that are clustered into the three first groups. The benefit of adding this fourth group of requirements is to avoid ignoring important needs from human factors.

The FBSE analysis is a consistent methodology for a systems architect as it allows the direct identification of all expected functions from the system. The functional requirements are the starting point to explore the solution space of a new system. These functions can be decomposed into sub-functions depending on the complexity of the functions. Decomposition of functions is a well-known process to manage complexity. Crawley et al. [CCS15] explain that two levels of decomposition are enough to describe a system; if more elements are needed for a function, it is advisable to treat it as a subsystem and perform the system engineering process separately. The architect organizes the functions and sub-functions making links among them using functional interfaces. The organization of the functions, sub-functions and functional interfaces is known as the functional architecture of a system [INC15].

After defining the functional architecture, the architect can start to propose logical components to fulfil each function and each interface. This is known as the logical architecture of the system. The logical architecture aims at defining a much detail as possible of the system without narrowing down the solution space to a specific technology, programming language or environment. The objective of the logical architecture is to present how the system will work to fulfil the expectations. Once the logical architecture is complete, an exploration of the solution space starts to identify suitable technologies, programming languages, methods, materials to fulfil the logical components.

The selection of these components is done at the software, hardware and structural levels of the system. This is known as physical architecture and states details of how the system will be developed and built. If several possible solutions to fulfil the logical architecture are identified, the architect must assess them based on the behavioural and structural requirements obtained from the FBSE analysis. The union of the functional architecture, logical architecture and physical architecture constitutes the complete system architecture.

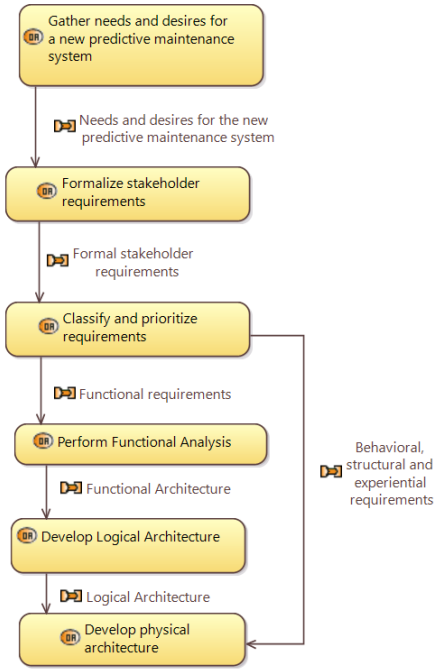


Figure 3.1: An approach to developing a system architecture in the concept stage

Recalling the refined research proposed at the end of Chapter 2:

1. How to address the design of predictive maintenance systems?
2. How to suggest a suitable approach for a predictive maintenance system solution?
3. How to select a suitable model or combination of models given a new predictive maintenance problem to solve?
4. How can a designer benefit from the experience of existing systems to develop new predictive maintenance solutions?

This chapter proposes a systematic approach to address the concept stage of predictive maintenance systems to provide an answer to the first refined research question. The different steps from the gathering of the initial needs and desires from the stakeholder until the logical architecture definition have been covered and different methods have been proposed to address them. However, the rest of the refined research questions are still to be answered. In predictive maintenance, the solution space is vast and complex as was shown in Chapter 2. There are no specific rules an architect can follow to select the suitable models and approaches to solve new predictive maintenance problems. In this research, an hypothesis to overcome this problem is proposed: the implementation of a Decision Support System (DSS) based on Case-Based Reasoning (CBR) and supported by ontologies could help the architect select suitable components based on past experiences extracted from successful implementations of predictive maintenance systems. The following three chapters are dedicated to the development of the DSS and how it fits in the proposed framework of predictive maintenance systems design. Chapter 4 and Chapter 5 explain the building blocks of the proposed framework: ontologies and CBR. Chapter 6 is then dedicated to explaining their integration in the DSS.

CHAPTER 4

Ontology development to support Case-based reasoning systems

"Your understanding of what you read and hear is, to a very large degree, determined by your vocabulary, so improve your vocabulary daily."

Zig Ziglar

Content

4.1 Building the framework vocabulary

Recalling the research statement proposed at the end of the Chapter 3, a Decision Support System (DSS) is proposed to suggest suitable components to fulfil the generic components of the logical architecture. The DSS is built upon two main technologies: Case-Based Reasoning (CBR) and ontologies. CBR is a reasoning paradigm that aims to solve new problems based on the experiences of similar problems solved in the past. CBR is addressed in Chapter 5, but a brief introduction is necessary to understand the role that plays the ontologies in the DSS developed in the current research. CBR systems are developed on a vocabulary framework [Alt+12]; [Sán+12]. This vocabulary is necessary to give structure to the stored cases of solved problems in a case base, to the similarity measures that compare the new problem with those in the case base, and to the necessary knowledge to adapt the retrieved solution for the new problem (see Figure 4.1. In the proposed framework, an ontology is chosen to serve as terminology framework. An ontology model provides the terms, definitions, and relations among terms that are used to build the case structure, the similarities, and adaptation knowledge in the DSS. This chapter is dedicated to the theoretical background and the development of the ontology model for the proposed DSS.

Section 4.2 provides a brief introduction to ontologies. Originally, an ontology was developed to study predictive maintenance terminology within the maintenance strategies domain. This was called the Ontology

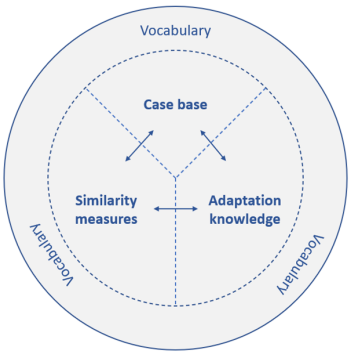


Figure 4.1: Case-Based Reasoning based on a vocabulary framework [Alt+12]

model for Maintenance strategy Selection and Assessment (OMSSA). In the later stages of the research, the benefits of ontologies as complementary knowledge models for CBR systems were recognized. To support the CBR system for predictive maintenance models recommendation, an ontology was created taking OMSSA as reference. Section 4.3 is dedicated to explaining the development of OMSSA, the terms, their relations and the methodology followed for its creation. It also includes the verification of the ontology using a simple case study. OMSSA has been consolidated in an article that has been accepted for publication in the Journal of Intelligent Manufacturing. In Section 4.4, a novel the ontology model to support the CBR system for predictive maintenance component selection is developed. This second ontology model received the name of Ontology for Predictive Maintenance Architecture and Design (OPMAD) and the description of its development includes the bridges for its integration with CBR in the DSS. The integration of ontologies and CBR is further addressed in Chapter 6 where the justification and benefits of their combination are also discussed.

4.2 Ontology

The word ontology finds its roots in philosophy where it can be defined as "a particular theory of being or existence" [RN12]. From an ontological point of view, all things are concepts whether concrete or abstract. In etymology, ontology is related to epistemology. While ontology is oriented to define the reality of things, "what exists in the world"; epistemology refers to the human perception of things through their senses, "what a person believes about the ontology". With the emergence of computer and information science, the word ontology has been adopted and its meaning has evolved. It conserves its roots in which all things are considered concepts but the scope and applications of the word is oriented to knowledge representation. To avoid ambiguity, all the content in this manuscript with regard to ontology (after this point) is seen in the light of the information science perspective.

In information science, an ontology is a formal explicit description of concepts in a domain of discourse, properties of each concept describing its features, attributes, and restrictions [NM01]. One of the most common goals in developing ontologies is “sharing a common understanding of the structure of information among people and software agents” [Gru93]; [Mus92]. This means that the vocabulary used by people in a specific domain of knowledge is enabled to be "machine-readable". All concepts in an ontology are represented by classes that are linked by properties (also called relations). Ontologies are defined using formal languages. One of the most recognized is the Web Ontology Language in its second version (OWL2) which is supported by the World Wide Web Consortium (W3C) [Wor12]. But also it is normal to graphically represent ontologies with graphs whose nodes are the ontology classes and the edges are the ontology relations among the classes. By only considering the graphical representation, ontologies can

be confused with semantic networks or frame-based representations. Frame-based representations can be seen as an extension of semantic networks and ontologies can also be seen as an extension of frame-based representations. Semantic nets could be used to represent anything (not just word concepts) and can be seen in many ways equivalent to generic graph representations [Hoe09]. The main contribution of the frame-based view was that it fixed a knowledge representation perspective. The frame proposal fixes the perspective on descriptions of situations in general, and objects and processes in a particular situation. However, semantic networks and frame-based models lack of formal logic-based semantics [HLP08]. Ontologies are based on Description Logics (DL) which provide logical formalism to overcome the deficiency in logic-based semantics of other knowledge representation methods [DMB16]; [HLP08].

The W3C supports the creation and use of ontologies because of their importance for the semantic web. The semantic web is an extension of the current web, where the information is well-structured and defined so that it can be processed by a machine [NB18]. The W3C recommends the use of the OWL2 language for the semantic web ontologies [MO+14]. Ontologies in OWL2 are compatible with information written in the Resource Description Framework (RDF). In RDF, information is represented in semantic triples: subject-predicate-object. OWL2 ontologies are primarily exchanged as RDF documents. This means that all knowledge stored in an ontology can be also represented in semantic triples. Two related classes will represent the subject and the object of the triple, while the relation between the two classes represents the predicate of the triple. RDF structure provides a flexible means to model, store and manage information; other methods requiring variable-length fields would require a more complicated implementation. Figure 4.2 presents the RDF structure of subject-predicate-object as well as the example of the representation of two sentences in RDF: e.g. "the machine has failure mode" and "failure mode has failure cause". For both sentences, the predicate (the relation of ownership between the subject and the object) is "has". For the first sentence, the subject is "the machine" and the object is "failure mode". For the second sentence "failure mode" is now the subject while the object is "failure cause". This shows that the concepts in RDF (classes in ontologies) can be subjects or objects depending on the predicate.

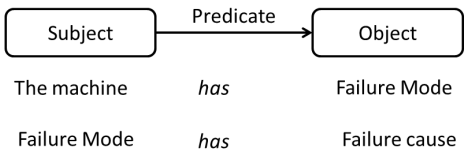


Figure 4.2: Semantic triple structure with examples

Ontologies help to make the knowledge explicit, defining the relations among different terms. They allow a deep analysis on domain knowledge, helping to identify semantic rules among the terms. These rules can be used to develop algorithms that perform automated inferences based on domain knowledge. Taking the example of Figure 4.2, as "the machine" has a "failure mode" and the "failure mode" has a "failure cause", one can infer that "the machine" has a "failure cause".

Ontologies enable the reuse of domain knowledge. For example, an ontology can represent all the terms related to time and another can represent all terms related to geographical locations; both ontologies can be reused to model the knowledge of another domain such as "sports events", which usually includes terms from time and geographical locations. The time and geographical locations ontologies can be also used to model the knowledge of "university courses". Even when "sports events" and "university courses" are different domains of knowledge, the definition of the terms related to time and the geographical location remains the same. Recent trends in ontology development are oriented towards the use of structured and standardized methodologies that include the use of generic ontologies (domain-neutral) such as BFO [Int20b] as a basis to build domain-specific ontologies or other reference ontologies. This boosts the reuse of knowledge already modelled in ontologies.

4.3 Ontology model for Maintenance Strategies selection and assessment (Article 3)

The content in this section corresponds to a published work in the Journal of Intelligent Manufacturing. Reprinted, with permission, from Juan José Montero Jimenez, Rob Vingerhoeds, Bernard Grabot, Sebastien Schwartz. “An Ontology Model for Maintenance Strategy Selection and Assessment”. Journal Intelligent Manufacturing, ***Accepted for publication*** [Mon+21]. This article is referred to as Article 3 in the current manuscript.

This article has been published in the
Journal of Intelligent Manufacturing

An Ontology Model for Maintenance Strategy Selection and Assessment

Juan José Montero Jimenez* [1,2], Rob Vingerhoeds[1], Bernard Grabot[3], Sébastien Schwartz[1]

** Corresponding author*

[1] *ISAE-SUPAERO, Université de Toulouse, 10 Avenue Edouard Belin, 31400 Toulouse, France*

[2] *TEC-Tecnológico de Costa Rica, Calle 15, Avenida 14., 1 km Sur de la Basílica de los Ángeles, Provincia de Cartago, Cartago, 30101, Costa Rica*

[3] *ENIT- INP Toulouse, 47, avenue d'Azereix - BP 1629 - 65016 Tarbes, France*

Abstract

Within maintenance management activities, engineers need to select maintenance strategies so to carry out the technical maintenance actions. A single equipment is composed of several components with different failure modes. There should be a maintenance strategy for each of them; while some of the components can be run-to-failure applying corrective maintenance, some others cannot afford a failure, and preventive or predictive strategies should be implemented. Selecting and assessing maintenance strategies is a complex task for which information from many sources should be retrieved. Information from a Failure Mode, Effects and Criticality Analysis (FMECA), a cost-benefit-risk analysis, Computational Maintenance Management Systems (CMMS), is often used by engineers to select and assess maintenance strategies. A selected strategy is often not evaluated over time to check its effectiveness. The strategy may need adjustments or substituted by a more efficient one, for example, a condition-based strategy substituting a time-based one. To facilitate maintenance strategies selection and assessment, the current study proposes an Ontology model for Maintenance Strategy Selection and Assessment (OMSSA). OMSSA serves as a formal terminology framework in maintenance strategies that can be used to develop smart computational agents that can help in the decision-making process for selecting and assessing maintenance strategies. To facilitate its future reuse and integration with other ontologies in the industrial domain, OMSSA builds following the state-of-the-art in ontology development by using a top-level domain-neutral ontology, the Basic Formal Ontology (BFO).

Keywords: Maintenance strategy, ontology, knowledge base, knowledge reuse.

Introduction

Maintenance, as defined by the standard BS EN 13306 (European Committee for Standardization 2017), is the “combination of all technical, administrative and management activities throughout the life cycle of an asset in order to maintain or restore it to the state where it can perform the intended functions." Maintenance exists since the first humans substituted the broken parts of their first tools, and it has evolved along with the increment of the complexity of

artificial assets. Today it plays a vital role in keeping technical systems in optimal operating conditions.

Within maintenance management activities of modern industrial systems, engineers need to select maintenance strategies so to carry out the technical maintenance actions. These maintenance strategies are directly linked to triggering events such as a failure (corrective maintenance), a safe operating time interval (preventive maintenance), or an abnormal symptom (predictive maintenance) that launches a maintenance action. Selecting the right maintenance strategy can be a complex task. It depends on several factors, such as the detrimental effects of an unforeseen failure, the maintenance costs, the maintenance personnel skills, and the access to technological tools to carry out the most advanced maintenance actions (Emovon et al. 2018). Information from different sources should be assessed to perform the right maintenance strategy selection, such as Failure Mode, Effects and Criticality Analysis (FMECA), a cost-benefit-risk analysis, Computational Maintenance Management Systems (CMMS). Besides, a selected strategy is often not assessed over time to confirm its effectiveness or suitability. Smart Decision Support Systems (DSS) can be developed to help engineers in such complex tasks. These smart DSS can be developed on the basis of a well-structured vocabulary framework. The definition of concepts, attributes, and relations among concepts can help to structure the knowledge around maintenance strategies, allowing their better selection and assessment.

For this aim, the current study proposes an Ontology model for Maintenance Strategy Selection and Assessment (OMSSA). From the practical point of view, the objective of OMSSA is to provide a vocabulary structure that can be used for the development of smart agents such as DSS to help engineers in the selection, assessment, and even in the implementation of maintenance strategies. From the scientific point of view, the OMSSA contribution can be seen as a structured meta-knowledge representation in the domain of maintenance strategies.

Ontologies are formal, explicit descriptions of concepts in a domain of discourse, the properties of each concept describing its features, attributes, and restrictions (Noy and McGuinness 2001). One of the most common goals in developing ontologies is "*sharing a common understanding of the structure information among people and software agents"* (Gruber 1993; Musen 1992). However, there are also several other motivations to create ontology models, such as enabling reuse and analysis of domain knowledge or making domain assumptions explicit. Ontologies are powerful tools for knowledge representation. The described concepts are represented by classes, while the properties represent the relations between the terms. An ontology with individual instances for its classes constitutes an ontology-based knowledge base (Noy and McGuinness 2001; Qin et al. 2016). This knowledge base can be used for several purposes in the artificial intelligence field. For example, it can provide a formal terminology framework allowing to perform reasoning based on semantic rules.

OMSSA provides a formal terminology framework in maintenance strategy selection and assessment, considering not only traditional approaches but also the current trends towards the use of advanced diagnosis and prognosis tools to trigger maintenance actions. Its development is part of a bigger research project in knowledge reuse for maintenance systems. For the OMSSA, the taxonomy of corrective maintenance, preventive maintenance, and predictive maintenance has been selected to classify the maintenance strategies. These three terms have been widely used in

the research community and across the industry (Emovon et al. 2018; Kothamasu et al. 2006; Montero Jimenez et al. 2020). Even when a good combination of the three strategies is vital to keep a technical system in nominal operation (Montero Jimenez and Vingerhoeds 2018), there is a trend to implement proactive techniques to avoid unforeseen failures that can affect a technical system operation. For OMSSA, special attention is given to the current trends in maintenance management towards the implementation of predictive maintenance systems to reach a safer, more reliable, and more efficient operation of technological systems.

The rest of the paper is organized as follows: "Sect. Maintenance strategies" introduces the topic of maintenance strategies, the selected taxonomy for maintenance strategies used in OMSSA, and the maturity model of the evolution of the maintenance strategies. "Sect. Ontology engineering" is dedicated to ontology engineering and the methodology used to develop OMSSA, based on the alignment to a top-level domain-neutral ontology to facilitate its future reuse. "Sect. Ontologies in maintenance" summarizes the related studies on ontologies in the maintenance domain and analyses their limitations for addressing the current trends of maintenance strategies. The requirements and vocabulary sources for OMSSA are introduced. "Sect. Ontology for Maintenance Strategy Selection and Assessment (OMSSA)" explains the most important terms and relations for maintenance strategy management using a graphical representation for different parts of the model. "Sect. Evaluation and discussion" is dedicated to the verification and validation of the ontological model, as well as its illustration on a case study. "Sect. Conclusion and future work perspectives" concludes this paper by presenting the final remarks and perspective for future work.

Maintenance strategies

Maintenance strategies are part of maintenance management. They aim to establish how and when maintenance action should be performed to restore or maintain a specific artifact, component, or system into its nominal operational state. Maintenance strategies are often established by experts who know the different failures that a system can encounter, taking into consideration the likelihood and the detrimental impacts (human lives, environmental damage, costs, production losses) of each failure occurrence. One classic approach to establish maintenance strategies is the Failure Modes, Effects, and Criticality Analysis (FMECA) (International Electrotechnical Commission (IEC) 2018), in which a maintenance strategy is assigned to each failure mode of each component of an equipment.

Maintenance strategies can be divided into three groups depending on the triggering event of the maintenance action: `corrective maintenance`, `preventive maintenance`, and `predictive maintenance`:

- `Corrective_maintenance` is triggered by a `failure`. However, there is an important difference between a run-to-failure strategy and a maintenance action produced by an unexpected failure. The second one cannot be considered as a strategy as it was not planned (Hodkiewicz et al. 2021). It is important not to confuse a corrective strategy with an undesired failure. The undesired failure is actually a sign that preventive or predictive strategies assigned to the failed component and should be re-evaluated.

- `Preventive_maintenance` actions are triggered by fixed time recommendations coming from safety operating time intervals or preventive scheduled stops of an `item`. Classic approaches such as Reliability Centered Maintenance (RCM) provide a set of strategies based on preventive action such as failure finding, fixed time restoration and fixed time replacement (Moubray 1997).
- `Predictive_maintenance` strategies propose an alternative to preventive actions. They aim at triggering the maintenance actions based on the condition of a machine. Predictive maintenance is carried out by specialized models or tools that help maintenance engineers for an intelligence-based decision-making process to trigger maintenance actions. Classic approaches such as RCM summarize these strategies as "condition-based," providing a limited insight on how these strategies are actually developed. Recent trends in maintenance show that predictive maintenance strategies can have three types of triggering events: early fault detection, health/degradation threshold overshoot, and future failure forecasting. These three triggering events could affect the scheduling of maintenance actions based on the `condition` of the `item`. Anticipating failures allows the optimization of maintenance schedules as the maintenance actions will be triggered accurately when needed (Montero Jimenez et al. 2020). Condition monitoring quality is an important issue for predictive maintenance strategies (Castaño et al. 2020). It is important to ensure accurate monitoring so that the recorded data can be used to train the specialized models that carry out the diagnosis and prognosis tasks.

From corrective maintenance to predictive maintenance, there is an evolution in the implementation of maintenance strategies, from the simplest ones to the most advanced. This evolution can be represented by a maturity model. The more advanced the strategy, the more benefits it could bring, but also the higher the implementation cost and the complexity. The final step in this maturity model is the strategies based on prognostics which remain a challenge for their implementation and are often included in the Prognostics and Health Management (PHM) discipline (Montero Jimenez et al. 2020). Fig. 1 shows the proposed maturity model for maintenance strategies considering the triggering event.

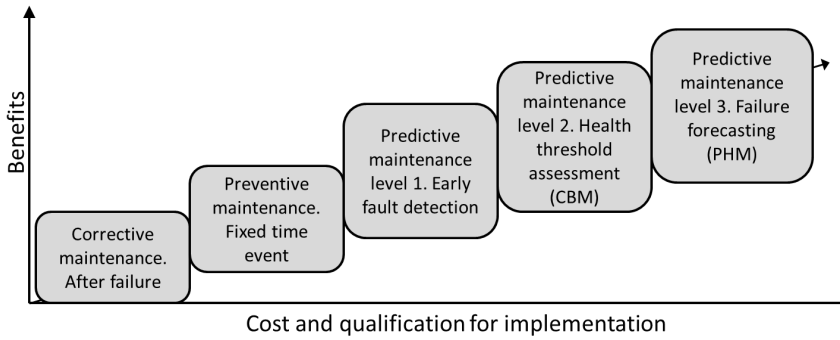


Fig. 1. Maintenance strategies maturity model

Ontology Engineering

Several specific methodologies exist for ontology development that are not simply interchangeable: two different methodologies will not lead to the same final ontology. (Keet 2018) presents an extensive comparison of the different methodologies and how they should be selected depending on the objectives of the ontology. As OMSSA aims at formalizing the knowledge of a specific domain, the selected methodology is a variant of "Ontology development 101" (Noy and McGuinness 2001). The main reason why this methodology was selected is because of its capability and simplicity for building new ontologies by reusing terms from top-level ontologies. Having a top-level ontology improves the interoperability of ontologies and thus facilitates their future reuse. For OMSSA, the selected top-level ontology is the Basic Formal Ontology (BFO) (Arp et al. 2016; International Organization for Standardization (ISO) 2020a, 2020b). To facilitate the alignment to BFO, some mid-level reference ontologies can be used. These ontologies provide a set of terms and relations that are common among different but related knowledge domains. For OMSSA, the alignment to BFO is performed using the Relation Ontology (RO) and the Common Core Ontologies (CCO) as these ontologies provide a good set of terms and relations to model ontologies in the industrial domain.

This section explains the different “blocks” used for OMSSA development. OMSSA was developed using the current version of the platform Protégé (Stanford University 2020). This platform supports the latest specifications of the OWL 2 set by the World Wide Web Consortium.

Ontology development 101

“Ontology Development 101” is a seven-step iterative methodology used to create ontologies (Noy and McGuinness 2001). Each step is explained hereafter:

1. Determine the scope of the ontology, meaning the main goal for which the terminology framework will be created. At this point, important aspects such as the ontology domain or the intended use of the ontology must be defined. To do so, a set of “competency questions” can be proposed to delimitate the ontology scope. Competency questions are a set of questions that an ontology-based knowledge base should be able to answer, emulating the way an expert would answer (Grüninger et al. 1995a).
2. Consider reusing existing ontologies. This step is vital to ensure the alignment of the new ontology with related ontologies that already exist. Some terms and relations can be retrieved from these existing ontologies. For OMSSA, some ontologies are being completely reused (imported) as upper-level ontologies (BFO, RO, CCO), and some important terms and relations linked to OMSSA scope have been extracted from other domain-specific ontologies (see Sect. Ontology engineering).
3. Enumerate terms. Here, the main terms related to the selected scope are listed. A classification of these terms is not needed at this step, as it will be performed in the following steps.
4. Define the classes. The first classification of the enumerated terms is performed to define the classes of the new ontology. Classes are the means to represent the different entities in ontologies. They must be written in the singular form.

5. Define properties of classes. Here, the internal structure of concepts is described. The properties of the classes represent the links among the classes. Verbs in third-person conjugation are often used to represent these properties.
6. Define constraints. The classes and properties among these classes can have several constraints, such as for example value type, allowed values, and cardinality.
7. Create instances. This last step aims at instantiating the new ontology. Once the ontology has individual instances for its classes, it becomes an ontology-based knowledge base. So, for many reference ontologies, the instantiation does not take place as they are intended to be as generic as possible. Sometimes, this instantiation is done for validation purposes, and later the ontology is published without the specific instances.

These steps can be iterative. Often, missing terms are identified in later steps of ontology development. This leads to the iteration of previous steps to make sure that all classes, properties, and constraints are properly defined for the new identified terms.

BFO as a top-level ontology

BFO is a top-level ontology that serves as starting point for the development of other reference ontologies and/or domain-specific ontologies (Arp et al. 2016; International Organization for Standardization (ISO) 2020b). It serves as a generic framework that offers a set of domain-neutral classes in which more specific terms can be clustered. Using a top-level ontology is advisable when developing domain-specific ontologies as it facilitates the interoperability with other related ontologies that use the same terms and relations (International Organization for Standardization (ISO) 2020a, 2020b). BFO has been successfully used by the Open Biological and Biomedical Ontology (OBO) Foundry (Arp et al. 2016). BFO is also used along with the Descriptive Ontology for Linguistic and Cognitive Engineering (DOLCE) by the Industrial Ontologies Foundry (IOF) to develop proof-of-concept ontologies (Karray et al. 2019; Sanfilippo et al. 2019). The future reuse and alignment of OMSSA to IOF ontologies motivated the selection of BFO as top-level ontology.

BFO starts from the most basic class `entity`, divided into two main subclasses: `continuant` and `occurrent` (Arp et al. 2016; International Organization for Standardization (ISO) 2020a, 2020b; Rodrigues and Abel 2019). Continuants refer to entities that continue to exist through time, for example objects, functions, and qualities. Occurrents refer to entities that occur, meaning that they are spread not only in space but also in time; for example, processes belong to this class. Based on these two main entities, BFO proposes a set of domain-neutral classes that can be used to model any domain-specific ontology. Fig. 2 shows the BFO 2.0 structure, which is a stable version since 2015 (Arp et al. 2016). In the current study, continuants are represented in blue and occurrents in green.

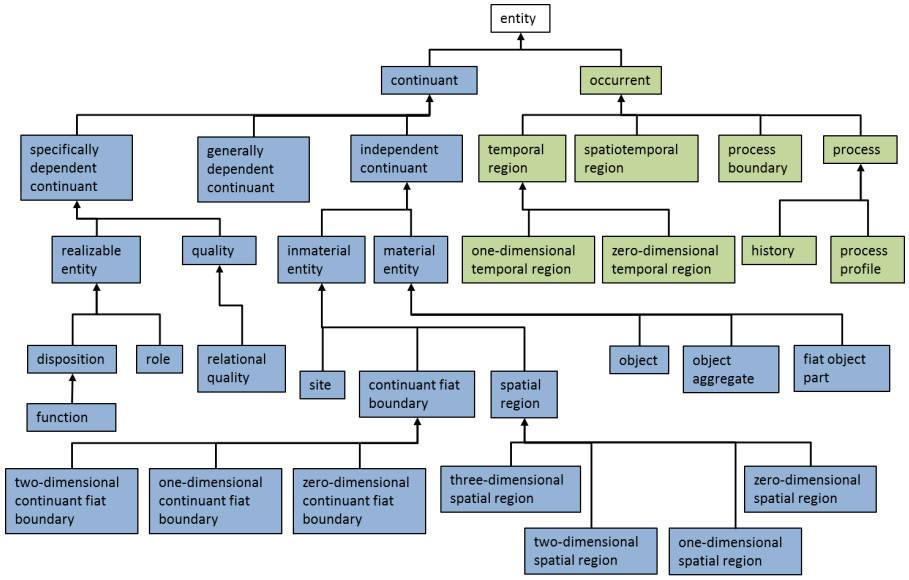


Fig. 2. Basic Formal Ontology structure, based on (International Organization for Standardization (ISO) 2020b)**.**

Building an ontology using a top-level ontology such as BFO could be complicated because all new domain-specific classes must be clustered under domain-neutral classes. These upper-level classes have their own properties and constraints that are inherited by the domain-specific classes. Some consistency issues might arise when creating the relations at the lower levels of the ontology. Nevertheless, having a top-level domain-neutral ontology promotes more important data interoperability and thus enables ontology reuse (International Organization for Standardization (ISO) 2020b).

Reuse of Relation Ontology and Common Core Ontologies

Relation Ontology (RO) (Foundry 2020) and Common Core Ontologies (CCO) (Rudnicki 2020a, 2020b) are mid-level domain-neutral BFO-compliant ontologies. RO provides domain-neutral relations among different classes, from which more specific relations can be derived in a specific domain. CCO is composed of eleven ontologies that have as objective the representation and taxonomy integration of generic classes and relations across all domains of interest. Within CCO, the following ontologies are found: Information Entity Ontology, Agent Ontology, Quality Ontology, Event Ontology, Artifact Ontology, Geospatial Ontology, Time Ontology, Units of Measure Ontology, Currency Unit Ontology, Extended Relation Ontology and Modal Relation Ontology (Rudnicki 2020a). The import structure of CCO in its version 1.3 is presented in Fig. 3.

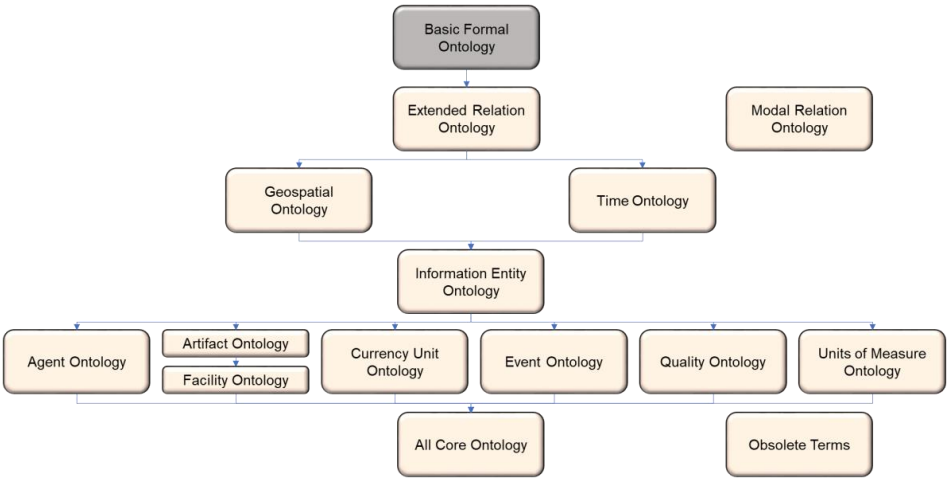


Fig. 3. Import structure of CCO version 1.3 (Rudnicki 2020a)

In maintenance management, many concepts can be clustered in the CCO class `Information_Content_Entity`. This class includes pieces of information used to describe, designate, or direct other entities. For maintenance, the concepts under this CCO class are used to describe the maintainable items attributes, condition data, costs, models, specifications, among others. To model all these information entities and the phenomena they describe in real life, OMSSA directly imports the Information Entity Ontology, the Artifact Ontology, and the Event Ontology from CCO. Due to the import structure of CCO, the Time Ontology, Geospatial Ontology, and the Extended Relation Ontology are indirectly imported from CCO, and consequently, RO and BFO are also imported.

It is important to mention that by importing the Information Entity Ontology, the “aboutness” paradigm to represent data properties is adopted for OMSSA. In contrast to UML, where attributes are listed inside each class, the "aboutness" paradigm represents these properties as separated classes. These classes are linked to the main class with the relation CCO: *isAbout*. This facilitates the management of data properties by machines (Smith and Ceusters 2015).

As for BFO, CCO and RO allow having a mid-level set of domain-neutral ontologies that facilitate the alignment of core or application ontologies. Having this set of mid-level ontologies aligned to BFO improves the interoperability of core and domain ontologies such as OMSSA with those of the industry domain that share the same structure. BFO and CCO serve as a generic ontology development structure to ensure uniformity in the use of generic terms.

Relations in OMSSA

One of the important aspects of ontologies concerns the object properties that gather the relations among the different classes. As mentioned in the previous section, the Relations Ontology RO and the extended relations ontology from CCO are implemented as a basis to represent the relations of the different classes for OMSSA. It is important to mention that more

specific relations could be used depending on the ontology domain but are contained as sub-relations of those in the RO or the Extended Relation Ontology from CCO.

Using these generic relations prevents inconsistency in the ontology. These relations are intended for specific types of BFO classes. Several inconsistencies might arise when developing the ontology. For example, the relation "*precedes*" can only be used to relate two occurents. In this context the assertion "*a fault precedes the failure*" cannot be directly proposed because a `fault` is considered as a `continuant` while the `failure` is an `occurrent` (definitions and classification of the terms are presented later in this article). To solve this inconsistency, a `fault` can be linked to another `occurrent` which is `degradation` using the relation "*participates in*", and the class `degradation process` is linked to the class `failure` with the relation "*precedes*". The assertions to link all these terms become "*a fault participates in the degradation*" and "*the degradation precedes the failure*". Having this fixed set of generic relations demands a deeper understanding of the terms of the ontologies. It turns out to be a complex task to link all terms in the ontology, but it also allows a better and more complete analysis of the used classes and their actual relations. It helps to identify missing terms and to prevent inconsistency in the ontology.

Only a few domain-specific relations have been added in OMSSA. They concern sub-relations of other domain-neutral object properties imported from RO and CCO. The added relations help to avoid inconsistency or ambiguity among different class properties. The sub-properties proposed in OMSSA help to distinguish the difference between two similar relations. The relations used in OMSSA, including their definition and the graphical representation in the current study, can be found in the Appendix.

Ontologies in maintenance

Ontologies describe the knowledge of a specific domain through a formal representation of concepts and relations (Nuñez and Borsato 2018). The use of ontologies can simplify the data exchange and interoperability among dissimilar systems, serving as a framework for answering queries about that data (Cao et al. 2019; Panov et al. 2014). Ontologies are often built upon the World Wide Web Consortium (W3C) standards such as the Ontology Web Language (OWL) and are often used to formalize vocabulary for the semantic web (Antoniou et al. 2012; Lu et al. 2019; Nuñez and Borsato 2018; Talhi et al. 2019). Ontologies provide a terminology framework that allows information from a specific domain of knowledge to be processed by a machine. Ontologies have already been used in the maintenance domain. Formal terminology frameworks have been developed to support tasks such as maintenance management, condition monitoring, and prognostics, and health management.

IMAMO (Industrial MAintenance Management Ontology) (Karray et al. 2012) presents a domain ontology for industrial maintenance management. The main goal was to establish a common and shared terminology set for maintenance support systems. The methodology used for its development is METHONTOLOGY (Fernandez et al. 1997). The authors used UML to graphically represent the selected terms and their relationships. IMAMO reasoning assessment was performed using the PowerLOOM tool. IMAMO integrates and reuses terms from other ontology

models such as the SMAC-model (Matsokis et al. 2010) and the MIMOSA-CRIS model (MIMOSA 2001). Maintenance strategies management is not included in the IMAMO approach.

An ontology-based model for prognostics and health management of machines OntoProg, is presented in (Nuñez and Borsato 2017) and (Nuñez and Borsato 2018). The scope of these ontologies is to detail the initial phase of Prognostics and Health Management (PHM) to support intelligent decision systems. These systems can potentially be developed to provide health condition of machines and support smart manufacturing. The ontology model was built following the "Ontology Development 101" (Noy and McGuinness 2001) with the OWL2 language (the current version of the Ontology Web Language) using the Protégé OWL editor. These ontologies reuse accurate terms from maintenance standards and norms that are also used in OMSSA. These ontologies are domain-specific and focus only on PHM, and thus, they leave the maintenance strategies terminology out of scope.

A core ontology for condition monitoring, CM-core, defines a basic set of concepts to understand the condition monitoring domain (Cao et al. 2019). Such a core ontology serves as a basis for other domain ontologies that can be used in several applications. It also builds upon the "Ontology Development 101" (Noy and McGuinness 2001) and uses UML to represent the ontology. As for the previous two ontologies, it focuses only on condition monitoring, which is part of current trends for maintenance strategies but not presented as such in CM-core. ROMAIN (Karray et al. 2019) is a Reference Ontology for industrial MAINtenance, compliant with the Basic Formal Ontology (BFO 2.0) (See Sect. BFO as a top-level ontology)(Arp et al. 2016; International Organization for Standardization (ISO) 2020a, 2020b). It aims at giving a standard vocabulary framework for maintenance management inspired by Reliability Centered Maintenance (RCM) strategies. The strong point of ROMAIN is the alignment with BFO, which is a top-level domain-neutral ontology. It allows the alignment of application ontologies with upper-level domain and reference ontologies. ROMAIN offers a robust set of classes for CMMS data, for example, the classes to define maintenance work orders records that describe maintenance actions. Many of these classes are reused in OMSSA since ROMAIN and OMSSA are BFO compliant. However, ROMAIN has a limited set of concepts related to maintenance strategies. ROMAIN aligns to a classical RCM classification for maintenance strategies, and the concepts involved in the selection of such strategies are not presented.

Another related work is presented in (Lupp et al. 2020), in which template libraries are suggested to build an ontology for the evaluation of maintenance strategies. This research is oriented on a novel methodology to build and maintain modular ontologies using maintenance strategies assessment as the use case. The terminology applied to describe the use case does not cover the current trends in maintenance strategies.

As part of a multidisciplinary effort to construct open access ontologies that can be used for industrial purposes, the Industrial Ontology Foundry (IOF) has been created in 2016 (IOF 2020). Within the IOF, there is a specific group working for maintenance management. At the moment of the creation of OMSSA, no stable versions of the ontologies from IOF were available. However, IOF ontologies are BFO compliant and use some mid-level reference ontologies that have been considered for OMSSA. This will allow a future alignment of OMSSA to other IOF ontologies. The top-level and mid-level ontologies reused in OMSSA are presented in chapter 4.

The above-mentioned ontologies have at least one of the following limitations:

- Their scope does not cover maintenance strategies or present a limited scope where important concepts for maintenance strategy selection are not considered.
- Their maintenance strategy taxonomy does not include the current trends in predictive maintenance. This leaves behind concepts that are currently important when selecting maintenance strategies.
- Their development framework does not consider the alignment to a top-level domain-neutral ontology which complicates their reuse and/or interoperability with other related ontologies.

OMSSA aims at covering the gaps in the above-mentioned ontologies by fulfilling the following requirements:

- Capture the core notions related to the maintenance strategies.
- Consider the current trends in maintenance strategies.
- Be aligned with a top-level domain-neutral ontology.
- Use the established terminology by norms, standards, and domain experts of maintenance strategies.

Given this research opportunity, the scope for the OMSSA was defined as maintenance strategy selection and assessment. As part of the ontology scope delimitation, a set of competency questions are proposed. These questions can be used for the model evaluation as they serve as a fidelity measure of the model with real-life phenomena (March and Smith 1995). Maintenance experts would be able to recommend a specific strategy to address a specific failure mode of an `item` or asset. The following competency questions are proposed to delimit the OMSSA scope:

1. What are the failure modes associated with an `item` or asset?
2. What is the cause of a failure mode?
3. What is the criticality of a failure mode?
4. What is the recommended maintenance strategy for a failure mode?
5. What is the triggering event specification for a specific maintenance action?
6. What is the benefit of implementing a specific maintenance strategy?
7. What is the risk of implementing a specific maintenance strategy?

As terms sources for OMSSA, several maintenance standards ((European Committee for Standardization 2017; International Electrotechnical Commission (IEC) 2018; International Organization for Standardization (ISO) 1997, 2003, 2012a; MIMOSA 2001)), expert knowledge and existing related ontologies have been consulted. Special attention is given to the terms that are aligned to BFO, as it is selected as the top-level ontology for the current model. An identifier has been assigned to the most relevant sources of terms. These identifiers are summarized in Table 1 and later used in the following section for traceability purposes of OMSSA terms.

Table 1. The identifier for the terms sources

Reference	Identifier	Reference	Identifier
(European Committee for Standardization 2017)	1	(International Electrotechnical Commission (IEC) 2018)	10
(Montero Jimenez et al. 2020)	2	(International Organization for Standardization (ISO) 2012b)	11
(Nuñez and Borsato 2018)	3	(Saxena et al. 2010)	12
(Karray et al. 2012)	4	(Keller et al. 2001)	13
(MIMOSA 2001)	5	(Kacprzynski et al. 2002)	14
(Karray et al. 2019)	6	(Rudnicki 2020a)	15
(Arp et al. 2016)	7	(INCOSE 2015)	16
(International Organization for Standardization (ISO) 1997)	8	(Rudnicki 2020b)	17
(International Organization for Standardization (ISO) 2003)	9	(Protter 2005)	18

Ontology for Maintenance Strategy selection and Assessment (OMSSA)

Maintenance management trends are now more oriented towards the analysis of data to establish the right moment to trigger maintenance actions based on the `condition` of the maintainable system. The `condition` is a `quality` that is normally used to describe the capabilities of an `item` to fulfill its `function`. `Condition` can be described by four different values: nominal, degraded, critical, and failed (Ramasso and Gouriveau 2010). For each `item`, there must be a specification to be able to assess its current `condition`:

- `Nominal_condition` refers to the range of parameters in which the `item` is intended to operate.
- `Degraded_condition` refers to an `item` that can perform its intended function but is not within the nominal operation parameters.
- `Critical_condition` is a sub-class of `degraded_condition` in which the `item` has overshot the safety degradation thresholds, and a `failure` is imminent.
- `Failed_condition` refers to the complete impossibility of the `item` to perform its intended `function` after a `failure`.

Fig. 4 shows a generic example of the evolution of the health/degradation of an `item`. The horizontal axis represents the life cycle of the `item`, and the vertical axis is an index that represents its health or `degradation`, for example, the reliability. The time in which the `item` works within the limits of `nominal_condition` is described by a process that ends when a fault appears. The `fault` will affect the `item` operation, accelerating its `degradation`. Within the accelerated degradation process, there is a safety degradation threshold that defines the starting point of the `critical_condition`. The safety degradation threshold is imposed by a safety operating margin to prevent the actual failure. Table 2 summarizes the definitions for all terms in Fig. 4.

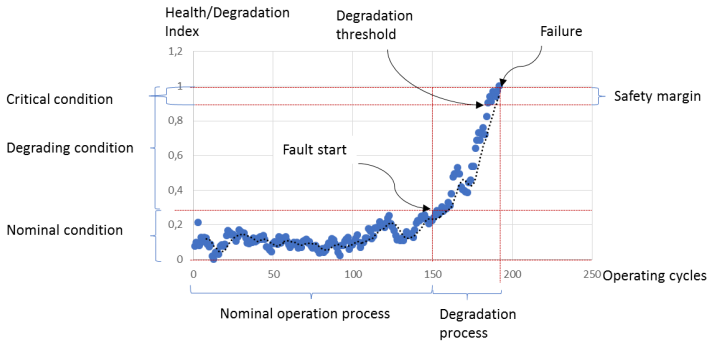


Fig. 4. Condition evolution over the item life cycle

Table 2. Definitions for important classes to describe the condition of an item

Class	OMSSA Formal definition	Based on (see Table 1)
Condition	A BFO: quality sub-class that an OMSSA: item bears. It qualifies the operation state of the item. It states the operational parameters of the item described by OMSSA: condition data and prescribed by OMSSA: condition specification.	1, 6
Condition data	A CCO: Descriptive Information Content Entity that consists of a set of propositions that describes OMSSA: Condition	3, 4, 8
Degradation	A CCO: Decrease of function. It represents a detrimental change in the operating condition of an item, normally as a result of a fault.	1, 6, 8, 11
Degraded condition	An OMSSA: Condition sub-class that qualifies an OMSSA: item operating out of its optimal parameters	1, 8, 11
Failed condition	An OMSSA: Condition sub-class that qualifies the impossibility of an OMSSA: Item to perform its intended function.	1, 8, 11
Fault	A BFO: quality sub-class. It qualifies a abnormal behavior or defect the OMSSA: item bears. It is described by OMSSA: symptoms	1, 3, 4, 11
Nominal condition	An OMSSA: Condition sub-class that qualifies the optimal operation of an OMSSA: Item	1, 8, 11
Nominal operation process	A CCO: stasis subclass that defines the time in which an item remains in nominal condition. This class is equivalent to CCO: Nominal stasis	1, 11
Item	A CCO: Object subclass equivalent to CCO: Artifact. An Object that was designed by some Agent to realize a certain function.	1, 4, 6, 11

`Condition` is described by the `condition_data`, which is at the same time the input for other processes to determine the triggering events for maintenance actions (see Fig. 5). These processes are carried out by a `predictive_maintenance_system` and can be used to detect incipient faults (`fault_detection`), assess degradation until a specific threshold (`degradation_threshold_overshoot`), and forecast the remaining useful life of the item or one of its components (`failure_forecasting`). Nevertheless, there are two other triggering events that do not use advanced predictive maintenance techniques but cannot be ignored: the `failure` of the `item` and a `fixed_time_recommendation` prescribed by a `preventive_maintenance_plan`.

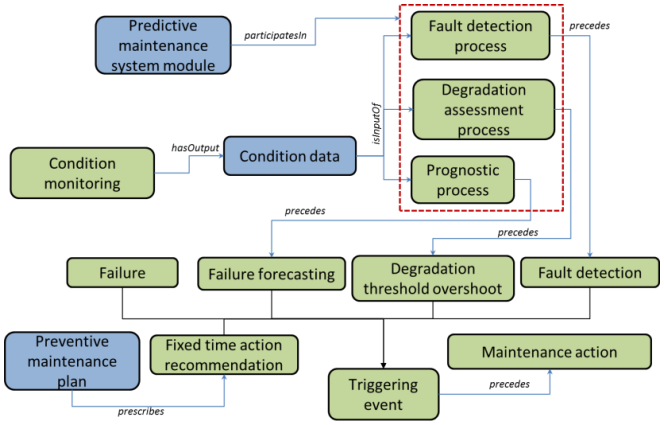


Fig. 5. Relations among the different triggering events

Table 3. Definitions of the terms in Fig. 5 and Fig. 6.

Class	OMSSA Formal definition	Based on (see Table 1)
Condition Monitoring	A BFO: a process that has as output OMSSA: Condition data	1, 10, 11
Corrective Maintenance	An OMSSA: maintenance strategy whose triggering event is the failure occurrence	1, 2, 10, 11
Degradation assessment process	A BFO: process performed on an OMSSA: item by an OMSSA: predictive maintenance module to assess degradation until this degradation overshoots a specific threshold.	1, 2, 7, 10, 11
Degradation threshold overshoot	An OMSSA: triggering event subclass related to predictive maintenance strategy. It is prescribed by a degradation assessment module of a predictive maintenance system.	1, 2, 10, 11
Failure	An OMSSA: triggering event subclass related to corrective maintenance strategy. The impossibility of an item to perform its intended function triggers a maintenance action	1, 2, 6, 10, 11
Failure forecast	An OMSSA: triggering event subclass related to predictive maintenance strategy. It is prescribed by a failure forecast module of a predictive maintenance system.	1, 2, 10, 11
Fault detection	An OMSSA: triggering event subclass related to predictive maintenance strategy. It is prescribed by a fault detection module of a predictive maintenance system.	1, 2, 10, 11
Fault detection process	A BFO: process performed on an OMSSA: item by an OMSSA: predictive maintenance module to detect incipient faults	1, 2, 7, 10, 11
Fixed time recommendation	An OMSSA: triggering event subclass related to preventive maintenance strategy. It is prescribed by a preventive maintenance plan. A recommendation based on fixed operation intervals or from fixed basic inspections triggers a maintenance action.	1, 2, 10, 11
Maintenance action	A BFO: process performed on an OMSSA: item to restore or keep it in its operational state.	1, 2, 7, 10, 11
Maintenance strategy	A CCO: directive information content entity resulting from maintenance strategy development process. It prescribes the expected triggering event for each maintenance action on an item	1, 2, 7, 10, 11
Predictive Maintenance	An OMSSA: maintenance strategy whose triggering events are based on the condition of the OMSSA: Item but before a failure. There are three possible triggering events	1, 2, 5, 10, 11
Predictive maintenance system	A CCO: information processing artifact used to analyze the item condition data and perform fault detection, health/degradation assessments, and/or life expectancy estimations.	1, 2, 10, 11
Preventive Maintenance	An OMSSA: maintenance strategy whose triggering event is a predefined fixed time event, after an operation time interval or at a given preventive maintenance date.	1, 2, 10, 11
Preventive Maintenance plan	A CCO: plan subclass. It includes all preventive recommendations to trigger maintenance actions	1, 2, 10, 11
Prognostic process	A BFO: process performed on an OMSSA: item by an OMSSA: predictive maintenance module to estimate the time to a future failure of an item or one of its components.	1, 2, 7, 10, 11
Triggering event	A BFO: process boundary that is the starting point for a maintenance action	1, 2, 7, 10, 11

Having a clearer idea of the different triggering events presented in Fig. 5 and the maintenance_strategy taxonomy introduced in "Sect. Maintenance strategies", the next step allows setting the relations between the triggering events and the maintenance strategies. Fig. 6 shows these relations in which a corrective_maintenance strategy is related to a failure, a preventive_maintenance strategy is related to a fixed_time_recommendation, and a predictive_maintenance strategy is related to fault_detection, degradation_threshold_overshoot, or a failure_forecast. These relations will allow classifying the different maintenance actions in the different maintenance strategies. Table 3 summarizes the concepts presented in Figs. 5 and 6.

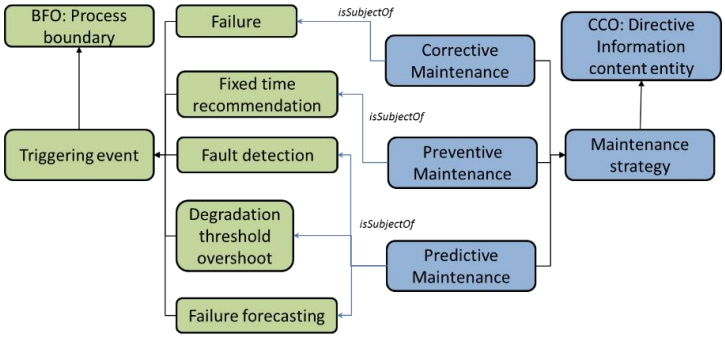


Fig. 6. Relations between triggering events and maintenance strategies

The maintenance_strategy_development_process is composed of different analyses and technical developments (see Fig. 7). There should be at least one maintenance_strategy for each failure_mode of an item. Failure modes and their impacts must be well known so that the most suitable strategy could be suggested. The Failure Mode, Effects and Criticality Analysis (FMECA) (International Electrotechnical Commission (IEC) 2018) is often used to assess the impact of failure modes of an item, their potential causes, risks (effects), and criticality (Zhou et al. 2015).

The FMECA can be used as the starting point for the maintenance strategy development. Several proactive strategies can be suitable to address a failure_mode, starting with fixed time recommendation and coming to a precise failure prognostic. The selection of the most suitable proactive strategy depends on the technology readiness for each application, on the operational context, the implementation cost, and the potential benefits and risks of implementing predictive maintenance. FMECA has a report (represented as OMSSA class) composed of the CCO: Information Content Entities explaining the different aspects that have been considered to select a specific maintenance strategy. Among these information entities, one can find the functional_failure of an item, the failure_mode, the cause_of_failure, the

failure_effect, the criticality, and failure_likeliness. The instances for all these entities are compulsory to be known when selecting a maintenance strategy using a FMECA.

A Cost-Benefit-Risk_Analysis is a complementary method that is used to assess the suitability of a strategy when several options are possible; especially when it includes the implementation of specialized systems for health monitoring, fault detection and identification, health assessment, and failure forecast that represent an important initial investment but are also attractive in terms of potential benefits. As (Saxena et al. 2010) explains, a Cost-Benefit-Risk_Analysis is vital to justify the implementation of more advanced maintenance strategies. As for FMECA, the Cost-Benefit-Risk_Analysis has a corresponding report class in OMSSA. This report contains important information content entities that are considered to select a new maintenance strategy, especially when justifying the transition of existing preventive strategies to predictive ones or when upgrading the type of predictive strategy. Among these information content entities, one can find the implementation_benefit of the new strategy, its related implementation_risk, and implementation_cost. It is important to mention that for the Cost-Benefit-Risk_Analysis some data should be gathered from other sources such as the FMECA and the CMMS. One of these data is the maintenance_cost that has been accumulated by an item in a specific interval of time; it is likely to justify the implementation of new maintenance strategies by the reduction of these costs.

The most advanced strategies that aim at performing accurate diagnosis and prognosis on the item rely on specialized tools, methods, and systems for decision support. These tools or systems demand engineering work for their design and implementation. That is why a technical development process is also included as part of the maintenance strategy development. oThe engineering work performed in this process leads to a preventive_maintenance_plan and/or a predictive_maintenance_system, depending on the item to be maintained and its operational context.

As it can be seen, the maintenance strategy development has many interdependencies among the different engineering activities. It is important to point out that there are other factors that can affect the development of a maintenance strategy and are not part of these engineering processes, such as production policies or authorities' regulations. These can be seen as "external" dependencies as the maintenance engineer may not directly intervene on them. These external dependencies are included in OMSSA in the class maintenance_strategy_dependency.

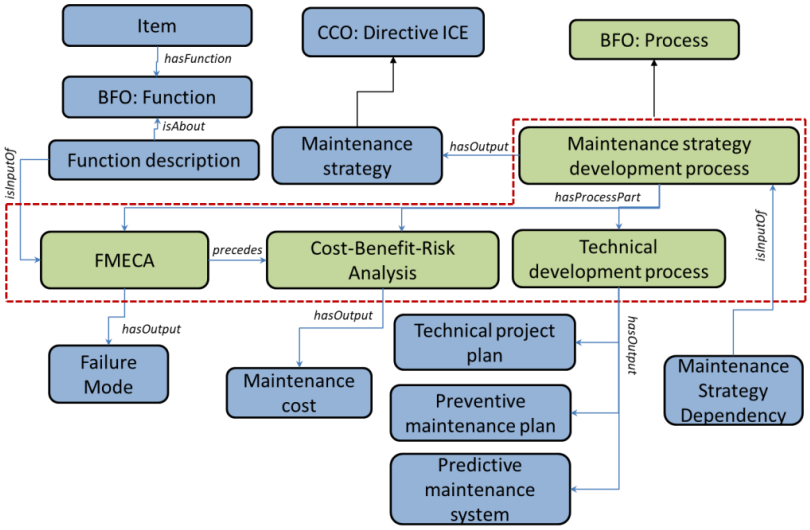


Fig. 7. Relations for the maintenance strategy development process

Once the `maintenance_strategy` is selected to address a `failure_mode` of an `item`, a `maintenance_action` is specified in which the expected `triggering_event` will be defined. This specification can be later compared to the actual `triggering_event` of a `maintenance_action` that is normally recorded in a work order report. This comparison between the specification and the report helps to validate and assess the `maintenance_strategy` over time. A knowledge base created with OMSSA can store all the maintenance strategies assigned to the components of an item, some queries on the maintenance records can automate the assessment on the proposed strategies allowing to re-adjust or change the strategy when it is no longer suitable. Fig.. 8 shows the classes and the relations that can be used to assess the `maintenance_strategy` for a given `failure_mode`. For OMSSA purposes, the concepts related to the work orders reports have been imported from ROMAIN (see "Sect. Ontology engineering") as it already offers a complete set of information content entities that can be used to describe a `maintenance_action`. OMSSA reuses these ROMAIN classes and aligns them to the newer version of CCO (version 1.3). An important class incorporated in OMSSA as part of the maintenance work order report is the `Triggering_event_report`. Table 4 summarizes the definitions of classes related to FMECA, cost-benefit-risk analysis, and maintenance reports used for maintenance strategies selection and assessment.

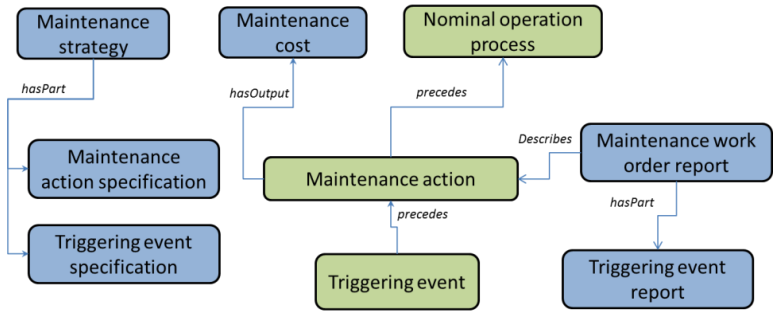


Fig.. 8. Relations between the classes to assess maintenance strategies

Table 4. Definitions of classes related to FMECA, cost-benefit-risk analysis, and maintenance reports used for maintenance strategies selection and assessment.

Class	OMSSA Formal definition	Based on (see Table 1)
Cause of failure	A CCO: Descriptive Information Content Entity that describes the cause of a failure mode. It is about a CCO: Cause that can lead to an OMSSA: Failure	1, 10, 11
Cost-Benefit-Risk analysis	A BFO: process, which is part of the strategy development process, it aims at evaluating the implementation of a more advanced maintenance strategy for an item compared to the current one. It considers the failure risk, the cost, and the benefits of its implementation.	12, 13, 14
Cost-Benefit-Risk analysis Report	A CCO: report subclass. It carries the information content entities that describe the result of an OMSSA: Cost_benefit_risk_analysis.	12, 13, 14
Criticality	A CCO: ordinal measurement information content entity that places the failure risk into some rank order that is used in the FMECA	1, 10, 11
Failure effect	A CCO: Descriptive Information Content Entity that describes the impacts of a failure in terms of safety, environment, and operation. It is normally measured by rank. It is about a CCO: Effect that results from an OMSSA: Failure	1, 10, 11
Failure likeliness	A CCO: ordinal measurement information content entity that states the likelihood of a failure occurrence, based statistical information, reliability analysis, or design information	1, 10, 11
Failure mode	It is a CCO: Descriptive information content entity that describes a failure of an item and the corresponding fault that can cause the failure. It is an output of the Failure Modes, Effects, and Criticality Analysis (FMECA).	1, 10, 11
Failure Modes, Effects, and Criticality Analysis	A BFO: process aiming at defining the possible failures of an item based on the non-fulfillment of its intended function. It defines the effects (consequences) and criticality of the failure.	1, 10, 11
Failure Modes, Effects, and Criticality Analysis Report	A CCO: report subclass. It carries the information content entities that describe the result of an OMSSA: Cost_benefit_risk_analysis.	12, 13, 14
Function	A function is a disposition that exists in virtue of the bearer's physical make-up, and this physical make-up is something the bearer possesses because it came into being, either through evolution (in the case of natural biological entities) or through intentional design (in the case of artifacts), in order to realize processes of a certain sort. (Axiom label in BFO2 Reference: [064-001])	6, 7, 9
Function description	A CCO: Descriptive Information Entity subclass that describes a BFO: Function	7, 9
Implementation benefit	A CCO: Descriptive information content entity sub-class that describes a benefit of implementing a preventive maintenance plan or a predictive maintenance system as part of the maintenance strategies.	12, 13, 14

Implementation cost	A CCO: Descriptive information content entity subclass that quantifies the cost of implementing a more advanced maintenance strategy by means of a preventive maintenance plan and/or a predictive maintenance system.	12, 13, 14
Implementation risk	A CCO: Descriptive information content entity. It describes the risk of implementing a new predictive maintenance strategy, plan, or system.	12, 13, 14
Maintenance work order record specification	A CCO: Descriptive information content entity that describes a maintenance action	1, 6, 11, 15
Maintenance action specification	A CCO: Directive information content entity that states the specifications of a maintenance action	1, 11, 15
Maintenance cost	A CCO: Descriptive information content entity subclass that quantifies the cost of any maintenance action performed on an OMSSA: item	1, 10, 11
Maintenance strategy development process	A BFO: process subclass. It includes all activities and sub-processes to select the right maintenance strategy to apply for the different failure modes of an item.	1, 10, 11
Technical development process	A BFO: process sub-class. It refers to all technical activities carried out to implement a predictive maintenance system or a preventive maintenance plan.	15, 16, 17
Technical project plan	A CCO: plan subclass. It describes the plan of the implementation of a technical project	15, 16, 17
Triggering event record	A CCO: Descriptive information content entity. It describes the actual triggering event for a specific maintenance action in a maintenance report.	6, 15, 16, 17
Triggering event specification	A CCO: Directive information content entity that states the expected triggering event for a maintenance action	6, 15, 17, 18

Evaluation and discussion

The evaluation of the model is composed of two main steps: verification and validation. The definitions of verification and validation are derived from (INCOSE 2015). The verification aims at providing evidence of the consistency of the model, meaning that it fulfills the technical requirements for its development. The validation provides evidence that the model when in use, fulfills the objectives for which it was developed.

As OMSSA was developed using the Protégé editor, the verification can be carried out using one of the reasoners embedded in the editor. Different reasoners can be installed and used to verify the model's consistency. These reasoners can be configured to verify different aspects of class inferences, object property inferences, data property inferences, and individual inferences such as hierarchy, domain, range, and conflicting disjoint assertions (Nuñez and Borsato 2018). It is important to make sure that when establishing relations among classes and instances, there are no contradictory or duplicated statements. The HermiT OWL reasoner (Glimm et al. 2014), embedded as a Protégé plug-in, was used to verify OMSSA consistency. During the verification process, several inconsistency alarms were detected, forcing the restructuration of several relations among OMSSA classes. The model presented in "Sect. BFO as a top-level ontology" corresponds to a stable version with no inconsistency warnings from the reasoners.

Once the verification of the model is fulfilled, the validation process to assess the fidelity of the model with its initial application requirements takes place. For OMSSA, the scope is limited to maintenance strategy selection and assessment. Ontologies are meant to be vocabulary frameworks for practical applications. It is important to understand how it can be integrated into practical applications (Ferrer et al. 2018). Fig. 9 shows the principle of the implementation of OMSSA in practical applications. OMSSA can be retrieved directly from its online repository and is

aimed to be instantiated with information from different sources such as FMECA, cost-benefit-risk analysis, or CMMS engineering information, among other sources of data. Once the ontology is populated, it becomes a knowledge base of a specific domain that can be used by smart agents such as Decision Support Systems (DSS). There exist ontology libraries that can be used to integrate the different components of the system. For example, the OWL API (Horridge and Bechhofer 2011) provides a Java-based platform to create the interfaces between the ontology and the data sources and between the knowledge base (instantiated ontology) and a smart agent such as the DSS. To do so, different types of queries can be implemented, such as Description Logic (DL) queries, SPARQL queries, and SQWRL queries (Muñoz-Hernández et al. 2021).

To validate its functionality, OMSSA was populated with the information of a generic FMECA based on a real air compressor. This 10HP air compressor supplies 10 l/s of air at 8 bar to a pneumatic cutting machine. The FMECA was developed by maintenance experts who determined the failure modes, their causes, effects, and criticality for all the air compressor components. These variables led the experts to suggest suitable maintenance strategies. As OMSSA was instantiated with the air compressor case study, it becomes a knowledge base on maintenance strategies for an air compressor (Noy and McGuinness 2001).

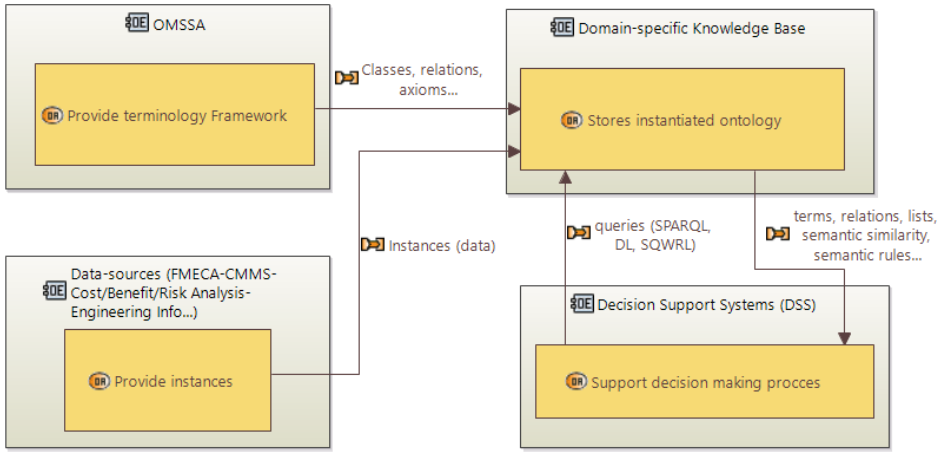


Fig. 9. Implementation of OMSSA for practical applications

To measure OMSSA fidelity to the real world, the ontology-based knowledge base (meaning the instantiated ontology) should be able to answer questions like an expert would do (Grüninger et al. 1995b; Heravi et al. 2014; Raad and Cruz 2015; Steiner and Albert 2017). To do so, the competency questions used to delimit the ontology scope can be used as a guideline. Validation of the knowledge modeling is performed by answering these competency questions with SPARQL queries on the instantiated ontology. To do so, the SPARQL plug-in for Protégé was used (Redmond 2012). As part of the SPARQL queries writing, a set of prefixes for OMSSA and all reused ontologies for its development (BFO, CCO, and RO). It simplifies the writing of the SPARQL queries for OMSSA validation. Table 5 summarizes the proposed prefixes for OMSSA validation using the SPARQL queries.

The air compressor is composed of four main items that are instantiated as: compressor_unit_1, electric_motor_1, drying_unit_1, and transmission_1. To illustrate the applicability of OMSSA, the first competency question (What are the failure modes associated with an item or asset?) will be answered by querying the instantiated OMSSA to obtain all failure modes related to the electric_motor_1. The result of the SPARQL query showed all registered failure modes for the electric motor (see Table 6 and Fig. 10). The other competency questions (questions 2 to 7 in the Ontologies in Maintenance section) are related to each failure mode. A query was performed to answer these competency questions for the failure mode Crank_damage (see Table 7 and Fig. 11). This query allowed to show the cause_of_failure, the criticality, the maintenance_strategy, and the expected triggering_event to perform the maintenance_action of changing the oil. Answering simple queries such as those in tables 6 and 7 will help validate the consistency of the modeled knowledge in the ontology. It is important to point out that OMSSA is not only limited to FMECA classes but also modeled the knowledge for the maintenance strategy assessment and upgrade to more advanced strategies. To do so, OMSSA contains classes that can be instantiated from CMMS and cost-benefit-risk analysis reports and other engineering information sources.

The four requirements proposed for OMSSA development have been satisfied. The first requirement established in Maintenance Ontology Section for OMSSA was to "capture the core notions related to the maintenance strategies". By performing these query-based tests, the consistency of the vocabulary related to maintenance strategies in OMSSA was validated. OMSSA includes the vocabulary related to advanced strategies triggered by diagnostics and prognostics; this validates the proposed requirement for OMSSA related to include the current trends in maintenance strategies. As OMSSA uses BFO as top-level ontology, CCO as mid-level ontologies, and its scope is limited by competency questions inspired in the domain knowledge, the proposed requirement related to OMSSA alignment is also met. As the terms for OMSSA were extracted from standards and norms and domain experts for maintenance strategies, the requirement with regards to the vocabulary sources for OMSSA is also satisfied.

Table 5. Prefixes to perform SPARQL queries on OMSSA

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
PREFIX OMSSA: <http://www.semanticweb.org/j.montero-jimenez/ontologies/2020/7/OMSSA#>
PREFIX bfo: <http://purl.obolibrary.org/obo/>
PREFIX ro:  <http://www.obofoundry.org/ro/ro.owl#>
PREFIX exro: http://www.ontologylibrary.mil/CommonCore/Upper/ExtendedRelationOntology#
PREFIX ieo: <http://www.ontologylibrary.mil/CommonCore/Mid/InformationEntityOntology#>
```

Table 6. SPARQL query to answer the first competency question on Compressor Unit 1

```
SELECT ?item ?FailureMode
        WHERE{    ?item rdfs:label “Compressor Unit 1”.
                  ?item OMSSA:has_failure_mode ?FailureMode}
```

item	FailureMode
Compressor Unit 1	Crank damage
Compressor Unit 1	Paper seal leak
Compressor Unit 1	Outlet valve damage
Compressor Unit 1	Frame damage
Compressor Unit 1	Cylinder rings damage
Compressor Unit 1	Piston damage

Fig. 10. Answer to the SPARQL query on Table 6

Table 7. SPARQL Query to answer the competency questions for the failure mode "Crank damage"

```
SELECT ?item ?FailureMode ?Cause ?Criticality ?MaintenanceStrategy ?TriggeringSpecification
        WHERE{ ?item exro:has_function ?Function.
                ?item OMSSA:has_failure_mode ?FailureMode.
                ?FailureMode rdfs:label “Crank damage”.
                ?FailureMode OMSSA:is_subject_of_failure_cause ?Cause.
                ?FailureMode OMSSA:is_subject_of_criticality ?Criticality.
                ?MaintenanceStrategy ieo:is_About ?FailureMode.
                ?MaintenanceStrategy OMSSA:has_maintenance_action ?MaintenanceaAction.
                ?MaintenanceAction OMSSA:has_triggering_specification ?Triggering
Specification}
```

item	FailureMode	Cause	Criticality	MaintenanceStrategy	TriggeringSpecification
Compressor Unit 1	Crank damage	Insufficient lubrication	High_criticality	Predictive oil change	Oil degradation detection

Fig. 11. Answer to SPARQL query on Table 7

OMSSA covers important terms related to the maintenance strategy selection and assessment, allowing the implementation of intelligent decision support systems that can perform automated tasks in the maintenance strategy domain. For example, ontologies enable the processing and sharing of knowledge that can be used at different tasks of Case-Based Reasoning (CBR) systems, such as representing the input problem, enhancing similarity assessments, case representation, case abstraction, and case adaptation (Prentzas and Hatzilygeroudis 2008). Some examples of ontological-based similarity retrieval applications can be found in (Fernández et al. 2011; Qin et al. 2016). The knowledge represented in OMSSA can be used to develop a smart system that can deal with the management of maintenance strategies, selecting the strategy to address different failure modes of an item, and assessing over time the efficiency and accuracy of the selected strategy. The creation of such reasoning among the OMSSA classes is out of the scope of the current article. As mentioned, OMSSA is part of a bigger research project in knowledge reuse for maintenance decision support systems. An extended version of OMSSA has already been used to support a DSS that uses CBR principles to select the suitable model or algorithm to implement predictive maintenance strategies given a specific use case (Montero Jiménez et al. 2021); OMSSA plays an important role in the semantic similarity computation which is needed for the CBR retrieve engine. This also validates the applicability of OMSSA for practical

implementations, not only supporting the maintenance strategy selection but also helping in the implementation of advanced strategies oriented to perform accurate diagnostics and prognostics of productive assets.

The structured methodology followed to develop OMSSA allows its alignment to other BFO-compliant ontologies, such as for example, those that are currently under development by the Industry Ontology Foundry. It is important to point out that ontologies evolve over time as knowledge in the different domains also evolves. The structured method used to develop OMSSA helps in the traceability of classes and relations, facilitating its future maintenance and update.

Conclusion and future work perspectives

The present study has considered the most important terms and the semantic relations among them in the maintenance strategy domain. All these terms and relations were used to build an ontological model called OMSSA (Ontology model for Maintenance Strategy Selection and Assessment) using the methodology “Ontology Development 101”. OMSSA is aligned to a top-level ontology (BFO) through some mid-level domain neutral ontologies (CCO). This alignment allows OMSSA to comply with standardized ontology development practices, which facilitates its future reuse. The ontology model was verified using semantic reasoners to prove its consistency among the different terms.

A concept of the implementation of OMSSA for practical applications was proposed. OMSSA aims to be instantiated with data from FMECA, cost-benefit-risk analysis reports, CMMS, among other types of engineering data sources, to provide enough information that can be used in maintenance strategy selection and assessment. OMSSA validation was performed using information from FMECA of an air compressor to instantiate some of the OMSSA classes. Using the competency questions as a guideline to perform some SPARQL queries on the knowledge base allowed to retrieve information and answer the questions as experts in the field would do. The classes and relations modeled in OMSSA provide an overview of the knowledge around maintenance strategies that was not fully covered before. Instantiated OMSSA can be used as a terminology framework for the creation of smart decision support systems in the domain of maintenance strategies. For example, these smart DSS could emulate the experts' reasoning by implementing rules or cases extracted from the ontology to assign maintenance strategies, assess the assigned ones, and justify the implementation of more advanced strategies based on accurate diagnosis and prognosis.

As part of the perspective of future work, a comparison can be performed between the smart agents developed with OMSSA for maintenance strategy selection and established methodologies such as Multi-Criteria Decision Making Methods (MCDM). OMSSA can also be extended and used as a reference ontology for a more specific ontology of predictive maintenance systems design. This terminology framework can serve as a protocol for a smart decision support system for the design of predictive maintenance systems.

This article has been published in the
Journal of Intelligent Manufacturing

Acknowledgments

The authors want to acknowledge the contribution of M.Eng. Carlos Piedra Santamaría from Tecnológico de Costa Rica for providing the FMECA analysis applied on the air compressor that was used as a case study for OMSSA validation.

Conflict of interest

Juan José Montero Jiménez, Bernard Grabot, Rob Vingerhoeds and Sebastien Schwartz declare no conflict of interest for the current study.

OMSSA repository

OMSSA can be accessed by the following link: https://github.com/jjmj128/OMSSA

References

Antoniou, G., Groth, P., Harmelen, F. van, & Hoekstra, R. (2012). *A Semantic Web Primer*. *The MIT Press* (Third edit.). MIT Press.

Arp, R., Smith, B., & Spear, A. D. (2016). *Building Ontologies with Basic Formal Ontology*. *Building Ontologies with Basic Formal Ontology*. https://doi.org/10.7551/mitpress/9780262527811.001.0001

Cao, Q., Zanni-Merk, C., & Reich, C. (2019). Towards a core ontology for condition monitoring. In *Procedia Manufacturing*. https://doi.org/10.1016/j.promfg.2018.12.029

Castaño, F., Haber, R. E., Mohammed, W. M., Nejman, M., Villalonga, A., & Martinez Lastra, J. L. (2020). Quality monitoring of complex manufacturing systems on the basis of model driven approach. *Smart Structures and Systems*, *26*(4), 495–506. https://doi.org/10.12989/sss.2020.26.4.495

Emovon, I., Norman, R. A., & Murphy, A. J. (2018). Hybrid MCDM based methodology for selecting the optimum maintenance strategy for ship machinery systems. *Journal of Intelligent Manufacturing*, *29*, 519–531. https://doi.org/10.1007/s10845-015-1133-6

European Committee for Standardization. (2017). *CEN EN 13306: Maintenance-Maintenance terminology*. *European Committee for Standardization. ICS 01.040.03; 03.080.10*.

Fernández, M., Cantador, I., López, V., Vallet, D., Castells, P., & Motta, E. (2011). Semantically enhanced Information Retrieval: An ontology-based approach. *Journal of Web Semantics*, *9*(1), 434–452. https://doi.org/10.1016/j.websem.2010.11.003

Fernandez, M., Gómez-Pérez, A., & Juristo, N. (1997). Methontology: from ontological art towards ontological engineering. In *Proceedings of the AAAI97 Spring Symposium Series on Ontological Engineering*.

Ferrer, B. R., Mohammed, W. M., Martinez Lastra, J. L., Villalonga, A., Beruvides, G., Castano, F., & Haber, R. E. (2018). Towards the Adoption of Cyber-Physical Systems of Systems Paradigm in Smart Manufacturing Environments. In *Proceedings - IEEE 16th International Conference on Industrial Informatics, INDIN 2018*. https://doi.org/10.1109/INDIN.2018.8472061

Foundry, T. O. (2020). Relation Ontology (RO). http://www.obofoundry.org/ontology/ro.html. Accessed 22 July 2020

Glimm, B., Horrocks, I., Motik, B., Stoilos, G., & Wang, Z. (2014). HermiT: An OWL 2 Reasoner. *Journal of Automated Reasoning*. https://doi.org/10.1007/s10817-014-9305-1

Gruber, T. R. (1993). A translation approach to portable ontology specifications. *Knowledge Acquisition*, *5*(2), 199–220. https://doi.org/10.1006/knac.1993.1008

Grüninger, M., Fox, M. S., & Gruninger, M. (1995a). Methodology for the Design and Evaluation of Ontologies. In *International Joint Conference on Artificial Inteligence (IJCAI95), Workshop on Basic Ontological Issues in Knowledge Sharing*. https://doi.org/citeulike-article-id:1273832

Heravi, B. R., Lycett, M., & de Cesare, S. (2014). Ontology-based standards development: Application of OntoStanD to ebXML business process specification schema. *International Journal of Accounting Information Systems*, *15*(3), 275–297. https://doi.org/10.1016/j.accinf.2014.01.005

Hodkiewicz, M., Lukens, S., Brundage, M. P., & Sexton, T. (2021). Rethinking Maintenance Terminology for an Industry 4.0 Future. *International Journal of Prognostics and Health Management*, *2021*(1), 14.

https://www.phmsociety.org/node/2794. Accessed 1 March 2021

Horridge, M., & Bechhofer, S. (2011). The OWL API: A Java API for OWL ontologies. *Semantic Web*, *2*(1), 11–21. https://doi.org/10.3233/SW-2011-0025

INCOSE. (2015). *Systems Engineering Handbook. A guide for system life cycle processes and activities. Fourth Edition.* Wiley.

International Electrotechnical Commission (IEC). (2018). *IEC60812, Analysis techniques for system reliability- Procedure for failure mode and effects analysis (FMECA)*.

International Organization for Standardization (ISO). (1997). Information technology — Vocabulary — Part 14: Reliability, maintainability and availability.

International Organization for Standardization (ISO). (2003). *ISO 13374-1:2003 Condition monitoring and diagnostics of machines — Data processing, communication and presentation — Part 1: General guidelines*.

International Organization for Standardization (ISO). (2012a). *ISO 13372 - Condition monitoring and diagnostics of machines − Vocabulary*.

International Organization for Standardization (ISO). (2012b). *ISO 13379-1:2012 - Condition monitoring and diagnostics of machines − Data interpretation and diagnostics techniques − Part 1: General guidelines*.

International Organization for Standardization (ISO). (2020a). *ISO/IEC 21838-1 Information technology - Top-level Ontologies (TLO) - Part 1: Requirements*.

International Organization for Standardization (ISO). (2020b). *ISO/IEC 21838-2 - Information technology - Top-level ontologies (TLO) - Part 2: Basic Formal Ontology (BFO)*.

IOF. (2020). Industrial Ontology Foundry. *https://www.industrialontologies.org/?page_id=164*. https://www.industrialontologies.org/?page_id=164. Accessed 6 October 2020

Kacprzynski, G. J., Roemer, M. J., & Hess, A. J. (2002). Health management system design: Development, simulation and cost/benefit optimization. In *IEEE Aerospace Conference Proceedings*. https://doi.org/10.1109/AERO.2002.1036148

Karray, M. H., Ameri, F., Hodkiewicz, M., & Louge, T. (2019). ROMAIN: Towards a BFO compliant reference ontology for industrial maintenance. *Applied Ontology*, *14*(2), 155–177. https://doi.org/10.3233/AO-190208

Karray, M. H., Chebel-Morello, B., & Zerhouni, N. (2012). A formal ontology for industrial maintenance. *Applied Ontology*. https://doi.org/10.3233/AO-2012-0112

Keet, M. (2018). *An Introduction to Ontology Engineering*. Texts in computing Vol 20. ISBN 978-1-84890-295-4.

Keller, K., Simon, K., Stevens, E., Jensen, C., Smith, R., & Hooks, D. (2001). A process and tool for determining the cost/benefit of prognostic applications. In *AUTOTESTCON (Proceedings)*. https://doi.org/10.1109/autest.2001.949432

Kothamasu, R., Huang, S. H., Verduin, W. H., Kothamasu, R., Huang, S. H., & Verduin, W. H. (2006). System health monitoring and prognostics -a review of current paradigms and practices. *International Journal of Advanced Manufacturing Technology*, *28*(9), 1012–1024. https://doi.org/10.1007/978-1-84882-472-0_14

Lu, Y., Wang, H., & Xu, X. (2019). ManuService ontology: a product data model for service-oriented business interactions in a cloud manufacturing environment. *Journal of Intelligent Manufacturing*, *30*, 317–334. https://doi.org/10.1007/s10845-016-1250-x

Lupp, D. P., Hodkiewicz, M., & Skjæveland, M. G. (2020). Template libraries for industrial asset maintenance: A methodology for scalable and maintainable ontologies. In *CEUR Workshop Proceedings* (Vol. 2757, pp. 49–64).

March, S. T., & Smith, G. F. (1995). Design and natural science research on information technology. *Decision Support Systems*, *15*, 251–266. https://doi.org/10.1016/0167-9236(94)00041-2

Matsokis, A., Karray, H. M., Chebel-Morello, B., & Kiritsis, D. (2010). An ontology-based model for providing Semantic Maintenance. In *1st IFAC Workshop on Advanced Maintenance Engineering, Services and Technology, Vol 43, Issue 3* (pp. 12–17). https://doi.org/10.3182/20100701-2-pt-4012.00004

MIMOSA. (2001). Open System Architecture for Condition-Based Maintenance (OSA-CBM). *Available on http://www.mimosa.org/mimosa-osa-cbm/*.

Montero Jimenez, J. J., Schwartz, S., Vingerhoeds, R., Grabot, B., & Salaün, M. (2020). Towards multi-model approaches to predictive maintenance: A systematic literature survey on diagnostics and prognostics. *Journal of Manufacturing Systems*, *56*, 539–557. https://doi.org/10.1016/j.jmsy.2020.07.008

Montero Jimenez, J. J., & Vingerhoeds, R. (2018). Enhancing operational fault diagnosis by assessing multiple operational modes. In *Proceedings - International Conference in Modelling, Optimization and Simulation MOSIM 2018, 27th-29th June* (pp. 237–244). Toulouse, France: MOSIM2018.

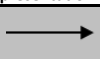
Montero Jiménez, J. J., Vingerhoeds, R., & Grabot, B. (2021). Enhancing predictive maintenance architecture process by using ontology-enabled Case-Based Reasoning. In *Proceedings - 7th IEEE International Symposium on System Engineering.* Virtual: IEEE.

Moubray, J. (1997). *RCM II: Reliability Centered Maintenance*. Industrial Press.

Muñoz-Hernández, H., Montero-Jiménez, J. J., & Vingerhoeds, R. (2021). Integrating ontologies and case-based reasoning for the development of knowledge-intensive systems. In *Proceedings - 35th European Simulation and*

Modelling Conference. Accepted for publication.
Musen, M. A. (1992). Dimensions of knowledge sharing and reuse. *Computers and Biomedical Research*, *25*(5), 435–467. https://doi.org/10.1016/0010-4809(92)90003-S
Noy, N. F., & McGuinness, D. L. (2001). *Ontology Development 101: A Guide to Creating Your First Ontology*. *Stanford Knowledge Systems Laboratory*. https://doi.org/10.1016/j.artmed.2004.01.014
Nuñez, D. L., & Borsato, M. (2017). An ontology-based model for prognostics and health management of machines. *Journal of Industrial Information Integration*, *6*, 33–46. https://doi.org/10.1016/j.jii.2017.02.006
Nuñez, D. L., & Borsato, M. (2018). OntoProg: An ontology-based model for implementing Prognostics Health Management in mechanical machines. *Advanced Engineering Informatics*, *38*, 746–759. https://doi.org/10.1016/j.aei.2018.10.006
Panov, P., Soldatova, L., & Džeroski, S. (2014). Ontology of core data mining entities. *Data Mining and Knowledge Discovery*, *28*, 1222–1265. https://doi.org/10.1007/s10618-014-0363-0
Prentzas, J., & Hatzilygeroudis, I. (2008). Combinations of case-based reasoning with other intelligent methods. In *CEUR Workshop Proceedings*. https://doi.org/10.3233/his-2009-0096
Protter, P. E. (2005). *Stochastic Integration and Differential Equations, Second Edition*. (B. Rozovski & M. Yor, Eds.). Springer.
Qin, F., Gao, S., Yang, X., Li, M., & Bai, J. (2016). An ontology-based semantic retrieval approach for heterogeneous 3D CAD models. *Advanced Engineering Informatics*, *30*, 751–768. https://doi.org/10.1016/j.aei.2016.10.001
Raad, J., & Cruz, C. (2015). A survey on ontology evaluation methods. In *IC3K 2015 - Proceedings of the 7th International Joint Conference on Knowledge Discovery, Knowledge Engineering and Knowledge Management*. https://doi.org/10.5220/0005591001790186
Ramasso, E., & Gouriveau, R. (2010). Prognostics in switching systems: Evidential Markovian classification of real-time neuro-fuzzy predictions. In *2010 Prognostics and System Health Management Conference, PHM '10*. https://doi.org/10.1109/PHM.2010.5413442
Redmond, T. (2012). SPARQL Query tab for Protégé. *Protégé wiki*. https://protegewiki.stanford.edu/wiki/SPARQL_Query. Accessed 22 October 2020
Rodrigues, F. H., & Abel, M. (2019). What to consider about events: A survey on the ontology of occurrents. *Applied Ontology*, *14*(4), 343–378. https://doi.org/10.3233/AO-190217
Rudnicki, R. (2020a). An Overview of the Common Core Ontologies 1.3. Buffalo, NY: National Institute of Standards and Technology (NIST). https://www.nist.gov/system/files/documents/2019/05/30/nist-ai-rfi-cubrc_inc_004.pdf
Rudnicki, R. (2020b). Common core ontologies. https://github.com/CommonCoreOntology/CommonCoreOntologies. Accessed 22 July 2020
Sanfilippo, E. M., Kitamura, Y., & Young, R. I. M. (2019). Formal ontologies in manufacturing. *Applied Ontology*, *14*(2), 119–125. https://doi.org/10.3233/AO-190209
Saxena, A., Roychoudhury, I., & Celaya, J. R. (2010). Requirements Specifications for Prognostics : An Overview. In *Proceedings of AIAA Infotech@Aerospace 2010*. https://doi.org/10.2514/6.2010-3398
Smith, B., & Ceusters, W. (2015). Aboutness: Towards foundations for the information artifact ontology. In *CEUR Workshop Proceedings*.
Stanford University. (2020). Protégé website. https://protege.stanford.edu/. Accessed 22 July 2020
Steiner, C. M., & Albert, D. (2017). Validating domain ontologies: A methodology exemplified for concept maps. *Cogent Education*, *4*(1). https://doi.org/10.1080/2331186X.2016.1263006
Talhi, A., Fortineau, V., Huet, J. C., & Lamouri, S. (2019). Ontology for cloud manufacturing based Product Lifecycle Management. *Journal of Intelligent Manufacturing*, *30*, 2171–2192. https://doi.org/10.1007/s10845-017-1376-5
Zhou, A., Yu, D., & Zhang, W. (2015). A research on intelligent fault diagnosis of wind turbines based on ontology and FMECA. *Advanced Engineering Informatics*, *29*(1), 115–125. https://doi.org/10.1016/j.aei.2014.10.001

Appendix. Relations used in OMSSA

Relation	Definition	Representation
is a	Entity A is an entity B means that A is subclass of B.	
RO: bearer of	A relation between an independent continuant (the bearer) and a specifically dependent continuant (the dependent), in which the dependent specifically depends on the bearer for its existence.	*bearerOf*
RO: has quality	For types E and Q where E is a type of Entity and Q is a type of Quality, E has quality Q if and only if for every instance e of E there is some instance q of Q such that e "has quality" q. Here "has quality" denotes the primitive instance level relation. Inverse of quality of.	*hasQuality*
CCO: is about	A primitive (i.e. undefined) relationship between an information entity and some entity.	*inAbout*
CCO: is subject of	The inverse of is about, which relates an Entity to some Information Content Entity.	*isSubjectOf*
OMSSA: is subject failure effect	A CCO: is_subject_of sub-property to define the relationship between a failure mode and a failure event	*isSubjectOf*
OMSSA: is subject of failure likeliness	A CCO: is_subject_of sub-property to define the relationship between a failure mode and its likeliness to happen	*isSubjectOf*
RO: prescribes	For all classes T1 and T2, if T1 prescribes T2, then there is some instance of T1, t1, that serves as a rule or guide to some instance of T2, t2 (if T2 is a type of BFO: occurrent) or that serves as a model for some instance of T2, t2 (if T2 is a type of BFO: continuant).	*prescribes*
OMSSA: prescribes implementation risk	A RO: prescribes sub-property to define the relationship between a technical project plan and its implementation risk	*prescribes*
OMSSA: prescribes implementation benefit	A RO: prescribes sub-property to define the relationship between a technical project plan and its implementation benefit	*prescribes*
OMSSA: prescribes implementation cost	A RO: prescribes sub-property to define the relationship between a technical project plan and its implementation cost	*prescribes*
RO: describes	For all classes T1 and T2, if T1 describes T2 then there is some instance of T1, t1 that presents the characteristics by which some instance of T2, t2 can be recognized or visualized.	*describes*
RO: has output	A relation between a processual entity and a Continuant such that the presence of the Continuant at the end of the processual entity is a necessary condition for the completion of the processual entity.	*hasOutput*
RO: is input of	Inverse of has input, a relation between a processual entity and a Continuant such that the presence of the Continuant at the beginning of the processual entity is a necessary condition for the start of the processual entity.	*isInputOf*
RO: has part	A core relation that holds between a whole and its part.	*hasPart*
OMSSA: has action specification	A RO: has_part sub-property the represents a relation between a maintenance strategy and its maintenance action specification	*hasPart*
OMSSA: has triggering specification	A RO: has part sub-property the represents a relation between a maintenance strategy and its triggering event specification	*hasPart*
RO: has function	A relation between an independent continuant (the bearer) and a function, in which the function specifically depends on the bearer for its existence.	*hasFunction*
RO: precedes	A relation between two occurrents. Occurrent x precedes occurrent y if and only if the time point at which x ends is before or equivalent to the time point at which y starts.	*preceeds*
RO: participates in	A relation between a continuant and a process, in which the continuant is somehow involved in the process.	*participatesIn*

4.4 Terminology framework for predictive maintenance components selection

The previous section explained the development of an Ontology Model for Maintenance Strategy Selection and Assessment (OMSSA). Once a specific strategy is selected, an important work remains to be done for the strategy implementation, especially for advanced strategies based on accurate diagnosis and prognosis. If a predictive maintenance strategy is selected to address a specific system of interest, several decisions remain to be made to select the suitable components of the predictive maintenance system that will address the selected strategy.

This section expands the scope of OMSSA to incorporate the vocabulary framework that can be used for the selection of suitable components for new predictive maintenance systems. The expanded ontology reuses several classes, relations and axioms from OMSSA, and is referred to as Ontology for Predictive Maintenance Architecture and Design (OPMAD). Figure 4.3 summarizes the import structure for OPMAD. The Basic Formal Ontology (BFO) is used as top-level ontology and the Common Core Ontologies (CCO) are used as mid-level ontologies. BFO and CCO provide a set of domain-neutral classes that facilitate future reuse and integration of ontologies using the same structure. OPMAD also imports some classes from ROMAIN [Kar+19] as it served as domain reference ontology for OMSSA.

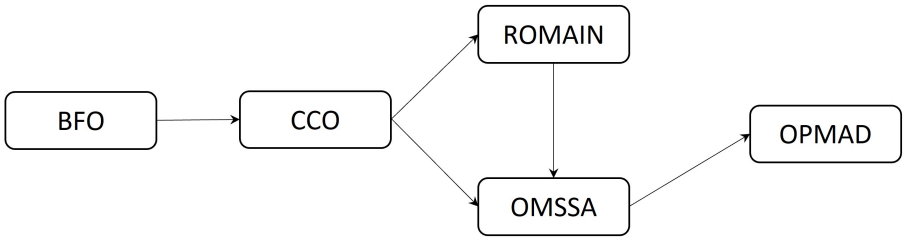


Figure 4.3: OPMAD import structure

4.4.1 OPMAD scope

OPMAD aims to support the DSS for component selection of new predictive maintenance systems. It follows the same development methodology as OMSSA. To delimit the scope of the ontology, the following competency questions haven been considered:

- What are the functions of a predictive maintenance system?
- What models have been used to fulfil each function of the system?
- What are the maintainable systems on which predictive maintenance has been implemented?
- For a given situation, is the predictive maintenance system implemented online or offline?
- What performance indicators have been used to assess a model that fulfils a predictive maintenance function?
- What data is analyzed by the predictive maintenance models?

- Given multi-model approaches, what is the models' configuration in the system?
- In which reference is the predictive maintenance implementation documented?

4.4.2 OPMAD terms and relations

Figure 4.4 presents the most important classes and relations in OPMAD. For layout purposes, the sub-classes are not shown in the figure; the full list of OPMAD classes can be found in Table 4.1. The acronym PdM is used for predictive maintenance in the figure. As the objective of OPMAD is to support the DSS to identify suitable models for predictive maintenance systems, the explanation for the ontology classes and their relations starts from the class `Predictive Maintenance Model`, and is based on the terms and relations presented in Figure 4.4. It is important to point out that the relations are adopted from BFO and CCO to comply with the upper level ontologies. A `Predictive Maintenance Model` is carried in a `Predictive Maintenance Module` which is a component of a `Predictive Maintenance System`. Each `Predictive Maintenance Module` has a `Predictive Maintenance Function`; this research is focused on diagnostics and prognostics functions.

The predictive maintenance model embedded in the module is directly linked to a `Maintainable item` which is the class that describes the technical maintainable system for which the predictive maintenance system is developed. The `Maintainable item` is classified by the class `Maintainable item type` which was added for similarity computation purposes in the DSS; this type of classes help to compare the new problem to be addressed with the solved problem in a case base of a CBR system. Maintainable items belonging to the same type share important degradation characteristics and thus the same predictive maintenance models can be proposed to solve the same predictive maintenance functions.

The `Maintainable item` has its own `Function` which is affected by a `Failure` that manifests itself through a `Failure Mode`. A `Maintainable item` has one or several failure modes which are also subject of the `Predictive Maintenance Model`. The predictive maintenance model has as input the `Condition Data` which is used to perform diagnostics and prognostics. Two important qualities for the implementation of a predictive maintenance model is its type and configuration. The `Predictive Maintenance Model Type` will classify the model within the families of knowledge-based, data-driven and physics-based models. This model type classification helps for similarity and preferences purposes in the DSS.

The `Predictive Maintenance Model Configuration` shows if a model should be complemented by other models to fulfil its function; the use of multiples models can improve performance but it increases development complexity. The `Predictive Maintenance Module` is qualified by the `Synchronization` and the `Performance indicator`. These two qualities provide useful information on how the predictive maintenance models embedded in the modules should be tested and synchronized with the `Maintainable item`.

All this information on predictive maintenance models, maintainable items, their failure models, etc, is gathered in `Predictive Maintenance Case` which is documented and published through a `Predictive Maintenance Article`, a special type of `Article` that is focused on predictive maintenance. From these predictive maintenance articles, some bibliometric indicators are also included in the ontology to be used in the CBR system. These bibliometric indicators are provided as solution attributes so that the engineer can easily find the information source of a specific case. The `Predictive Maintenance Article Title`, `Predictive Maintenance Article Identifier` and `Predictive Maintenance Article Publication Year` can be used for similarity purposes and/or as information source for further details of the predictive maintenance models and their implementation.

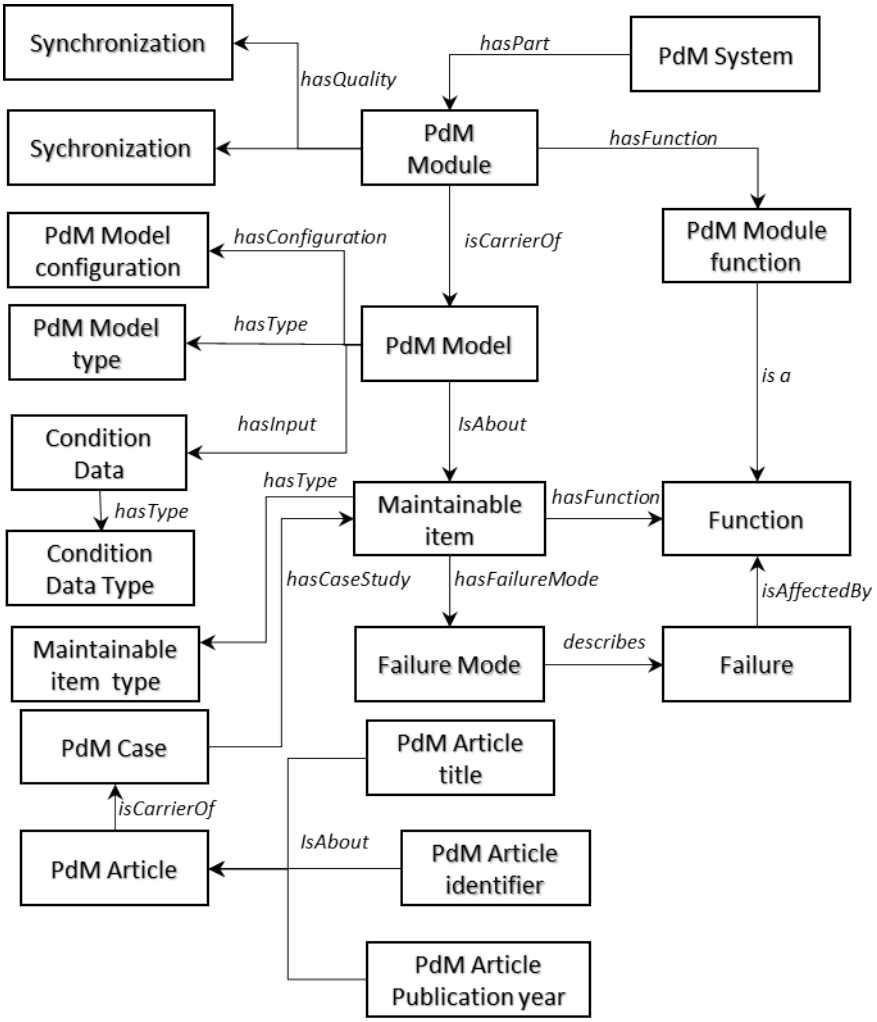


Figure 4.4: Classes and relations in OPMAD

Table 4.1: Terms of the Ontology for Predictive Maintenance Architecture and Design.

Begin of Table 4.1			
Class	Sub-class of	Definition	Source
Article identifier	CCO: Artifact identifier	An Artifact Identifier entity that designates some article	Based on CCO: Artifact identifier definition
Article title	CCO: Designative name	A Designative Name for some article	Based on CCO: Designative name definition
BFO: Function	BFO: Disposition	See BFO definition	BFO
Case	CCO: Designative name	A designative information content entity that designates a name to a case in a case base.	Based on CCO: Journal Article and the case definition for Case-Based Reasoning
Case-base	CCO: Information bearing artifact	An information Bearing Artifact that is designed to bear a set of cases for case-based reasoning. A CCO: Database subclass	Based on CCO: Journal Article and the case-base definition for Case-Based Reasoning

Continuation of Table 4.1			
Class	Sub-class of	Definition	Source
CCO: Artifact	BFO: Independent continuant	See CCO definition	CCO [Rud20a]; [Rud20b]
Data variable	CCO: Descriptive ICE	A descriptive information content entity that describes the variable in the maintainable item data	Domain-specific definition
Design detail	BFO: quality		Domain specific definition
Failure mode	CCO: Descriptive ICE	Manner in which a failure occurs	IEC 60812:2018
Fault detection	OPMAD: Predictive maintenance module function	A function whose purpose is the search of faults by assessing some Maintainable item parameters	Domain-specific definition [MV19]; [MIM01]
Fault feature extraction	OPMAD: Predictive maintenance module function	A function whose purpose is purpose is the identification of symptoms that correspond to a specific fault	Domain specific definition [MV19]; [MIM01]
Fault identification	OPMAD: Predictive maintenance module function	A function whose purpose is the comparison of the symptoms against known criteria to identify a specific fault	[MV19]; [MIM01]
Future state forecast	OPMAD: Predictive maintenance module function	A function whose purpose is the forecast of the condition for the next cycle operational cycle based on the current state of the maintainable item	Domain-specific definition [MV19]; [MIM01]
Health assessment	OPMAD: Predictive maintenance module function	A function whose purpose is to rate the current condition of a maintainable item against a predefined scale	Domain-specific definition [MV19]; [MIM01]
Health modelling	OPMAD: Predictive maintenance module function	A function whose purpose is to establish a scale to rate the condition of a maintainable item	Domain-specific definition [MV19]; [MIM01]
Maintainable item data	CCO: Information bearing artifact	An information Bearing Artifact that is designed to bear information from the maintainable item that can be used for predictive maintenance	Domain-specific definition
Item type	CCO: Descriptive ICE	A descriptive information content entity that describes the family of an item from the predictive maintenance perspective	Domain-specific definition
Maintainable item	BFO: Independent continuant	Subject being considered for maintenance	IEC 60812:2018 [Int18]

Continuation of Table 4.1			
Class	Sub-class of	Definition	Source
Model type	CCO: Descriptive ICE	A descriptive information content entity that describes the family of an item from the predictive maintenance perspective	Domain-specific definition
Models configuration	CCO: Descriptive ICE	An arrangement of models in a predictive maintenance module	Based on Oxford definition of configuration. Adapted for predictive maintenance models applications
Module synchronization	BFO: quality	A quality of a predictive maintenance module with regards the operation synchronization with the Maintainable item	Domain-specific definition
Number of input variables	BFO: quality	A quality that states the number of input variables that are used in a predictive maintenance model for a specific item in a predictive maintenance case.	Domain-specific definition
Number of failure modes	BFO: quality	A quality that states the number of failures modes that are known from a maintainable item	Domain-specific definition
Performance indicator	CCO: Descriptive ICE	A Directive Information Content Entity that describes the way to measure the performance of a predictive maintenance module	Domain-specific definition
Performance value	CCO: Descriptive ICE	A numerical or string value to rate the quality Performance indicator	Domain-specific definition
Predictive Maintenance Article	CCO: Information bearing artifact	An Information Bearing Artifact that is designed to bear a specific brief composition on predictive maintenance topic as part of a Journal Issue or conference proceedings	Based on CCO: Journal Article definition
Predictive maintenance Model	CCO: Directive ICE	A Directive Information Content Entity that consist of a set of propositions for predictive maintenance purposes	Based on CCO: Directive Information Content Entity definition adapted to the context of predictive maintenance
Predictive maintenance module	CCO: Information processing artifact	A component of a predictive maintenance system that fulfils one specific function	Domain-specific definition
Predictive maintenance module function	BFO: function	An realizable entity that is the purpose of a predictive maintenance module	Derived from BFO: function definition
Predictive maintenance system	CCO: Information processing artifact	A set of components that work together for predictive maintenance purposes	Domain-specific definition

Continuation of Table 4.1			
Class	Sub-class of	Definition	Source
Publication year	CCO: Descriptive ICE	A integer value that represents the Gregorian year in which an article was published	Domain-specific definition
Remaining useful life estimation	OPMAD: Predictive maintenance module function	A function whose purpose is the forecast of the remaining operational life based on the current state of the maintainable item	Domain-specific definition
End of Table 4.1			

4.4.3 OPMAD instantiation

The last step in ontology development is its instantiation. It consists of including individuals in the ontology classes. An instantiated ontology can be seen as a knowledge base of a specific knowledge domain. In the context of the current research, a knowledge base built from OPMAD will contain the data of successful implementations of predictive maintenance systems. The developed knowledge base is in fact the case base that is going to be used for the DSS. OPMAD has been populated with information coming from the publications considered in a recent structured literature survey about predictive maintenance [Mon+20]. Articles analysis and instantiation are manual processes. These tasks demand a deep understanding of the predictive maintenance domain as the vocabulary in the articles is very heterogeneous and not always standardized. Further explanation on how the different classes were instantiated and their purpose for the CBR system is provided in Chapter 5 as part of the case structure definition and case base creation.

4.5 Lessons learnt

This chapter addressed the creation of ontologies in the field of maintenance strategies, giving special attention to predictive maintenance solutions and their design. The chapter explained the principles of ontologies, the state-of-the-art and best practices for their development. The developed ontologies use the Basic Formal Ontology (BFO) as top-level ontology and the Common Core Ontologies (CCO) as mid-level ontologies. The Ontology Development 101 methodology was selected to carry out the ontologies creation. An Ontology model for Maintenance Strategy Selection and Assessment (OMSSA) has been created as a terminology framework for maintenance strategies. An extension to OMSSA, the Ontology for Predictive Maintenance Architecture and Design (OPMAD) was proposed as the terminology framework for the proposed knowledge reuse framework to select predictive maintenance models. Both ontologies have been created using the Protégé ontology editor and their consistency has been tested using the Hermit reasoner embedded in the editor. Further explanations about the integration of the ontology and the CBR system can be found in the next chapter. The developed ontologies aim at storing the case base and provide the ontological semantic similarity for some of the case attributes. The creation of the ontology is itself a contribution of the current thesis as it can be reused in other industrial applications

Intentionally left blank

CHAPTER 5

Case-based Reasoning systems development

"Bad reasoning, as well as good reasoning, is possible; and this fact is the foundation of practical side of logic."

Charles Sanders Pierce

Content

5.1 Reasoning considering the past experiences

Case-Based Reasoning (CBR) is a paradigm that leverages past problem-solving experience, in form of concrete solving cases, when it comes to solving new problems [RS89]; [Kol93]. Case-based reasoning was briefly introduced in Chapter 2. It is used by case-based models which are part of the knowledge-based models family. Case-based reasoning has a vast field of applications spanning from medical to industrial applications; CBR has been used to benefit from previous experiences (cases) that have been solved in the past. In the current thesis framework, CBR is implemented to retrieve possible components (models) that can be used in predictive maintenance systems. This section provides further insights into CBR, its phases and its characteristics.

5.2 The case-based reasoning paradigm

Case-based reasoning is a memory-based artificial intelligence solving methodology [De +05]. Case-Based reasoning was developed under the philosophy that human beings think and reason using analogies and

examples, rather than IF-THEN structures, the latter forming the basis for rule-based reasoning. A good example can be seen in the way a mechanic solves a problem with a car. If for example a mechanic is confronted with a car that does not accelerate properly, they remind of several comparable previous problems. The mechanic might recall previous similar situations, where for example brakes were clamping to the wheel or where the carburettor was not functioning optimally. It might be that the current situation reminds the mechanic of a problem with a chain wheel of a motorbike. Starting from this previous experience (knowledge), they can derive ideas for repairing the carburettor. From this earlier knowledge, the technician can deduce where and how to approach the problem, even though with the current type of car, they never actually encountered this problem.

Case-based reasoning tries to implement this way of reasoning. The following definition is used here:

Solve a new problem by remembering a previous similar situation and by reusing information and knowledge of that situation.

Case-based reasoning is therefore a paradigm in which specific knowledge of previously experienced problem situations is being used to solve a new problem. This is being done by finding similar previous cases and reusing previous experiences. In fact, it leads to a form of incremental, sustained learning, where information from new situations is kept for future use. The essence of case-based reasoning is that knowledge of previous experiences is represented in the form of "cases", instead of rules or constraints relations. A case is a declaration of a set of characteristic features, such as:

- the specific previously experienced problem situation,
- the previously applied solution,
- and sometimes also the procedure or method to obtain this solution.

In the example of the mechanic, 3 cases were mentioned. Features to represent the problem of a case are the bad acceleration, type of the vehicle and engine. The feature to describe the solution can have descriptions of the clamping brakes, the carburettor or the chain wheel respectively. Notice that the knowledge about the problem-solving process is implicitly available in the cases. No explicit relations, such as why the solutions were applied, are declared. New arising problems are declared in the occurring problem features, such as the bad acceleration. Cases with similar features as the occurring problem are searched, their solutions and methods are retrieved from the case-base and applied to the current problem.

5.2.1 Case-based reasoning cycle

Solving the problem is performed in a cyclic process of several steps: the CBR-cycle [AP94]. This cycle is composed of four phases: retrieve, reuse, revise and retain. Figure 5.1 shows the steps of the CBR cycle. This figure already shown in the Article 1, but it is recalled in this section to facilitate the CBR cycle explanations .

The CBR cycle is triggered when a new problem is encountered. This first phase aims at retrieving the most similar cases from a knowledge base that stores all previous cases. The target (new) case is compared to the existing cases in the knowledge base using different similarity measurements. The closest retrieved case is proposed as a possible solution in the reuse phase. Some adaptation might be needed to implement the solution in the target case. After suggesting and implementing the solution the revision phase takes place. If the suggested solution achieves to solve the problem, it is confirmed and in the last phase, it is retained in

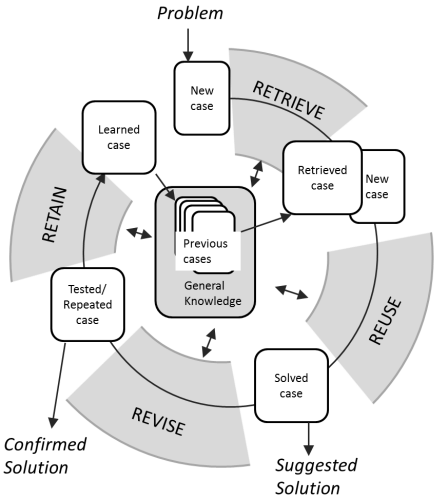


Figure 5.1: Case-based reasoning cycle. Inspired on [AP94]

the knowledge base so that it can be reused in future similar problems. Further details of each CBR phase is provided in the following sub-sections.

5.2.1.1 Case retrieval

Traditionally the case retrieval is performed by the comparison of attributes from the target case and the stored cases in the case base. If the case is composed by different heterogeneous attributes, a decomposition of the similarity measures by attribute is often performed. Later, the global similarity is obtained by an amalgamation function that gives a weight value to each individual similarity. As [De +05] explains, similarity can be divided into surface similarity and structural similarity. For surface similarity, each case attribute is represented as a real number in [0,1] and the similarity is computed according to a similarity measure. Structured similarity deepens in the case similarity assessment by extensive use of the domain knowledge. It can be computationally expensive but more relevant cases can be retrieved.

Recent trends in case retrieval include not only similarity measures when identifying the most suitable case. Sometimes the closest case retrieved by similarity measures can be impossible to adapt to the new problem. One alternative is adaptation-guided retrieval. Here the assessment is not only oriented on how "similar" a case is but how "useful" it can be [Ber+01]. Another alternative to pure similarity-based retrieval is diversity-conscious retrieval. Very often, the most similar retrieved cases are very similar to one another and this may result in a limited offer of possible solutions to the CBR system user. Proposing a set of diverse similar cases can contribute to exploring the possible solutions to the problem, improving the traditional similarity-based case retrieval.

5.2.1.2 Case reuse

Now that a case has been selected to solve the problem, it has to be decided what part of the case can be re-used. This will depend on the differences between the retrieved case and the problem. Here as well, much domain knowledge is used. In particular, rule-based and model-based reasoning is used in this stage. Two ways of re-use of the solution can be seen:

- transformational reuse
- derivational reuse

Transformational reuse entails copying the solution described in the retrieved case. The solution can be adapted based on differences between the features of the problem and the retrieved case. The focus is on the equivalence of the solutions. This requires a strong domain-dependent model. Derivational reuse, in contrast, implies the reuse of the method or procedure used to construct the retrieved case to construct a new solution. This means that not the solution as such is copied, but the methodology behind the solution.

5.2.1.3 Case revision

At this stage, the solution is applied to the problem. From this, new information ((partial) success, failure, etc.) and new knowledge emerge. The results have to be evaluated, to see if a further adaptation of the solution is necessary to solve the problem. The differences between the expected results and the real results have to be explained and possible repairs or improvements to the solution have to be derived to really solve the problem.

5.2.1.4 Case retain

In the final stage of the CBR cycle, learning takes place. This learning entails remembering successful solutions as positive experiences and failing solutions as negative experiences. In this way, the system can benefit both in a positive and negative manner from its own behaviour. Case-based adjustment can be done automatically, but usually it is done offline with the help of human experts. Learning can be done by introducing new cases in the case base, explaining the new experience, fine-tuning existing case bases, based on new information and assigning importance to certain cases/features. In fact, learning is a "by-product" of problem-solving. Case-based reasoning offers an integrated problem solving and learning paradigm. New trends in artificial intelligence aim at automating this learning process by the incorporation of other models such as neural networks or rule-based systems. This automates the case-base maintenance, improving the case update, case addition and the elimination of obsolete cases.

5.2.2 MyCBR introduction: workbench and SDK

MyCBR is an open-source similarity-based retrieval tool for Case-Based Reasoning (CBR) [Alt+12]. It is a joint effort of the Competence Centre CBR at the German Research Center for Artificial Intelligence, and the School of Computing and Technology at the University of West London. MyCBR offers two possible options for CBR solutions. The first one is a stand-alone application called myCBR Workbench in which it is possible to model and test highly sophisticated, knowledge-intensive similarity measures in a friendly user interface. The second one is an open-source Software Development Kit (SDK). The SDK is written in Java and includes all classes and methods to develop CBR applications and integrate them into other systems. Projects created in the Workbench are easily modifiable and run using the SDK. MyCBR workbench can be used to fast prototype CBR retrieval tools that are later integrated with the SDK. The SDK is intended to integrate the CBR engines created wit MyCBR to Java or Android-based applications. Figure 5.2 shows the intended use of MyCBR workbench and MyCBR SDK. It is important to point out that some advanced capabilities such as string-based similarity functions are only available in the SDK and can not be prototyped in the workbench.

When developing a CBR retrieval system using myCBR, the first step is determining the case attributes. Cases are represented by a coupled vector of attributes Case = [Problem attributes, Solution Attributes]. The problem attributes are used to measure the similarity of a target case and the cases in a case base. The solution attributes are stored as part of the solution information for the cases.

Once the attributes have been determined, the second step is the local similarity assignation to each problem attribute. MyCBR provides several options to compute similarity for each problem attribute. Within the available options, three similarity functions have been used for the current research:

1. Integer/Float similarity: this similarity function is used for numeric attributes. The similarity is obtained by a difference between a reference value and the input values of the function. A mathematical function is needed to define how the similarity decreases as the input values get further from the reference value. This mathematical function can be linear, exponential or determined by discrete points in the Cartesian plane. Figure 5.3 shows the procedure to edit simple integer similarity functions in MyCBR Workbench [Alt+12].

2. Symbol: this similarity measure is advisable for variables with a fixed set of options. These options are organized in a similarity matrix and some numerical values are given to establish the similarity among the options. Figure 5.4 presents a generic example of a symbol similarity matrix and how it is managed in MyCBR Workbench. This similarity function has been modified for some attributes in the current research to interact with the ontology model. The similarity matrix values are automatically obtained from the classes and relations of an ontology using feature-based similarity approach proposed by [Sán+12]. This feature-based similarity between two terms a and b is equal to 1 minus the normalized dissimilarity between a and b, as it is shown in equation 5.1.

$$sim_{norm}(a,b) = 1 - \log_2\left(1 + \frac{|\phi(a)\backslash\phi(b)| + |\phi(b)\backslash\phi(a)|}{|\phi(a)\backslash\phi b)| + |\phi(b)\backslash\phi(a)| + |\phi(b)\cap\phi(a)|}\right) \tag{5.1}$$

   where $|\phi(a)\backslash\phi(b)|$ are the features of a that are not present in b, $|\phi(a)\backslash\phi(b)|$ are the features of b that are not present in a, and $|\phi(b)\cap\phi(a)|$ are the features that both a and b have in common.

3. String: attributes similarity is obtained based on open text strings. Unlike the `Symbol` type attribute

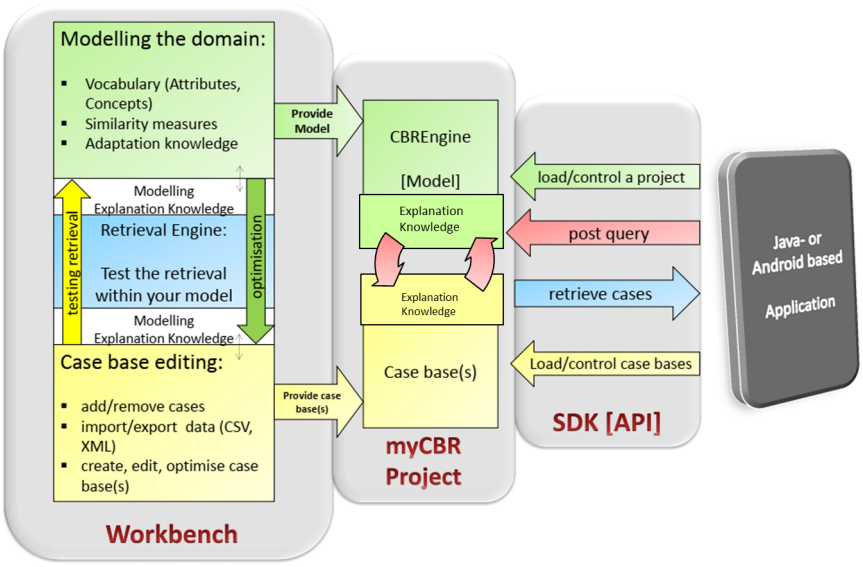


Figure 5.2: MyCBR platform description, [Alt+12]

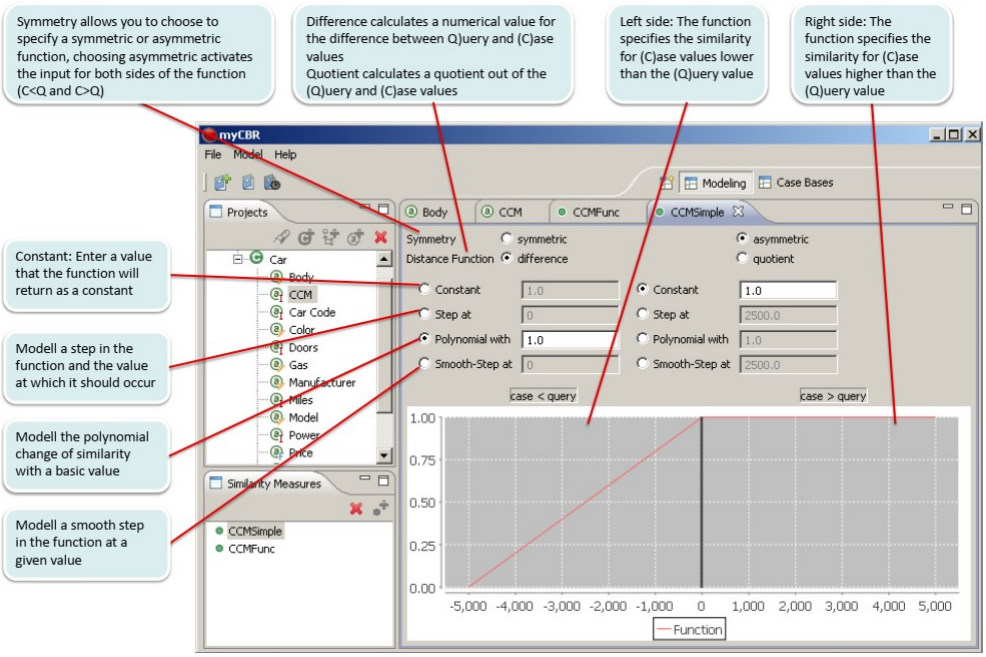


Figure 5.3: Simple integer similarity example, [Alt+12]

that has a certain number of categories, `String` type has only the restriction of having sentences or words as input. MyCBR offers three options to compute string-based similarity which are: `Equality`, `Ngram` and `Levenshtein`. For the current case study, the `Levenshtein` function was selected to compute the similarity string-based attributes. `Levenshtein` function offers a flexible means to compute similarity based on each character of the string [Lev66]. This is especially useful when there is a vast set of options that are unknown when building similarities. It also tolerates little orthographic errors from the user when typing the maintainable item of the target case. MyCBR offers string-based similarity only in its SDK; this similarity function is not yet available in MyCBR workbench.

The CBR system developed in this research was developed using the MyCBR SDK. The SDK was privileged over the Workbench for three main reasons:

1. String-based similarity functions are only available in the SDK.
2. The SDK provides access to modify the similarity functions. This allows the implementation of ontology-based similarity which is not originally included in MyCBR.
3. The SDK provides a flexible means to integrate the developed CBR retrieval solutions with other components of a Decisions Support System (DSS).

The complete development of the retrieval engine using the SDK and its integration with other parts of the DSS has been documented in a JavaDoc HTML and in the coding guide in Appendix C

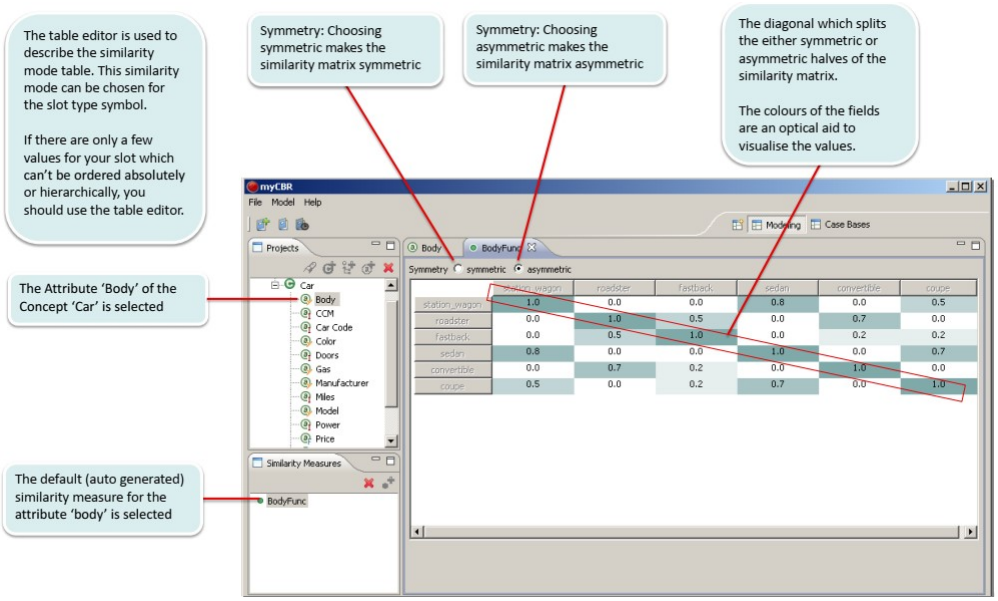


Figure 5.4: Symbol similarity matrix example, [Alt+12]

5.3 Development of the retrieval engine for predictive maintenance components selection

While the previous section provided the theoretical background of Case-Based Reasoning (CBR), this section explains the development of the retrieval engine for the predictive maintenance (PdM) component selection which is part of the DSS proposed in this thesis framework.

5.3.1 Case representation

"A case is a piece of knowledge in a particular context representing an experience that teaches an essential lesson to reach the goal of the reasoner" [Kol93]. Cases are often represented in a coupled form [problem, solution], in which similarities are applied on the "problem" part so that the "solution" part is retrieved. Case representation is composed of three main parts [BKP05]:

1. Defining the attributes of the case.
2. Defining the structure for the case content.
3. Organizing the case base.

5.3.1.1 Defining the attributes of the case

For the use case of predictive maintenance design, the cases are composed of three main attributes and some other complementary attributes. A case consists of a model that fulfils a specific predictive maintenance function on a determined system of interest. These three attributes are represented in OPMAD as:

- `Predictive maintenance model`: Every recorded case will have one or several models used to fulfil a specific `Predictive maintenance function`.
- `Predictive maintenance module function`: this attribute includes the function or functions that are fulfilled by predictive maintenance system for a specific maintainable item.
- `Maintainable item`: this class gathers the different machines, equipment, components or technological systems on which the model was applied and validated on each consulted article. The instances included in this class can match the subclasses of CCO: Artifact. The CCO: Artifact class and its subclasses structure were not directly reused to define the predictive maintenance case as CCO: Artifact has a more general purpose that goes beyond maintainable items.

Some complementary attributes have also been included to support the similarity assessment and the solution suggestion:

- `Maintainable item type`: the use case for the new predictive maintenance system may not be the same as the recorded implementations in the case base; however, they might share some similarities that could be considered to adjust a potential solution. Case studies may find some similarities if they are within the same "family" of applications as for example rotary machines or electronics devices.
- `Condition data`: This attribute gathers the instances that describe the inputs for a `predictive maintenance model`. Each model can be linked to several instances of the class condition data. For example, a neural network to fulfil a fault detection function of jet engines can have as condition data variables temperature, pressure, spinning speed, health index, reliability among others.
- `Condition data type`: this attribute aims helping in the similarity assessment. It is a complementary measure to `Condition data` class. It divides the condition data into different categories (see case base creation section).
- `Predictive maintenance model type`: this attribute aims at providing additional information to the proposed solution. It clusters the models into different categories (see case base creation section).
- `Predictive maintenance model configuration`: this attribute provides important information about the implementation of predictive maintenance models. It is part of the solution attributes.
- `Module synchronization`: this class aims at providing information of the synchronization between the `Predictive maintenance module` and the `maintainable item`. This is a problem attribute based on the structural requirements of a new predictive maintenance system. Online applications deal with real-time data which is constantly assessed to perform diagnostics or prognostics. Off-line applications gather information from an operative cycle to be assessed later. Online models normally are embedded in small control and monitoring boards with limited processing and storage memory while the offline application may be carried out by high-performance computers. Online scopes look for fast results in real time to trigger immediate actions in operation. Off-line scopes may target the maximum accuracy as there is more freedom in terms of time to perform the computations.
- `Predictive maintenance article title`: It is part of the solution attributes and its purpose is to provide traceability to a `predictive maintenance case` information source.
- `Predictive maintenance article identifier`: this attribute is intended to be part of the solution attributes of the case as it provides the traceability link to find the case information source. This source can be needed to find more detailed information about the case.
- `predictive maintenance article publication year`: this attribute is intended to be part of the similarity measures for the retrieval engine. Newer publications are privileged over older ones.

Some other attributes are part of the case but they are not intended to be part of the similarity assessment attributes or the solution suggestion. These attributes have been added for consistency in the ontology:

- `Predictive maintenance system`: this attribute aims at naming the system that has one of several predictive maintenance modules to fulfil one or several predictive maintenance functions on a `Maintainable item`.
- `Predictive maintenance module`: this attribute has been created to link the `Predictive maintenance model` with the `Predictive maintenance model` complying with BFO and CCO standards. The edges between these classes help compute some local similarities for the CBR system (see local similarities section).
- `Predictive maintenance article`: this attribute aims to be the link between the `predictive maintenance case` and information entities extracted from research articles such as `predictive maintenance article title`, `predictive maintenance article identifier`, and `predictive maintenance article publication year`.

5.3.1.2 Defining the structure for the case content

An extensive review on case representation techniques can be found in [ESE15]. The selection of the case representation is directly related to the similarity assessment that will be performed. Traditional methods for case representation include:

- Feature vector representation: this is the simplest representation of a case in which the cases are composed by a set of attributes that are used to describe the problem and the solution. The case retrieval is performed by the weighted sum of the similarities between the target case attributes and the cases in the case base. This case representation has limited capabilities to support semantic similarity as there is no representation of the knowledge domain.
- Frame-based representation: Frames provide a natural way for structured and concise knowledge representation. Frames are composed of different slots to organize knowledge. Each case is represented by a frame and each frame slot represents a case attribute. Frame-based representations have been (partially) formalized by description logic [BKP05]. The notion of “cases as terms" [Pla95] argues that viewing structured cases as terms in feature logics (a particular brand of description logics) helps in better understanding several aspects of case-based reasoning.
- Object-oriented representation: this case representation is suitable for complex case data structures. Cases are represented as a set of objects that have their own set of attributes.
- Textual representation: this case representation is suitable when cases are written in natural language texts. Specialized methods can be used to address these cases in an automated of semi-automated way.
- Hierarchical representation: In this approach, a case is represented at multiple levels of detail, possibly using multiple vocabularies. When solving a new problem, similar cases are retrieved from the case base at different levels of abstraction, their solutions are then combined and refined until achieving a final solution [BW96].
- Predicate-based representation: a predicate is a relation among objects, and it consists of a condition part and an action part, IF (condition) and THEN (action). Predicates that have no conditional part are facts. Cases can be represented as a collection of predicates [PS04]. The advantage of predicate representation is that it uses both rules and facts to represent a case, and it enables a case-based designer to build hybrid systems that are integrated rule/case-based [BKP05].

The above-mentioned case representation techniques have limitations in knowledge representation. They have few (if any) structures to describe the relations and constraints among the case features [BKP05]. Case representations using ontologies aims at solving this problem as ontologies provide not only the set of terms that constitute a case but also the relations among these attributes. Ontologies can be useful for designing knowledge-intensive CBR applications because they have powerful capabilities in knowledge acquisition, representation, and semantic understanding [GPH13]. Taking advantage of the created ontology for the current study, the case representation and structure is described using the proposed ontology in Chapter 4. Figure 4.4 showed the different attributes (classes in ontology) of the case represented in OPMAD. These ontology classes can be divided into two different groups: the problem attributes and the solution attributes:

- **`Problem Attributes`** = [`PdM Function`, `Maintainable Item`, `Maintainable Item Type`, `Condition Data Type`, `Module synchronization`, `PdM Article Publication year`]
- **`Solution Attributes`** = [`PdM Model`, `PdM Model Configuration`, `PdM Model Type`, `Module Performance Indicator`, `PdM Article Identifier`, `PdM Article Title`]

5.3.1.3 Organizing the case base

The case base corresponds to an instantiated version of OPMAD. The OPMAD structure provides the guidelines for the case base organization. The ontology has been populated using predictive maintenance cases retrieved from an extensive literature review about implemented predictive maintenance systems. The review presented in Chapter 2 was the starting point. An explanation of how the articles were consulted to instantiate OPMAD classes is presented hereafter:

1. `Predictive maintenance case`: the instances of this class are recorded in the form of "Case*N*", where N is an integer value starting from 1 that denotes the order in which the cases have been recorded. A case of a paper is identified when three interrelated attributes are found: the `Predictive maintenance function` that is fulfilled by a predictive maintenance system, the `Model` that fulfils the function, and the `Maintainable item` for which the predictive maintenance system has been developed. In a single article, it is likely to find several cases because of different factors such as:
    - There is a comparison of different models to fulfil the same PdM function. If several models are developed to fulfil the same PdM function, there is an instance in the database for each implemented model.
    - There are different PdM functions addressed by different models. The study can address more than one PdM function in a single PdM system. If every function is addressed by a different model, there is an instance for each addressed function.
    - A model that fulfils a PdM function has been implemented on different maintainable items. Some studies apply their approach to many case studies. An instance is recorded for each case study.
2. `Predictive maintenance system`: the instances of this class are named in the form of "System*N*", where N is a consecutive number according to the order in which the articles that explain these systems were consulted.
3. `Predictive maintenance module`: the instances of this class do not provide much information for the case retrieval or for the solution suggestion. These instances of `Predictive maintenance module` are named in the format of "Module*N*" where N is the case number it belongs to.

4. `Predictive maintenance module function`: the predictive maintenance functions can be clustered in eight different types (sub-classes). The instances receive the name of their subclass plus an integer number that records the order in which they were recorded. For example, instances of the sub-class `Fault detection` will be named as “faul detection1", “fault detection2", ..., “fault detection*N*", where *N* is the total number of instances of the sub-class `Fault detection`. The `Predictive maintenance function` sub-classes are:

    4.1. `Features extraction`: this PdM function identifies features from data that can be used for diagnostics or prognostics.

    4.2. `Fault detection`: this PdM function aims at detecting a faulty condition on a technical system given some symptoms.

    4.3. `Fault identification/isolation`: this PdM function allows to identify the source of the faulty condition when at least two faults present similar symptoms.

    4.4. `Degradation modelling`: a fault is not static; it evolves over time until it becomes a failure. This progression of the fault is usually called degradation. This PdM function aims at providing a representation of such degradation. Reliability and health indexes are some examples of values that can be used to model the degradation on maintainable items.

    4.5. `Health assessment/degradation analysis`: this PdM function aims at determining the current state of maintainable items based on a pre-existing degradation model.

    4.6. `Next state forecasting`: this PdM function aims at forecasting the health state of a maintainable item. It is a applicable when the life cycle of the maintainable item can be divided into discrete steps. This prognostics function aims at determining one-step-ahead or multiple-step-ahead if a failure is likely to happen.

    4.7. `Remaining useful life (RUL) calculation`: this PdM function aims at calculating the remaining work cycles the technical system or one of its components can work before failure.

5. `Predictive maintenance model`: the instances of this class are not limited by any type of restrictions. The instances of this class have been left as "open-text" as it is normal to find articles proposing new models that have been named by the article authors. Some of the instances in the `Model` class can be used in the future as subclasses in a more detailed version of OPMAD that has a more detailed level of instantiation.

6. `Maintainable item`: the instances of this class have been left as "open-text" as it is normal to find articles proposing new models that have been named by the article authors. Some of the instances in the `Maintainable item` class can be used in the future as subclasses in a more detailed version of OPMAD that has a more detailed level of instantiation.

7. `Maintainable item type`: the instances of this class aim at helping in the similarity measure between a target case and the cases in the case base. It is a complementary measure to `Maintainable item` class. The instances for this class have for the moment been limited to the following values:

    (a) Rotary machines

    (b) Reciprocating machines

    (c) Electrical components

    (d) Structures

    (e) Energy cells and batteries

    (f) Lubricants

8. `Condition data`: the instances of this class include all variables that are input for a `Predictive maintenance model`. The instances of this class can be found in different cases. Special attention must be paid to the instances of this class if one article includes more than one.

9. `Predictive maintenance case`: each case can have different data used as input for the `predictive maintenance model`. For example, there could be a model which aims to model degradation and later performs health assessment. For the first function, the model might use sensors data such as temperature and pressure. The output of the model would be a health index which is at the same time part of the inputs for the health assessment.

10. `Condition data type`: the instances of this class are limited to the following four options:

    (a) Signals: direct measurements from the technical system sensors whose reads are signals. Signal processing models are usually needed to extract features that can be used for diagnostics and prognostics. These signals can come from different types of sensors, such as: temperature, vibrations, pressure, spinning speed among others.

    (b) Time series (Discrete measurements or values): a single value is obtained to describe the state of input from the technical system at a specific work cycle. It can be obtained from sensors, models outputs or other data sources.

    (c) Structured text-based: this type of variable is similar to time series. The data is recorded in discrete events, but the variables are text-based with a limited number of possible options for the instances. It allows an easier analysis.

    (d) Text-based maintenance/operation logs: declared knowledge obtained from those who operate or maintain the technical system. Includes symptoms that are not measured by any other means but the human perception. Specialized techniques are needed to address natural language processing.

11. `Predictive maintenance model type`: it clusters the single-model approaches into three main instances: knowledge-based models, data-driven models and physics-based models. For multi-model approaches, there could be any combination of the proposed instances.

12. `Predictive maintenance model configuration`: the instances in this class provide information about the implementation of a `Predictive maintenance model`. In a first attempt, the instances are limited to two possible options: single model approach and multi-model approach. A more detailed analysis is possible by adding the specific configuration of the multi-model approaches as they can be implemented in parallel, in series, or embedded one in another [Mon+20].

13. `Module synchronization`: the instances of this class are limited to three possible options: “online", “off-line", and “not mentioned".

14. `Module performance indicator`: this class aims at storing the instances of performance indicators used to validate a model that fulfils a specific predictive maintenance function. For example, fault detection is usually assessed with the percentage of accurate detection, remaining useful life estimation is validated with a standard deviation of the results. More than one performance indicator can be included for each instance. No limited number of options are set for the instances in this class.

15. `Predictive maintenance article`: the instances of this class are named in the form of “Article*N*", where *N* is the consecutive number that keeps track of the order the articles were consulted. The instances of this class do not provide much information for the solution suggestion as the traceability to the case information source is done through the mentioned information entities of the article. However, its existence is justified for ontology consistency and compliance with BFO standards.

16. `Predictive maintenance article title`: the instances of these classes are strings that provide the article title given by its authors.

17. `Predictive maintenance article identifier`: the instances are strings that gather the alpha-numeric code used as publication identifier; for example, the DOI, HAL Id, conference Id, among many other options. The DOI will be the privileged one if more than one ID is available.

18. `Predictive maintenance article publication year`: the instances of this class are recorded as integer values.

5.3.2 Similarity assessment

The similarity assessment is performed in two steps. First, a local similarity is performed for each of the problem attributes, and in the end, all these local similarities are consolidated in a single value called global similarity.

5.3.2.1 Local similarities

Once the case structure is defined for each problem attributes, a similarity measure must be assigned. Table 5.1 summarizes the problem attributes and the similarity functions that have been assigned to each of them. Seven attributes define the problem but the retrieval engine user is able to provide as input six of them. The `PdM Article Publication year` is automatically computed by the system. An integer similarity function based on discrete points has been assigned to the `PdM Article Publication year`. It provides a local similarity of 1 to cases that are 5 years old or less. For older cases, the similarity decreases linearly until it reaches the value of 0 at 40 years old. Cases older than 40 years will receive a local similarity of 0 for the `PdM Article Publication year` attribute.

Table 5.1: Similarity functions assignation

Attibute	**Similarity function**
`PdM Function`	Symbol (Ontology)
`Maintainable item`	String (Levenshtein)
`Maintainable item type`	Symbol (Equality)
`Condition Data`	Symbol (Ontology)
`Condition Data Type`	Symbol (Equality)
`Module synchronization`	Symbol (Equality)
`PdM Article Publication Year`	Integer (Points function)

The attribute `PdM function` has been assigned with a symbol similarity function in MyCBR. The similarity matrix has been filled out using the feature-based ontological similarity presented in [Sán+12]. For this attribute the ontological similarity is computed based on the classes `PdM model`, `PdM module`, and `PdM function` of the ontology (see Figure 5.5). A `PdM Module` is linked to the `PdM model` by the relation *isCarrierOf* and at the same time is linked to the `PdM function` by the relation *hasFunction*. An inferred relation has been obtained between `PdM model` and `PdM function`. Using this inferred relation, the similarity is computed based on the models that have been used to fulfil a specific function. For example, a model such as Support Vector Machines (SVM) has been used in several cases for fault detection or fault identification; this increases the similarity between these two PdM functions.

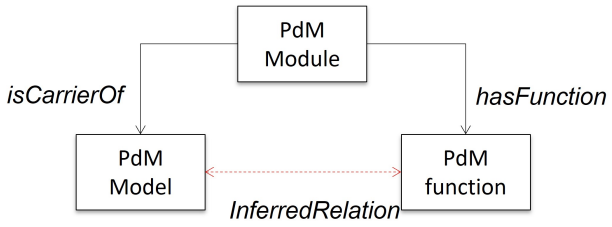


Figure 5.5: Classes and relations used to compute the ontological similarity for `PdM function`

Similarly, the attribute `condition data` has been assigned with a symbol similarity function whose

matrix has been filled out using the feature-based ontological similarity presented in [Sán+12]. For this attribute, the similarity is built based on an inferred relation between `PdM module` and `condition data` (see Figure 5.6). A `PdM module` of a specific case can have a list of variables that define the `condition data` (several instances for each case). The user will enter the list of variables of the target case and there will be similarity assessment for each condition data variable against the corresponding lists in the stored cases of the case base. In the end, the local similarity is based on the amount of condition data variables that match between the target case and the stored cases. For example, a target case has the variables “temperature", and “pressure", the closest case in the case base has the variables “pressure", “temperature", and “spinning speed", the ontological similarity obtained with equation 5.1 would be approximately 0,70 as there is only one different element between each list. For this similarity function, an additional typo error tolerance has been added. The DSS user enters the condition data variables manually and a typo error is likely to happen. The similarity assessment among the variables is done using the Levenshtein distance [Lev66]. In the first attempt, a limit of two different characters has been selected for typo error tolerance. This means that if a user writes for example “tepmerature" instead of “temperature" the function is still able to perform the similarity assessment correctly.

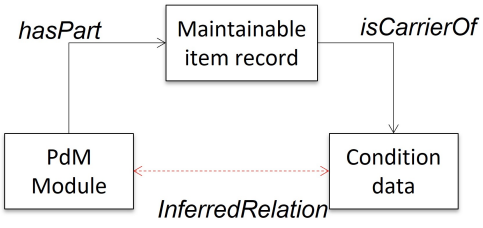


Figure 5.6: Classes and relations used to compute the ontological similarity for `condition data`

The attributes `Maintainable item type`, `Condition Data Type`, and `Module synchronization` have been assigned a symbol similarity function as provided by MyCBR. For each of these attributes, all the possible options are known and can be directly selected by the retrieval engine user. The equality in the symbol function means that the similarity is binary, if the user selects one option from the available ones for each of these attributes, there will be a local similarity equal to 1 to each case that has the same attribute value and 0 when the case has a different value. No partial similarity has been assigned among the options in the similarity matrix of the symbol function. This can be changed in the future as more relations from the ontology can be used to compute partial similarity among the attribute instances.

The attribute `Maintainable item` has been assigned with a string-based similarity function as provided by MyCBR. Levenshtein distance is used to compare a string entered by the retrieval engine user and the corresponding case attribute in the stored cases.

5.3.2.2 Global similarity: aggregation functions

After the similarity computation for each of the problem attributes, the combination of all these individual similarities in global similarity takes place. Each attribute has a weight and an amalgamation function calculates the final similarity based on local similarities and weights. The weights aim to give special importance to specific problem attributes. In a first attempt, all attributes get the weight of 1, meaning they have the same importance. MyCBR offers two different options to compute the global similarity: weighted sum and euclidean distance.

- **`Weighted sum`**: as the name says, is the sum of similarities considering the weight of each one. The equation 5.2 shows the calculation performed to find the global similarity.

$$sim_{global} = \sum_{i=1}^{n} w_i \cdot sim_i \tag{5.2}$$

  The values of n, w_i and sim_i are respectively the numbers of variables, the weight and the similarity of variable i.

- **`Euclidean distance`**: the euclidean distance between two points in an euclidean space is a number, measuring the length of a line segment between the two points. In the scope of the project, the distance is equal to global similarities. The equation 5.3 shows the calculation performed to find the similarity.

$$sim_{global} = \sqrt{\sum_{i=1}^{n} w_i \cdot sim_i^2} \tag{5.3}$$

In the current research, both amalgamation functions can be used by the user. It is important to point out that to compute the global similarity, only the available attributes are considered. This means that if the architect counts only on two or three attributes out of the seven that describe the problem, the global similarity will be computed based on those two or three attributes.

5.3.3 Retrieval engine user interface

The retrieval engine developed with MyCBR is the core of the DSS for predictive maintenance component selection in this research framework. A Graphical User Interface (GUI) has been developed to help in the verification and validation of the system. Figure 5.7 shows the developed GUI for the developed retrieval engine.

It is important to recall that the CBR system development and the ontology development were done in parallel. The ontology classes that define the problem attributes received different names compared to those used for the CBR system variables. Table 5.2 shows the OPMAD classes and the corresponding names in the retrieval engine code. The distinction of these names has helped to avoid ambiguity in the code.

The GUI has a pull-down menu for those problem attributes with a fixed number of options. Free-text cells have been added for the problem attributes `maintainable item` and `condition data`. The weight for each problem attribute can be directly assigned in the GUI (by default these weights are equal to 1). In the GUI it is possible to selected how many cases to retrieve from the case base and the aggregation (amalgamation) function to compute the global similarity between the target case and the retrieved cases. The list of the most similar cases is shown in the white field just below the attributes that the user can modify. The user triggers the retrieval by clicking on the ***SUBMIT QUERY*** button at the bottom of the GUI.

5.4 Lessons learnt

This chapter addressed the creation of the CBR retrieval engine using the MyCBR platform. The principles of CBR have been introduced including the different phases of the CBR cycle. MyCBR is intended for the retrieval phase of CBR systems. The platform offers a stand-alone application called MyCBR workbench for fast prototyping of retrieval engines. MyCBR also offers an open-source System Development Kit (SDK)

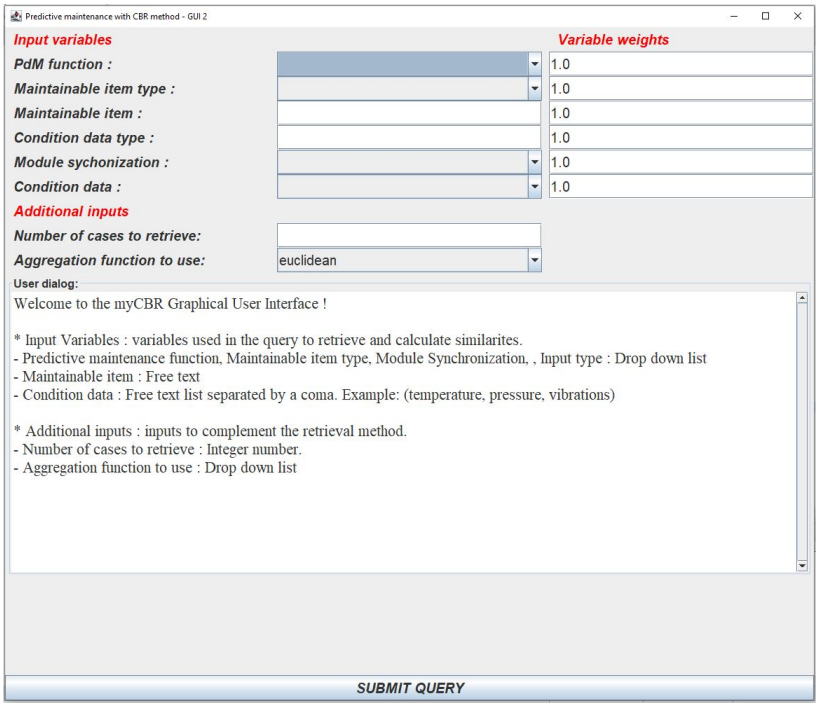


Figure 5.7: Retrieval engine Graphical User Interface (GUI)

Table 5.2: OPMAD classes and corresponding variables in the retrieval engine

OPMAD class	**CBR variable**
`PdM Function`	Task
`Maintainable item`	Case study
`Maintainable item type`	Case study type
`Condition Data`	Input for the model
`Condition Data Type`	Input type
`Module synchronization`	Online/Off-line
`PdM Article Publication Year`	Publication year

that allows the integration of the retrieval engines with other systems in a Java-based environment. As the SDK has been selected over the workbench to develop the retrieval engine in this research as the SDK provides more capabilities than the workbench. The case structure and the case base have been defined using the OPMAD classes defined in Chapter 4. The case base corresponds to an instantiated version of OPMAD. The process to instantiate OPMAD from an extensive literature review includes the definition of the different variables to search and the possible options for each variable; this allows a better case base structure and facilitates the retrieval. For each problem attribute, a local similarity is selected among the possible options: integer, symbol, ontology-based, open text. Two different aggregation functions have been added to compute the global similarity between a target case and those cases stored in the case base. A GUI has been developed to facilitate the verification and validation of the retrieval engine.

The developed retrieval engine is the core of the DSS for the selection of predictive maintenance components. The next chapter is oriented on showing the complete framework of the ontology-enabled CBR system for the selection of predictive maintenance components. The verification and validation of the retrieval engine developed in this section are addressed in Chapters 6 and 7. Development details of the retrieval engine are provided in the code guide in Appendix C.

CHAPTER 6

Building a framework for predictive maintenance models selection

"Manufacturing is more than just putting parts together. It's coming up with ideas, testing principles and perfecting the engineering as well as final assembly."

James Dyson

Content

6.1 Making the parts work together

Chapters 4 and 5 introduced the two technologies that are used to develop the Decision Support System (DSS) for predictive maintenance component selection. The proposed DSS is intended to overcome the problems encountered when selecting suitable approaches and models in the systems engineering approach to predictive maintenance design presented in Chapter 3. As complex systems may represent an important investment at high risk, the architect can not only rely on an immediate vision of a possible solution (unstructured creativity). The architect needs to explore the solution space looking for suitable components and their possible combinations (structured creativity) to propose a systems architecture able to meet the initial requirements of the system. Exploring the solution space of a new system can be a long-lasting task, especially when the architect has several of options from which they can select logical components. The proposed DSS can help the architect to save time in the exploration of the solution space to find suitable components to fulfill the logical architecture.

This chapter presents the framework for the proposed DSS. It explains how the proposed ontology and the CBR retrieval engine are integrated, and how the DSS fits in the creative work performed by the systems architect. The framework has been consolidated in a conference article. This chapter also includes a complementary section that extends the cross-validation performed to test the DSS capabilities.

6.2 Enhancing predictive maintenance architecture process by using ontology-enabled Case-Based Reasoning (Article 4)

Enhancing predictive maintenance architecture process by using ontology-enabled Case-Based Reasoning

Juan José Montero-Jiménez
Tecnológico de Costa Rica
Cartago, Costa Rica
ISAE-SUPAERO
Toulouse, France
Email: juan.montero@itcr.ac.cr

Rob Vingerhoeds
ISAE-SUPAERO, France
Toulouse, France
Email: rob.vingerhoeds@isae-supaero.fr

Bernard Grabot
ENIT
Tarbes, France
Email: bernard.grabot@enit.fr

***Abstract*—A common milestone in systems architecture development is the logical architecture. It provides a detailed overview of the system components and their interfaces but keeps the architecture as generic as possible, meaning that no component is bound to a specific technology. Subsequently, the architect searches for physical/informational components to fulfill the logical architecture and can apply structured creativity to look for innovative solutions. This search can turn out to be a difficult and long-lasting task depending on the system complexity. Too many options may be available to fulfill the logical system components and not always the most suitable ones are identified. This problem is for instance encountered in the design of new predictive maintenance systems, especially when selecting the components to carry out the diagnostics and prognostics. The current study proposes to support the choice of suitable components combining case-based reasoning and ontologies. A domain ontology has been developed as a terminology framework to support the case base, case structure and similarity measures for a case-based reasoning Decision Support System (DSS). The DSS uses attributes of the new problem to solve and suggests the most similar cases from past experiences. The retrieved solutions can be adapted to develop a new predictive maintenance architecture. The decision support system has been tested with data coming from proved predictive maintenance solutions documented in scientific publications.**

***Index Terms*—System architecture, case-based reasoning, predictive maintenance, knowledge reuse, structured creativity, decision-support system.**

I. Introduction

The life cycle of complex systems is composed of several stages starting from the concept phase [1], which can be seen as a very crucial stage. Following a systems engineering approach, the concept phase starts by gathering the needs and desires from all stakeholders, that are then translated into a formal set of requirements to be classified and prioritized to facilitate their analysis. Once the set of stakeholders' requirements is agreed by all parties a creative process starts to find the solution or solutions that meet the requirements. This is then formalized in a system architecture before a detailed design and the system implementation start in the development stage. The system architecture formally describes and represents the system elements and their relationships [1].

Developing a system architecture can be a complex task in which many options are available. Several methods exist to carry out this architecture process. Very often these methods cover a functional analysis of the system that includes a functional decomposition allowing to handle complexity [1]–[3]. Logical components are proposed to fulfill each sub-function. Once the logical components are defined, physical/informational components are selected and allocated to the logical components to complete the systems architecture.

Proposing suitable components to fulfill the logical architecture is an important challenge for the system architect. Sometimes, the architect may have an immediate vision of a solution, yet this does not always lead to the most efficient one. As the development of complex systems may represent an important investment, structured creativity is increasingly used in the concept stage [2]. This means the architect explores the solution space of the system using a structured methodology, identifying and proposing several possible solutions and performing trade-off analysis to identify the most suitable ones. Exploring the solution space of a new system can be long-lasting task, specially when the architect has many options for suitable components. Also, no explicit design rules exist to perform such component selection.

To overcome this challenge, this study proposes a searching tool that combines Case-Based Reasoning (CBR) and Ontologies. CBR is a problem solving paradigm based on previous experiences that can be used for design purposes. Ontologies are semantic knowledge representations that can model the terminology of a specific domain. The proposed approach aims at allowing the architect to save time when exploring the solution space and at the same time providing inspiration by offering diverse possible solutions to fulfill the logical components. In contrast to other implementations of CBR for systems design [4], [5], the proposed tool incorporates a domain ontology that serves as terminology framework for the CBR system. It helps to model the case structure, store the case base and compute semantic similarity for retrieval purposes in CBR.

This paper is organized as follows: section II explains the context of the research by introducing the building blocks of the proposed framework: Case-Based Reasoning (CBR), ontologies, and ontology-enabled CBR systems. Section III presents the design of predictive maintenance systems and

their architectures, domain to which the proposed approach is applied. Section IV explains the proposed approach of the use of an ontology-enabled CBR system to enhance structured creativity. Section V presents the implementation of this approach for predictive maintenance systems design. Section VI describes the results of the resulting Decision Support System (DSS) and section VII concludes the paper by summarizing the lessons learnt and providing perspectives of future work.

II. Background

The current study aims at implementing an ontology-based CBR system that can be used to retrieve suitable components for predictive maintenance (PdM) systems. The objective is to propose to system architects a means to explore the solution space more efficiently so not to miss important potential solutions. This section provides a background of CBR, ontologies, and their combination in recent knowledge modelling and reuse works.

A. Case-based Reasoning (CBR)

Case-based reasoning is a paradigm that leverages past problem solving experience, in form of concrete solving cases, when it comes to solving new problems [6]. Case-based reasoning tries to implement this way of reasoning. The following definition for CBR is used here:

Solve a new problem by remembering a previous similar situation and by reusing information and knowledge of that situation.

Case-based reasoning is a paradigm in which specific knowledge of previously experienced problem situations is being used to solve a new problem, by finding close previous cases, adapting and reusing those previous experiences. It leads to a form of incremental, sustained learning, where information from new situations is kept for future use.

Solving a problem with CBR is performed in a cyclic process of several steps, the so-called CBR-cycle [7], triggered by a new problem: solving the target case. The first phase aims at retrieving the most similar cases from a knowledge base that stores all previous cases. The target case is compared to the stored cases in a the case base using different similarity measurements. The most similar retrieved case is proposed as possible solution in the reuse phase. Some adaptation may be needed to implement the solution for the target case. Subsequently, revision phase takes place to ensure that the suggested solution achieves to solve the problem. Once the solution is validated, in a last phase it is stored in the knowledge base so that it can be reused in future similar problems.

B. Ontology

In information science, an ontology is a formal explicit description of concepts in a domain of discourse, properties of each concept describing its features, attributes and restrictions [8]. One of the goals in developing ontologies is “sharing a common understanding of the structure information among people and software agents” [9]. This means that the vocabulary used by people in a specific domain of knowledge is ”machine readable”. All concepts in an ontology are represented by classes which are linked by properties (also called relations). Ontologies are built using formal languages. One of the most recognized languages is the Web Ontology Language in its second version (OWL2) which is recommended by the World Wide Web Consortium (W3C) [10] because of its importance for the semantic web. The semantic web is an extension of the current web, where the information is well-structured and well-defined so that it can be processed by a machine [11].

Ontologies in OWL2 are compatible with information written in Resource Description Framework (RDF) [10]. In RDF, information is represented in semantic triples: subject-predicate-object. OWL2 ontologies are primarily exchanged as RDF documents; all the knowledge stored in an ontology can be also represented by semantic triples. Two related classes will represent the subject and the object of the triple, while the relation between the two classes represents the predicate of the triple. RDF structure provides a flexible means to model, storage and manage information [10]; other methods requiring variable-length fields would require a more complicated implementation.

Ontologies can help to make the knowledge explicit, defining the relations among different terms. This allows deep analysis on domain knowledge, helping to identify semantic rules among the terms. These rules can be used to develop algorithms that perform automated inferences based on the domain knowledge.

C. State-of-the-art of ontology-enabled CBR

For any CBR application, a very important step is the definition of a domain vocabulary that will serve to model the cases, the case base, the similarity measures and the solution adaptation knowledge [12], [13]. CBR was formalized before ontologies gained importance in the artificial intelligence field. Recent research trends incorporate ontologies to facilitate natural language modelling and processing. For CBR, ontologies can be used to enhance case indexing and retrieval, to improve semantic similarity estimation, to improve case representation, to improve case adaptation, and to improve case retention in a case base [14]. Ontologies provide the terminology framework to better describe the case attributes. In [15], an ontology was used as vocabulary framework for a case-based reasoning system that automates emergency response services. The authors use the ontology in the information extraction process, during the lexical analysis. Later, the ontology is used in the case retrieval by helping to determine the similarities for the case attributes.

Ontologies provide the possibility to compute the similarity between two semantic terms by the use of the ontology-based similarity. As [16] explains, ontology-based similarity can be determined by two different manners: by computing the ontological distance or by comparing the features of the terms. As ontologies can be represented by graphs in which the

terms are in the nodes and the relations among the terms are the edges, the ontological distance between two terms is the shortest edges path from one term to another. Feature-based similarity is obtained by comparing the properties of the classes. This feature based similarity can vary depending on the scope. Selecting the different features will yield to different similarities.

Ontologies can also be used to model the cases and the case base in CBR. The ontology classes can be instantiated with the different cases of the case base. An instantiated ontology can be be referred to as a knowledge base of the a specific domain [8]. This knowledge base stores the information in the Resource Description Framework (RDF) which can be easily retrieved and managed. In [17], an ontology is presented to support a CBR system for mechanics tolerance specification in which the case base is built on top of the ontology. Another example uses an ontology to build the case base of a CBR system for decision support on manufacturing process selection [18]. Specifically in systems design, [4] proposes an integrated approach using an ontology and a CBR system to propose design solutions based on the initial requirements and preferences. The authors propose a generic design approach in which the cases are built from requirement attributes and the solution obtained from previous experiences with similar requirements.

Current trends in systems architecture aim at incorporating structured creativity methods to explore the solution space in the concept phase of complex systems. The current study aims at integrating CBR and a domain ontology to facilitate the architect work of retrieving suitable components for predictive maintenance systems.

III. Predictive maintenance systems

Within the maintenance strategies to trigger maintenance actions, three terms are commonly used: corrective maintenance, preventive maintenance and predictive maintenance. Corrective maintenance triggers the maintenance actions once the failure of a component or system has occurred. Preventive maintenance trigger the maintenance actions by using fixed operation intervals of the component or system; such as time, cycles, kilometres, flights, among others. Predictive maintenance (PdM) aims at determining the right moment to trigger the maintenance actions based on the condition of the maintainable system under consideration [19]. Current fast expanding tools and technologies such as machine learning, internet of things, Industry 4.0, big data, boost predictive maintenance implementation to reach safer, more reliable and more efficient technical systems. Predictive maintenance is often studied within the disciplines of Condition-Based Maintenance (CBM) and Prognostics and Health Management (PHM).

Predictive maintenance is carried out by specialized systems whose main function is to estimate the precise moment to trigger maintenance actions. This main function can be decomposed into six different sub-functions (Figure 1: collect data, pre-process data, detect faults, assess degradation, compute remaining useful life and make recommendation report). This functional decomposition is widely used to develop the architecture of predictive maintenance systems.A new predictive maintenance system may require components to fulfill all sub-functions or a smaller subset of them [20]. For example, a system can be intended only to cover a fault detection, then the architecture should have components to collect data (F1), pre-process data (F2), detect the fault (F3) and make a recommendation report (F6).

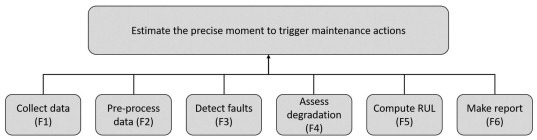


Fig. 1. Functional decomposition for a predictive maintenance system. [20]

Specifically for the diagnosis and prognosis sub-functions, there exist several models that can fulfill the requirements. These models can be implemented individually following a single model approach for each sub-function or many of them can be combined in a multi-model approach [19]. These models can be divided into three main families:

- *Knowledge-based models*: these models are built from expert knowledge and experience from which it is possible to explicitly define rules, cases and constraints that can be used to perform reasoning. These explicit models highly rely on the access to experts which can be a challenge for their development.
- *Physics-based models*: these models use the laws of physics to assess the degradation of components. They demand high skills on mathematics and physics of the phenomena for the application.
- *Data-driven models*: these models use data records that often consist of time series. Data-driven models have gained a lot of importance in recent years thanks to the improved availability of computational power and the large amounts of data produced every day by technical systems.

Predictive maintenance systems accurately illustrate the problem for system architects to select suitable components that fulfill the logical architecture. There are no explicit rules that guide the architect throughout the components selection to fulfill the logical architecture when developing a new predictive system. There exist several models that can be used to carry out the diagnosis and prognosis functions of the system; and these models are often combined as one single model hardly addresses a single task of a predictive maintenance system [19]. A decision support system can help the architect to select the model or combination models that have been used in the past accentuates the need for supporting architects for components selection.

IV. Ontology enabled Case-Based Reasoning for systems architecture

When developing a new system architecture, the architect often looks for inspiration from previous experiences and related systems that already exist. The idea to integrate Case-Based Reasoning (CBR) and ontologies for systems architecture comes from an analogy between CBR and the structured creativity process, as well as the need to formally model the domain vocabulary that allows the knowledge reuse. This section explains the analogy between CBR and structured creativity. A first concept of the integration of a ontology-enabled CBR Decision Support System (DSS) is introduced as means to help the architect to look for inspiration from previous experiences. The analogy and the concept of the DSS, here intended for predictive maintenance systems, are generic and can be used for the development of other types of complex systems as well.

A. The analogy between structured creativity and CBR for systems architecture

A common milestone in the systems architecture development is the logical architecture. Structured creativity at the logical architecture is based on the combination of different possible physical/informational components that can fulfill the logical components. This allows the exploration of the solution space and may help the architect to identify innovative solutions for a new system. If several possible solutions are identified a trade-off analysis may be necessary to select the most suitable one. The selected architecture serves as a basis for detailed design of the systems. The knowledge gathered from the new implemented system can be used by the architect to develop future systems.

There is an analogy between the structured creativity work performed by the architect to select suitable components to fulfill the logical architecture and the four phases of Case-Based Reasoning: retrieve, reuse, revise and retain. A graphical representation of this analogy is presented in Figure 2 (in Capella notation [3]). All activities before the logical architecture are outside the scope for the current study and for layout simplification, these activities have been summarized in a single box on Figure 2.

- The proposed analogy relates the retrieve phase of CBR with the search performed by the architect on previous related systems that may serve as inspiration for the development of the new system.
- The reuse phase of CBR would be the allocation of the identified components to fulfill the new system architecture.
- The revise phase of CBR would start by the trade-off analysis done by the architect to identify the most suitable components. This phase continues until the verification and validation of the implemented system which are usually performed by other actors than the architect.
- After validation of the new system, the architect may keep the records to be used in the future; this would be the retain phase of CBR.

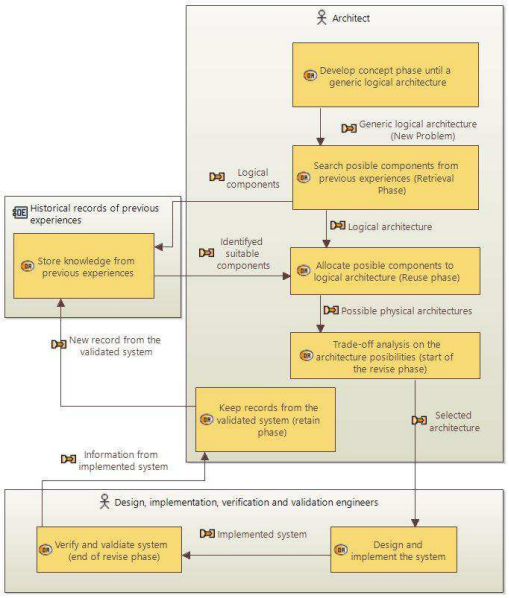


Fig. 2. Analogy between case-based reasoning and the tasks performed by a systems architect, in Capella notation [3]

This analogy remains generic in terms of the system to be developed. All the activities are carried out "manually" by the architect and the records from previous experiences may not be structured to facilitate their reuse. This manual approach may work when the amount of previous experiences is limited so that the solution space to be explored by the architect remains manageable. When the number of previous experiences is high or the requirements complex, the selection of the most suitable components may not be easily achieved. Retrieving knowledge from an important number of previous architectures may consume too much time and important options may be missed.

B. A Decision Support System concept using CBR and ontologies

Considering this analogy between CBR and the structured creativity work, different algorithms can be proposed to support the architect in the different phases of the architecture work. In a first step, the current research focused on the development of a Decision Support System (DSS) able to perform the search and recommendation of suitable physical/informational components. This can help the architect save time in the concept phase, and allows a broader analysis done by machines, analysis that can be hardly addressed by humans.

Figure 3 presents a concept of the DSS and how it fits in the analogy presented on Figure 2. This concept is composed of three main parts: the case base, the retrieve engine, and the domain specific ontology. The case base stores the cases from the past in a structured format to facilitate reuse. The

architect will present to the retrieve engine the information from the new system under development and will obtain a set of the most similar cases retrieved from the case base that will serve as inspiration when selecting suitable components for the logical architecture.

Within this structure the ontology plays a vital role. The cases, attributes of these cases and the similarities among the different text based variables are often described in natural language. Modeling this natural language and making it machine readable is necessary to automate the case retrieval.

V. Implementation on the Predictive Maintenance Case Study

The proposed approach assumes that the logical architecture of the system has been developed. A systematic approach to develop new predictive maintenance systems has been proposed in [20]. The ontology-enabled CBR Decision Support System (DSS) proposed here is for predictive maintenance models (components) selection to fulfill the logical architecture of a new system, especially to fulfill the diagnosis and prognosis functions. The stakeholder requirements for the DSS are as follows:

1) The system shall suggest suitable models to fulfill a specific diagnosis and prognosis function.
2) The system shall provide a list of diverse potential solutions for each logical component.
3) The system shall provide a structured framework to store the knowledge form previous predictive maintenance systems.
4) The system shall base the recommendation on known attributes of the new predictive maintenance system such as function to fulfill, the maintainable system and the available condition data from the maintainable system.
5) The system shall provide design information from the suitable models such as: performance indicators, input variables and models configuration.

A. Domain ontology development

The ontology for this study was developed using the methodology *Ontology Development 101* [8]. To delimit the scope of the ontology a set of competency questions should be considered. The ontology must be able to answer these questions. In this study, these competency questions have been defined in cooperation with experts in predictive maintenance and aim at gathering useful information that can be used in the design of new predictive maintenance systems. In a first attempt, the scope of the ontology is limited by the following competency questions:

- What are functions of a predictive maintenance system?
- What are the systems on which predictive maintenance has been implemented?
- What models have been used to fulfill each function of the system?
- For a given predictive maintenance case, is the predictive maintenance system implemented online or off-line?
- What performance indicators have been used to assess a model that fulfills a predictive maintenance function?
- What data is analyzed by the predictive maintenance models?
- If several models are being used to fulfill a specific function, what is the models configuration in the system?
- Where is the predictive maintenance implementation documented?

The ontology model was developed using a top-level domain-neutral ontology, the Basic Formal Ontology (BFO) [21]. It also uses a set of mid-level domain-neutral ontologies, the Common Core Ontologies (CCO) [22]. BFO and CCO

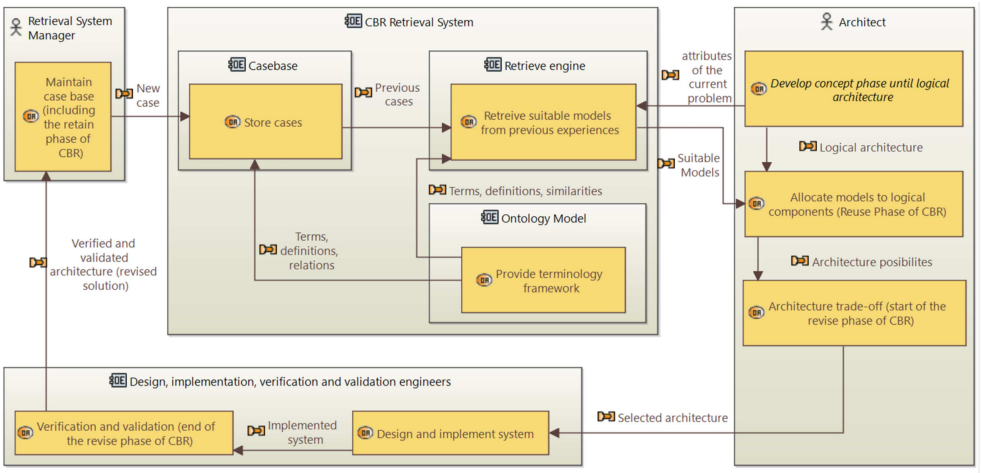


Fig. 3. Incorporation of an CBR retrieve system to facilitate logical components selection in the architecture process

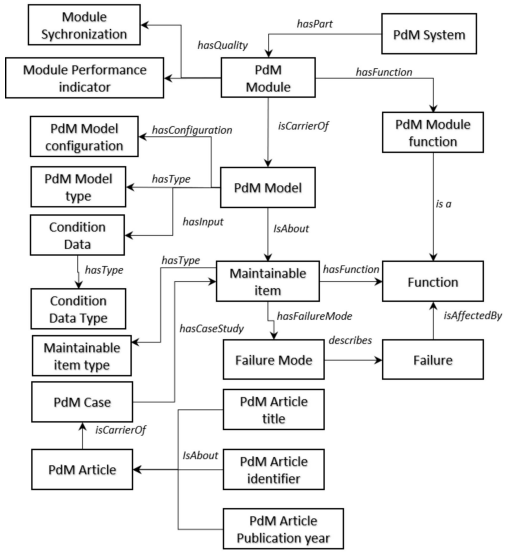


Fig. 4. Classes and relations in the ontology to support CBR for PdM component selection

have also been used in other ontologies in the industrial domain. The standardization on the developed ontology by adopting these upper level ontologies facilitates its future reuse and integration with other ontologies in the industrial domain. The Ontology was created using `Protégé©` (version 5.5.0) and its consistency was verified using the ontology reasoner HermiT OWL (version 1.4.3.456).

Figure 4 summarizes the most important classes and relations in the ontology for CBR decision support system for predictive maintenance components selection. For the validation of the current approach, the ontology was populated with information coming from the publications considered in a recent structured literature survey about predictive maintenance [19]. The OWL API [23] was used to populate the ontology and to provide the classes and methods for the integration of the ontology and the CBR system.

B. CBR decision support using MyCBR SDK

MyCBR is an open-source similarity-based retrieval tool for Case-Based Reasoning (CBR) [12] that offers two possible options for CBR solutions: a stand-alone application called myCBR Workbench and a Software Development Kit (SDK). The SDK is written in Java and includes all classes and methods to develop CBR applications and integrate them to other systems.

When developing a CBR retrieval system using myCBR, the first step is to determine the case structure. Cases are represented by a coupled vector of attributes Case = [Problem attributes, Solution Attributes]. The problem attributes are used to measure the similarity of a target case and the cases stored in the case base. The solution attributes are stored as part of the solution information for the cases. Taking as reference the classes in the domain ontology, for predictive maintenance design the case vector will have the following structure:

- **`Problem Attributes`** = [`PdM Function`, `Maintainable Item`, `Maintainable Item Type`, `Condition Data Type`, `Module synchronization`, `PdM Article Publication year`]
- **`Solution Attributes`** = [`PdM Model`, `PdM Model Configuration`, `PdM Model Type`, `Module Performance Indicator`, `PdM Article Identifier`, `PdM Article Title`]

Once the case structure is defined, a similarity measure must be assigned to each problem attribute. In myCBR the similarity for each problem attribute is referred to as local similarity. MyCBR offers a complete set of similarity measures to compute local similarities. For the current study three main types of similarities have been used: integer/float similarity, symbol similarity and string similarity [12]. The domain ontology plays a vital role in the computation of some of the local similarities. For example, for the predictive maintenance (PdM) functions an ontological similarity approach based on the ontological features is adopted to obtain the similarity among the options [16]. This similarity is computed based on the models that have been historically used to fulfill each function. All the cases stored in the ontology are used to compute this similarity and if the case base is updated with new cases in the future, the similarity will be automatically updated. This similarity measure was selected for the attributes `PdM Function`, `Maintainable Item Type`, `Condition Data Type`, and `Module synchronization`.

After the similarity computation for each attribute of the problem, the combination of all these individual similarities in a global similarity takes place. Each attribute has a weight and an amalgamation function calculates the final similarity based on local similarities and weights. MyCBR offers two different options to compute the global similarity: weighted sum and Euclidean distance.

- `Weighted sum`: the sum of similarities considering the weight of each one (equation 1).

$$sim_{global} = \sum_{i=1}^{n} w_i \cdot sim_i \tag{1}$$

  The values of n, w_i and sim_i are respectively the number of variables, the weight and the similarity of variable i.
- `Euclidean distance`: distance between two points in Euclidean space. It represents the length of a line segment between the two points (equation 2).

$$sim_{global} = \sqrt{\sum_{i=1}^{n} w_i \cdot sim_i^2} \tag{2}$$

For both amalgamation functions there is a re-scaling normalization between 0 and 1, with 1 being the maximum global similarity between the target case and a retrieved case. This

maximum global similarity is achieved when the target case attributes are identical to those of the retrieved case. This maximum global similarity can be achieved independently from the number of problem attributes that are introduced to the DSS. If the architect uses only on two or three attributes to describe the target case, a global similarity of 1 can be obtained based on those two or three attributes.

VI. Results and discussion

The case base stored in the domain ontology was instantiated with 263 cases obtained from the 135 research papers consulted in [19]. For a cross-validation of the ontology-enabled CBR Decision Support System (DSS), the cases were randomly divided into two sets. A set of 200 cases was used as case base in the ontology, the other 63 cases were used as test set.

A. Verification

The DSS verification checked how each local similarity performs. For this, several searches were realized using a reduced set of problem attributes. The verification results are described hereafter:

- With searches using one single attribute it was possible to validate the similarity measures assigned to the problem attributes: `PdM Function`, `Maintainable Item`, `Maintainable Item Type`, `Condition Data Type` and `Module synchronization`. These are the variables that the user can directly introduce as inputs for the DSS. When presenting only one out of these five possible attributes to define the target case, the DSS retrieved with similarity of 1 every case from the case base that shared the same attribute value as for the target case, independently from the assessed attribute.
- When increasing one by one the number of attributes of the target case, this behaviour was consistent. Several combinations using a reduced set of attributes were tested to confirm the DSS consistency to retrieve the most similar cases.
- The more attributes shown to the DSS, the fewer cases with similarity of 1 or close to one were retrieved. This is an expected behaviour because the stored cases in the case base are diverse. It was rare to find two cases in the case base with the same problem attributes and when it happened, the solution attributes of the retrieved cases were completely different.
- For some of target cases in the testing set, the maximum similarity reached was between 0.7 and 0.8 proving that when no identical cases were found an thus, the maximum global similarity of one can not be reached.
- The searches using a reduced number of attributes also helped verify the similarity measure implemented for the `PdM Article Publication year`, the only attribute that the user does not specify in the DSS to describe the target case and it is used to compute the global similarity. The current year is automatically retrieved from the operative system and it is compared against the year of publication of each PdM article. Models applications in newer papers are favored over similar applications from the past. With a reduced number of problem attributes it was possible to confirm that cases from recent publications had a higher similarity to the target case compared to those from older publications.

B. Validation

The cross-validation of the DSS checked that the stakeholder requirements of section V were met.

- Requirement 1 states that the DSS must suggest suitable components to fulfill diagnosis and prognosis functions in predictive maintenance systems. In practice, an architect needs at least one suitable model to develop the architecture but may look for inspiration from the retrieved cases with highest similarity. In the first attempt the acceptance criteria for the DSS is to provide at least 1 suitable model that can fulfill the PdM function of the target case. For each of the 63 cases it was possible to retrieve the similarity to the 200 cases in the case base. A list of the 5 most similar cases for each target case in the testing set was retrieved. For all the test cases at least one suitable model with similarity above 0.7 was retrieved. For each test case, the implemented PdM model is known. In 40% of the tested cases the implemented PdM model matched to one of proposed solutions by the DSS. In 40% of the tested cases all the proposed solutions by the DSS were implemented for the same PdM function. For the other 60% there was at least one recommended PdM model that was originally used for a different PdM function but being possible to adapt it to the target case. For example, for a target case to fulfill a fault detection function, the DSS proposed a classification model that was used for fault identification in the retrieved case. Classification techniques are suitable for both PdM functions. With these results, the acceptance criteria of the first requirement has been fulfilled; the capability of ontology-enabled CBR decision support system to recommend suitable models to fulfill a specific function of a predictive maintenance system has been confirmed.
- The validation highlights an important point of improvement with regard to the diversity in the retrieved cases. This is related to the second requirement. In at least half of the tested cases, within the list of the 5 most similar cases retrieved by the DSS, two or three cases proposed the same PdM model as potential solution. It is important to notice that this is not a problem of repeated cases in the case base, the similarities of the retrieved cases are different among them but sometimes they propose the exact same solution. Diversity in retrieved cases is a common problem in CBR [13], especially when several different potential solutions are expected from the retrieval phase. Diversity in the retrieved cases can help architects to develop innovative solutions. The identified problem narrows down the solution space recommended

by the ontology-enabled CBR system to the architect and represents an improvement of the DSS for future work.

- As the case base is stored in the domain ontology the third requirements is met. The ontology is a formal and structured knowledge representation that stores the predictive maintenance implementations of the past that can be reused.
- The case structured in the DSS was determined in such a way to fulfill requirements 4 and 5. The problem attributes are based on known parameters of the new predictive maintenance system and the solution attributes of the retrieved cases provide useful information for the design of the system.

For the verification and validation of the proposed DSS, both amalgamation functions were tested with no significant differences in the results. For this current study, all the attributes were assumed to have the same importance, so no special weights were assigned to the local similarities. As such they had a default value of 1.

VII. Conclusion and future work perspectives

The current study presents the use of a ontology-enabled Case-Based Reasoning (CBR) recommendation system to select suitable components for predictive maintenance systems architecture. The proposed approach benefits from the semantic knowledge modelled using a domain ontology. It allows to build the similarity measures for different attributes of the CBR system. The ontology stores the case base for the CBR that was created using the myCBR platform. The validation shown that the ontology-enabled CBR system is capable to retrieve suitable components to fulfill specific predictive maintenance functions. The retrieval capability was validated with all problem attributes that have been introduced individually and combined to the recommendation system. The retrieved cases can help architects as inspiration to develop innovative solutions for new predictive maintenance systems.

Future work perspectives include the refinement of the case retrieval functions, case base maintenance to improve the diversity in the retrieved cases, and the integration of a trade-off analysis that can help the architect perform the selection among the retrieved components. Future perspective also include the incorporation of algorithms to address other CBR phases such as adaptation of the retrieved cases for the target problem and case base maintenance after retaining newer cases. As the analogy of CBR and structured creativity was proposed as a generic approach, its implementation in other case studies is also considered in the future work perspectives.

Acknowledgment

The authors would like to thank the students Johanna Mazouzi from ISAE-ENSMA, Augusto Miyagawa and Hugo Muñoz-Hernández from ISAE-SUPAERO, for their collaboration in this research.

References

[1] INCOSE, *Systems Engineering Handbook. A guide for system life cycle processes and activities. Fourth Edition.* Wiley, 2015.

[2] E. Crawley, B. Cameron, and D. Selva, *System Architecture: Strategy and Product Development for Complex Systems*. Pearson Higher Education, Inc., 2015.

[3] P. Roques, *Systems Architecture Modeling with the Arcadia Method 1st Edition*. ISTE Press, 2018.

[4] J. C. Romero Bejarano, T. Coudert, E. Vareilles, L. Geneste, M. Aldanondo, and J. Abeille, "Case-based reasoning and system design: An integrated approach based on ontology and preference modeling," *Artificial Intelligence for Engineering Design, Analysis and Manufacturing: AIEDAM*, vol. 28, pp. 49–69, 2014.

[5] A. Tidemann, F. O. Bjørnson, and A. Aamodt, "Case-based reasoning in a system architecture for intelligent fish farming," in *Frontiers in Artificial Intelligence and Applications*, 2011.

[6] J. Kolodner, *Case based reasoning*. Morgan Kaufmann, 1993.

[7] A. Agnar and E. Plaza, "Case-Based reasoning: Foundational issues, methodological variations, and system approaches," *AI Communications*, vol. 7, no. 1, pp. 39–59, 1994.

[8] N. F. Noy and D. L. McGuinness, "Ontology Development 101: A Guide to Creating Your First Ontology," Tech. Rep., 2001.

[9] T. R. Gruber, "A translation approach to portable ontology specifications," *Knowledge Acquisition*, vol. 5, no. 2, pp. 199–220, 1993.

[10] World Wide Web Consortium, "OWL 2 Web Ontology Language," 2012. [Online]. Available: http://www.w3.org/TR/2012/REC-owl2-overview-20121211/

[11] D. L. Nuñez and M. Borsato, "OntoProg: An ontology-based model for implementing Prognostics Health Management in mechanical machines," *Advanced Engineering Informatics*, vol. 38, pp. 746–759, 2018.

[12] K. Althoff, T. Roth-Berhofer, K. Bach, and C. Severin, "Documentation: myCBR," 2006. [Online]. Available: http://mycbr-project.org/%0Adownloads/myCBR_3_tutorial_slides.pdf

[13] R. L. De Mantaras, D. Mcsherry, D. Bridge, D. Leake, B. Smyth, S. Craw, B. Faltings, M. L. Maher, M. T. Cox, K. Forbus, M. Keane, A. Aamodt, and I. Watsoni, "Retrieval, reuse, revision and retention in case-based reasoning," *The Knowledge Engineering Review*, vol. 20, no. 3, pp. 215–240, 2005.

[14] J. Prentzas and I. Hatzilygeroudis, "Combinations of case-based reasoning with other intelligent methods," in *CEUR Workshop Proceedings*, 2008.

[15] K. Amailef and J. Lu, "Ontology-supported case-based reasoning approach for intelligent m-Government emergency response services," *Decision Support Systems*, vol. 55, pp. 79–97, 2013.

[16] D. Sánchez, M. Batet, D. Isern, and A. Valls, "Ontology-based semantic similarity: A new feature-based approach," *Expert Systems with Applications*, vol. 39, pp. 7718–7728, 2012.

[17] Y. Qin, W. Lu, Q. Qi, X. Liu, M. Huang, P. J. Scott, and X. Jiang, "Towards an ontology-supported case-based reasoning approach for computer-aided tolerance specification," *Knowledge-Based Systems*, vol. 141, pp. 129–147, 2018.

[18] M. M. Mabkhot, A. M. Al-Samhan, and L. Hidri, "An ontology-enabled case-based reasoning decision support system for manufacturing process selection," *Advances in Materials Science and Engineering*, 2019.

[19] J. J. Montero Jimenez, S. Schwartz, R. Vingerhoeds, B. Grabot, and M. Salaün, "Towards multi-model approaches to predictive maintenance: A systematic literature survey on diagnostics and prognostics," *Journal of Manufacturing Systems*, vol. 56, pp. 539–557, 2020.

[20] J. J. Montero Jiménez and R. Vingerhoeds, "A System Engineering Approach to Predictive Maintenance Systems: from needs and desires to logical architecture." in *5th IEEE Int. Symposium on Systems Engineering 2019,*, Edinburgh, 2019.

[21] International Organization for Standardization (ISO), *ISO/IEC 21838-2 - Information technology - Top-level ontologies (TLO) - Part 2: Basic Formal Ontology (BFO)*, 2020.

[22] R. Rudnicki, "An Overview of the Common Core Ontologies 1.3," Buffalo, NY, p. 29, 2020. [Online]. Available: https://www.nist.gov/system/files/documents/2019/05/30/nist-ai-rfi-cubrc_inc_004.pdf

[23] I. Palmisano, "OWLAPI Documentation," 2020. [Online]. Available: https://mvnrepository.com/artifact/net.sourceforge.owlapi/owlapi-distribution/5.1.17

6.3 Cross-validation

This section extends the cross-validation discussion presented in Article 4 included in Section 6.2. The cross-validation of the ontology-enabled Case-Based Reasoning (CBR) system (the Decision Support System) allows demonstrating that this knowledge reuse technique can be used for the component selection of predictive maintenance systems. It is important to recall that in this first step, the validation is oriented to show that a Decision Support System (DSS) is capable to propose suitable components to fulfil specific predictive maintenance functions. No trade-off analysis has been implemented yet to assess the models proposed by the DSS. The validation focuses on the fact that the suggested models can be adapted to fulfil a specific function and that the list of suggested solutions covers all the solution space.

Cross-validation is a technique that can be used to test the effectiveness of trained artificial intelligence models. For the current approach, a train-test split approach is selected to perform the cross validation. For the test set, 63 out of the 263 cases in the case base were randomly extracted and the rest have been left in case base as training set. For each of the 63 extracted cases, the problem attributes were presented to the DSS using a GUI. The attention was centred on the 10 most similar cases when performing the retrieval. Figure 6.1 shows an example of a retrieval test in which all the target case problem attributes are matched with the corresponding attributes of case 35. This full match between the attributes turns out in a global similarity of 1.

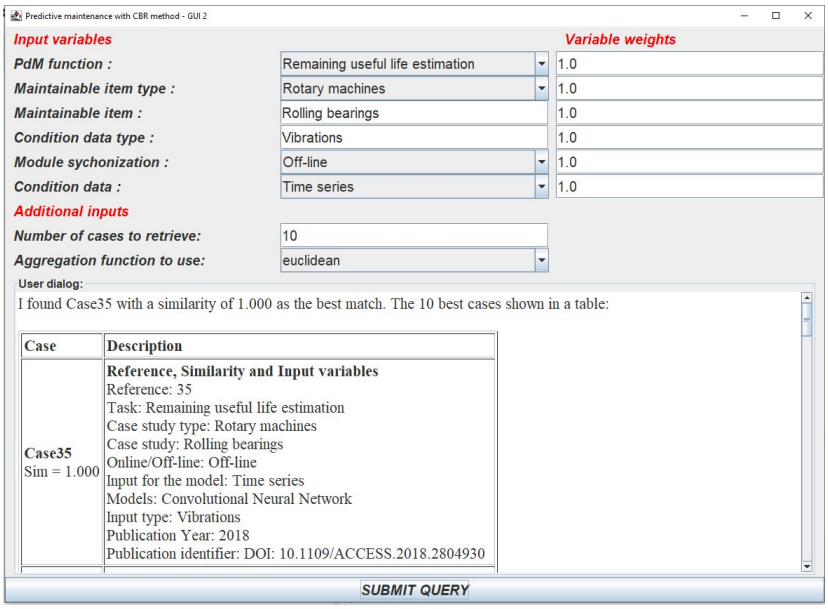


Figure 6.1: Retrieval example using the GUI of the DSS

The solution space of the predictive maintenance models can be divided into two groups as explained in [Mon+20]. The first group is composed of single model approaches, divided into three main categories: knowledge-based models, data-driven models and physics-based models. The second group is composed of multi-model approaches, in which at least two models from any of the three mentioned categories are combined to fulfil a specific function of the PdM system. Predictive maintenance is composed of diagnostics and prognostics tasks. According to the literature review [Mon+20], diagnostics tasks such as fault detection, fault identification, and health state modelling can be addressed by models of the three categories of single model approaches or multi-model approaches. In contrast, for prognostics tasks, it is difficult to find knowledge-based models applications. For example, prognostics functions such as remaining useful life estimation and next state forecast functions are normally fulfilled by physics-based models, data-driven

models, and multi-model approaches combining physics-based and data-driven models. The cross-validation of the DSS allowed to confirm this behaviour. The retrievals for diagnostics tasks proposed single model and multi-model solutions considering models from the three categories of models. The retrievals for prognostics also proposed single model and multi-model solutions, but the proposed models were only from data-driven and physics-based categories. This helps to confirm that the solution space is well covered.

For each test case, the retrieved cases are ranked based on the similarity of problem attributes. For some of the tests, two or more retrieved cases had the same maximum similarity value. From a systems engineering perspective this represents a limitation because no further information is provided to perform the trade-off analysis and selected the most suitable one. Another limitation was identified with regards the diversity in the retrieved cases. For some target cases the DSS retrieved two or more cases that proposed the same model to fulfil a specific PdM function. An architect looking for inspiration to develop innovative solutions will need the DSS to propose a diverse set of models. Retrieving two cases with the exact same solution is not useful for the solution space exploration.

The above mentioned limitations are improvement points for the DSS to be considered in the future. A more detailed analysis can also be performed including the effort to adapt the solution of a retrieved case for the target case. This can help to perform the trade-off analysis on the retrieved options and consequently improve the performance of the DSS.

6.4 Lessons learnt

This chapter presented the Decision Support System framework and can help the architect in the selection of predictive maintenance components and approaches. The chapter explains the integration of OPMAD and the CBR retrieval engine developed with myCBR platform. It also explains how the DSS can fit in the architecture process and how an architect can benefit from it to automate the solution space exploration allowing a more accurate structured creativity by only retrieving the most suitable components for a new predictive maintenance system.

Recalling the refine research questions introduced in Chapter 2:

1. How to address the design of predictive maintenance systems?
2. How to suggest a suitable approach for a predictive maintenance system solution?
3. How to select a suitable model or combination of models given a new predictive maintenance problem to solve?
4. How can a designer benefit from the experience of existing systems to develop new predictive maintenance solutions?

The presented framework attempts to answer the first refined research question. It combines the systems engineering approach for the concept stage presented in Chapter 3 and the use of a DSS for the selection of suitable components for the logical architecture. The DSS implementation attempts to answer the rest of the research questions. It allows to store the past experiences of predictive maintenance implementations and retrieves the most suitable ones depending on the new predictive maintenance system to develop. The DSS not only suggests the models that can fulfil a specific predictive maintenance function but also other important aspects that can be used for the detailed design and implementation. To further validate the proposed framework and the DSS capabilities, the next chapter presents a practical example on which the framework and the DSS are used. An implementation of one of the suggested models by the DSS is used to indirectly validate the accuracy of the DSS recommendations.

CHAPTER 7

Validation approach and discussion

"When you want to know how things really work, study them when they're coming apart."

William Gibson

Content

7.1 Validating the proposed Decision Support System in a practical example

As part of the framework validation, a use case example has been developed. This example uses an aircraft engine data set [Cha+21]. The data set contains the run-failure records of 128 aircraft jet engines under real flight conditions and has been generated with the Commercial Modular Aero-Propulsion System Simulation (CMAPSS) model developed by NASA. This data set is called the N-CMAPSS. The purpose is not only to check the capabilities of the proposed ontology-enabled CBR system but also to identify the improvement points for the DSS. The objective for this validation is to implement one of the models proposed by the DSS to fulfil a predictive maintenance function for the N-CMAPSS database. This implementation helps to demonstrate that the DSS is capable to suggest suitable components for predictive maintenance systems, complementing the cross-validation in Chapter 6.

7.2 Use case example: Design of a predictive maintenance system for aircraft engine run-to-failure data set under real flight conditions

The N-CMAPSS data set was published in January 2021 and has an objective to facilitate the development of specialized algorithms for predictive maintenance applications by providing a complete set of run-to-failure data with different failure modes. CMAPSS has been used to simulate other well-known data sets for

prognostics purposes, such as PHM08 data [Sax+08]. The N-CMAPSS data set supposes an improvement in the level of fidelity between the simulated data versus the real-life data. Each flight in the N-CMAPSS data set is fully recorded from take-off to landing. Previous versions of CMAPSS data sets offer a simpler approach that only provides one single discrete measure for each engine flight. Another improvement can be seen in the number of failure modes. Previous CMAPSS data sets only had one or two imposed failure modes, and for those sets with two failure modes, there were no explicit records that could be used to discriminate the failure mode that affected each jet engine. The N-CMAPSS data set has up to seven different failure modes and there is an explicit record of the failure mode that affected each engine. Each failure mode has its own set of symptoms that can be identified by the loss of Flow (F) or Efficiency (E) in the rotary components of the engine such as the fan, the Low-Pressure Compressor (LPC), the High-Pressure Compressor (HPC), the Low-Pressure Turbine (LPT), and the High-Pressure Turbine (HPT). This improvement extends the use of the N-CMAPSS data set, not only for prognostics like the previous CMAPPS data sets but also for diagnostics.

According to [Cha+21], five main steps have been followed to generate the data set (see Figure 7.1):

1. The flight data are defined as recorded onboard real commercial jets.
2. The degradation of the engine components is imposed in the simulation so that it is possible to keep track of the failed component in each run-to failure engine.
3. The resulting degraded flight is simulated using the CMAPSS.
4. The health condition is evaluated and the unit continues flying with increasing degradation until the health index of the engine has reached zero.
5. Sensor noise is added to the simulated engine response in order to make the simulated data closer to real-life data.

The flight conditions have been divided into three flight classes depending on the flight length. Class 1 includes all flights that last from one to three hours. Class 2 includes all flights between three to five hours of duration. Class 3 includes all flights that last more than 5 hours. The data set is divided into 8 sub-sets with different failure modes and a different number of engine units. Table 7.1 shows the overview of the N-CMAPSS data set. Each sub-set has its own failure mode except for the sub-set DS02 which has 2 failure modes which are the ones from the sub-sets SD01 and DS03 correspondingly. The overview shows the symptoms that describe each failure mode. For example, the failure mode of sub-set DS01 can be detected by a decrease of the efficiency in the HPC, while the failure mode of sub-set DS04 can be detected by a decrease of the efficiency and flow of the engine fan.

Each unit in the N-CMAPSS data set has unknown initial degradation, meaning that the records did not start at the same point for all of them. The records for each engine in the N-CMAPSS finish when the engine reaches the failed state. Each engine goes through a normal degradation stage in which the fault symptoms are not evident. If a fault appears in the engine, there will be a transition from normal degradation to abnormal degradation that is faster than normal degradation to reach the failed state. The data is composed of 46 variables divided in 5 different groups:

1. Scenario descriptors: variables that are independent of the engine operation that are used to describe the operational modes of the engine.
2. Measurements: Sensors that describe the engine operation.
3. Virtual sensors: complementary measures offered by CMAPSS to assess the engine operation

4. Model health parameters: complementary variables that show the symptoms of the different failure modes.
5. Auxiliary variables: variables for the data set consistency.

Further explanation about the N-CMAPSS data set can be found in [Cha+21] and [FDL07].

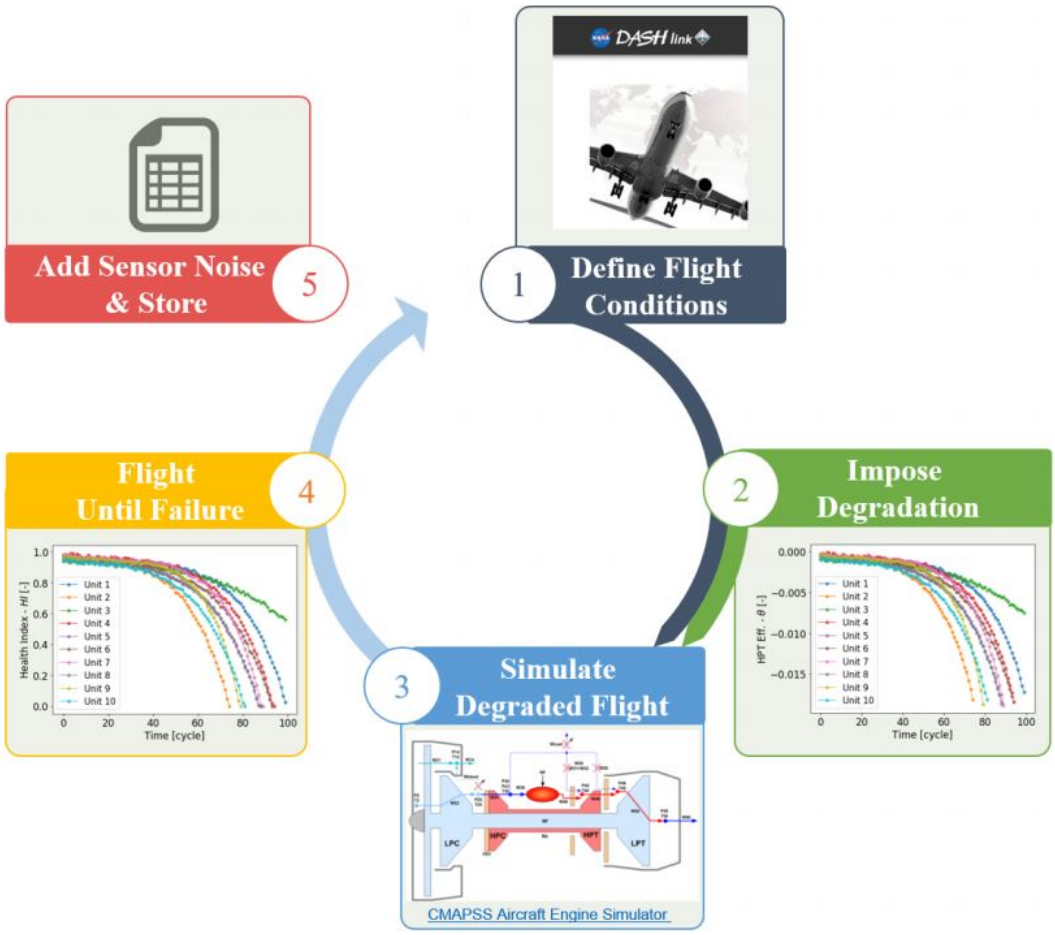


Figure 7.1: N-CMAPSS data creation process [Cha+21]

7.3 The concept phase of a predictive maintenance system for the N-CMAPSS database

Given that N-CMAPSS is intended for research purposes, it is up to the researchers to define the list of needs and desires for the new predictive maintenance system. For the current research framework, the following list of needs and desires is proposed:

1. Read the N-CMAPSS data format.
2. Identify the variables needed to perform the diagnostics and prognostics.

Table 7.1: Overview of N-CMAPSS data set [Cha+21]

Name	# Units	Flight classes	Failure modes	Fan		LPC		HPC		HPT		LPT		Size (measures)
				E	F	E	F	E	F	E	F	E	F	
DS01	10	1, 2, 3	1							✓				7.6M
DS02	9	1, 2, 3	2							✓		✓	✓	6.5M
DS03	15	1, 2, 3	1							✓		✓	✓	9.8M
DS04	10	2, 3	1	✓	✓									10.0M
DS05	10	1, 2, 3	1					✓	✓					6.9M
DS06	10	1, 2, 3	1			✓	✓	✓	✓					6.8M
DS07	10	1, 2, 3	1									✓	✓	7.2M
DS08	54	1, 2, 3	1	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	35.6M

3. Pre-process and reduce the N-CMAPSS data set.
4. Detect the transition from normal degradation to abnormal degradation.
5. Identify the failure mode that leads the engine to fail.
6. Determine the current state of a jet engine.
7. Estimate the remaining useful life of a jet engine.
8. Provide a report of the results

The previous list of needs and desires are translated into the following list of functional requirements:

1. The system shall read the N-CMAPSS data set.
2. The system shall model the health of the jet engine.
3. The system shall detect incipient faults in the engine.
4. The system shall assess the health state of the engine.
5. The system shall estimate the remaining useful life of the engine.
6. The system shall provide a report of the predictive maintenance results.

In the first concept of the predictive maintenance systems, the data pre-process and data reduction are not included and they have been performed manually. As can be seen, only functional requirements have been determined. The N-CMAPSS does not provide any performance needs, structural constraints or experiential needs which means that no behavioural, structural or experiential requirements have been added. As the data set comes from an academic case study, no experiential needs or desires have been added. It is important to notice that in practical applications, these three types of requirements are important to be considered as they provide the requirements related to the expected performance of the system, interfaces with other related systems and interface with the system user.

Given the functional requirements, the architecture process presented in Chapter 3 has been followed until obtaining a logical architecture of the new predictive maintenance system. Figure 7.2 presents the logical architecture developed from the functional requirements previously listed. A first component will collect the processed data from the jet engine records. This data will be used to develop the health model of the engine. Later, the engine data and the health model are used for the components in charge of assessing the health state, detect incipient faults, and identify these faults. The outputs of these three components are

the input for the system component in charge of the remaining useful life estimation. The health assessment, fault detection, fault identification and remaining useful life computation will provide the information needed to make the predictive maintenance report by the last system component. The functions of each predictive maintenance system component have been assigned to match with the definitions established in the ontology and the Decision Support System (DSS) so that it can be possible to retrieve the corresponding recommendations.

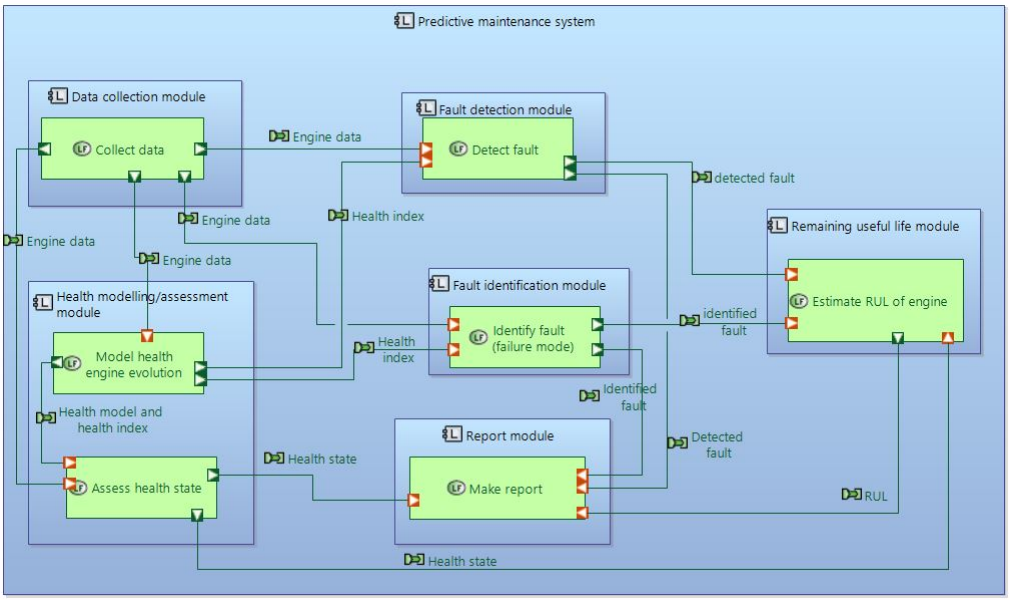


Figure 7.2: The logical architecture of a predictive maintenance system for the N-CMAPSS data set

7.4 Component selection using an ontology-enabled case-based recommendation system

Considering the logical architecture presented in Figure 7.2, there are four different logical components for which the Decision Support System can provide suggestions:

- Health modelling/assessment module
- Fault detection module
- Fault identification module
- Remaining useful life module

For each of these four logical components, a retrieval of possible solutions using the proposed DSS was performed. The results of the retrievals are summarized in Table 7.2. The DSS is capable to provide the similarity to each of the cases in the case base, but the most similar cases will have more relevance. For the current analysis and layout purposes of Table 7.2 only the five highest similarities are considered for each predictive maintenance function.

The results in Table 7.2 demonstrate the capabilities of the DSS to retrieve suitable solutions for each of the logical components. All the proposed models can be adapted to fulfil the intended function. All the

similarities between the target case and retrieved cases were higher than 0.776, meaning that the global similarity between proposed models and the target case was at least of 77.6%. An improvement opportunity for future research can be seen for those retrieved cases with exact same similarity; additional information would be needed to select the most suitable one. For example, for remaining useful life estimation module the DSS proposed five possible models with the same similarity value, no model is ranked higher than the other. An architect will need need further information to perform the trade-off analysis and select the most suitable model to be implemented. Further similarity measures can be added to refine the retrieval process. It is important not to forget that in a first attempt the DSS is intended to provide suitable solutions, but there is a fair amount of remaining work so that the DSS is capable to suggest the most suitable among the others.

It is important to notice that the ontological similarity allowed to propose some models originally used for fault identification to fulfil fault detection and vice-versa. This inferred similarity is accurate as both functions are related to classification tasks. By adding more cases to the case base, more semantic similarities like this one can be found. The current values of similarity are low as the diversity in the case base is high compared to the case base size. Further explanations about the semantic similarity can be found in [MHMJV21], a conference publication performed in the framework of the current research, included in Appendix B.

In order to further validate the DSS recommendations, one of the suggested models has been taken as an example and has been implemented to fulfil the corresponding predictive maintenance function. Taking advantage of the research team experience in Self-Organizing Maps (SOM), an implementation of the health modelling component is performed using the SOM. The DSS recommends the SOM and logistic regression for health modelling as the most suitable models (similarity equal to 0.897) for the N-CMAPSS case study. The following section provides further explanations about the SOM example implementation and its preliminary results.

The methodology to implement the Self-Organizing Maps (SOM) for health modelling has been adopted from [Sch+20], (see Appendix A). Self-Organizing Maps are artificial neural networks with unsupervised training that are capable to cluster instances of data depending on the instance attributes. SOM is normally composed of a square layer of neurons and the different clusters after the SOM training can be graphically represented on the maps as well defined regions. It has been successfully used to model the degradation process of different machines such as jet engines [MV18]. In these cases, the neurons represent the health or degradation of the machine at a specific operational mode. The trained SOM will have a single region but a transition from white (optimal state) to black (failed state). When assessing the health or degradation of a machine using the trained SOM, a neuron will be excited on the map showing how advanced the degradation is or how much the health has decreased. This is the actual goal of the current implementation: to obtain a trained SOM capable to show a transition from optimal state to failed state.

In a first attempt, a simplified implementation is selected. To train the SOM, only sub-set of the N-CMAPSS data is used. The DS01 has been selected as it has only one failure mode. The first challenge in the adaptation of the SOM solution to the N-CMAPSS database is related to the operational modes. To train the SOM, the different operational modes must be distinguishable from one another; unfortunately, the N-CMAPSS is not focused on the different operational modes of the engine but in the complete flight envelope. To solve this challenge an operational mode has been defined when the engine reaches 10000 feet of altitude when the aircraft is climbing. At that operational mode, the throttle-resolver angle will be always higher than 70% and the flight Mach number is always in the same interval. These three variables are presented in the N-CMAPSS as scenario descriptors and are independent of the engine operation.

A second challenge in the adaptation of the SOM for the N-CMAPSS was related to the variables selected for the SOM training. The N-CMAPSS is composed by 45 different variables of different natures: scenario descriptors, operational engine measurements, virtual sensors and model health parameters. For the current implementation, only the scenario descriptors and the operational engine measures (see Table 7.3) have been

Table 7.2: Models retrieval for the N-CMAPSS examples

Target case function	Case	PdM Function	Model	Similarity
Fault detection	166	Fault detection	Gaussian process classifier	0.859
	211	Fault detection	Piecewise linear model (PWL), Hybrid Kalman Filter, OBEM model	0.834
	49	Fault identification	Support vector machines	0.776
	48	Fault identification	Deep belief network	0.776
	50	Fault identification	Multi-layer perceptron neural network	0.776
Fault identification	48	Fault identification	Deep belief network	0.859
	50	Fault identification	Multi-layer perceptron neural network	0.859
	49	Fault identification	Support vector machines	0.859
	212	Fault detection	Piecewise linear model (PWL), Hybrid Kalman Filter, OBEM model	0.834
	166	Fault detection	Gaussian process clasifier	0.794
Health modelling /assessment	12	Health modelling	Self-organizing maps	0.897
	1	Health modelling	Logistic regression	0.897
	57	Health modelling	Statistical regression model	0.854
	159	Health modelling	Hidden Markov Chains	0.852
	168	Health modelling	Copula based sampling	0.838
Remaining useful life estimation	51	Remaining useful life estimation	LSTM (Long-Short Term Memory Neural Network)	0.895
	53	Remaining useful life estimation	Recurrent Neural Network	0.895
	54	Remaining useful life estimation	Gated recurrent unit network	0.895
	62	Remaining useful life estimation	Relevance vector machine	0.895
	64	Remaining useful life estimation	Bayesian linear regression	0.895

selected for the training as they represent the variables that can be obtained from real aircraft engines. The virtual sensors and the model health parameters are part of the simulation and model consistency but they would not be available in real jet engines.

Table 7.3: Operational Measurements N-CMAPSS

Symbol	Description	Units
Wf	Fuel flow	pps
Nf	Physical fan speed	rpm
Nc	Physical core speed	rpm
T24	Total temperature at LPC outlet	°R
T30	Total temperature at HPC outlet	°R
T48	Total temperature at HPT outlet	°R
T50	Total temperature at LPT outlet	°R
P2	Total pressure at fan inlet	psia
P15	Total pressure in bypass-duct	psia
P21	Total pressute at fan outlet	psia
P24	Total pressure at LPC outlet	psia
Ps30	Static pressure at HPC outlet	psia
P40	Total pressure at burner outlet	psia
P50	Total pressure at LPT outlet	psia

LPC: Low Pressure Compressor
HPC: High Pressure Compressor
LPT: Low Pressure Turbine
HPT: High Pressure Turbine

To increase the convergence changes in the SOM training, it is advisable to add the most representative inputs. A good practice is to delete all variables that present a binary or a constant behaviour as they do no provide any useful information about the degradation. A correlation test of the variables is also recommended to avoid adding redundant variables to the SOM training. Both of these tests have been performed on the operational measurements. No variables presented constants or binary records. Figure 7.3 shows a correlation matrix for the operational measures in which the same variables are in the horizontal and vertical axes; the correlation value is shown in the intersection of a vertical and horizontal axis. A correlation threshold has been set at 0.9. If two or more variables have a correlation higher than the threshold, only one of them will be selected at random as they will provide almost the same information. It is important to point out that before the correlation test, all variables were normalized between 0 and 1 which is also a requirement for the SOM training [Sch+20]. After performing the variables assessment, only four of them were selected to train the map, which represents a reduction in the number of variables compared to the study in Appendix A:

1. Total temperature at Low-Pressure Compressor (LPC) outlet.
2. Total temperature at High-Pressure Turbine (HPT) outlet.
3. Total pressure at fan inlet.
4. Total pressure at Low-Pressure Turbine (LPT) outlet

Once the variables have been selected and normalized, the training of the SOM can start. Given the data size a map of 5x5 neurons in square architecture has been selected. Several SOM's were trained and their convergence and behaviour were confirmed in the results. Figure 7.4 shows an example of a trained

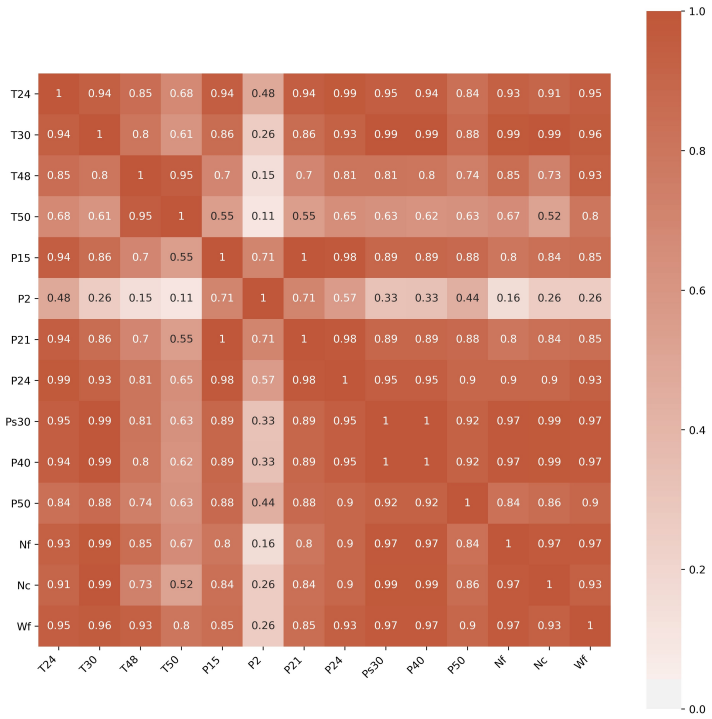


Figure 7.3: Correlation matrix of the sensors of the N-CMAPSS data set to be used as input for the SOM

map using the extracted data from the N-CMAPSS database. The clearest neuron represents the optimal operation condition of the engine and the darkest red neuron represents the condition of the engine just before its failure. An assessed engine with this SOM will excite a neuron between these two limits and its degradation can be estimated using the weights of the neuron. A transition from clear to darks is observable in the graphical representation of the SOM. This represents the degradation of the engine at the selected operational mode. For further explanations about the training and interpretation of the SOM, please read the article in Appendix A.

7.5 Discussion

It is important to recall that the validation of a Decision Support System (DSS) is a non-trivial task. The validation of the implemented model suggested by the DSS can be used to indirectly validate the DSS but further implementations would be required to confirm the capabilities and accuracy of the proposed DSS for predictive maintenance models selection. In the current implementation, the N-CMPASS represents the target case for which a new predictive maintenance system has to be developed. The ontology-enabled CBR recommendation system (also called DSS in this manuscript) proposed different models to fulfil each predictive maintenance function. One of the models proposed by the DSS for the function health modelling was the Self-Organizing Map (SOM). The SOM implementation successfully showed the degradation trend of jet-engines from nominal operation to failed condition using a subset of the N-CMPASS. The trained SOM can be also used to assess the health/ degradation of other engines of the same type and under the same operation conditions. It is important to clarify that this validation aims to demonstrate the suitability of the model proposed by the DSS, but further comparisons are needed among the suggested models are necessary to determine the best model. While performing the cross-validation presented in Chapter 6 and the current validation based on the SOM implementation, several improvement points have been identified:

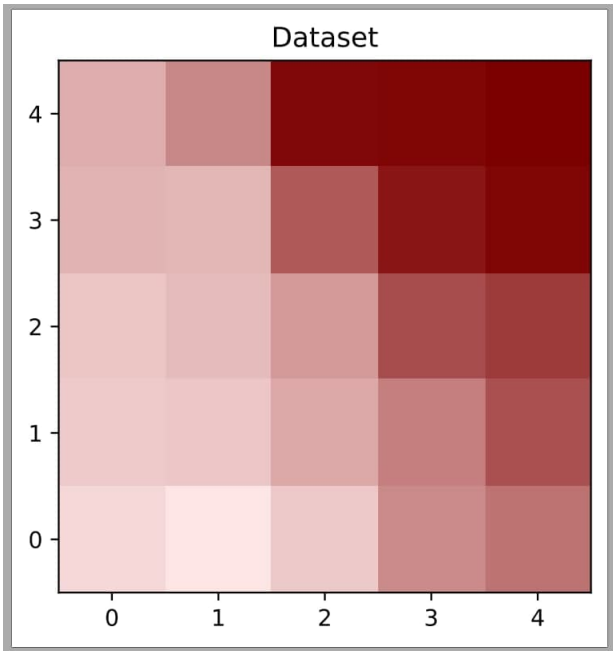


Figure 7.4: Trained SOM for the sub-set DS01

- Problems with the diversity in the retrieved cases: Even if it was not the case for this N-CMAPSS case study, in some retrievals of the cross-validation, the DSS proposed the same model several times with different references in the case base. This is a problem from a systems engineering point of view. An architect looking for innovative solutions needs a diverse list of possible solutions from the DSS. Further work can be done to avoid this diversity problem. Cases generalization and some other case diversity techniques can be applied to overcome this problem [De +05]. Such a situation could also be attributed to a potential problem of the coverage of the solution space. If the stored cases correspond only to a small portion of the solution space, the models suggested by the DSS can be very similar. For the current study this is not the case; models from all over the solution space have been considered. The state-of-the-art study in Chapter 2 allowed determining that the solution space for predictive maintenance systems is composed of data-driven models, knowledge-based models, physics-based models and multi-model approaches that combine at least two models from any of the three mentioned model families. The case base for the DSS was built considering all the solution space but the validation showed that this can be improved. A refinement of the solution space can be done with regards to each predictive maintenance function. For diagnostics functions such as for example “feature extraction" only data-driven models have been identified while creating the the case base. Another example can be seen for the function “remaining useful life estimation" where only data-driven and physics-based models have been identified. A refinement of the solution space considering each predictive maintenance function separately can help to improve the case base and the diversity in the models retrieved with the DSS.

- Additional information to perform the trade-off analysis: Several retrieved models for the same predictive maintenance function have the same similarity. Implement several solutions using different models that have the same similarity may not always be feasible for time and resources limitations. This poses a challenge for the architect to select the most suitable. Some other attributes could be added to the case so that the retrieved information can be used to perform a trade-off analysis so that the DSS can help the architect to make an accurate decision with regard to the model selection. Performance indicators, computational power required, implementation complexity, implementation cost, are some examples of complementary variables that can help to assess the models proposed by

the DSS. The additional attributes can be also used to add the preferences or initial constraint set by the DSS user from the beginning. Some users may be interested only in data-driven models, then the solution could be limited before the retrieval of models.

- Information about the adaptation of the proposed model: The current version of the DSS can be improved by including additional information for the adaptation of the suggested model in the target case. This is not only useful for the trade-off analysis, but also for the detailed design and implementation stage of the predictive maintenance system. The implementation of the SOM for health modelling has shown that some hyper-parameters of the models must be changed. A complementary guide for the adaptation of the suggested models can be added to facilitate its implementation in the target case. An expert system may guide the architect with the adaptation steps for each of the models. For time limitations, this is not possible for the current research.

7.6 Lessons learnt from the DSS validation using the N-CMAPSS

The validation of the proposed DSS for predictive maintenance component selection has been presented in this chapter. The capabilities of the DSS have been tested with the implementation of one of the suggested models for health modelling for the N-CMAPSS case study. This validation helped to demonstrate not only that the proposed DSS is capable to suggest suitable components for predictive maintenance systems, but also it allowed to identify improvement points of the DSS. A DSS is difficult to validate with a limited amount of data and with a limited number of implementations of the suggested models, but the success in the implementation of the SOM for health modelling of the N-CMAPSS case study indirectly validates the DSS capabilities. In the following and last chapter, the lessons learnt in the research are summarized in the conclusions and the perspectives of future work proposed.

Intentionally left blank

CHAPTER 8

Conclusion and perspective for future work

"No tengas miedo de la perfección,
nunca la alcanzarás."
"Have no fear of perfection,
you'll never reach it."

Salvador Dalí

Content

8.1 The journey is coming to an end, but a new one starts...

Research never ends but the current manuscript has already covered a wide field in the design of predictive maintenance systems and how the knowledge reuse from previous experiences can enhance the architecture process of such systems. Acknowledging that for the addressed topics there is still a long way to be covered, this chapter concludes the content of this manuscript by summarizing the lessons learnt, the limitations encountered, the research contributions, and more importantly the perspectives of future work that can serve as inspiration for future research lines.

8.2 Lessons learnt

To summarize the lessons learnt during this research it is necessary to come back to the research questions that initially motivated this work. In the introduction the first research questions was introduced:

1. What are the current trends in diagnostics and prognostics in predictive maintenance?
2. What are the main challenges in predictive maintenance?
3. What are the main research opportunities in the field of predictive maintenance?

These questions motivated an extensive literature review, that allowed to understand that predictive maintenance is carried out by specialized tools that incorporate models able to perform diagnostics and prognostics. The implementation of these models can be divided in two different approaches: single-model approaches and multi-model approaches. Single model approaches can be divided into three main model families: data-driven models, knowledge-based models and physics-based models. Multi-model approaches include at least two models from any of the mentioned model families. Depending on the multi-model approaches configuration, they can be called hybrid models. Recent trends in predictive maintenance push towards the implementation of multi-model approaches as one single model hardly addresses all the predictive maintenance functions for a complex system. The literature review allowed to determine the current challenges in predictive maintenance which include:

- Lack of a systematic method to design predictive maintenance systems.
- The extrapolation of existing solutions to complex system applications.
- The fusion of large and different sources of condition monitoring data.
- The incorporation of external influence data.
- Uncertainty management.

The main part of the challenges are focused in the concept stage and the design of new predictive maintenance systems. This motivated a refinement of the research questions:

1. How to address the design of predictive maintenance systems?
2. How to suggest a suitable approach for a predictive maintenance systems solution?
3. How to select a suitable model or combination of models given a new predictive maintenance system to solve?
4. How can a designer benefit from the experience of existing systems to develop new predictive maintenance solutions?

With regards to the first refined question, a systems engineering approach is proposed in the current research to address the concept stage of new predictive maintenance systems. This systematic approach starts by gathering all initial needs and desires from the stakeholders. The needs and desires are translated into a formal set of stakeholder requirements which are divided into four main categories: functional requirements, behavioural requirements, structural requirements and experiential requirements. The classified requirements are prioritized and later used at different phases of the system architecture process.

While developing this systems engineering approach, a challenge was identified at the logical architecture which also matches with the other refined research questions. The logical architecture shows as much detail as possible of the new system but without engaging it to any specific technology, meaning that the logical architecture still remains generic. The selection of predictive maintenance models to fulfil the logical components became the main scope of the current research. While performing the literature review, hundreds

of publications with successful cases of implemented predictive maintenance systems were consulted; the information from these implementations is not being efficiently used. The hypothesis was proposed: a Case-Based Reasoning (CBR) system can help an architect to retrieve efficiently a suitable model or set of models to perform a structured creativity process and fulfil the logical components in a new predictive maintenance system.

The CBR paradigm was selected as it offers means of reasoning to solve problems based on previous experiences. It is more flexible than rule-based reasoning and it is suitable for the current research as no specific rules exist to link the predictive models to a specific function in the system or to a specific case study. To develop a CBR system one of the most important aspects is a strong vocabulary basis from which the cases and the similarities can be modelled. For the current research, the state-of-the-art in vocabulary development for CBR was considered. It concerns ontologies, which are formal terminology representations that have a wide field of applications, especially in the semantic web. The research in ontologies allowed to identify an opportunity to make a contribution in the topic. Two Ontologies have been developed for the current research, the first one called OMSSA corresponds to a terminology framework to select and assess maintenance strategies. The second one, called OPMAD, is an OMSSA extension and covers the vocabulary involved in the design of predictive maintenance systems. OPMAD is the ontology that is used as a vocabulary framework for the proposed DSS.

It is important to mention that CBR and ontologies not always were implemented together. In recent years, both technologies have been increasingly used together because ontologies boost the use of the vocabulary basis for CBR applications. In the proposed framework, an ontology-enabled CBR recommendation system (a DSS) was developed to help the architect in the selection of suitable predictive maintenance models. A crossed validation helped to demonstrate that the proposed DSS was capable to suggest suitable components for specific predictive maintenance functions. To further validate the DSS, an implementation example was performed using the N-CMAPPS case study. The DSS proposed the Self-Organizing Maps (SOM) among the possible options to address health modelling. During the implementation of the SOM, it was possible not only to confirm the capabilities of the DSS but also to identify its improvement points. These improvement points are part of the perspectives of future works presented in this chapter.

In general, the research allowed to understand the principles of knowledge reuse with the CBR paradigm and how it can be used for architecture and design purposes. The implementation of the DSS allowed suggesting suitable components to fulfil generic components of a logical architecture of a predictive maintenance system.

8.3 Limitations encountered

As for any other research project, several limitations have been found on the way. This section aims at sharing part of the personal experience when performing the current research. Two main limitations have been faced. These are not specific for the current research but rather generic in the current context:

The first one and most important would be the limitation of time. It is understood that research never ends and at the end of any PhD thesis there will always remaining works to be done. For the current research, this is not an exception. Besides the normal workload that a PhD thesis represents, the time frame in which it was developed was also characterized by a global pandemic that forced to close the laboratories in France and all over the world. This affected the research schedule and forced the research objectives refinement. The research topic had to change in the middle of the second year and impacts due to the pandemic were considered to remake the research schedule.

The second limitation was related to the lack of access to practical examples that can be used to further

validate the DSS capabilities. There were conversations with different companies and groups able to provide practical examples that could have been included in this research validation. Bureaucracy and lack of interest from the external actors did not allow to have the case studies on time to be added to the current manuscript. Conversations with public companies in Costa Rica still undergoing so that the research can be continued in the near future.

Having faced these limitations enriches the research experience. It helps the researcher to be resilient and creative when facing problems out of their hands.

8.4 Contributions summary

The current manuscript is composed of four articles that have been published or submitted for being considered for publication in international journals and/or conferences. The main contributions of the PhD research have been consolidated in the articles as follows:

1. The review article summarized the state-of-the-art in diagnostics and prognostics. A proposal to differentiate hybrid models from multi-model approaches has been provided. The tendencies towards the implementation of multi-model approaches have been pointed out. By the time this manuscript was finished, the review article was cited more than thirty times. The literature review can be seen as a scientific contribution as it provides a starting point for those researchers interested in predictive maintenance.

2. The systems engineering approach to predictive maintenance systems design has been the first scientific contribution published in an international conference during the PhD development. In contrast to existing generic architectures for the design of predictive maintenance systems, the proposed systematic approach addresses the concept stage of predictive maintenance systems from the initial needs and desires until the logical architecture. It helps the architect to accurately determine the components of the system and keep traceability with the initial needs and desires obtained from the stakeholders.

3. Even if ontologies were not the main scope of the current research, a fair amount of research work has been performed in the domain and it has produced its own research outcomes. The creation of OMSSA, an ontology model for maintenance strategy selection and assessment provides a terminology framework that can be used by smart agents to automate the complex tasks of maintenance strategies management. An extension to OMSSA (OPMAD) is the ontology used to create the case base and the similarity measures of the DSS for predictive maintenance component selection. The research work performed in creating OMSSA has been consolidated in a journal article accepted for publication at the moment this manuscript was written.

4. The main scope of the current research has produced a decision support system able to query successful cases of predictive maintenance system implementations that can help a systems architect to select suitable predictive maintenance models to fulfil diagnostics and prognostics tasks. The first outcomes of this research have been published in a conference paper which has been published at an international conference while this manuscript was being completed.

8.5 Perspectives of future work

While working in research it is not possible to explain and prove everything. In order to accomplish the research objectives in a fixed time frame, some of the intermediate research steps have to be simplified and sometimes pragmatic decisions are made. When accomplishing the results, it is important to put things in perspective to spot the improvement points and list them as perspectives of future work. Research never ends, and one of the main parts of any PhD manuscript is to indicate what was missing and expected to be done in order to continue the research line. The perspectives of future work are organized in three main groups:

1. Perspectives related to the systems engineering approach to predictive maintenance systems design
2. Perspectives related to the ontologies developed in the current research
3. Perspective related to the ontology-enabled case-based recommendation system for predictive maintenance component selection.

8.5.1 Perspectives related to the systems engineering approach to predictive maintenance systems design

- Refine the list of predictive maintenance system needs, desires and requirements: after publishing the article related to the concept stage of predictive maintenance systems several needs, desires and requirements were identified that were not originally considered. The list of possible needs, desires and requirements for new predictive maintenance systems can be improved. These lists can be stored in a knowledge base that can be used by automated smart agents to help the engineers assess the initial needs and desires for a predictive maintenance system and suggest the corresponding stakeholder requirements. A DSS could be developed to define the correct stakeholder requirements based on the initial needs and desires for the new predictive maintenance system.
- Include trade-off analysis technique in the systematic framework: Trade-off analyses are present in several points during the concept stage. Due to time constraints, it was not addressed in this thesis. The integration of a trade-off analysis tool can help to improve the DSS and facilitate the selection of the components of the architecture.
- Include the formalization of system requirements: an important aspect of a systems engineering approach is the elicitation of system requirements. Even when the creation of these requirements is in theory before the architecture process, in practice these requirements are normally established in parallel with the development of the system architecture or even at the end when all the performances for the different logical components are known. These requirements are important for the detailed design stage. The elicitation of these requirements was out of the scope of the current research but it is an interesting complementary topic for future research.

8.5.2 OMSSA and OPMAD perspectives

- Refine classes and relations: ontology development is a fast evolving topic. There are several initiatives to provide top-level and mid-level ontologies in different domains. The industrial domain is not the exception. Further work will be needed to align OMSSA and OPMAD to these standardized ontologies so that to boost their integration and reuse in other applications. This work includes the refinement of classes and relations among them.

- Extend reasoning in the ontology: OMSSA and OPMAD have been useful in the current research but as ontologies, they have been underutilised. Ontologies have several capabilities that were not exploited. One of them is the implementation of semantic rules. This extends the reasoning options and can be used as a complementary means to the CBR system.

8.5.3 Ontology-enabled case-based reasoning system perspectives

- Expand the use of the ontology for more local similarity functions: as it was already mentioned, the ontology has been useful but underutilised. Some other similarity measures that have been assigned a binary symbol similarity measure can be rearranged to use ontology-based similarities.
- Use the ontology to estimate the weights for the global similarity computation: related to the previous remark. the ontology can also be used to compute the weights of each local similarity when computing the global similarity. In the first attempt of global similarity computation performed in this research, all local similarities received the same weight. An improvement in the global similarity computation could be by giving a rank of importance to each local similarity. The weights can be computed using the populated ontology.
- Expand the Decision Support System by developing other phases of the CBR cycle, such as the adaptation phase: The scope for the DSS was on the retrieval phase of the CBR cycle. The system can be expanded to facilitate the adaptation phase of the suggested components for the new predictive maintenance system. This extension can also include the trade-off analysis on different components suggested by the DSS. The results of the current research shown several improvement opportunities that should be addressed in the future before eventual deployment of the DSS.
- Refine the instantiation process to avoid diversity problems but keeping all the solution space covered: As it was mentioned in the validation chapter, the instantiation of the case base should be refined. An improvement opportunity can be to divide the solution space according to the different predictive maintenance functions and make sure that the solution space is fully covered for each of them. Generalized cases for each predictive maintenance function can solve the problem of diversity in the retrieval cases.
- Trade-off analysis to better select the technique among the proposed ones: it was already mentioned as the perspective of future work for the systematic approach to design predictive maintenance systems. Regarding the DSS, the trade-off analysis can help the architect discriminate among the suggested components by the DSS. In the validation it was mentioned that sometimes the DSS suggested two different components with the same similarity. Adding some extra attributes such as performance indicators can help the architect make the right decision for their problem.
- Compare against other techniques for DSS: In a first attempt, the objective of the current research was to prove that case-based reasoning can be combined with ontologies to develop a Decision Support System for component selection for new predictive maintenance systems and it worked. Further research would be dedicated to assess the performance of the proposed DSS against other technologies that can be used for the same purposes such as machine learning algorithms. Such comparison was not considered as it exceeded the scope of the current research.

8.6 Epilogue

The content of the current manuscript has come to an end. It attempted to summarize the research experiences accumulated over the last three years specifically in the topic of predictive maintenance systems design, an interesting topic that still leaves a lot challenges to be solved. A systematic methodology to address the concept phase of such systems has been proposed and within this framework a more specific challenge was identified for suitable components selection. A Decision Support System composed of an ontology-enabled cased-based reasoning retrieval engine has been proposed to overcome the challenge of suitable component selection based on past experiences. The results demonstrated the capabilities of the proposed DSS but also allowed to spot important improvement opportunities that should be considered in future research.

Intentionally left blank

Bibliography

[AA10] Edgar J. Amaya and Alberto J. Alvares. "SIMPREBAL: An expert system for real-time fault diagnosis of hydrogenerators machinery". In: *Proceedings of the 15th IEEE International Conference on Emerging Technologies and Factory Automation, ETFA 2010*. 2010. ISBN: 9781424468508. DOI: 10.1109/ETFA.2010.5641302.

[AGC18] Panagiotis Aivaliotis, Konstantinos Georgoulias, and George Chryssolouris. "A RUL calculation approach based on physical-based simulation models for predictive maintenance". In: *2017 International Conference on Engineering, Technology and Innovation: Engineering, Technology and Innovation Management Beyond 2020: New Challenges, New Approaches, ICE/ITMC 2017 - Proceedings*. 2018. ISBN: 9781538607749. DOI: 10.1109/ICE.2017.8280022.

[AH17] Sylvestre A. Aye and Philippus S. Heyns. "An integrated Gaussian process regression for prediction of remaining useful life of slow speed bearings based on acoustic emission". In: *Mechanical Systems and Signal Processing* 84.A (2017), pp. 485–498. ISSN: 10961216. DOI: 10.1016/j.ymssp.2016.07.039.

[AH18] Sylvester A. Aye and Stephan Heyns. "Prognostics of slow speed bearings using a composite integrated Gaussian process regression model". In: *International Journal of Production Research* 56.14 (2018), pp. 4860–4873. ISSN: 1366588X. DOI: 10.1080/00207543.2018.1470340.

[AIC18] Ronke M. Ayo-Imoru and Anthonie C. Cilliers. "Continuous machine learning for abnormality identification to aid condition-based maintenance in nuclear power plant". In: *Annals of Nuclear Energy* 118 (2018), pp. 61–70. ISSN: 18732100. DOI: 10.1016/j.anucene.2018.04.002.

[AKC15] Dawn An, Nam H. Kim, and Joo Ho Choi. "Practical options for selecting data-driven or physics-based prognostics algorithms with reviews". In: *Reliability Engineering and System Safety* 133 (2015), pp. 223–236. ISSN: 09518320. DOI: 10.1016/j.ress.2014.09.014.

[AL13] Khaled Amailef and Jie Lu. "Ontology-supported case-based reasoning approach for intelligent m-Government emergency response services". In: *Decision Support Systems* 55 (2013), pp. 79–97. ISSN: 01679236. DOI: 10.1016/j.dss.2012.12.034.

[Alt+12] Klaus Althoff et al. *Documentation: myCBR*. 2012. URL: http://www.mycbr-project.org/3.1-doc/index.html.

[Ant+12] Grigoris Antoniou et al. *A Semantic Web Primer*. Third edit. MIT Press, 2012. ISBN: 9780262018289.

[AP94] Agnar Aamodt and Enric Plaza. "Case-Based reasoning: Foundational issues, methodological variations, and system approaches". In: *AI Communications* 7.1 (1994), pp. 39–59. ISSN: 09217126. DOI: 10.3233/AIC-1994-7104.

[ASF14] Pekka Aarnio, Ilkka Seilonen, and Mats Friman. "Semantic repository for case-based reasoning in CBM services". In: *19th IEEE International Conference on Emerging Technologies and Factory Automation, ETFA 2014*. 2014. ISBN: 9781479948468. DOI: 10.1109/ETFA.2014.7005195.

[ASS16] Robert Arp, Barry Smith, and Andrew D. Spear. *Building Ontologies with Basic Formal Ontology*. 2016. DOI: 10.7551/mitpress/9780262527811.001.0001.

[AX17] Suzan Alaswad and Yisha Xiang. "A review on condition-based maintenance optimization models for stochastically deteriorating system". In: *Reliability Engineering and System Safety* 157 (2017), pp. 54–63. ISSN: 09518320. DOI: 10.1016/j.ress.2016.08.009.

[Bag+15] Behrad Bagheri et al. “A Stochastic Asset Life Prediction Method for Large Fleet Datasets in Big Data Environment”. In: *ASME 2015 International Mechanical Engineering Congress and Exposition. Volume 14: Emerging Technologies; Safety Engineering and Risk Analysis; Materials: Genetics to Structures*. 2015. ISBN: 978-0-7918-5757-1. DOI: 10.1115/IMECE2015-52458.

[Bai+15] Chris Bailey et al. “Prognostic and health management for engineering systems: a review of the data-driven approach and algorithms”. In: *The Journal of Engineering* 2015.7 (2015), pp. 215–222. ISSN: 2051-3305. DOI: 10.1049/joe.2014.0303.

[Baj+09] Gautam Bajracharya et al. “Optimization of maintenance for power system equipment using a predictive health model”. In: *2009 IEEE Bucharest PowerTech: Innovative Ideas Toward the Electrical Grid of the Future*. 2009. ISBN: 9781424422357. DOI: 10.1109/PTC.2009.5281928.

[BAJ17] Oguz Bektas, Amjad Alfudail, and Jeffrey A. Jones. “Reducing Dimensionality of Multi-regime Data for Failure Prognostics”. In: *Journal of Failure Analysis and Prevention* 17 (2017), pp. 1268–1275. ISSN: 15477029. DOI: 10.1007/s11668-017-0368-2.

[BB09] Donald W Benbow and Hugh W Broome. *The Certified Reliability Engineer Handbook*. 2009. ISBN: 978-0-87389-721-1. DOI: 10.1017/CBO9781107415324.004. arXiv: arXiv:1011.1669v3.

[BB+17] Diana Barraza-Barraza et al. “An adaptive ARX model to estimate the RUL of aluminum plates based on its crack growth”. In: *Mechanical Systems and Signal Processing* 82 (2017), pp. 519–536. ISSN: 10961216. DOI: 10.1016/j.ymssp.2016.05.041.

[BB18] Toufik Berredjem and Mohamed Benidir. “Bearing faults diagnosis using fuzzy expert system relying on an Improved Range Overlaps and Similarity method”. In: *Expert Systems with Applications* 108 (2018), pp. 134–142. ISSN: 09574174. DOI: 10.1016/j.eswa.2018.04.025.

[BBM18] Marius Baban, Calin Florin Baban, and Beniamin Moisi. “A Fuzzy Logic-Based Approach for Predictive Maintenance of Grinding Wheels of Automated Grinding Lines”. In: *2018 23rd International Conference on Methods and Models in Automation and Robotics, MMAR 2018*. 2018. ISBN: 9781538643259. DOI: 10.1109/MMAR.2018.8486144.

[Ber+01] Ralph Bergmann et al. “Utility-Oriented Matching: A New Research Direction for Case-Based Reasoning”. In: *Proceedings of the Ninth German Workshop on Case-Based Reasoning*. 2001.

[Ber+16] Maitane Berecibar et al. “Online state of health estimation on NMC cells based on predictive analytics”. In: *Journal of Power Sources* (2016). ISSN: 03787753. DOI: 10.1016/j.jpowsour.2016.04.109.

[BKP05] Ralph Bergmann, Janet Kolodner, and Enric Plaza. “Representation in case-based reasoning”. In: *Knowledge Engineering Review* 20.3 (2005), pp. 209–213. ISSN: 02698889. DOI: 10.1017/S0269888906000555.

[Bos+13] Kosta P. Boshnakov et al. “Predictive maintenance model-based approach for objects exposed to extremely high temperatures”. In: *2013 Signal Processing Symposium, SPS 2013*. 2013. ISBN: 9781467363198. DOI: 10.1109/SPS.2013.6623621.

[Bra+18] Frances Brazier et al. “Design, Engineering and Governance of Complex Systems”. In: *Projects and People - Mastering success*. Ed. by H.L.M. Bakker and J.P. Kleynen. NAP Foundation Press, 2018, pp. 34–59.

[But96] Karen L. Butler. “An Expert System Based Framework for an Incipient Failure Detection and Predictive Maintenance System”. In: *Proceeding of the Int Conf on Intelligent Sys Application to Power Sys* (1996). ISSN: 0020-7543. DOI: 10.1080/002075400188933. arXiv: arXiv:1011.1669v3.

[BW96] Ralph Bergmann and Wolfgang Wilke. “On the role of abstraction in case-based reasoning”. In: *Proceeding of the 3rd European Workshop on Case-Based Reasoning.* Berlin: Springer, 1996, pp. 29–43.

[CA+17] Vicente Climente-Alarcon et al. “Combined Model for Simulating the Effect of Transients on a Damaged Rotor Cage”. In: *IEEE Transactions on Industry Applications* (2017). ISSN: 00939994. DOI: 10.1109/TIA.2017.2691001.

[Cas+20] Fernando Castaño et al. “Quality monitoring of complex manufacturing systems on the basis of model driven approach”. In: *Smart Structures and Systems* 26.4 (2020), pp. 495–506. ISSN: 17381991. DOI: 10.12989/sss.2020.26.4.495. URL: http://techno-press.org/content/?page=article{\&}journal=sss{\&}volume=26{\&}num=4{\&}ordernum=7.

[CCN19] Duan Chaoqun, Deng Chao, and Li Ning. “Reliability assessment for CNC equipment based on degradation data”. In: *The International Journal of Advanced Manufacturing Technology* 100 (2019), pp. 421–434.

[CCS15] Edward Crawley, Bruce Cameron, and Daniel Selva. *System Architecture: Strategy and Product Development for Complex Systems*. Pearson Higher Education, Inc., 2015.

[Cer+16] Mariela Cerrada et al. “Fault diagnosis in spur gears based on genetic algorithm and random forest”. In: *Mechanical Systems and Signal Processing* 70-71 (2016), pp. 87–103. ISSN: 10961216. DOI: 10.1016/j.ymssp.2015.08.030.

[CFZ17] Yang Chang, Huajing Fang, and Yong Zhang. “A new hybrid method for the prediction of the remaining useful life of a lithium-ion battery”. In: *Applied Energy* 206 (2017), pp. 1564–1578. ISSN: 03062619. DOI: 10.1016/j.apenergy.2017.09.106.

[CH11] Jamie B. Coble and Wesley Hines. “Applying the General Path Model to Estimation of Remaining Useful Life”. In: *International Journal of Prognostics and Health Management* 2.1 (2011), p. 13. ISSN: 21532648.

[Cha+21] Manuel Arias Chao et al. “Aircraft Engine Run-to-Failure Dataset under Real Flight Conditions for Prognostics and Diagnostics”. In: *Data 2021, Vol. 6, Page 5* 6.1 (2021), p. 5. DOI: 10.3390/DATA6010005. URL: https://www.mdpi.com/2306-5729/6/1/5/htmhttps://www.mdpi.com/2306-5729/6/1/5.

[Che+16] Peter Chemweno et al. “I-RCAM: Intelligent expert system for root cause analysis in maintenance decision making”. In: *2016 IEEE International Conference on Prognostics and Health Management, ICPHM 2016*. 2016. ISBN: 9781509003822. DOI: 10.1109/ICPHM.2016.7542830.

[Che+17] Zhen Chen et al. “Hidden Markov model with auto-correlated observations for remaining useful life prediction and optimal maintenance policy”. In: *Reliability Engineering and System Safety* 184 (2017), pp. 123–136. ISSN: 09518320. DOI: 10.1016/j.ress.2017.09.002.

[Chi+19] Juan Chiachío et al. “A knowledge-based prognostics framework for railway track geometry degradation”. In: *Reliability Engineering and System Safety* 181 (2019), pp. 127–141. ISSN: 09518320. DOI: 10.1016/j.ress.2018.07.004.

[Cho+19] Michael E. Cholette et al. “Degradation modeling and condition-based maintenance of boiler heat exchangers using gamma processes”. In: *Reliability Engineering and System Safety* 183 (2019), pp. 184–196.

[CNY10] Wahyu Caesarendra, Gang Niu, and Bo Suk Yang. “Machine condition prognosis based on sequential Monte Carlo method”. In: *Expert Systems with Applications* 37.3 (2010), pp. 2412–2420. ISSN: 09574174. DOI: 10.1016/j.eswa.2009.07.014.

[CRW08] Jiehua Chen, Clive Roberts, and Paul Weston. "Fault detection and diagnosis for railway track circuits using neuro-fuzzy systems". In: *Control Engineering Practice* 16 (2008), pp. 585–596. ISSN: 09670661. DOI: 10.1016/j.conengprac.2007.06.007.

[CYC12] Cong Cheng, Ling Yu, and Liu Jie Chen. "Structural nonlinear damage detection based on ARMA-GARCH model". In: *Applied Mechanics and Materials* 204-208 (2012), pp. 2891–2896. ISSN: 16609336. DOI: 10.4028/www.scientific.net/AMM.204-208.2891.

[CZMR19] Qiushi Cao, Cecilia Zanni-Merk, and Christoph Reich. "Towards a core ontology for condition monitoring". In: *Procedia Manufacturing*. 2019. DOI: 10.1016/j.promfg.2018.12.029.

[De +05] Ramon Lopez De Mantaras et al. "Retrieval, reuse, revision and retention in case-based reasoning". In: *The Knowledge Engineering Review* 20.3 (2005), pp. 215–240. ISSN: 02698889. DOI: 10.1017/S0269888906000646.

[De +18] Massimiliano De Benedetti et al. "Anomaly detection and predictive maintenance for photovoltaic systems". In: *Neurocomputing* 310 (2018), pp. 59–68. ISSN: 18728286. DOI: 10.1016/j.neucom.2018.05.017.

[DHK13] Nadjette Dendani-Hadiby and M. Tarek Khadir. "A fault diagnosis application based on a combination case-based reasoning and ontology approach". In: *International Journal of Knowledge-Based and Intelligent Engineering Systems* 17.4 (2013), pp. 305–317. ISSN: 13272314. DOI: 10.3233/KES-130280.

[DLS17] Dong Dong, Xiao Yang Li, and Fu Qiang Sun. "Life prediction of jet engines based on LSTM-recurrent neural networks". In: *2017 Prognostics and System Health Management Conference, PHM-Harbin 2017 - Proceedings*. 2017. ISBN: 9781538603703. DOI: 10.1109/PHM.2017.8079264. arXiv: arXiv:1512.04143v1.

[DMB16] Landon T. Detwiler, Jose L.V. Mejino, and James F. Brinkley. "From frames to OWL2: Converting the Foundational Model of Anatomy". In: *Artificial Intelligence in Medicine* 69 (2016), pp. 12–21. ISSN: 18732860. DOI: 10.1016/j.artmed.2016.04.003.

[DMD18] Chaoqun Duan, Viliam Makis, and Chao Deng. "An integrated framework for health measures prediction and optimal maintenance policy for mechanical systems using a proportional hazards model". In: *Mechanical Systems and Signal Processing* 111 (2018), pp. 285–302. ISSN: 10961216. DOI: 10.1016/j.ymssp.2018.02.029.

[Dow+19] Austin Downey et al. "Physics-based prognostics of lithium-ion battery using non-linear least squares with dynamic bounds". In: *Reliability Engineering and System Safety* 182 (2019), pp. 1–12.

[Dra+09] Otilia Elena Dragomir et al. "Review of prognostic problem in condition-based maintenance". In: *European Control Conference, (ECC'09)*. 2009. ISBN: 9783952417393. DOI: 10.1128/JB.00591-09.

[DS18] Samalis Santini De León and Daniel Selva. "A rule-based tool for science traceability of Mars exploration mission architectures". In: *IEEE Aerospace Conference Proceedings*. 2018. ISBN: 9781538620144. DOI: 10.1109/AERO.2018.8396804.

[DZD14] Xiaofei Du, Yuanjun Zhou, and Shiliang Dong. "Residual life prediction from statistical features and a GARCH modeling approach for aircraft generators". In: *Proceedings of the Institution of Mechanical Engineers, Part G: Journal of Aerospace Engineering* 228.1 (2014), pp. 137–146. ISSN: 20413025. DOI: 10.1177/0954410012472838.

[ECJ14] Omer Faruk Eker, Fatih Camci, and Ian K Jennions. "A Similarity-based Prognostics Approach for Remaining Useful Life Prediction". In: *Second European Conference of the Prognostics and Health Management Society 2014*. 2014.

[ECJ16] Omer F. Eker, Fatih Camci, and Ian K. Jennions. "Physics-based prognostic modelling of filter clogging phenomena". In: *Mechanical Systems and Signal Processing* 75 (2016), pp. 395–412. ISSN: 10961216. DOI: 10.1016/j.ymssp.2015.12.011.

[EER16] Hatem M. Elattar, Hamdy K. Elminir, and Alaa el-din Mohamed Riad. "Prognostics: a literature review". In: *Complex & Intelligent Systems* 2.2 (2016), pp. 125–154. ISSN: 2199-4536. DOI: 10.1007/s40747-016-0019-3.

[ENM18] Ikuobase Emovon, Rosemary A. Norman, and Alan J. Murphy. "Hybrid MCDM based methodology for selecting the optimum maintenance strategy for ship machinery systems". In: *Journal of Intelligent Manufacturing* 29 (2018), pp. 519–531. ISSN: 15728145. DOI: 10.1007/s10845-015-1133-6.

[ESE15] Shaker El-Sappagh and Mohammed Elmogy. "Case Based Reasoning: Case Representation Methodologies". In: *International Journal of Advanced Computer Science and Applications* 6.11 (2015), pp. 192–208. ISSN: 2158107X. DOI: 10.14569/ijacsa.2015.061126.

[Eur09] European standard NF EN 13306X60-319. *Maintenance — Terminologie de la maintenance*. 2009.

[Eur17] European Committee for Standardization. *CEN EN 13306: Maintenance-Maintenance terminology*. 2017.

[FDL07] Dean K Frederick, Jonathan DeCastro, and Jonathan Litt. *User's guide for the Commercial Modular Aero-Propulsion System Simulation (C-MAPSS)*. Tech. rep. Washington, DC: NASA, 2007.

[Fer+11] Miriam Fernández et al. "Semantically enhanced Information Retrieval: An ontology-based approach". In: *Journal of Web Semantics* 9.1 (2011), pp. 434–452. ISSN: 15708268. DOI: 10.1016/j.websem.2010.11.003.

[Fer+18] Borja Ramis Ferrer et al. "Towards the Adoption of Cyber-Physical Systems of Systems Paradigm in Smart Manufacturing Environments". In: *Proceedings - IEEE 16th International Conference on Industrial Informatics, INDIN 2018*. 2018. ISBN: 9781538648292. DOI: 10.1109/INDIN.2018.8472061.

[FGPJ97] Mariano Fernandez, Asuncion. Gómez-Pérez, and Natalia Juristo. "Methontology: from ontological art towards ontological engineering". In: *Proceedings of the AAAI97 Spring Symposium Series on Ontological Engineering*. 1997.

[Fou20] The OBO Foundry. *Relation Ontology (RO)*. 2020. URL: http://www.obofoundry.org/ontology/ro.html (visited on 07/22/2020).

[Fra57] Alexander S. Fraser. "Simulation of Genetic Systems by Automatic Digital Computers I". In: *Australian Journal of Biological Science* 10 (1957), pp. 484–491.

[Fre91] Bernd Freyermuth. "Knowledge based incipient fault diagnosis of industrial robots". In: *IFAC Proceedings Volumes (IFAC-PapersOnline)* 24.6 (1991), pp. 369–375.

[FZ15] Liu Fang and Huang Zhaodong. "System Dynamics Based Simulation Approach on Corrective Maintenance Cost of Aviation Equipments". In: *Procedia Engineering*. Vol. 99. Elsevier Ltd, 2015, pp. 150–155. DOI: 10.1016/j.proeng.2014.12.519.

[Gan+15] Ma Gang et al. "A model of intelligent fault diagnosis of power equipment based on CBR". In: *Mathematical progblems in engineering* Article ID 203083 (2015).

[Ger90] John S. Gero. "Design Prototypes: A knowledge Representation Schema for Design". In: *AI Magazine* 11.4 (1990), pp. 26–36.

[GF95] Michael Grüninger and Mark Stephen Fox. "Methodology for the Design and Evaluation of Ontologies". In: *International Joint Conference on Artificial Inteligence (IJCAI95), Workshop on Basic Ontological Issues in Knowledge Sharing*. 1995. DOI: citeulike-article-id:1273832.

[GK07] John S. Gero and Udo Kannengiesser. “A function-behavior-structure ontology of processes”. In: *Artificial Intelligence for Engineering Design, Analysis and Manufacturing: AIEDAM*. 2007. DOI: 10.1017/S0890060407000340.

[GMZ16] Rafael Gouriveau, Kamal Medjaher, and Noureddine Zerhouni. *From Prognostics and Health Systems Management to Predictive Maintenance 1: Monitoring and Prognostics*. 2016. ISBN: 9781119371052. DOI: 10.1002/9781119371052.

[Goy+16] Deepam Goyal et al. “Intelligent predictive maintenance of dynamic systems using condition monitoring and signal processing techniques-A review”. In: *Proceedings - 2016 International Conference on Advances in Computing, Communication and Automation, ICACCA 2016*. 2016. ISBN: 9781509006731. DOI: 10.1109/ICACCA.2016.7578870.

[GPH13] Yuan Guo, Yinghong Peng, and Jie Hu. “Research on high creative application of case-based reasoning system on engineering design”. In: *Computers in Industry* 64.1 (2013), pp. 90–103. ISSN: 0166-3615. DOI: 10.1016/J.COMPIND.2012.10.006.

[Gru93] Thomas R. Gruber. “A translation approach to portable ontology specifications”. In: *Knowledge Acquisition* 5.2 (1993), pp. 199–220. ISSN: 10428143. DOI: 10.1006/knac.1993.1008.

[Guo+17] Liang Guo et al. “A recurrent neural network based health indicator for remaining useful life prediction of bearings”. In: *Neurocomputing* 240 (2017), pp. 98–109. ISSN: 18728286. DOI: 10.1016/j.neucom.2017.02.045.

[GV17] Jakub Gajewski and David Vališ. “The determination of combustion engine condition and reliability using oil analysis by MLP and RBF neural networks”. In: *Tribology International* 115 (2017), pp. 557–572. ISSN: 0301679X. DOI: 10.1016/j.triboint.2017.06.032.

[Han+19] Houman Hanachi et al. “Hybrid sequential fault estimation for multi-mode diagnosis of gas turbine engines”. In: *Mechanical Systems and Signal Processing* 115 (2019), pp. 225–268. ISSN: 10961216. DOI: 10.1016/j.ymssp.2018.05.054.

[HB11] Matthew Horridge and Sean Bechhofer. “The OWL API: A Java API for OWL ontologies”. In: *Semantic Web* 2.1 (2011), pp. 11–21. ISSN: 15700844. DOI: 10.3233/SW-2011-0025.

[HLC14] Bahareh Rahmanzadeh Heravi, Mark Lycett, and Sergio de Cesare. “Ontology-based standards development: Application of OntoStanD to ebXML business process specification schema”. In: *International Journal of Accounting Information Systems* 15.3 (2014), pp. 275–297. ISSN: 14670895. DOI: 10.1016/j.accinf.2014.01.005.

[HLP08] Frank van Harmelen, Vladimir Lifschitz, and Bruce Porter. *Handbook of Knowledge Representation*. 2008. ISBN: 9780444522115. DOI: 10.1016/S1574-6526(07)03013-1.

[Hod+21] Melinda Hodkiewicz et al. “Rethinking Maintenance Terminology for an Industry 4.0 Future”. In: *International Journal of Prognostics and Health Management* 2021.1 (2021), p. 14. URL: https://www.phmsociety.org/node/2794.

[Hoe09] Rinke Hoekstra. *Ontology representation: Design patterns and ontologies that make sense*. IOS Press, 2009. ISBN: 9781607500131. DOI: 10.3233/978-1-60750-013-1-i.

[HSC18] Moinul Shaidul Haque, Mohammad Noor Bin Shaheed, and Seungdeog Choi. “RUL Estimation of Power Semiconductor Switch using Evolutionary Time series Prediction”. In: *2018 IEEE Transportation and Electrification Conference and Expo, ITEC 2018*. 2018. ISBN: 9781538630488. DOI: 10.1109/ITEC.2018.8450131.

[HT18] Ahmed Zakariae Hinchi and Mohamed Tkiouat. “Rolling element bearing remaining useful life estimation based on a convolutional long-short-Term memory network”. In: *Procedia Computer Science* 127 (2018), pp. 123–132. ISSN: 18770509. DOI: 10.1016/j.procs.2018.01.106.

[Hu+12] Chao Hu et al. “Ensemble of data-driven prognostic algorithms for robust prediction of remaining useful life”. In: *Reliability Engineering and System Safety* 103 (2012), pp. 120–135. ISSN: 09518320. DOI: 10.1016/j.ress.2012.03.008.

[Hu+18] Ya-Wei Hu et al. “Sequential Monte Carlo Method Toward Online RUL Assessment with Applications”. In: *Chinese Journal of Mechanical Engineering* 31.5. https://doi.org/10.1186/s10033-018-0205-x (2018). ISSN: 1000-9345. DOI: 10.1186/s10033-018-0205-x.

[Hus+15] Akhtar Hussain et al. “An expert system for acoustic diagnosis of power circuit breakers and on-load tap changers”. In: *Expert Systems with Applications* 42.24 (2015), pp. 9426–9433. ISSN: 09574174. DOI: 10.1016/j.eswa.2015.07.079.

[INC15] INCOSE. *Systems Engineering Handbook. A guide for system life cycle processes and activities. Fourth Edition.* Wiley, 2015.

[Int03] International Organization for Standardization (ISO). *ISO 13374-1:2003 Condition monitoring and diagnostics of machines — Data processing, communication and presentation — Part 1: General guidelines*. 2003.

[Int12a] International Organization for Standardization (ISO). *ISO 13372 - Condition monitoring and diagnostics of machines Vocabulary*. 2012.

[Int12b] International Organization for Standardization (ISO). *ISO 13379-1:2012 - Condition monitoring and diagnostics of machines Data interpretation and diagnostics techniques Part 1: General guidelines*. 2012.

[Int18] International Electrotechnical Commission (IEC). *IEC60812, Analysis techniques for system reliability- Procedure for failure mode and effects analysis (FMECA)*. 2018.

[Int20a] International Organization for Standardization (ISO). *ISO/IEC 21838-1 Information technology - Top-level Ontologies (TLO) - Part 1: Requirements*. 2020.

[Int20b] International Organization for Standardization (ISO). *ISO/IEC 21838-2 - Information technology - Top-level ontologies (TLO) - Part 2: Basic Formal Ontology (BFO)*. 2020.

[Int97] International Organization for Standardization (ISO). *Information technology — Vocabulary — Part 14: Reliability, maintainability and availability*. 1997.

[IOF20] IOF. *Industrial Ontology Foundry*. 2020. URL: https://www.industrialontologies.org/?page{_}id=164 (visited on 10/06/2020).

[ISO11] ISO/IEC/IEE. *42010:2011 Systems and Software Engineering — Architectural Description*. Ed. by ISO/IEC 42010:2011 International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC). 2011.

[JGZ17] Kamran Javed, Rafael Gouriveau, and Noureddine Zerhouni. “State of the art and taxonomy of prognostics approaches, trends of prognostics applications and open issues towards maturity at different technology readiness levels”. In: *Mechanical Systems and Signal Processing* 94 (2017), pp. 214–236. ISSN: 10961216. DOI: 10.1016/j.ymssp.2017.01.050.

[Jin+15] Wenjing Jin et al. “Development and evaluation of health monitoring techniques for railway point machines”. In: *2015 IEEE Conference on Prognostics and Health Management: Enhancing Safety, Efficiency, Availability, and Effectiveness of Systems Through PHAf Technology and Application, PHM 2015*. 2015. ISBN: 9781479918935. DOI: 10.1109/ICPHM.2015.7245016.

[JLB06] Andrew K.S. Jardine, Daming Lin, and Dragan Banjevic. “A review on machinery diagnostics and prognostics implementing condition-based maintenance”. In: *Mechanical Systems and Signal Processing* 20.7 (2006), pp. 1483–1510. ISSN: 08883270. DOI: 10.1016/j.ymssp.2005.09.012. arXiv: 0208024 [gr-qc].

[Kar+19] Mohamed Hedi Karray et al. “ROMAIN: Towards a BFO compliant reference ontology for industrial maintenance”. In: *Applied Ontology* 14.2 (2019), pp. 155–177. ISSN: 18758533. DOI: 10.3233/AO-190208.

[KCL18] Dongdong Kong, Yongjie Chen, and Ning Li. “Gaussian process regression for tool wear prediction”. In: *Mechanical Systems and Signal Processing* 104 (2018), pp. 556–574. ISSN: 10961216. DOI: 10.1016/j.ymssp.2017.11.021.

[KCMZ12] Mohamed Hedi Karray, Brigitte Chebel-Morello, and Noureddine Zerhouni. “A formal ontology for industrial maintenance”. In: *Applied Ontology* 7.3 (2012), pp. 1–20. ISSN: 15705838. DOI: 10.3233/AO-2012-0112.

[KCMZ15] Racha Khelif, Brigitte Chebel-Morello, and Noureddine Zerhouni. “Experience Based Approach for Li-ion Batteries RUL Prediction”. In: *IFAC-PapersOnLine*. 2015. DOI: 10.1016/j.ifacol.2015.06.174.

[Kee18] Maria Keet. *An Introduction to Ontology Engineering*. Texts in computing Vol 20, 2018, p. 289.

[Kel+01] Kirby Keller et al. “A process and tool for determining the cost/benefit of prognostic applications”. In: *AUTOTESTCON (Proceedings)*. 2001. DOI: 10.1109/autest.2001.949432.

[KH07] Ranganath Kothamasu and Samuel H. Huang. “Adaptive Mamdani fuzzy model for condition-based maintenance”. In: *Fuzzy Sets and Systems* 158.24 (2007), pp. 2715–2733. ISSN: 01650114. DOI: 10.1016/j.fss.2007.07.004.

[Kin+18] Jakob Kinghorst et al. “Hidden Markov model-based predictive maintenance in semiconductor manufacturing: A genetic algorithm approach”. In: *IEEE International Conference on Automation Science and Engineering*. 2018. ISBN: 9781509067800. DOI: 10.1109/COASE.2017.8256274.

[KKL12] Kiyoshi Kobayashi, Kiyoyuki Kaito, and Nam Lethanh. “A statistical deterioration forecasting method using hidden Markov model for infrastructure management”. In: *Transportation Research Part B: Methodological* 46.4 (2012), pp. 544–561. ISSN: 01912615. DOI: 10.1016/j.trb.2011.11.008.

[Kol93] Janet Kolodner. *Case based reasoning*. Morgan Kaufmann, 1993, p. 612.

[Kol94] Janet L. Kolodner. “Understanding creativity: A case-based approach”. In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. 1994. ISBN: 9783540583301. DOI: 10.1007/3-540-58330-0_73.

[Kot+06] Ranganath Kothamasu et al. “System health monitoring and prognostics -a review of current paradigms and practices”. In: *International Journal of Advanced Manufacturing Technology* 28.9 (2006), pp. 1012–1024. ISSN: 02683768. DOI: 10.1007/978-1-84882-472-0_14. arXiv: arXiv:1011.1669v3.

[KRH02] Gregory J. Kacprzynski, Michael J. Roemer, and Andrew J. Hess. “Health management system design: Development, simulation and cost/benefit optimization”. In: *IEEE Aerospace Conference Proceedings*. 2002. ISBN: 078037231X. DOI: 10.1109/AERO.2002.1036148.

[KS12a] Erik E. Kostandyan and John D. Sorensen. “Physics of failure as a basis for solder elements reliability assessment in wind turbines”. In: *Reliability Engineering and System Safety* 108 (2012), pp. 100–107. ISSN: 09518320. DOI: 10.1016/j.ress.2012.06.020.

[KS12b] Prakash Kumar and R. K. Srivastava. “An expert system for predictive maintenance of mining excavators and its various forms in open cast mining”. In: *2012 1st International Conference on Recent Advances in Information Technology, RAIT-2012*. 2012. ISBN: 9781457706974. DOI: 10.1109/RAIT.2012.6194607.

[LDS18] Xiang Li, Qian Ding, and Jian Qiao Sun. “Remaining useful life estimation in prognostics using deep convolution neural networks”. In: *Reliability Engineering and System Safety* 172 (2018), pp. 1–11. ISSN: 09518320. DOI: 10.1016/j.ress.2017.11.021.

[Le +13] Khanh Le Son et al. “Remaining useful life estimation based on stochastic deterioration models: A comparative study”. In: *Reliability Engineering and System Safety* 112 (2013), pp. 165–175. ISSN: 09518320. DOI: 10.1016/j.ress.2012.11.022. arXiv: 1011.1669.

[Leg+17] Václav Legát et al. “Preventive maintenance models – Higher operational reliability. Maintenance and Reliability”. In: *Eksploatacja i Niezawodnosc* 19.1 (2017), pp. 134–141. ISSN: 15072711. DOI: 10.17531/ein.2017.1.19.

[Lei+18] Yaguo Lei et al. “Machinery health prognostics: A systematic review from data acquisition to RUL prediction”. In: *Mechanical Systems and Signal Processing* 104 (2018), pp. 799–834. ISSN: 10961216. DOI: 10.1016/j.ymssp.2017.11.016.

[Lev66] Vladimir Levenshtein. “Binary codes capable of correcting deletions, insertions, and reversals”. In: *Soviet Physics Doklady* 10.8 (1966), pp. 707–710.

[LFS18] Sai Li, Huajing Fang, and Bing Shi. “Multi-Step-Ahead Prediction with Long Short Term Memory Networks and Support Vector Regression”. In: *Chinese Control Conference, (CCC)*. 2018. ISBN: 9789881563941. DOI: 10.23919/ChiCC.2018.8484066.

[LGS13] Kaibo Liu, Nagi Z. Gebraeel, and Jianjun Shi. “A Data-level fusion model for developing composite health indices for degradation modeling and prognostic analysis”. In: *IEEE Transactions on Automation Science and Engineering* 10.3 (2013), pp. 652–664. ISSN: 15455955. DOI: 10.1109/TASE.2013.2250282.

[LHS20] Daniel P Lupp, Melinda Hodkiewicz, and Martin G Skjæveland. “Template libraries for industrial asset maintenance: A methodology for scalable and maintainable ontologies”. In: *CEUR Workshop Proceedings*. Vol. 2757. 2020, pp. 49–64.

[LHZ18] Huan Luo, Miaohua Huang, and Zhou Zhou. “Integration of Multi-Gaussian fitting and LSTM neural networks for health monitoring of an automotive suspension component”. In: *Journal of Sound and Vibration* 428 (2018), pp. 87–103. ISSN: 10958568. DOI: 10.1016/j.jsv.2018.05.007.

[Li+12] Sha Li et al. “Health condition-based maintenance decision intelligent reasoning method”. In: *Proceedings of 2012 International Conference on Quality, Reliability, Risk, Maintenance, and Safety Engineering, ICQR2MSE 2012*. 2012. ISBN: 9781467307888. DOI: 10.1109/ICQR2MSE.2012.6246263.

[Li+16] Qi Li et al. “Remaining useful life estimation for deteriorating systems with time-varying operational conditions and condition-specific failure zones”. In: *Chinese Journal of Aeronautics* 29.3 (2016), pp. 662–674. ISSN: 10009361. DOI: 10.1016/j.cja.2016.04.007.

[Li+17] Gaoyang Li et al. “Failure Prognosis of High Voltage Circuit Breakers with Temporal Latent Dirichlet Allocation”. In: *Energies* 10.11 (2017), p. 1913. ISSN: 1996-1073. DOI: 10.3390/en10111913.

[Li+18] Jianfeng Li et al. “Three-dimensional Simulation and Prediction of Solenoid Valve Failure Mechanism Based on Finite Element Model”. In: *IOP Conference Series: Earth and Environmental Science*. 2018. DOI: 10.1088/1755-1315/108/2/022035.

[Liu+09] Hao Liu et al. “A review on fault prognostics in integrated health management”. In: *ICEMI 2009 - Proceedings of 9th International Conference on Electronic Measurement and Instruments*. 2009. ISBN: 9781424438624. DOI: 10.1109/ICEMI.2009.5274082.

[Liu+18] Datong Liu et al. “An on-line state of health estimation of lithium-ion battery using unscented particle filter”. In: *IEEE Access* (2018). ISSN: 21693536. DOI: 10.1109/ACCESS.2018.2854224.

[LK18] Changyong Lee and Daeil Kwon. “A similarity based prognostics approach for real time health management of electronics using impedance analysis and SVM regression”. In: *Microelectronics Reliability* 83 (2018), pp. 77–83. ISSN: 00262714. DOI: 10.1016/j.microrel.2018.02.014.

[LLS19] Yanting Li, Shujun Liu, and Lianjie Shu. “Wind turbine fault diagnosis based on Gaussian process classifiers applied to operational data”. In: *Renewable Energy* 134 (2019), pp. 357–366. ISSN: 18790682. DOI: 10.1016/j.renene.2018.10.088.

[LMM18] Yazid Laib dit Leksir, Moufid Mansour, and Abdelkrim Moussaoui. “Localization of thermal anomalies in electrical equipment using Infrared Thermography and support vector machine”. In: *Infrared Physics and Technology* 89 (2018), pp. 120–128. ISSN: 13504495. DOI: 10.1016/j.infrared.2017.12.015.

[LQ+19] Hu Li-Qiang et al. “Track circuit fault predicition method based on grey theory and expert systems”. In: *Journal of Visual Communication and Image Representation* 58 (2019), pp. 37–45.

[LWX19] Yuqian Lu, Hongqiang Wang, and Xun Xu. “ManuService ontology: a product data model for service-oriented business interactions in a cloud manufacturing environment”. In: *Journal of Intelligent Manufacturing* 30 (2019), pp. 317–334. ISSN: 15728145. DOI: 10.1007/s10845-016-1250-x.

[LYKS18] Kenisuomo C. Luwei, Akilu Yunusa-Kaltungo, and Yusuf A. Sha’aban. “Integrated Fault Detection Framework for Classifying Rotating Machine Faults Using Frequency Domain Data Fusion and Artificial Neural Networks”. In: *Machines* 6.59 (2018).

[LYM16] Lei Lu, Jihong Yan, and Yue Meng. “Dynamic Genetic Algorithm-based Feature Selection Scheme for Machine Health Prognostics”. In: *Procedia CIRP* 56 (2016), pp. 316–320. ISSN: 22128271. DOI: 10.1016/j.procir.2016.10.026.

[Mab+18] Mohammed M. Mabkhot et al. *Requirements of the smart factory system: A survey and perspective*. 2018. DOI: 10.3390/MACHINES6020023.

[MAK14] Nadakatti Mahantesh, Parida Aditya, and Uday Kumar. “Integrated machine health monitoring: A knowledge based approach”. In: *International Journal of Systems Assurance Engineering and Management* 5 (2014), pp. 371–382. ISSN: 09764348. DOI: 10.1007/s13198-013-0178-1.

[MASH19] Mohammed M. Mabkhot, Ali M. Al-Samhan, and Lotfi Hidri. “An ontology-enabled case-based reasoning decision support system for manufacturing process selection”. In: *Advances in Materials Science and Engineering* (2019). ISSN: 16878442. DOI: 10.1155/2019/2505183.

[Mat+10] Aristeidis Matsokis et al. “An ontology-based model for providing Semantic Maintenance”. In: *IFAC Proceedings Volumes (IFAC-PapersOnline)*. 2010. ISBN: 9783902661784. DOI: 10.3182/20100701-2-pt-4012.00004.

[MBZ95] Mary Lou Maher, M. Bala Balachandran, and Dong Mei Zhang. *Case-Based Reasoning in Design*. 1st Editio. Psychology Press, 1995.

[McD+18] Darren McDonnell et al. “Predicting the unpredictable: Consideration of human and organisational factors in maintenance prognostics”. In: *Journal of Loss Prevention in the Process Industries* 54 (2018), pp. 131–145. ISSN: 09504230. DOI: 10.1016/j.jlp.2018.03.008. arXiv: 1612.08814.

[MD97] Mary Lou Maher and Andrés Gómez De Silva Garza. “Case-based reasoning in design”. In: *IEEE Expert-Intelligent Systems and their Applications* 12.2 (1997), pp. 34–41. ISSN: 08859000. DOI: 10.1109/64.585102.

[Men+15] Sandeep Menon et al. “Evaluating covariance in prognostic and system health management applications”. In: *Mechanical Systems and Signal Processing* 58-59 (2015), pp. 206–217. ISSN: 10961216. DOI: 10.1016/j.ymssp.2014.10.012.

[MHMJV21] Hugo Muñoz-Hernández, Juan José Montero-Jiménez, and Rob Vingerhoeds. “Integrating ontologies and case-based reasoning for the development of knowledge-intensive systems”. In: *Proceedings of the 35th annual European Simulation and Modelling Conference*. 2021, pp. 29–36. ISBN: 978-9-492859-18-1.

[MIM01] MIMOSA. *Open System Architecture for Condition-Based Maintenance (OSA-CBM)*. 2001.

[MLK17] Josey Mathew, Ming Luo, and Chee Khiang Pang. “Regression kernel for prognostics with support vector machines”. In: *22nd IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*. 2017.

[MM90] Vidosav D. Majstorovic and Vladimir R. Milacic. “Expert Systems for Maintenance in the CIM Concept”. In: *Computers in Industry* 15 (1990), pp. 83–93.

[MMZ16] Ahmed Mosallam, Kamal Medjaher, and Noureddine Zerhouni. “Data-driven prognostic method based on Bayesian approaches for direct remaining useful life prediction”. In: *Journal of Intelligent Manufacturing* 27.5 (2016), pp. 1037–1048. ISSN: 15728145. DOI: 10.1007/s10845-014-0933-4.

[MNK13] Nader Meskin, Esmaeil Naderi, and Khashayar Khorasani. “A multiple model-based approach for fault diagnosis of jet engines”. In: *IEEE Transactions on Control Systems Technology* 21.1 (2013), pp. 254–262. ISSN: 10636536. DOI: 10.1109/TCST.2011.2177981.

[MO+14] Gabriela Medina-Oliva et al. “Predictive diagnosis based on a fleet-wide ontology approach”. In: *Knowledge-Based Systems* 68 (2014), pp. 40–57. ISSN: 09507051. DOI: 10.1016/j.knosys.2013.12.020.

[Mon+20] Juan José Montero Jimenez et al. “Towards multi-model approaches to predictive maintenance: A systematic literature survey on diagnostics and prognostics”. In: *Journal of Manufacturing Systems* 56 (2020), pp. 539–557. ISSN: 02786125. DOI: 10.1016/j.jmsy.2020.07.008.

[Mon+21] Juan José Montero Jiménez et al. “An Ontology Model for Maintenance Strategy Selection and Assessment”. In: *Journal of Intelligent Manufacturing.* (2021). DOI: 10.1007/s10845-021-01855-3.

[Mou09] Marcelo Nascimento Moutinho. “Fuzzy diagnostic systems of rotating machineries, some Eletronorte’s applications”. In: *2009 15th International Conference on Intelligent System Applications to Power Systems, ISAP ’09*. 2009. ISBN: 9781424450985. DOI: 10.1109/ISAP.2009.5352882.

[Mou97] John Moubray. *RCM II: Reliability Centered Maintenance*. Industrial Press, 1997, p. 423. ISBN: 978-0831130787.

[MS95] Salvatore T. March and Gerald F. Smith. “Design and natural science research on information technology”. In: *Decision Support Systems* 15 (1995), pp. 251–266. ISSN: 01679236. DOI: 10.1016/0167-9236(94)00041-2.

[MU11] Kamran S. Moghaddam and John S. Usher. “Sensitivity analysis and comparison of algorithms in preventive maintenance and replacement scheduling optimization models”. In: *Computers and Industrial Engineering* 61.1 (2011), pp. 64–75. ISSN: 03608352. DOI: 10.1016/j.cie.2011.02.012.

[Mus92] Mark A. Musen. “Dimensions of knowledge sharing and reuse”. In: *Computers and Biomedical Research* 25.5 (1992), pp. 435–467. ISSN: 00104809. DOI: 10.1016/0010-4809(92)90003-S.

[MV18] Juan José Montero Jimenez and Rob Vingerhoeds. “Enhancing operational fault diagnosis by assessing multiple operational modes”. In: *Proceedings - International Conference in Modelling, Optimization and Simulation MOSIM 2018, 27th-29th June*. Toulouse, France: MOSIM2018, 2018, pp. 237–244.

[MV19] Juan José Montero Jiménez and Rob Vingerhoeds. “A System Engineering Approach to Predictive Maintenance Systems: from needs and desires to logical architecture.” In: *5th IEEE Int. Symposium on Systems Engineering 2019,* Edinburgh, 2019. DOI: 10.1109/ISSE46696.2019.8984559.

[MVG21] Juan José Montero Jiménez, Rob Vingerhoeds, and Bernard Grabot. “Enhancing predictive maintenance architecture process by using ontology-enabled Case-Based Reasoning”. In: *7th IEEE International Symposium on System Engineering*. 2021. DOI: 10.1109/ISSE51541.2021.9582535.

[MXJ19] Seyed M.Mehdi Hassani.N, Jin Xiaoning, and Ni Jun. “Physics-based Gaussian process for the health monitoring for a rolling bearing”. In: *Acta Astronatica* 154 (2019), pp. 133–139.

[MZ18] Jianing Man and Qiang Zhou. “Prediction of hard failures with stochastic degradation signals using Wiener process and proportional hazards model”. In: *Computers and Industrial Engineering* 125 (2018), pp. 480–489. ISSN: 03608352. DOI: 10.1016/j.cie.2018.09.015.

[NB17a] David Lira Nuñez and Milton Borsato. “An ontology-based model for prognostics and health management of machines”. In: *Journal of Industrial Information Integration* (2017). ISSN: 2452414X. DOI: 10.1016/j.jii.2017.02.006.

[NB17b] David Lira Nuñez and Milton Borsato. “An ontology-based model for prognostics and health management of machines”. In: *Journal of Industrial Information Integration* 6 (2017), pp. 33–46. ISSN: 2452414X. DOI: 10.1016/j.jii.2017.02.006.

[NB18] David Lira Nuñez and Milton Borsato. “OntoProg: An ontology-based model for implementing Prognostics Health Management in mechanical machines”. In: *Advanced Engineering Informatics* 38 (2018), pp. 746–759. ISSN: 14740346. DOI: 10.1016/j.aei.2018.10.006.

[NM01] Natalya F. Noy and Deborah L. McGuinness. *Ontology Development 101: A Guide to Creating Your First Ontology*. Tech. rep. 2001. DOI: 10.1016/j.artmed.2004.01.014.

[NO17] Vladislavs Nazaruks and Janis Osis. “A survey on domain knowledge representation with frames”. In: *ENASE 2017 - Proceedings of the 12th International Conference on Evaluation of Novel Approaches to Software Engineering*. 2017. ISBN: 9789897582509. DOI: 10.5220/0006388303460354.

[Nom+18] Mohammed A. Noman et al. *Overview of predictive condition based maintenance research using bibliometric indicators*. 2018. DOI: 10.1016/j.jksues.2018.02.003.

[NSG12] Fachri P. Nasution, Svein Sævik, and Janne K.Ø. Gjøsteen. “Fatigue analysis of copper conductor for offshore wind turbines by experimental and FE method”. In: *Energy Procedia* 24 (2012), pp. 271–280. ISSN: 18766102. DOI: 10.1016/j.egypro.2012.06.109.

[Nyu+18] Ladislav Nyulászi et al. “Fault Detection and Isolation of an Aircraft Turbojet Engine Using a Multi-Sensor Network and Multiple Model Approach”. In: *Acta Polytechnica Hungarica* 15.2 (2018), pp. 189–209.

[Oko+14] Caxton Okoh et al. “Overview of Remaining Useful Life prediction techniques in Through-life Engineering Services”. In: *Procedia CIRP* 16 (2014), pp. 158–163. ISSN: 22128271. DOI: 10.1016/j.procir.2014.02.006.

[One+18] Melis Onel et al. “Simultaneous Fault Detection and Identification in Continuous Processes via nonlinear Support Vector Machine based Feature Selection”. In: *Computer Aided Chemical Engineering* 44 (2018), pp. 2077–2082. ISSN: 15707946. DOI: 10.1016/B978-0-444-64241-7.50341-4.

[ORM17] Criston. Okoh, Rajkumar Roy, and Jorn Mehnen. “Predictive Maintenance Modelling for Through-Life Engineering Services”. In: *Procedia CIRP*. 2017. ISBN: 22128271 (ISSN). DOI: 10.1016/j.procir.2016.09.033. arXiv: 0208024 [gr-qc].

[PBG12] Ashok Prajapati, James Bechtel, and Subramaniam Ganesan. “Condition based maintenance: A survey”. In: *Journal of Quality in Maintenance Engineering* 18.4 (2012), pp. 384–400. ISSN: 13552511. DOI: 10.1108/13552511211281552. arXiv: 2014WR016527 [10.1002].

[PD11] Paul Phillips and Dominic Diston. “A knowledge driven approach to aerospace condition monitoring”. In: *Knowledge-Based Systems* 24.6 (2011), pp. 915–927. ISSN: 09507051. DOI: 10.1016/j.knosys.2011.04.008.

[PDZ10] Ying Peng, Ming Dong, and Ming Jian Zuo. “Current status of machine prognostics in condition-based maintenance: A review”. In: *International Journal of Advanced Manufacturing Technology* 50.1-4 (2010), pp. 297–313. ISSN: 02683768. DOI: 10.1007/s00170-009-2482-0. arXiv: 0208024 [gr-qc].

[Pey+09] Flavien Peysson et al. “Expert knowledge impact on damage trajectory analysis based prognostics”. In: *IFAC Proceedings Volumes (IFAC-PapersOnline)*. 2009. ISBN: 9783902661463. DOI: 10.3182/20090630-4-ES-2003.0231.

[PH08] Jim Prentzas and Ioannis Hatzilygeroudis. “Combinations of case-based reasoning with other intelligent methods”. In: *CEUR Workshop Proceedings*. 2008. DOI: 10.3233/his-2009-0096.

[Pla95] Enric Plaza. “Cases as terms: A feature term approach to the structured representation of cases”. In: *First International Conference, ICCBR-95*. Vol. 1. Springer Verlag, 1995, pp. 265–276. DOI: 10.1007/3-540-60598-3_24.

[PMK13] Bahareh Pourbabaee, Nader Meskin, and Khashayar Khorasani. “Multiple-Model Based Sensor Fault Diagnosis Using Hybrid Kalman Filter Approach for Nonlinear Gas Turbine Engines”. In: *2013 American Control Conference (Acc)*. 2013. ISBN: 9781479901760. DOI: 10.1109/TCST.2015.2480003.

[Pro05] Philip E. Protter. *Stochastic Integration and Differential Equations, Second Edition*. Ed. by B Rozovski and M Yor. Springer, 2005.

[PS04] Sankar Pal and Simon Shiu. *Foundations of soft case-based reasoning*. Wiley, 2004, p. 299.

[PSD14] Panče Panov, Larisa Soldatova, and Sašo Džeroski. “Ontology of core data mining entities”. In: *Data Mining and Knowledge Discovery* 28 (2014), pp. 1222–1265. ISSN: 13845810. DOI: 10.1007/s10618-014-0363-0.

[QA+17] Santiago Quintana-Amate et al. “A new knowledge sourcing framework for knowledge-based engineering: An aerospace industry case study”. In: *Computers and Industrial Engineering* 104 (2017), pp. 35–50. ISSN: 03608352. DOI: 10.1016/j.cie.2016.12.013.

[Qin+16] Feiwei Qin et al. “An ontology-based semantic retrieval approach for heterogeneous 3D CAD models”. In: *Advanced Engineering Informatics* 30 (2016), pp. 751–768. ISSN: 14740346. DOI: 10.1016/j.aei.2016.10.001.

[Qin+18] Yuchu Qin et al. “Towards an ontology-supported case-based reasoning approach for computer-aided tolerance specification”. In: *Knowledge-Based Systems* 141 (2018), pp. 129–147. ISSN: 09507051. DOI: 10.1016/j.knosys.2017.11.013.

[RA19] Fabrício Henrique Rodrigues and Mara Abel. “What to consider about events: A survey on the ontology of occurrents”. In: *Applied Ontology* 14.4 (2019), pp. 343–378. ISSN: 18758533. DOI: 10.3233/AO-190217.

[Ram14] Emmanuel Ramasso. “Investigating computational geometry for failure prognostics”. In: *Int. Journal on Prognostics and Health Management* 5.005 (2014), p. 18. ISSN: 21532648.

[Ram15] Luis Ramos. “Semantic Web for manufacturing, trends and open issues: Toward a state of the art”. In: *Computers and Industrial Engineering* 90 (2015), pp. 444–460. ISSN: 03608352. DOI: 10.1016/j.cie.2015.10.013.

[Ras03] Carl Edward Rasmussen. “Gaussian Processes in Machine Learning”. In: *Advanced Lectures on Machine Learning*. Ed. by Olivier Bousquet, Ulrike von Luxburg, and Gunnar Rätsch. Springer, 2003, pp. 63–71.

[RC15] Joe Raad and Christophe Cruz. “A survey on ontology evaluation methods”. In: *IC3K 2015 - Proceedings of the 7th International Joint Conference on Knowledge Discovery, Knowledge Engineering and Knowledge Management*. 2015. ISBN: 9789897581588. DOI: 10.5220/0005591001790186.

[Red12] Timothy Redmond. *SPARQL Query tab for Protégé*. 2012. URL: https://protegewiki.stanford.edu/wiki/SPARQL{_}Query (visited on 10/22/2020).

[RFG14] Paula Potes Ruiz, Bernard Kamsu Foguem, and Bernard Grabot. “Generating knowledge in maintenance from Experience Feedback”. In: *Knowledge-Based Systems* 68 (2014), pp. 4–20. ISSN: 09507051. DOI: 10.1016/j.knosys.2014.02.002.

[RG10] Emmanuel Ramasso and Rafael Gouriveau. “Prognostics in switching systems: Evidential Markovian classification of real-time neuro-fuzzy predictions”. In: *2010 Prognostics and System Health Management Conference, PHM ’10*. 2010. ISBN: 9781424447565. DOI: 10.1109/PHM.2010.5413442.

[RN12] Stuart Russel and Peter Norvig. *Artificial intelligence—a modern approach 3rd Edition*. 2012. ISBN: 9780136042594. DOI: 10.1017/S0269888900007724. arXiv: 9809069v1 [arXiv:gr-qc].

[Rom+14] Juan Camilo Romero Bejarano et al. “Case-based reasoning and system design: An integrated approach based on ontology and preference modeling”. In: *Artificial Intelligence for Engineering Design, Analysis and Manufacturing: AIEDAM* 28 (2014), pp. 49–69. ISSN: 14691760. DOI: 10.1017/S0890060413000498.

[Roq18] Pascal Roques. *Systems Architecture Modeling with the Arcadia Method 1st Edition*. ISTE Press, 2018. ISBN: 9781785481680.

[RS89] Christopher K Riesbeck and Roger C Schank. *Inside case-based reasoning*. 1989. ISBN: 0-89859-767-6 (Hardcover).

[Rud20a] Ron Rudnicki. *An Overview of the Common Core Ontologies 1.3*. Buffalo, NY, 2020. URL: https://www.nist.gov/system/files/documents/2019/05/30/nist-ai-rfi-cubrc{_}inc{_}004.pdf.

[Rud20b] Ron Rudnicki. *Common core ontologies*. 2020. URL: https://github.com/CommonCoreOntology/CommonCoreOntologies (visited on 07/22/2020).

[RW06] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian process for machine learning*. the MIT Press, 2006.

[SA08] Abdel-Badeeh M Salem and Marco Alfonse. “Ontology versus semantic networks for medical knowledge representation”. In: *Proceedings of the 12th WSEAS international Conference on COMPUTERS*. 2008. ISBN: 9789606766855.

[SA17] Christina M. Steiner and Dietrich Albert. “Validating domain ontologies: A methodology exemplified for concept maps”. In: *Cogent Education* 4.1 (2017). ISSN: 2331186X. DOI: 10.1080/2331186X.2016.1263006.

[Sán+12] David Sánchez et al. “Ontology-based semantic similarity: A new feature-based approach”. In: *Expert Systems with Applications* 39 (2012), pp. 7718–7728. ISSN: 09574174. DOI: 10.1016/j.eswa.2012.01.082.

[Sax+08] Abhinav Saxena et al. “Damage propagation modeling for aircraft engine run-to-failure simulation”. In: *2008 International Conference on Prognostics and Health Management, PHM 2008*. 2008. ISBN: 9781424419357. DOI: 10.1109/PHM.2008.4711414.

[SC15] Barry Smith and Werner Ceusters. “Aboutness: Towards foundations for the information artifact ontology”. In: *CEUR Workshop Proceedings*. 2015.

[Sch+20] Sebastien Schwartz et al. “A fault mode identification methodology based on self-organizing map”. In: *Neural Computing and Applications* 32 (2020), pp. 13405–13423.

[SHM11] Joanna Z. Sikorska, Melinda Hodkiewicz, and Lin Ma. “Prognostic modelling options for remaining useful life estimation by industry”. In: *Mechanical Systems and Signal Processing* 25.5 (2011), pp. 1803–1836. ISSN: 08883270. DOI: 10.1016/j.ymssp.2010.11.018.

[Shu+17] Jin Shuangshuang et al. “The Remaining Life Prediction of the Fan Bearing Based on Genetic Algorithm and Multi-parameter Support Vector Machine”. In: *5th International Conference on Mechanical, Automotive and Materials Engineering*. 2017.

[Si+11] Xiao Sheng Si et al. “Remaining useful life estimation - A review on the statistical data driven approaches”. In: *European Journal of Operational Research* 213.1 (2011), pp. 1–14. ISSN: 03772217. DOI: 10.1016/j.ejor.2010.11.018.

[SKY19] Emilio M. Sanfilippo, Yoshinobu Kitamura, and Robert I.M. Young. “Formal ontologies in manufacturing”. In: *Applied Ontology* 14.2 (2019), pp. 119–125. ISSN: 18758533. DOI: 10.3233/AO-190209.

[SMS17] Karanvir Singh, Hasmat Malik, and Rajneesh Sharma. “Condition monitoring of wind turbine gearbox using electrical signatures”. In: *2017 International Conference on Microelectronic Devices, Circuits and Systems, ICMDCS 2017,* 2017. ISBN: 9781538617168. DOI: 10.1109/ICMDCS.2017.8211718.

[SRC10] Abhinav Saxena, Indranil Roychoudhury, and Jose R Celaya. “Requirements Specifications for Prognostics : An Overview”. In: *Proceedings of AIAA Infotech@Aerospace 2010*. 2010. ISBN: 9781600867439 (ISBN). DOI: 10.2514/6.2010-3398.

[Sta20] Stanford University. *Protégé website*. 2020. URL: https://protege.stanford.edu/ (visited on 07/22/2020).

[Sul+10] Greg P. Sullivan et al. *Operations & Maintenance Best Practices: A Guide to Achieving Operational Efficiency*. U. S. Departament of Energy, Federal energy management program, 2010. ISBN: 18773373463. DOI: 10.2172/1034595.

[SW15] Bernard Schmidt and Lihui Wang. “Predictive Maintenance: Literature review and future trends”. In: *Conference: Proceedings of the 25th International Conference on Flexible Automation and Intelligent Manufacturing* 1 (2015), pp. 232–239.

[SW18] Nazmus Sakib and Thorten Wuest. “Challenges and Opportunities of Condition-based Predictive Maintenance: a Review”. In: *6th CIRP Global Web Conference: “Envisaging the future manufacturing, design, technologies, and systems in innovation era”*. Elseiver B.V., 2018, pp. 267–272.

[Tal+19] Asma Talhi et al. “Ontology for cloud manufacturing based Product Lifecycle Management”. In: *Journal of Intelligent Manufacturing* 30 (2019), pp. 2171–2192. ISSN: 15728145. DOI: 10.1007/s10845-017-1376-5.

[TL14] Tiedo Tinga and Richard Loendersloot. “Aligning PHM, SHM and CBM by understanding the physical system failure behaviour”. In: *Proceedings of the European Conference of the Prognostics and Health Management Society*. 2014. ISBN: 978-1-936263-16-5.

[TSY18] Diyin Tang, Wubin Sheng, and Jinsong Yu. "Dynamic condition-based maintenance policy for degrading systems described by a random-coefficient autoregressive model: A comparative study". In: *Eksploatacja i Niezawodnosc* 20.4 (2018), pp. 590–601. ISSN: 15072711. DOI: 10.17531/ein.2018.4.10.

[Vac+07] George Vachtsevanos et al. *Intelligent Fault Diagnosis and Prognosis for Engineering Systems*. 2007. ISBN: 047172999X. DOI: 10.1002/9780470117842.

[VBD17] Kim Verbert, Robert Babuška, and Bart De Schutter. "Bayesian and Dempster–Shafer reasoning for knowledge-based fault diagnosis–A comparative study". In: *Engineering Applications of Artificial Intelligence* 60 (2017), pp. 136–150. ISSN: 09521976. DOI: 10.1016/j.engappai.2017.01.011.

[VD18] Wim J.C. Verhagen and Lennaert W.M. De Boer. *Predictive maintenance for aircraft components using proportional hazard models*. 2018. DOI: 10.1016/j.jii.2018.04.004.

[VDB15] Kim Verbert, Bart De Schutter, and Robert Babuška. "Reasoning under uncertainty for knowledge-based fault diagnosis: A comparative study". In: *IFAC-PapersOnLine* 48 (2015), pp. 422–427. ISSN: 24058963. DOI: 10.1016/j.ifacol.2015.09.563.

[Vep91] Rajan Vepa. "Introduction to fuzzy logic and fuzzy sets". In: *Application of artificial intelligence in process control. (L. Boullart, A. Krijgsman and R.A. Vingerhoeds; editors)*. 1991, pp. 146–163.

[Vin+95] Rob A. Vingerhoeds et al. "Enhancing off-line and on-line condition monitoring and fault diagnosis". In: *Control Engineering Practice* 3.11 (1995), pp. 1515–1528. ISSN: 09670661. DOI: 10.1016/0967-0661(95)00162-N.

[VM18] David Vališ and Dariusz Mazurkiewicz. "Application of selected Levy processes for degradation modelling of long range mine belt using real-time data". In: *Archives of Civil and Mechanical Engineering* 18.4 (2018), pp. 1430–1440. ISSN: 16449665. DOI: 10.1016/j.acme.2018.05.006.

[Von+18] Alexander Von Birgelen et al. "Self-Organizing Maps for Anomaly Localization and Predictive Maintenance in Cyber-Physical Production Systems". In: *Procedia CIRP* 72 (2018), pp. 480–485. ISSN: 22128271. DOI: 10.1016/j.procir.2018.03.150.

[VWD14] Gregory W. Vogl, Brian Weiss, and Alkan Donmez. "Standards for Prognostics and Health Management (PHM) Techniques within Manufacturing Operations". In: *Annual Conference of the Prognostics and Health Management Society*. 2014. ISBN: 9781936263172.

[Wan+08] Tianyi Wang et al. "A similarity-based prognostics approach for remaining useful life estimation of engineered systems". In: *2008 International Conference on Prognostics and Health Management, PHM 2008*. 2008. ISBN: 9781424419357. DOI: 10.1109/PHM.2008.4711421.

[Wan+14] Yuanhang Wang et al. "A corrective maintenance scheme for engineering equipment". In: *Engineering Failure Analysis* 36 (2014), pp. 269–283. ISSN: 13506307. DOI: 10.1016/j.engfailanal.2013.10.006.

[Wan+18] Anping Wan et al. "Prognostics of gas turbine: A condition-based maintenance approach based on multi-environmental time similarity". In: *Mechanical Systems and Signal Processing* 109 (2018), pp. 150–165. ISSN: 10961216. DOI: 10.1016/j.ymssp.2018.02.027.

[Wel92] Gordon Wells. "An introduction to neural networks". In: *Application of artificial intelligence in process control. (L. Boullart, A. Krijgsman and R.A. Vingerhoeds; editors)*. 1992, pp. 164–200.

[WHF18] Zhao Qiang Wang, Chang Hua Hu, and Hong Dong Fan. "Real-Time Remaining Useful Life Prediction for a Nonlinear Degrading System in Service: Application to Bearing Data". In: *IEEE/ASME Transactions on Mechatronics* 23.1 (2018), pp. 211–222. ISSN: 10834435. DOI: 10.1109/TMECH.2017.2666199.

[Wor12] World Wide Web Consortium. *OWL 2 Web Ontology Language*. 2012. URL: http://www.w3.org/TR/2012/REC-owl2-overview-20121211/ (visited on 02/01/2020).

[WTM17] Dong Wang, Kwok Leung Tsui, and Qiang Miao. "Prognostics and Health Management: A Review of Vibration Based Bearing and Gear Health Indicators". In: *IEEE Access* 6 (2017), pp. 665–676. ISSN: 21693536. DOI: 10.1109/ACCESS.2017.2774261.

[Wu+18] Zhenyu Wu et al. "K-PdM: KPI-Oriented Machinery Deterioration Estimation Framework for Predictive Maintenance Using Cluster-Based Hidden Markov Model". In: *IEEE Access* Vol 6.DOI: 10.1109/ACCESS.2018.2859922 (2018), pp. 41676 –41687. ISSN: 21693536. DOI: 10.1109/ACCESS.2018.2859922.

[Yan+04] Bo Suk Yang et al. "Case-based reasoning system with Petri nets for induction motor fault diagnosis". In: *Expert Systems with Applications* 27.2 (2004), pp. 301–311. ISSN: 09574174. DOI: 10.1016/j.eswa.2004.02.004.

[YN18] Ali Yahyatabar and Amir Abbas Najafi. "Condition based maintenance policy for series-parallel systems through Proportional Hazards Model: A multi-stage stochastic programming approach". In: *Computers and Industrial Engineering* 126 (2018), pp. 30–46. ISSN: 03608352. DOI: 10.1016/j.cie.2018.09.014.

[YSJ17] Kam Chuen Yung, Bo Sun, and Xiaopeng Jiang. "Prognostics-based qualification of high-power white LEDs using Lévy process approach". In: *Mechanical Systems and Signal Processing* 82 (2017), pp. 206–216. ISSN: 10961216. DOI: 10.1016/j.ymssp.2016.05.019.

[ZBD16] Deyi Zhang, Andrew D. Bailey, and Dragan Djurdjanovic. "Bayesian Identification of Hidden Markov Models and Their Use for Condition-Based Monitoring". In: *IEEE Transactions on Reliability* (2016). ISSN: 00189529. DOI: 10.1109/TR.2016.2570561.

[Zer+17] Noureddine Zerhouni et al. "Prognostics and Health Management for Maintenance Practitioners-Review, Implementation and Tools Evaluation". In: *Article in International Journal of Prognostics and Health Management* 8.60 (2017), p. 31. ISSN: 22129685. DOI: 10.1016/j.euprot.2015.07.015.

[Zha+17] Rui Zhao et al. "Machine Health Monitoring Using Local Feature-based Gated Recurrent Unit Networks". In: *IEEE Transactions on Industrial Electronics* 65.2 (2017), pp. 1539 –1548. ISSN: 0278-0046. DOI: 10.1109/TIE.2017.2733438.

[Zha+18a] Zhengxin Zhang et al. "Degradation data analysis and remaining useful life estimation: A review on Wiener-process-based methods". In: *European Journal of Operational Research* 271.3 (2018), pp. 775–796. ISSN: 03772217. DOI: 10.1016/j.ejor.2018.02.033.

[Zha+18b] Shuai Zhao et al. "Evaluation of Reliability Function and Mean Residual Life for Degrading Systems Subject to Condition Monitoring and Random Failure". In: *IEEE Transactions on Reliability* 67.1 (2018), pp. 13–25. ISSN: 00189529. DOI: 10.1109/TR.2017.2779322.

[Zho+10] Zhi Jie Zhou et al. "A model for real-time failure prognosis based on hidden Markov model and belief rule base". In: *European Journal of Operational Research* 207.1 (2010), pp. 269–283. ISSN: 03772217. DOI: 10.1016/j.ejor.2010.03.032.

[Zho+14] Zhi Jie Zhou et al. "A model for online failure prognosis subject to two failure modes based on belief rule base and semi-quantitative information". In: *Knowledge-Based Systems* (2014). ISSN: 09507051. DOI: 10.1016/j.knosys.2014.06.026.

[ZM07a] Yimin Zhan and Chris K. Mechefske. "Robust detection of gearbox deterioration using compromised autoregressive modeling and Kolmogorov-Smirnov test statistic-Part I: Compromised autoregressive modeling with the aid of hypothesis tests and simulation analysis". In: *Mechanical Systems and Signal Processing* 21.5 (2007), pp. 1953–1982. ISSN: 08883270. DOI: 10.1016/j.ymssp.2006.11.005.

[ZM07b] Yimin Zhan and Chris K. Mechefske. “Robust detection of gearbox deterioration using compromised autoregressive modeling and Kolmogorov-Smirnov test statistic. Part II: Experiment and application”. In: *Mechanical Systems and Signal Processing* 21.5 (2007), pp. 1983–2011. ISSN: 08883270. DOI: 10.1016/j.ymssp.2006.11.006.

[ZYX17] Qiang Zhou, Ping Yan, and Yang Xin. “Research on a knowledge modelling methodology for fault diagnosis of machine tools based on formal semantics”. In: *Advanced Engineering Informatics* 32 (2017), pp. 92–112. ISSN: 14740346. DOI: 10.1016/j.aei.2017.01.002.

[ZYZ15] Anmei Zhou, Dejie Yu, and Wenyi Zhang. “A research on intelligent fault diagnosis of wind turbines based on ontology and FMECA”. In: *Advanced Engineering Informatics* 29.1 (2015), pp. 115–125. ISSN: 14740346. DOI: 10.1016/j.aei.2014.10.001.

APPENDIX A

A fault mode identification methodology based on self-organizing maps

---

Intentionally left blank

Neural Computing and Applications (2020) 32:13405–13423
https://doi.org/10.1007/s00521-019-04692-x

ORIGINAL ARTICLE

A fault mode identification methodology based on self-organizing map

Sébastien Schwartz[1,2] · Juan José Montero Jimenez[2,3] · Michel Salaün[2] · Rob Vingerhoeds[2]

Received: 14 January 2019 / Accepted: 18 December 2019 / Published online: 1 January 2020

Abstract
One of the main goals of predictive maintenance is to be able to trigger the right maintenance actions at the right moment in time building upon the monitoring of the health status of the concerned systems and their components. As such, it allows identifying incipient faults and forecasting the moment of failure at the earliest stage. Many different data-driven methods are used in such approaches (Naderi and Khorasani in 2017 IEEE 30th Canadian conference on electrical and computer engineering (CCECE), Windsor, ON, IEEE, pp 1–6, 2017. https://doi.org/10.1109/ccece.2017.7946715; Sarkar et al. in J Eng Gas Turbines Power 1338(8):081602, 2011. https://doi.org/10.1115/1.4002877; Svärd et al. in Mech Syst Signal Process 45(1):170–192, 2014. https://doi.org/10.1016/j.ymssp.2013.11.002; Pourbabaee et al. Mech Syst Signal Process 76–77:136–156, 2016. https://doi.org/10.1016/j.ymssp.2016.02.023). This work uses the self-organizing maps (SOMs) or Kohonen map, thanks to its ability to emphasize underlying behavior such as fault modes. An automatic fault mode detection is presented based on a SOM network and the kernel density estimation with as less as possible prior knowledge. The different SOM development steps are presented and the suitable solutions proposed to structure the approach are accompanied by mathematical methods. The generated maps are then used with kernel density analysis to isolate fault modes on them. Finally, a methodology is presented to identify the different fault modes. The work is illustrated with an aircraft jet engines case study.

Keywords Diagnostic · Fault identification · Predictive maintenance · Self-organizing map

1 Introduction

Maintenance departments are confronted with three types of maintenance: corrective maintenance (i.e., correcting systems that break down or have a deteriorated functional behavior), preventive maintenance (i.e., maintenance actions at regular intervals, to avoid break down or deterioration) and predictive maintenance (i.e., performing specific maintenance actions based on indications derived from fine analysis on data, crew reports, etc.). Predictive maintenance has seen a huge rise over the last years, essentially due to the application of neural networks to identify incipient faults and to forecast the moment of failure through the diagnostic phase. Depending on the monitored data, there are different types of classification for diagnostic system (Fig. 1). Machine fault diagnostic approaches are grouped into model-based [1–3] and data-driven [4–7] techniques.

In this paper, a hybrid approach for a "process history-based" diagnosis with quantitative data through the combination of a neural network (NN) with a probability density function (PDF) is scoped. In particular, an unsupervised neural network (UNN) type, the self-organizing map (SOM), or Kohonen map, is used. This NN has already shown its effectiveness in the past [9] but requires substantial knowledge on the neural network type itself, making it complex to comprehend and frequently requiring "manual" rework. Therefore, the presented novel approach

✉ Sébastien Schwartz
sebastien.schwartz@isae-supaero.fr

[1] SOGETI High Tech, R&D Dpt., Aeropark, 3 Chemin de Laporte, 31100 Toulouse, France

[2] ISAE-SUPAERO, Université de Toulouse, 10 Avenue Edouard Belin, 31400 Toulouse, France

[3] Tecnológico de Costa Rica Institute of Technology, Calle 15, Avenida 14., 1 km Sur de la Basílica de los Ángeles, Provincia de Cartago, Cartago 30101, Costa Rica

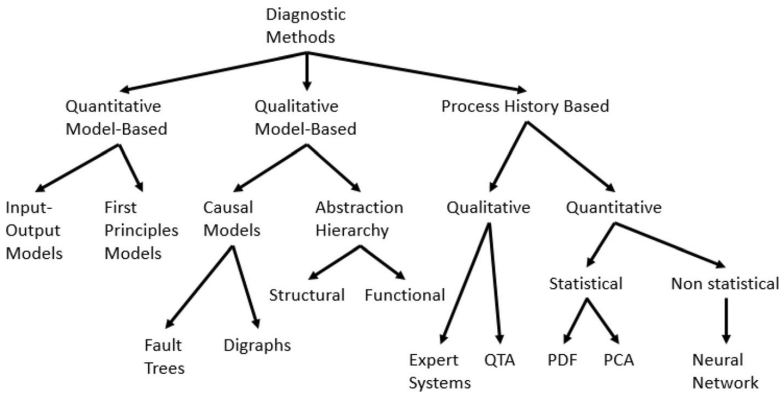


Fig. 1 Classification of diagnostic methods based on [8]. *QTA* qualitative trend analysis, *PDF* probability density function, *PCA* principal component analysis

aims at automating as much as possible for the development of the SOM. The first objective is to reduce to the minimum level of the interaction between the expert knowledge and the network itself. The complete process is reviewed, and, for each step in the process, mathematical methods are proposed. It leads to a more structured and automatable procedure with an intent to make the method more accessible and autonomous. The second objective is to manage automatically the output of the SOM with PDFs to identify fault modes.

The goal of this paper is to present a fully autonomous toolchain that identifies the fault modes from input sensors by reducing as much as possible the prior knowledge and the expert intervention.

The paper is organized as follows: In the second section, predictive maintenance is introduced. In the third section, the self-organizing map is presented, as well as the different possibilities to enhance the approach and a methodology to identify the fault modes. The fourth section exposes and applies the methodology on the case study of aircraft engines. The paper concludes with some general observations and indications for future work.

2 Predictive maintenance

Keeping a technical system in optimal operational conditions is key for a successful and an efficient use of the system. Interruptions of operations not only have a negative impact (e.g., delayed flights, internet connections not being available, etc.), but may also have worst consequences (e.g., image of the company impacted, people will tend to privilege other suppliers, loss of income, etc.). Maintaining a system in optimal state of operation also means having timely maintenance actions to ensure the intended functionality and to avoid potential failures to occur. A good combination of corrective, preventive and predictive maintenance is required [10]. Whereas corrective maintenance is based on alarm handling, troubleshooting for corrective actions, etc., predictive maintenance relies on offline diagnostic task analyses such as recorded data, crew reports, maintenance logs and other data recordings on the actual health state of the system. Such information is then used by operation departments to assess the health state and derive necessary maintenance actions and their planning if necessary.

Condition monitoring is used to assess the current health state of the system at hand. It uses pattern recognition in time series of monitored data and classifies those patterns as known conditions. While this is used to be done by human experts [9], requiring great skill and experience from the expert, software tools have appeared to support engineers in such activities.

As predictive maintenance aims to define the best possible moment to trigger maintenance actions [9], it gained more and more attention over the last few years. One of the solutions discussed in the literature for early detection and classification of failures during the diagnostic phase is the self-organizing map (SOM) originally proposed by Kohonen [11]. This approach allows to detect degradation patterns and the nature of the problem and to derive the remaining useful life [9, 10, 12]. This network has been widely used on various application fields for its particular abilities [13–18].

Visualization helps humans to understand diagnostic tasks. According to [19], the visualization of the data helps to gain an understanding of an unknown dataset. For limited amounts of dimensions in data, humans can do it, but the perception is limited to three dimensions. High-dimensional data visualization with more than three features is therefore unreachable. To address this issue, several visualization techniques were developed such as principal component analysis (PCA) [20], self-organized maps (SOMs) [11] or Sammon mapping [21]. Those techniques

rely on dimension reduction to visualize on 2D or 3D plots. This dimension reduction therefore generates new knowledge to be labeled for the analysis. Labeling data using prior knowledge or human reasoning become complicated on high dimensions [22], which could induce errors. Approaches relying on unsupervised learning are interesting thanks to their abilities to deal with high-dimensional data without prior knowledge. Therefore, a SOM network answers to the requirements: visualize high-dimensional data on 2D maps and obtain knowledge generated from these maps.

The successful implementation of SOM for diagnostic tasks requires an in-depth analysis of data obtained from the system at hand. It involves the assessment of data interdependency, the analysis on how many different faults/failures can be identified in the data, the distinction of eventual operational modes (if necessary) and, finally, successful training and subsequent validation of the SOM. Such analysis requires a good knowledge on the application domain itself and the measured data, in addition to strong knowledge on SOMs. In the next section, the fault mode diagnosis approach is presented. Each step of the SOM neural network is revisited and analyzed to see "whether and how" improvements may be obtained to automatize the use of SOM neural networks and to reduce the need for prior knowledge. Then, probability density function on the neural network output is used to identify faults of the supervised system.

3 Fault mode diagnosis using self-organizing maps

3.1 Overview

As presented previously, fault diagnosis requires human intervention and prior knowledge. The proposed methodology (Fig. 2) attempts to perform an automatic fault mode diagnosis with as less as possible prior knowledge. The self-organizing map neural network is the core of the methodology as a tool to emphasize the input data through a map representation. The underlying information such as faults becomes more accessible. The approach has been structured into three phases.

The first step is "input data management". Input data (raw monitored sensors) have to be formatted and used with the neural network. This sensor management involves the choice of useful variables to decrease the complexity and the size the neural network. In addition, the data are normalized to facilitate the network training.

The second step is "system map." The formatted input data are presented to the neural network. For the training phase, the network generates maps representing the input data. During the testing phase, the network outputs a localization on the previously trained map.

The last step is the "fault mode identification". The localization on the map (output of the previous stage) enables the identification of the fault mode thanks to a mathematical procedure based on the probability density function.

In the following sections, each step (Fig. 2) will be described more in detail.

3.2 Input data management

As presented previously, a selection among raw monitored data is performed. This procedure is called feature selection (see [23–26]). It is used on structured data to select features that explain most of the system behaviors by eliminating inappropriate and redundant data [23]. This paper will only focus on time-series data. Some basic approaches provide good means to address the feature selection. For example, the variance is a good way to eliminate features with little or no evolution. The correlation coefficient is powerful to identify feature that have the same behavior. Visual analysis highlights features that have unusual trend.

For better analysis purpose, a normalization of the input data is performed. It provides a common scale for the features. Two main methods are used: rescaling and standardizing. The rescaling method is the simplest one and consists to scale data on the range $[0, 1]$ with the following formula:

$$\bar{x}_{ij} = \frac{x_{ij} - \min_i x_{ij}}{\max_i x_{ij} - \min_i x_{ij}} \tag{1}$$

where x_{ij} and $\bar{x}_{ij}$ are, respectively, the original and normalized data values for a sample j of a feature i from the input dataset, with $i = 1 \ldots p$, where p is the number of network input.

The standardizing method uses the following formula:

$$\bar{x}_{ij} = \frac{x_{ij} - \mu_i}{\sigma_i} \tag{2}$$

where μ_i and σ_i are, respectively, the mean and the standard deviation of a feature i. Even if this method provides a uniform scale, normalized inputs data do not belong to the same common scale. That is why the rescaling method will be used to have all input data on the same range $[0, 1]$.

3.3 System map

3.3.1 Overview

Self-organizing maps are neural networks using unsupervised learning inspired from human brain way [11]. They

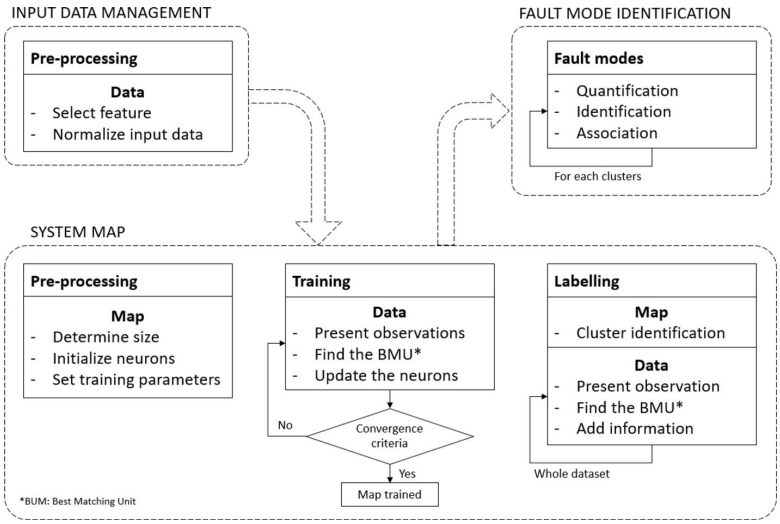


Fig. 2 Fault mode diagnosis methodology

are suitable to produce a low-dimensional representation of the input space of training samples, called a map, to visualize high-dimensional data [27]. SOMs are therefore useful for dimensionality reduction and representation in which the similarity relations between input data are preserved [10]. Its competitive learning capability and the use of the neighborhood function preserve the topological properties of the inputs. The competitive approach aims to put output neurons in competition with each other to be activated, and the winning neuron is the only one that can be activated. As most artificial neural networks, SOMs are developed in two subsequent phases: a training phase and a testing phase. Thanks to the characteristics of this particular neural network, the testing phase can also be used as a labeling part, which is the assignment of information to specific clusters on the map, such as specific faults, or system operational conditions. The goal of the training phase is to teach the algorithm with the dataset in such a way that similar data features are clustered on specific topological regions on the map [28]. Then, the generated map provides clusters, and a health index (HI) is estimated for each node, depicting the degradation status of the studied system.

The SOM neural network building is divided into three stages: preprocessing, training and labeling. A lot of manual work done by experts is needed to perform these tasks.

3.3.2 Preprocessing

SOM topologies can be in one, two or even three dimensions [29–33]. The neurons are localized at lattice nodes. The original SOM [11] is a 2D hexagonal map. Then, successively, 1D lines, 2D rectangular grids or more complex structures, such a star lattices [34] (Fig. 3), have been created. For our case study, a 2D square lattice is used to visualize it as picture.

A square lattice has $n \times n$ neurons of m weights. The number of weights per group (i.e., m) corresponds to the number of inputs to the network. According to [35], a size of the map can be determined by calculating the number of neurons from the number of observations in the dataset such as:

$$M \cong 5\sqrt{N} \tag{3}$$

where M is the number of neurons, and N is the number of observations. A square lattice will have $n = \sqrt{M}$. For example, with about 10,000 observations, Eq. (3) leads to a size $M \cong 500$. For a square lattice, the closest dimension would be a 23×23 matrix.

The next step is the map initialization. There are various ways to set initial weights, such as input vectors randomly selected [36], principal components of the input space [36], large hypercube [37] or random values. A uniform distribution in the range $[0, 1]$ with a probability density function of 1 is considered for usefulness to set neuron weight vectors $w_{ij} = \left(w_{ij1}, w_{ij2}, \ldots, w_{ijm}\right)$ with $i, j = 1 \ldots n$.

3.3.3 Training phase

After the input data are processed and the map is initialized, the map is trained using preprocessed data. Training algorithms related to SOMs are various. Stochastic training as Algorithm 1 is one of the most classical algorithms [38]. Alternatives such as fast batch SOM [39] or growing self-

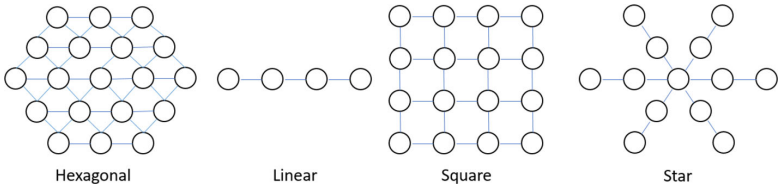


Fig. 3 Examples of lattice structures

organizing map (GSOM) [40] can be faster but are more complex to use. They all rely on the determination of the best matching unit (BMU), which is the smallest distance between the input vector and the weight vector of the map nodes.

The learning process is iterative, until a stopping criterion is met. Examples of criteria are an error estimation such as the quantization error [36] or the so-called "rule of thumb" where the number of steps must be at least 500 times the number of neurons in the map [38]. This last criterion will be used in this paper.

Algorithm 1. Randomly input selection for self- organizing map

```
For t from 1 to Number_Of_Iteration do
  | Pick up a random input vector among the input dataset
  | For each node in the map do
  |   | Calculate the distance between input vector and map node weight vector
  |   | Track the node with the smallest distance (i.e. BMU)
  | End
  | Update the weight vector of the neighborhood of the BMU node and itself
End
```

The weight vector is updated at each iteration as follows:

$$\begin{cases} w_{ij}(t+1) = w_{ij}(t) + h_{kl,ij}(t)\left[x(t) - w_{ij}(t)\right] & \forall n_{ij} \in \mathbb{E}_{BMU} \\ w_{ij}(t+1) = w_{ij}(t) & \forall n_{ij} \notin \mathbb{E}_{BMU} \end{cases} \tag{4}$$

where w_{ij} is the vector weight, t is the t^{th} iteration, $h_{kl,ij}$ is the neighborhood function, $x(t)$ is the input observed, n_{ij} is the node on the map, ij are the node coordinates on a 2D lattice, kl are the BMU node coordinates on the same 2D lattice and $\mathbb{E}_{BMU}$ is the space of BMU neighborhood node and itself. This space is defined by the width of the neighborhood function, also called the BMU radius. A smooth Gaussian kernel is mostly used for the neighborhood function [36, 41]:

$$h_{kl,ij}(t) = \eta(t) \cdot e^{\frac{-w_{kl}(t) - w_{ij}(t)_2^2}{2\sigma^2(t)}} \tag{5}$$

where $\eta(t)$ and $\sigma(t)$ are, respectively, the learning rate and the width of the kernel, which are the decreasing functions of time [36, 38]. This function decreases through the time to improve the neighborhood identification.

The BMU node n_{kl} is defined by:

$$\left\{ n_{kl} | x(t) - w_{kl2}^2 = \arg\min_{ij} x(t) - w_{ij}(t)_2^2 \right\} \tag{6}$$

The SOM training speed is linked to the map size, the number of inputs and the number of samplings. The number of weights could be large, which leads to a slow convergence due to the amount of weight updates involved in each iteration. Several mechanisms have been developed to address this problem, such as optimizing the width of the neighborhood function or learning rate function. According to [36, 41], the Gaussian kernel is a good candidate. The width of the neighborhood function $\sigma(t)$ is chosen as:

$$\sigma(t) = \sigma_0 \cdot e^{-t/\tau_1} \tag{7}$$

where σ_0 is an initial variance set to the map size divided by two [38], and τ_1 is a positive constant. The learning rate function $\eta(t)$ is chosen as:

$$\eta(t) = \eta_0 \cdot e^{-t/\tau_2} \tag{8}$$

where η_0 is an initial learning rate set to 0.9 [38], and τ_2 is a positive constant. The function is limited to a minimum set at 0.01 [38].

For convenience, τ_1 and τ_2 are equal and follow the relation:

$$\tau_i = t_{\max} / \ln \sigma_0 \quad \text{with } i = 1, 2 \tag{9}$$

where $t_{\max}$ is the maximal number of iterations. Those constants lead the exponential decay function radius to 1 when t reaches it maximum value, which is the maximal number of iterations [42].

With this proposed training, the network is able to adapt automatically to the presented input data. There is no need for an objective function as in supervised learning. The main disadvantage is related to the map size. For the training phase, the computational needs increase exponentially with the map size. The inference phase has lower computational needs compared to the training phase. The output is a map that depicts the used dataset as a representation in lower dimension.

3.3.4 Labeling phase

The goal of the labeling phase is to attribute additional information, such as the name, the color or the number of a cluster. Additional information relies on user's needs and is linked to the application. The generated map has observable clusters. Instead of identifying them manually, an automatic cluster identification phase has been created to do so.

The cluster identification phase wants to identify nodes that make up clusters and assign an information to the cluster to which they belong. The classification of map nodes is performed with Algorithm 2. It generates automatically clusters surrounded by boundaries, and an identifier is assigned to them. For example, if the node 43 with the coordinate (4,3) is localized inside the cluster "2," then this value is attributed to the node.

$$H_{ij} = \frac{w_{ij2} - \min_{(i,j)\in C} w_{ij2}}{\max_{(i,j)\in C} w_{ij2} - \min_{(i,j)\in C} w_{ij2}} \tag{10}$$

in which scale node values of each cluster are in the range $[0, 1]$. Those values represent the current state of the studied system. Each cluster has a degradation trend. For a node, a high HI value represents a healthy condition, whereas a low HI indicates a high degradation or a failure.

When a database with more than one fault mode is used to train the network, several subregions could appear on some clusters. Those subregions are linked to different fault modes related to part failures. To identify fault modes automatically, input data need to be labeled through the diagnostic phase.

The diagnostic phase aims at creating knowledge for the fault identification phase. The input data from the training set are again presented to the map. BMU searching provides the cluster number to which they belong and the associated HI. It leads to the input association using Algorithm 3. The building of the fault mode indication becomes possible by knowing exactly which sample appears in which clusters to localize occurring faults.

Algorithm 2. Cluster identification

```
For each Node on the Map not seen do
  | While Node is not a Boundary And has NeighborhoodNodes which are not Boundary do
  |   | Attribute ClusterLabel
  | End
  | Increment ClusterLabel
End
```

Algorithm 3. Diagnostic of input data

```
For each Sample in Input Data do
  | Present the sample to the map and localize the exited BMU node
  | Association to the sample : the cluster number and HI of the BMU node
End
```

A *Node* is considered as seen if it has been assigned to a cluster identifier, called *ClusterLabel*. *NeighborhoodNodes* are nodes that touch it in all four directions: up, down, left and right.

In the end, a map is generated in which cluster regions appear and are defined by boundaries surrounding them. Let us recall that weight vectors are associated with each node of the map. Then, for each cluster, a health index is built for nodes (i, j) that belong to cluster C by Eq. (10)

3.4 Fault modes identification

The identification of the fault modes gives an insight on the system state, such as the probability that a specific fault occurs, or its evolution through the time. Without prior knowledge about datasets, the number of faults is estimated through the fault quantification phase. Their area on the map is approximated thanks to a straightforward methodology based on probabilistic theory during the fault

subregions identification phase. Then, by presenting iteratively datasets with one associated fault to a map, that has one or more unknown fault, the fault modes association phase identifies the unknown faults.

3.4.1 Fault quantification

The fault quantification phase attempts to evaluate automatically the number of different system faults from a dataset (e.g., pressure drop and over-temperature are two different errors). Within a given time series of data referred to as cycles of a specific system, it can be assumed that the last cycle before non-recoverable error corresponds to the error state and can be used to identify faults [43]. At this cycle, the system is considered to be in a defect state and is taken out of service for maintenance actions. Before the stopping of the system, the advanced degradation of parts that were about to fail took place. This should be visible in the dataset. So, the last cycles of the system before breakdown are presented to the SOM and the labeling phase provides the best matching unit (BMU) (i.e., the hit node on the SOM map). The hit BMU can be the same for several instances of the system (for example, different aircraft jet engines belonging to the same family). The quantity of instances hitting this BMU indicates the hit number.

Two ways to estimate the hit number are now introduced:

- H1: using the last cycle of the system
- H2: using the last cluster hit of the system

In the first case, H1, there is only one last hit on a specific cluster for the system. For example, in a dataset with 249 systems, there are 249 last hits, shared by all map clusters, representing the final (most likely) faulty condition in which the system is found to be itself before it was stopped for maintenance. It represents a "sure" faulty condition.

In the second case, H2, the last hit for each system in each cluster is taken into consideration. Then, in the dataset with 249 systems, there are 249 last hits on each cluster. In the case of a six clusters map, this leads to 1494 last hits, representing faulty conditions for those operational conditions.

In the next section, it is shown that H2 provides more information and reliability than H1, and it is a decent approximation. H2 is used for the case study.

The frequency of those hits over each map cluster is linked to the fault number. Indeed, they tend to gather in areas that can be distinguished separately. Those hits are managed with tools from probability theory to build a representation of those subregions. A good candidate is the probability density function (PDF). The kernel density estimator [44] provides the estimation of the PDF such as:

$$\hat{f}_h(\vec{x}) = \frac{1}{n \cdot h} \sum_{i=1}^{N} K\left(\frac{\vec{x} - \overrightarrow{x_i}}{h}\right) \tag{11}$$

where $\vec{x} = (x^1, x^2, \ldots, x^p)$ are real values, $\overrightarrow{x_i}$ are random samples from an unknown distribution, N is the number of observation, K is the kernel smoothing function, which is a Gaussian kernel, and h is the bandwidth. In this study, the bandwidth has been selected at 1% of the SOM map size, leaving out of consideration the boundary nodes between fault clusters. For a map of 25 $\times$ 25 nodes without cluster, $h = 0.25$. According to the targeted application, the rule could evolve. Other kernel parameters are automatically estimated by the algorithm [45]. The PDF represents the probability distribution using the data samples where the kernel distribution sums the smoothing functions for each data value to produce a smooth, continuous probability curve. A 3D-PDF generation is used for each subregion, with node coordinate (x, y) and the hit number as a frequency as z coordinate. The generated function can be estimated at any (x, y) point. The number of peaks of the PDF leads to the number of faults inside each map cluster. Therefore, if a dataset has one or two fault modes, the method should lead, respectively, to one and two peaks for each cluster on the map. The goal of this approach is to be able to get an overview on the number of fault modes that are present in a dataset, without relying on a priori information.

3.4.2 Fault subregion identification

A cluster is surrounded by boundaries, and several small subregions can be estimated inside, related to fault modes. Fault subregions are the extracted regions from a cluster. They are generated from the separation of PDF peaks for a cluster and are used to define each fault area. Indeed, PDF uses Gaussian functions, which can be separated geometrically. However, all cluster nodes are not necessarily classified in a fault area. This is the case for nodes with weak PDF value, far away from the peak center. To address this problem, the PDF of a cluster is estimated at every cluster node. A custom threshold is applied on each estimated PDF, and fault areas are then generated with their own self-defined boundaries. The remaining nodes inside each fault area, after applying the threshold, represent the failure. So, if the node (4,3) is inside the subregion Failure 1, then this node is attributed to it.

3.4.3 Fault modes association

The association of fault modes (i.e., subregions of cluster) with a physical part is performed with similar data, which

present a known defect mode. Prior knowledge about fault modes, which is previously identified, is used. For example, a dataset with one fault mode is presented to a generated map that has been trained with a dataset with two fault modes. The presented dataset will excite nodes from one of the two subregions previously determined. This subregion will correspond to the known fault mode of the presented dataset.

4 Case study on aircraft jet engines

4.1 Overview

To illustrate the present work, a case study on diagnosing jet engines is used. Engine condition monitoring (ECM) allows for regular assessment of the jet engine health state, based on in-flight measured variables on the engine itself, as well as its environment (the aircraft) in its flight conditions. Specific parameter trend evolutions have shown to be early indications for engine degradations, failures and/or malfunctions [9]. Engine condition monitoring consists of a wide range of activities assessing the jet engine health, from the mounting on-wing until its removal. After every flight, performance engineers evaluate the evolution of engine critical parameters and derive from those analyses to anticipate or to avoid incidents, to evaluate the effects of incidents or to provide a clear "no problem for the next few flights" indication. Whenever an engine gets into a much deteriorated health state, no longer allowing operation within regulatory limits, the performance engineer recommends its removal and a precise planning. Actions of the performance engineer aim not only to keep the engine in its optimum operational condition, but also to correct in an early stage any detected malfunction, allowing for staying within safe operation and also reducing fuel consumption and increasing operational punctuality. Therefore, early fault mode identification provides relevant information for the maintenance program. The use of the SOM neural network for this application is particularly interesting for its ability to map the input data without prior knowledge on fault modes. In general, the monitored system does not provide labeled data related to fault modes, whereas in the case of only one fault mode, the situation is straightforward. In the case of multiple fault modes, it becomes more complicated without a proper monitoring to identify them.

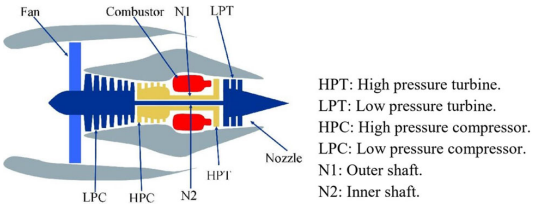


Fig. 4 Simplified diagram of the 90 K engine [46]

4.2 Input data

In this paper, datasets are generated [43] by using the C-MAPSS software [46]. C-MAPSS is a tool for the simulation of a realistic large commercial turbofan engine (Fig. 4) for the 90,000 lb thrust class. Thanks to editable input parameters, it is possible to specify operational profile, closed-loop controllers, and environmental conditions such as altitude. Furthermore, various degradations can be managed in different sections of the engine system.

Using this simulation environment, five datasets were generated by [43]. One of them was used for the prognostics challenge competition at International Conference on Prognostics and Health Management in 2008 (PHM08). In those datasets, the simulated engines have one or six operational conditions (flight phases such as Take-off, Cruise, etc.) driven by engine control settings (altitude, Mach number and Throttle Resolver Angle) and one or two fault modes. In PHM08 (Table 1), there are three datasets with one fault (i.e., #1, #2, and #5) and two with two faults (i.e., #3 and #4). The fault, corresponding to a failed system part, is, respectively, the HPC and the HPC and the fan (see Fig. 4). All dataset characteristics are summarized in Table 1.

Each dataset (i.e., #1 to #5) consists of multivariate time series and is divided into a training set and a testing set, generated by [43]. The database provides those sets in separate files: five training files and five testing files. The training subset is only used for training of the neural network (the learning), whereas the testing subset, with

Table 1 C-MAPSS dataset characteristics

Id	Name	Operational conditions	Fault modes	Failed system part	Number of engines
#1	FD001	1	1	HPC	100
#2	FD002	6	1	HPC	260
#3	FD003	1	2	HPC, Fan	100
#4	FD004	6	2	HPC, Fan	549
#5	FD005	6	1	HPC	218

Table 2 Output variables from C-MAPSS tool

Sensor id	Symbol	Description	Units
1	T2	Total temperature at fan inlet	°R
2	T24	Total temperature at LPC outlet	°R
3	T30	Total temperature at HPC outlet	°R
4	T50	Total temperature at LPT outlet	°R
5	P2	Pressure at fan inlet	psia
6	P15	Total pressure in bypass duct	psia
7	P30	Total pressure at HPC outlet	psia
8	Nf	Physical fan speed	rpm
9	Nc	Physical core speed	rpm
10	epr	Engine pressure ration (P50/P2)	–
11	Ps30	Static pressure at HPC outlet	psia
12	Phi	Ratio of fuel flow to Ps30	pps/psi
13	NRf	Corrected fan speed	rpm
14	NRc	Corrected core speed	rpm
15	BPR	Bypass ration	–
16	farB	Burner fuel–air ratio	–
17	htBleed	Bleed enthalpy	–
18	Nf_dmd	Demanded fan speed	rpm
19	PCNfR_dmd	Demanded corrected fan speed	rpm
20	W31	HPT coolant bleed	lbm/s
21	W32	LPT coolant bleed	lbm/s

Table 3 Extract of the dataset #4

Engine	Cycle	CS1	CS2	CS3	S1	S2	S3	S4	S5	S6	S7	S8	S9
1	1	42.0049	0.84	100	445	549.68	1343.43	1112.93	3.91	5.7	137.36	2211.86	8311.32
1	2	20.002	0.7002	100	491.19	606.07	1477.61	1237.5	9.35	13.61	332.1	2323.66	8713.6
1	3	42.0038	0.8409	100	445	548.95	1343.12	1117.05	3.91	5.69	138.18	2211.92	8306.69
1	4	42	0.84	100	445	548.7	1341.24	1118.03	3.91	5.7	137.98	2211.88	8312.35
1	5	25.0063	0.6207	60	462.54	536.1	1255.23	1033.59	7.05	9	174.82	1915.22	7994.94
1	6	34.9996	0.84	100	449.44	554.77	1352.87	1117.01	5.48	7.97	193.82	2222.77	8340
1	7	0.0019	0.0001	100	518.67	641.83	1583.47	1393.89	14.62	21.58	552.45	2387.92	9050.5
1	8	41.9981	0.84	100	445	549.05	1344.16	1110.77	3.91	5.69	137.13	2211.92	8307.28
1	9	42.0016	0.84	100	445	549.55	1342.85	1101.67	3.91	5.7	138.02	2211.9	8307.81
1	10	25.0019	0.6217	60	462.54	536.35	1251.91	1041.37	7.05	9.01	174.7	1915.23	8005.83
1	11	20.0016	0.7	100	491.19	606.88	1478.02	1233.07	9.35	13.61	333.22	2323.7	8709.62
1	12	34.9993	0.84	100	449.44	554.53	1365.99	1122.73	5.48	7.98	193.67	2222.78	8337.46
1	13	24.9986	0.62	60	462.54	536.32	1257.84	1040.87	7.05	9.01	174.53	1915.28	8000.07
1	14	20.0056	0.7008	100	491.19	607.32	1470.33	1242.41	9.35	13.61	333.71	2323.72	8714.35

Engine	Cycle	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	S21
1	1	1.01	41.69	129.78	2387.99	8074.83	9.3335	0.02	330	2212	100	10.62	6.367
1	2	1.07	43.94	312.59	2387.73	8046.13	9.1913	0.02	361	2324	100	24.37	14.6552
1	3	1.01	41.66	129.62	2387.97	8066.62	9.4007	0.02	329	2212	100	10.48	6.4213
1	4	1.02	41.68	129.8	2388.02	8076.05	9.3369	0.02	328	2212	100	10.54	6.4176

Table 3 (continued)

Engine	Cycle	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	S21
1	5	0.93	36.48	164.11	2028.08	7865.8	10.8366	0.02	305	1915	84.93	14.03	8.6754
1	6	1.02	41.44	181.9	2387.87	8054.1	9.3346	0.02	330	2223	100	14.91	8.9057
1	7	1.3	46.94	520.48	2387.89	8127.92	8.396	0.03	391	2388	100	38.93	23.4578
1	8	1.01	41.6	129.65	2387.97	8075.99	9.3679	0.02	329	2212	100	10.55	6.2787
1	9	1.02	41.44	129.65	2388	8071.13	9.3384	0.02	328	2212	100	10.63	6.3055
1	10	0.94	36.24	164.08	2028.13	7869.41	10.9141	0.02	305	1915	84.93	14.34	8.6119
1	11	1.07	43.86	312.96	2387.83	8050.06	9.1667	0.02	363	2324	100	24.63	14.6705
1	12	1.02	41.45	181.71	2387.86	8056.31	9.3041	0.02	332	2223	100	14.68	8.8752
1	13	0.94	36.42	163.67	2028.14	7865.15	10.8388	0.02	305	1915	84.93	14.41	8.6062
1	14	1.07	43.92	313.3	2387.85	8051.34	9.2272	0.02	364	2324	100	24.3	14.7105

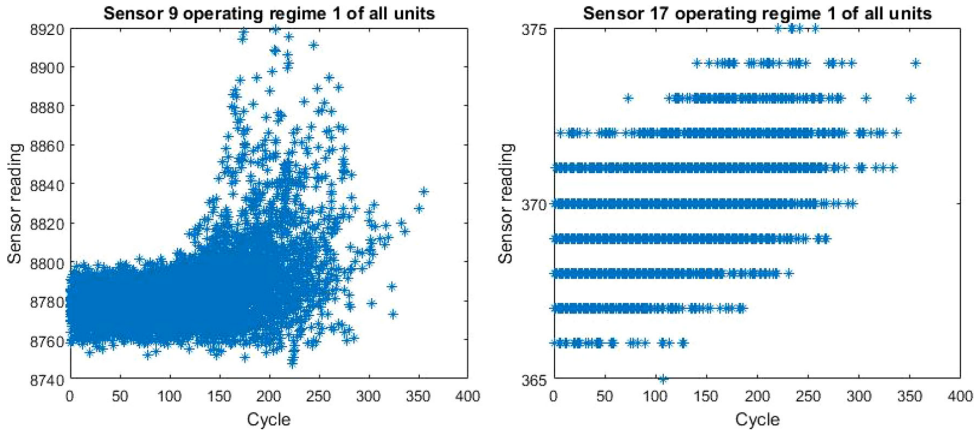


Fig. 5 Trend of sensors in a selected regimes. (Left) Inconsistent end-life trends. (Right) Piecewise trends

different data, is only used for the network validation. In this case study, a dataset contains three input variables representing the engine operational settings, that generate one or six operational conditions and 21 output sensors (Table 2). Dataset is comparable between themselves whether they have the same operation settings, generating the same number of operational conditions. Thus, FD001 and FD003 are comparable as well as FD002 and FD004. However, the FD005 dataset cannot be compared with FD002 or FD004 because the values of the operational settings are not compatible, even if it has six operational conditions.

Table 3 shows an extract of available data for the dataset #4. The other datasets follow the same format.

A reduction in the number of sensors could lead to a drastic reduction in the computational time for the training of the neural network. Following the work of [10, 22, 47] for the PHM08 dataset number #5, the only relevant seven sensors were found to be: 2, 3, 4, 7, 11, 12 and 15.

In fact, among the 21 sensors, constant or binary trend is observed on several sensors that do not provide degradation behaviors. Sensors 1, 5, 6, 10, 16, 17, 18 and 19 are concerned and not considered from the selection. Others sensors provide similar information such as sensors 8 and 13

Table 4 SOM information for all datasets

Id	Number of observations	Features	Map size	Iterations[a]
#1	20,631	3	7	24,500
#2	53,759	3	9	40,500
#3	24,720	3	7	24,500
#4	61,249	3	9	40,500
#5	45,918	3	8	32,000

[a]Estimated following the rule of thumb

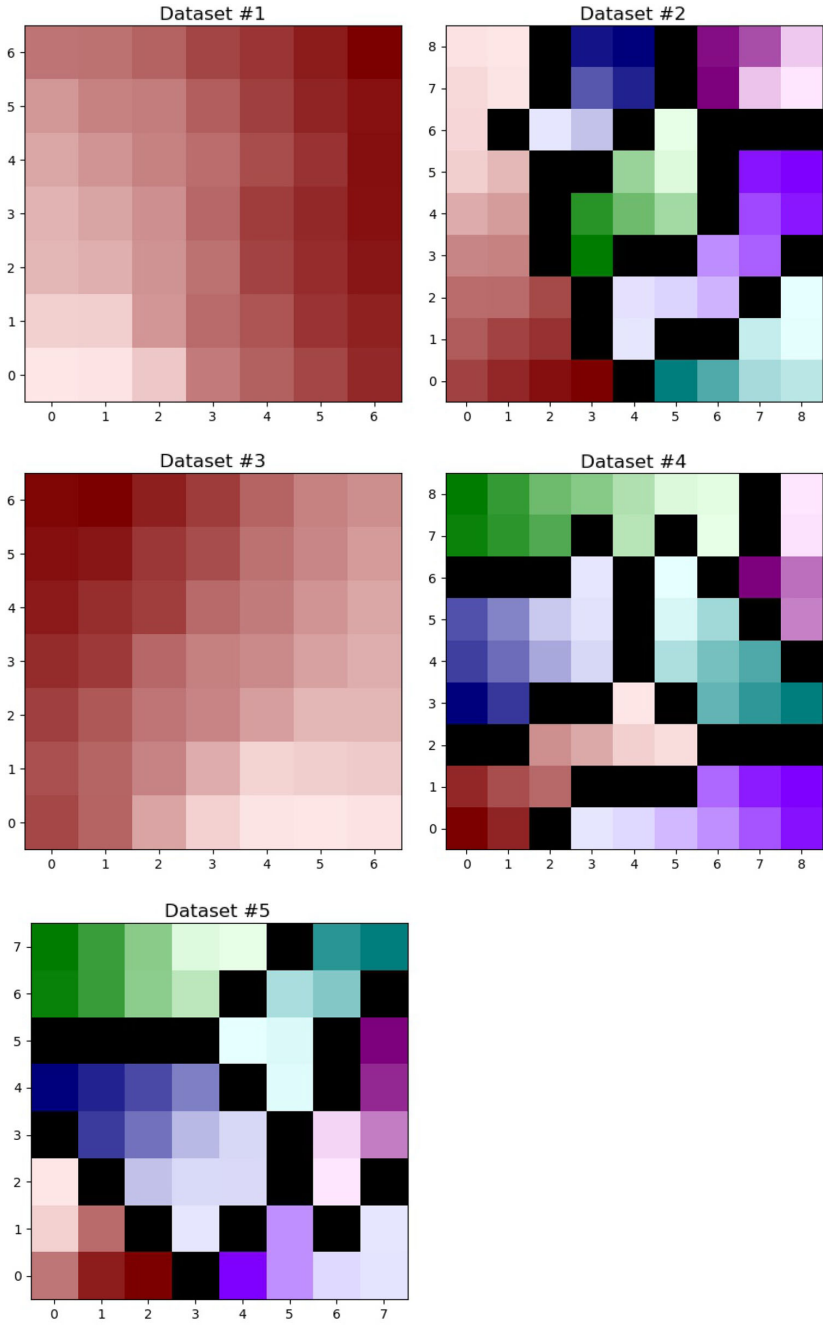


Fig. 6 SOM maps from datasets #1 to #5 with three operation conditions as features

with the sensor 11 by looking at the correlation coefficient with a threshold of 85%. Sensors 9 and 14 show inconsistent end-life trends among the engines (Fig. 5, left), and the sensor 17 is a piecewise constant function (Fig. 5, right). Finally, the sensors 20 and 21 do not bring a clear trend throughout the unit's life according to [22]. Through those steps, the final seven sensors are determined. The

Table 5 Training SOM information for all datasets

Id	Number of observations	Features	Map size	Iterations[a]
#1	20,631	7	19	180,500
#2	53,759	7	24	288,000
#3	24,720	7	20	200,000
#4	61,249	7	25	312,500
#5	45,918	7	23	264,500

[a]Estimated following the rule of thumb

same selected sensors have been taken into account for the datasets #1, #2, #3 and #4.

4.3 Operational mode labeling

The case study contains five datasets generated by [43] (see Table 1), where it is known that one or six different operational conditions are used. According to the complexity of the case study, manual operational mode labeling may not be possible by hand. To demonstrate the power of the automated SOM, they are performed following the system map process introduced in the previous section (see Fig. 2).

The three operational settings (altitude, Mach number and Throttle Resolver Angle) in the dataset are used as inputs to the SOM. Due to the number of inputs, the map size is determined with Eq. (3) and the result is divided by four, custom factors established through multiple empirical experiments. A bigger (or smaller) map leads to an increase (and decrease) in the cluster numbers and may lead to an inconsistency in the representation of information. Further development will be done to address this empirical estimation. The convergence criteria of the "rule of thumb" are used, leading to a number of iterations of 500 times the number of neurons. Table 4 summarizes SOM information used.

The training phase generates maps in Fig. 6.

The maps reveal one cluster for the datasets #1 and #3, whereas the datasets #2, #4 and #5 show six clusters. This means that there is one operational condition for #1 and #3 and six operational conditions for #2, #4 and #5, which is in line with Table 4. The diagnostic phase from the system map process provides exactly the same operational mode labeling as what was obtained manually.

4.4 System map generation

4.4.1 Preprocessing and training

The SOM will now be trained with the seven sensors identified in the previous section. The number of neurons is determined with Eq. (3) and divided by two to reduce the computational time. Table 5 summarizes SOM information used.

The training phase generates the maps in Fig. 7.

For each dataset in Fig. 7, the number of clusters is identical to Fig. 6, corresponding to the number of operational modes. The cluster identification phase provides the same cluster information with the seven selected sensors, compared to the operational mode labeling in Sect. 4.3. It provides reliability in the unsupervised approach.

It appears that the maps for the datasets #3 and #4 show two darker colors on each cluster, which means that there are two fault modes in each cluster. This matches the information of Table 1. The color degradation corresponds to the evolution of the HI (i.e., degradation status). Lighter colors correspond to a healthy system, whereas the darker colors mean an advanced degradation. Other datasets have only one dark colors on each cluster; they have one fault mode. To confirm that, mathematical tools are now introduced.

4.5 Fault mode identification

4.5.1 Fault quantification

The kernel density estimator Eq. (11) is applied on each cluster of each map under H2. It generates a PDF to identify the number of fault modes. Thanks to the SOM map (Fig. 7), the minimum probability density function kernel bandwidth without cluster boundaries can be obtained (see Table 6).

Figure 8 presents the PDF generated for a particular cluster. On those figures, PDF values of node coordinates (x, y) and hit numbers z are normalized. The number of fault modes is easily identifiable visually as well as automatically. This procedure is performed for each generated cluster on each map, and results are compared with the information provided by the datasets. It results that the fault number is well identified, with two fault modes for the database #3, #4 and one for the others.

In Sect. 3.4.1, two ways to evaluate the hit number were presented. Here, we would like to evaluate whether H1 is more pertinent than H2. H1 is relevant in terms of interpretation. However, the dataset used provides few engines (Table 1). Table 7 summarizes the number of samples according to hypotheses presented in Sect. 3.4.1.

Under hypothesis H1, due to a lack of samples, only one out of two fault modes is identified. For dataset #4, it has two fault modes and six operational conditions; the 249 samples of H1 represent around 21 samples per fault per operational conditions. Following the philosophy of H2, around 100 samples per fault per operational conditions are needed for proper identification.

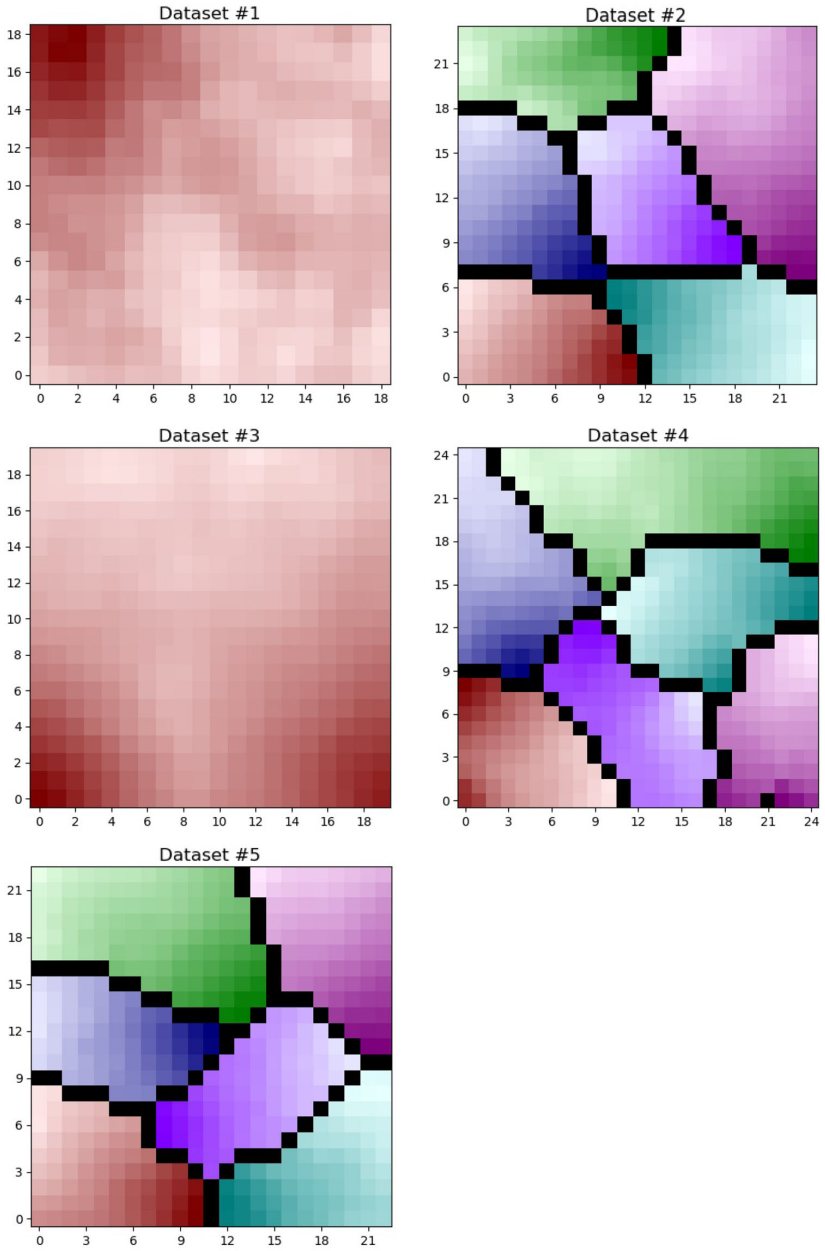


Fig. 7 SOM map from datasets #1 to #5 with 7 physical sensor data as features

For each engine, the last flight cycle spent on each operational condition (H2) is compared to the last flight cycle before a fault occurs (H1) to quantify the reliability of H2. For example, the engine 12 (Table 8) has six operational conditions. Table 8 summarizes the flight cycle for both cases and the error of H2 compared to H1. The

Table 6 PDF kernel bandwidth

Id	Map size	Bandwidth h
#1	19	0.19
#2	24	0.22
#3	20	0.20
#4	25	0.22
#5	23	0.21

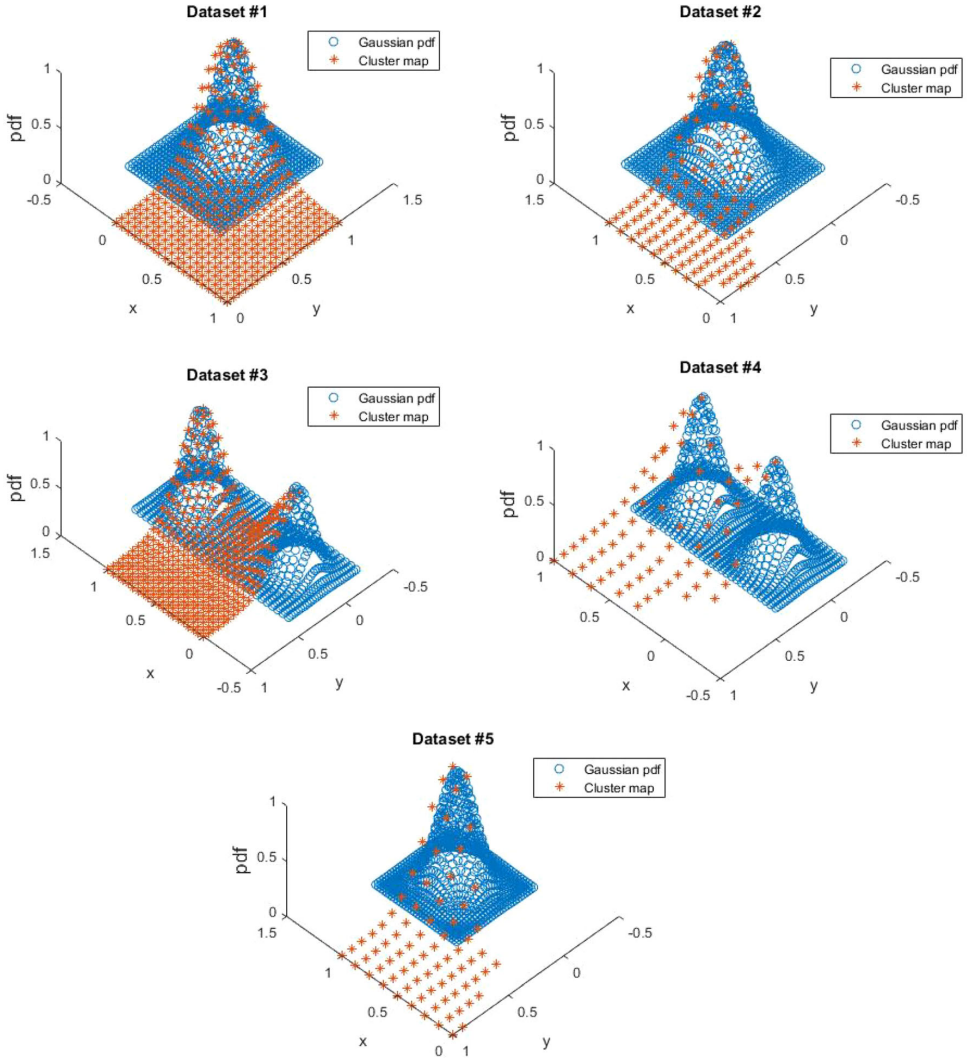


Fig. 8 PDF on a cluster for datasets #1 to #5

Table 7 Number of sample for datasets #1 to #5

Id	H1	H2
#1	100	100
#2	260	1560
#3	100	100
#4	249	1494
#5	218	1308

error is above 10% (custom threshold) for the clusters 1 and 5. That means engine 12 belongs to the group of engines, where H1 is more relevant than H2.

Table 8 Example of comparison for engine 12

Engine 12			
Cluster	H1	H2	Error (%)
1	320	260	18.75[a]
2		320	0.00
3		300	6.25
4		289	9.69
5		277	13.44[a]
6		310	3.13

[a]Error above 10%

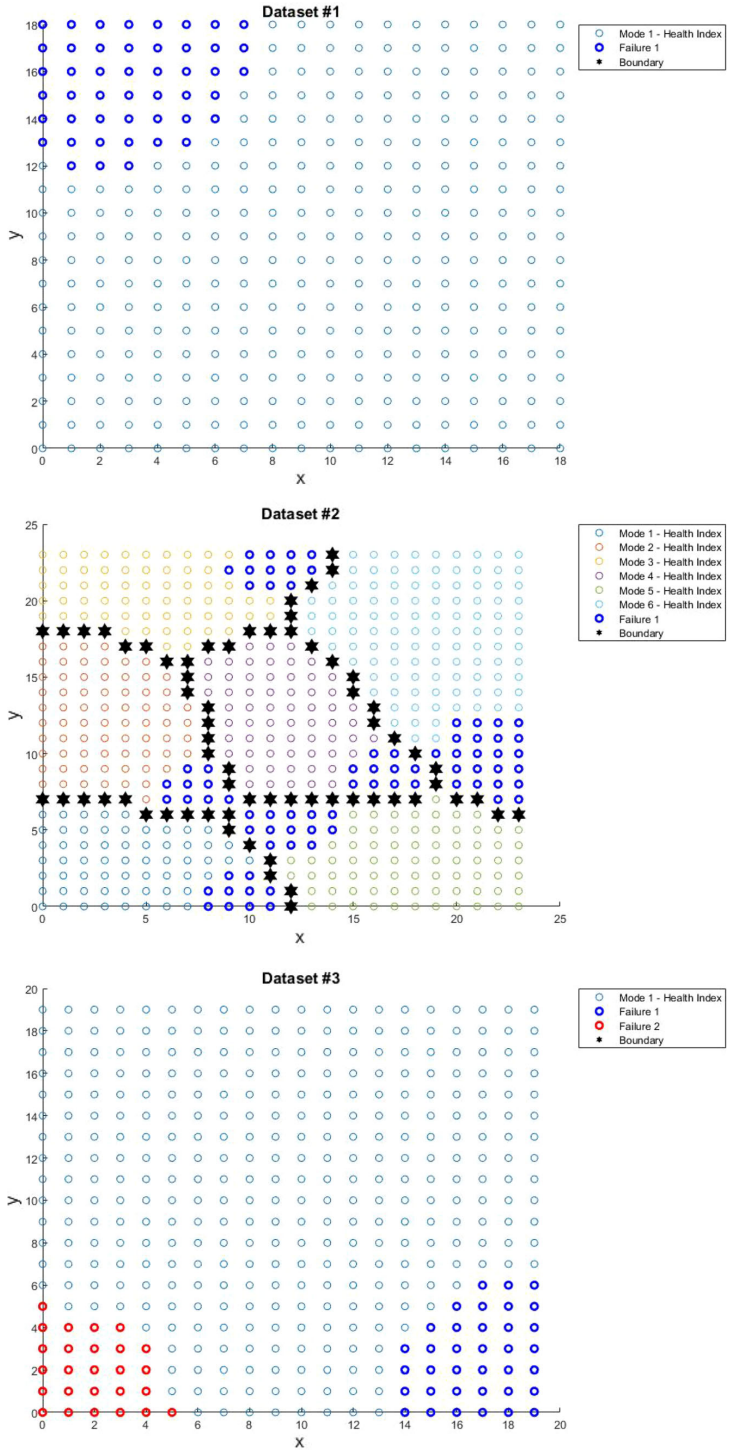
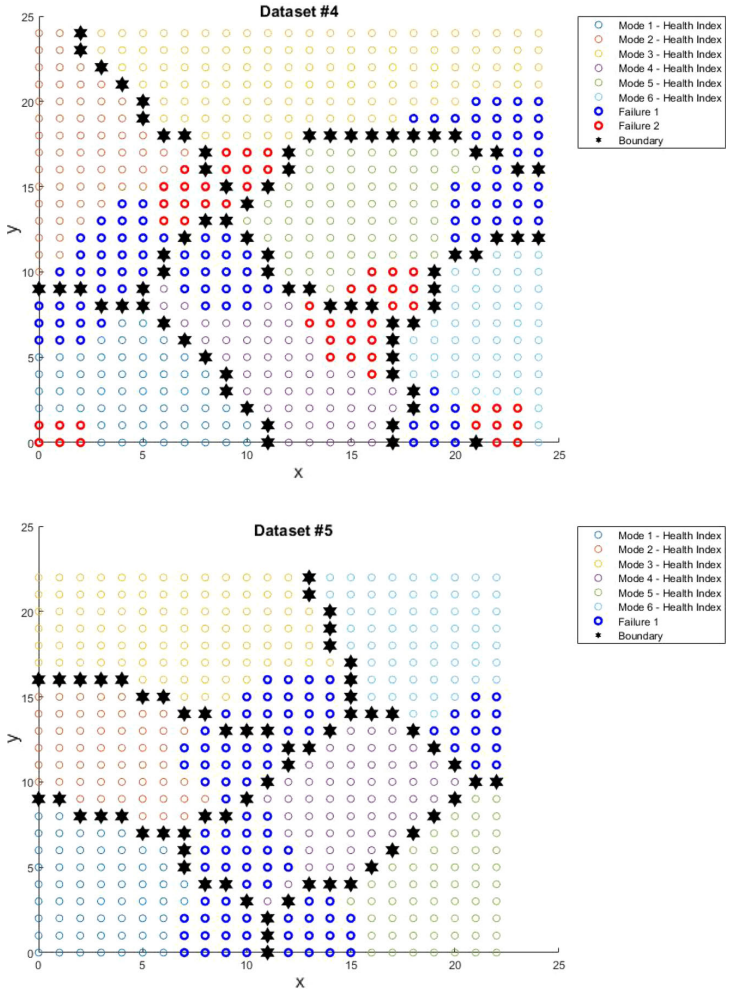


Fig. 9 Failure modes for datasets #1 to #5

Fig. 9 continued



This evaluation is performed for all datasets with a custom threshold of 10%. It results that there are around 3% of engines where H2 is not relevant. For a dataset of 249 engines, H1 is more relevant than H2 for only eight engines. H2 is therefore acceptable for this case study.

4.5.2 Fault subregions identification

For the fault subregions identification, only map nodes that are included in the PDF shape are retained. The user customizes the threshold according to the required precision. This is linked to the probability of training engines of which their last flight cycle has hit the fault mode area. A threshold of 0.40 was used, meaning that all nodes with a probability density lower than 40% are removed. The results are shown in Fig. 9.

When an engine degrades up to a point a failure mode is likely to happen, fault mode area is crossed on each operational mode. Based on this observation, the fault mode is considered to be the same on each map cluster and labeled as failure in Fig. 9.

4.5.3 Fault modes association

The fault modes association phase can only be performed on similar datasets as explained in Sect. 4.2. Thus, the datasets #1 and #3 are similar (similar operational settings and operation conditions) as well as the datasets #2 and #4. The use of datasets with one fault mode (i.e., the datasets #1 and #2) to identify the same failure on the maps trained with two failures (i.e., the datasets #3 and #4) results in a

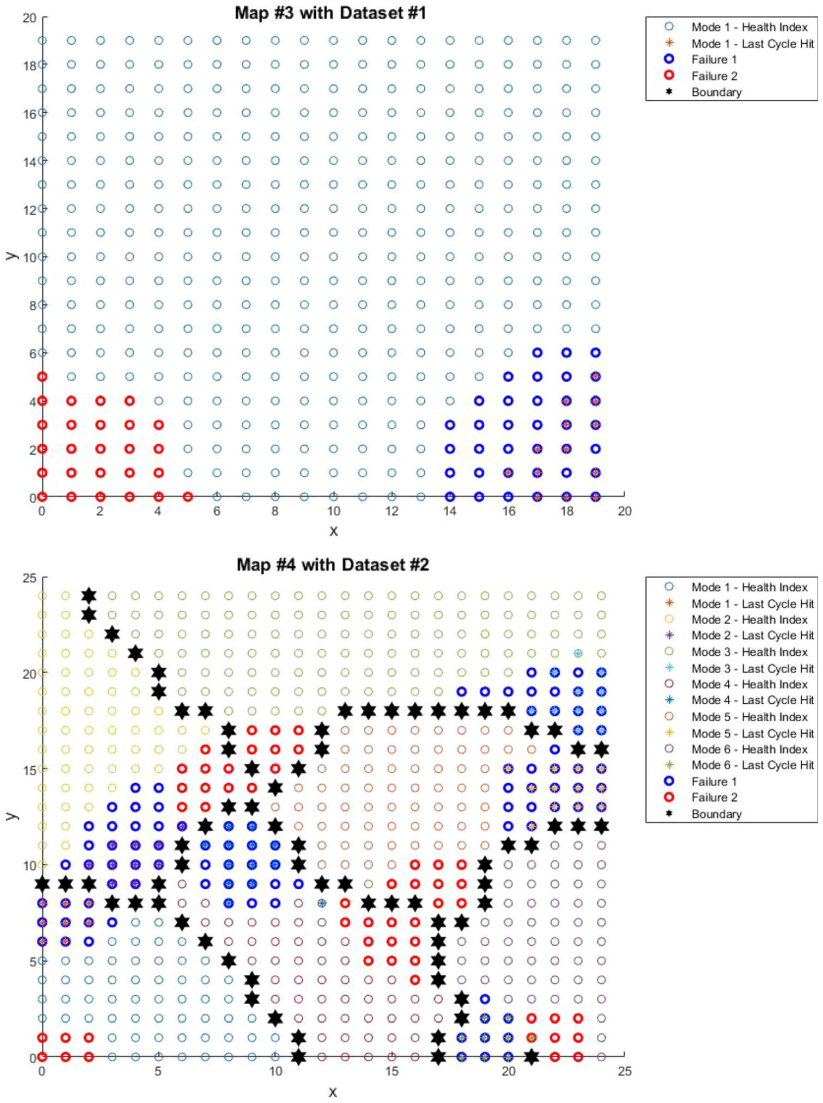


Fig. 10 Fault modes identification for maps #3 and #4

clear and unambiguous identification. The found fault mode is associated with the corresponding failure.

As shown in Fig. 9, the datasets #3 and #4 have both two fault modes, named, respectively, failures 1 and 2. Yet, the faulty system part is unknown. As mentioned, datasets #1 and #2 are used to identify one of the two failures on maps trend with datasets #3 and #4.

Figure 10 shows that all engines from the datasets #1 and #2 are, respectively, in the cluster failure 1 of the datasets #3 and #4, following the hit number estimation 'H2' (see Sect. 3.4.1). Knowing that #1 and #2 have HPC fault modes (Table 1), failures 1 and 2 are, respectively, identified as HPC and fan fault mode. With this knowledge, the fault modes of each engine in the datasets #3 and #4 can be determined.

However, on the map #4 (Fig. 10), some engines are out of a failure area, such as in modes 3 and 4, or misclassified, such as in mode 6. With the hit number estimation 'H2,' it represents 0.19% of error, whereas with the hit number estimation 'H1,' all engines are perfectly classified for this study. The failure identification procedure is then considered satisfactory.

5 Conclusion

This study addresses early detection and classification of faults through an unsupervised learning approach, without prior knowledge, based on self-organizing maps (SOMs). This neural network is at the core of the automatized approach. A complete process to comprehend the concept and to use SOM has been presented. The SOM has some advantage such as unsupervised training and is useful for dimensionality reduction and representation of complex and large datasets thanks to the map visualization. A methodology has been described to build and to configure the SOM according to the case study. The article highlights the possibility to identify the operational mode and fault modes inside generated maps with a methodology relying on the kernel density estimation. The methodology has been illustrated on a case study for diagnosing jet engine datasets. Without prior knowledge on the faults, the proposed algorithm was able to identify the number of operational modes as well as the fault mode number for the five datasets. Furthermore, the fault subregions identification estimates fault mode areas on the map, leading to failures identification on each cluster.

The unsupervised classifier used is a SOM neural network. There exist different types of unsupervised clustering techniques such as hierarchical or Bayesian clustering that could provide different results according to the case study [48]. Another candidate for future work could be a network based on restricted Boltzmann machines (RBM) [49] that will provide a probability distribution over its set of inputs.

In the current study, the automatic fault mode identification was experimented up to two fault modes. For future work, the study should be extended to more than two failure modes on the same case study. Two ways to evaluate the hit number have been explored. Other hypothesis could be examined. The used feature for #1 to #4 was supposed to be same than for #5. A generic approach to get automatically the best set of feature for different types of data will be a good solution to consider, as well as for the custom factor for the map size reduction. A study could be performed to explore the use of the presented approach with a different case study.

Acknowledgements The author affiliated to Sogeti High Tech and ISAE-SUPAERO gratefully acknowledges his colleagues who provided insight and expertise through this paper.

Compliance with ethical standards

Conflict of interest Sébastien Schwartz, Juan José Montero Jimenez, Michel Salaün and Rob Vingerhoeds declare that they have no conflict of interest.

References

1. Simon DL, Rinehart AW (2014) A model-based anomaly detection approach for analyzing streaming aircraft engine measurement data. In: Volume 6: Ceramics; controls, diagnostics and instrumentation; education; manufacturing materials and metallurgy, Düsseldorf, Germany. ASME, p V006T06A032. https://doi.org/10.1115/gt2014-27172
2. Naderi E, Meskin N, Khorasani K (2012) Nonlinear fault diagnosis of jet engines by using a multiple model-based approach. J Eng Gas Turbines Power 134(1):011602. https://doi.org/10.1115/1.4004152
3. Zeng D, Zhou D, Tan C, Jiang B (2018) Research on model-based fault diagnosis for a gas turbine based on transient performance. Appl Sci 8(1):148. https://doi.org/10.3390/app8010148
4. Naderi E, Khorasani K (2017) Data-driven fault detection, isolation and estimation of aircraft gas turbine engine actuator and sensors. In: 2017 IEEE 30th Canadian conference on electrical and computer engineering (CCECE), Windsor, ON. IEEE, pp 1–6. https://doi.org/10.1109/ccece.2017.7946715
5. Sarkar S, Jin X, Ray A (2011) Data-driven fault detection in aircraft engines with noisy sensor measurements. J Eng Gas Turbines Power 133(8):081602. https://doi.org/10.1115/1.4002877
6. Svärd C, Nyberg M, Frisk E, Krysander M (2014) Data-driven and adaptive statistical residual evaluation for fault detection with an automotive application. Mech Syst Signal Process 45(1):170–192. https://doi.org/10.1016/j.ymssp.2013.11.002
7. Pourbabaee B, Meskin N, Khorasani K (2016) Robust sensor fault detection and isolation of gas turbine engines subjected to time-varying parameter uncertainties. Mech Syst Signal Process 76–77:136–156. https://doi.org/10.1016/j.ymssp.2016.02.023
8. Venkatasubramanian V (2005) Prognostic and diagnostic monitoring of complex systems for product lifecycle management: challenges and opportunities. Comput Chem Eng 29(6):1253–1263. https://doi.org/10.1016/j.compchemeng.2005.02.026
9. Vingerhoeds RA, Janssens P, Netten BD, Aznar Fernández-Montesinos M (1995) Enhancing off-line and on-line condition monitoring and fault diagnosis. Control Eng Pract 3(11):1515–1528. https://doi.org/10.1016/0967-0661(95)00162-N
10. Montero Jimenez JJ, Vingerhoeds R (2018) Enhancing operational fault diagnosis by assessing multiple operational modes. In: MOSIM'18—Conférence Internationale de Modélisation, Optimisation et Simulation, Toulouse
11. Kohonen T (1982) Self-organized formation of topologically correct feature maps. Biol Cybern 43(1):59–69. https://doi.org/10.1007/BF00337288
12. Germen E, Başaran M, Fidan M (2014) Sound based induction motor fault diagnosis using Kohonen self-organizing map. Mech Syst Signal Process 46(1):45–58. https://doi.org/10.1016/j.ymssp.2013.12.002
13. Côme E, Cottrell M, Verleysen M, Lacaille J (2010) Aircraft engine health monitoring using self-organizing maps. In: Perner P (ed) Advances in data mining. Springer, Berlin, pp 405–417. https://doi.org/10.1007/978-3-642-14400-4_31
14. Cottrell M, Gaubert P, Eloy C, François D, Hallaux G, Lacaille J, Verleysen M (2009) Fault prediction in aircraft engines using self-organizing maps. In: Príncipe JC, Miikkulainen R (eds) Advances in self-organizing maps, vol 5629. Springer, Berlin, pp 37–44. https://doi.org/10.1007/978-3-642-02397-2_5
15. Katunin A, Amarowicz M, Chrzanowski P (2015) Faults diagnosis using self-organizing maps: a case study on the

DAMADICS benchmark problem, pp 1673–1681. https://doi.org/10.15439/2015f26
16. Yu H, Khan F, Garaniya V (2015) Risk-based fault detection using self-organizing map. Reliab Eng Syst Saf 139:82–96. https://doi.org/10.1016/j.ress.2015.02.011
17. Chen X, Yan X (2012) Using improved self-organizing map for fault diagnosis in chemical industry process. Chem Eng Res Des 90(12):2262–2277. https://doi.org/10.1016/j.cherd.2012.06.004
18. Dharshini R, Hemanandhini S (2016) Brain tumor segmentation based on self organising map and discrete wavelet transform. In: 2016 international conference on computer communication and informatics (ICCCI), Coimbatore, India. IEEE, pp 1–9. https://doi.org/10.1109/iccci.2016.7479960
19. Peel L (2008) Data driven prognostics using a Kalman filter ensemble of neural network models. In: 2008 international conference on prognostics and health management. https://doi.org/10.1109/phm.2008.4711423
20. Jolliffe I (2011) Principal component analysis. Springer, Berlin. https://doi.org/10.1007/b98835
21. Sammon JW (1969) A nonlinear mapping for data structure analysis. IEEE Trans Comput 100(5):401–409. https://doi.org/10.1109/t-c.1969.222678
22. Wang T, Jianbo Y, Siegel D, Lee JA (2008) Similarity-based prognostics approach for remaining useful life estimation of engineered systems. In: 2008 international conference on prognostics and health management. IEEE, pp 1–6. https://doi.org/10.1109/phm.2008.4711421
23. Guyon I, Elisseeff A (2003) An introduction to variable and feature selection. J Mach Learn Res 3:1157–1182
24. Jovic A, Brkic K, Bogunovic N (2015) A review of feature selection methods with applications. In: 2015 38th international convention on information and communication technology, electronics and microelectronics (MIPRO), Opatija, Croatia. IEEE, pp 1200–1205. https://doi.org/10.1109/mipro.2015.7160458
25. Saeys Y, Inza I, Larranaga P (2007) A review of feature selection techniques in bioinformatics. Bioinformatics 23(19):2507–2517. https://doi.org/10.1093/bioinformatics/btm344
26. Visalakshi S, Radha V (2014) A literature review of feature selection techniques and applications: review of feature selection in data mining. In 2014 IEEE international conference on computational intelligence and computing research. IEEE, Coimbatore, India, pp 1–6. https://doi.org/10.1109/iccic.2014.7238499
27. Kohonen T (1997) Springer series in information sciences, vol 30. Springer, Berlin. https://doi.org/10.1007/978-3-642-97966-8
28. Kohonen T (2014) Unigrafia, Helsinki, Finland
29. Zin ZM (2014) Cluster and visualize data using 3D self-organizing maps. In: 2014 11th international conference on ubiquitous robots and ambient intelligence (URAI). IEEE, pp 163–168. https://doi.org/10.1109/urai.2014.7057523
30. Azcarraga A, Manalili S (2011) Design of a structured 3D SOM as a music archive, pp 188–197. https://doi.org/10.1007/978-3-642-21566-7_19
31. Gorricha J, Lobo V (2012) Improvements on the visualization of clusters in geo-referenced data using self-organizing maps. Comput Geosci 43:177–186. https://doi.org/10.1016/j.cageo.2011.10.008
32. El Tobely T, Salem A (2005) Position detection of unexploded ordnance from airborne magnetic anomaly data using 3-D self organized feature map. In: Proceedings of the fifth IEEE international symposium on signal processing and information technology, 2005. IEEE, pp 322–327. https://doi.org/10.1109/isspit.2005.1577117
33. Fujimura K, Masuda K, Fukui Y (2006) A consideration on the multi-dimensional topology in self-organizing maps. In: 2006 international symposium on intelligent signal processing and communications, IEEE, pp 825–828. https://doi.org/10.1109/ispacs.2006.364772
34. Côme E, Cottrell M, Verleysen M, Lacaille J (2010) Self organizing star (SOS) for health monitoring. In: European conference on artificial neural networks, pp 99–104
35. Tian J, Azarian MH, Pecht M (2014) Anomaly detection using self-organizing maps-based k-nearest neighbor algorithm. In: European conference of the prognostics and health management society
36. Engelbrecht AP (2007) Computational intelligence: an introduction, 2nd edn. Wiley, Hoboken. https://doi.org/10.1002/9780470512517
37. Su M-C, Liu T-K, Chang H-T (1999) An efficient initialization scheme for the self-organizing feature map algorithm. In: IJCNN'99. International joint conference on neural networks. Proceedings (Cat. No. 99CH36339), vol 3. IEEE, pp 1906–1910. https://doi.org/10.1109/ijcnn.1999.832672
38. Kohonen T (2001) Springer series in information sciences, vol 30, 3rd edn. Springer, Berlin. https://doi.org/10.1007/978-3-642-56927-2
39. Kaski S, Venna J, Kohonen T (2000) Coloring that reveals cluster structures in multivariate data. Aust J Intell Inf Process Syst 6:82–88
40. Alahakoon D, Halgamuge SK, Srinivasan B (2000) Dynamic self-organizing maps with controlled growth for knowledge discovery. IEEE Trans Neural Netw 11(3):601–614. https://doi.org/10.1109/72.846732
41. Natita W, Wiboonsak W, Dusadee S (2016) Appropriate learning rate and neighborhood function of self-organizing map (SOM) for specific humidity pattern classification over Southern Thailand. Int J Model Optim 6(1):61–65. https://doi.org/10.7763/IJMO.2016.V6.504
42. Zhang W, Wang J, Jin D, Oreopoulos L, Zhang Z (2018) A deterministic self-organizing map approach and its application on satellite data based cloud type classification. In: Conference IEEE Big Data
43. Saxena A, Goebel K, Simon D, Eklund N (2008) Damage propagation modeling for aircraft engine run-to-failure simulation. In: 2008 international conference on prognostics and health management. IEEE, pp 1–9. https://doi.org/10.1109/phm.2008.4711414
44. Kafadar K, Bowman AW, Azzalini A (1999) Applied smoothing techniques for data analysis: the kernel approach with S-PLUS. J Am Stat Assoc 94(447):982. https://doi.org/10.2307/2670015
45. Bowman AW, Azzalini A (1997) Applied smoothing techniques for data analysis: the kernel approach with S-Plus illustrations, vol 18. Oxford University Press, Oxford
46. Frederick DK, DeCastro JA, Litt JS (2007) User's guide for the commercial modular aero-propulsion system simulation (C-MAPSS)
47. Hu C, Youn BD, Wang P, Taek Yoon J (2012) Ensemble of data-driven prongostic algorithms for robust prediction of remaining useful life. Reliab Eng Syst Saf 103:120–135. https://doi.org/10.1016/j.ress.2012.03.008
48. Fusco G, Perez J (2019) Bayesian network clustering and self-organizing maps under the test of Indian Districts. A comparison. Cybergeo. https://doi.org/10.4000/cybergeo.31909
49. Zhang X, Yao L, Wang X, Monaghan J, Mcalpine D, Zhang Y (2019) Cs Eess Q-Bio. arXiv:1905.04149

Intentionally left blank

APPENDIX B

Ontology and CBR integration article

The content in this appendix corresponds to work accepted for publication in The 35th annual European Simulation and Modelling Conference, 2021. Reprinted, with permission, Hugo Muñoz Hernández, Juan José Montero Jiménez, Rob Vingerhoeds. Integrating ontologies and case-based reasoning for the development of knozledge intensive intelligent systems. Accepted for publication.

Intentionally left blank

INTEGRATING ONTOLOGIES AND CASE-BASED REASONING FOR THE DEVELOPMENT OF KNOWLEDGE-INTENSIVE INTELLIGENT SYSTEMS.

Hugo Muñoz-Hernández
Rob Vingerhoeds
ISAE-SUPAERO
10 Avenue Edouard Belin 31055 Toulouse, France
hugo.munoz-hernandez@student.isae-supaero.fr
rob.vingerhoeds@isae-supaero.fr

Juan José Montero-Jiménez
TEC-Tecnológico de Costa Rica
Calle 15, Avenida 14
1 km al sur Basílica de los Ángeles
Provincia de Cartago, Cartago, 30101, Costa Rica
juan.montero@itcr.ac.cr

KEYWORDS
Ontology, Case-Based Reasoning (CBR), similarity function.

ABSTRACT

Case-Based Reasoning (CBR) allows emulating the human inference of solutions to problems profiting from previous experience. The integration of CBR with ontologies, structured organization of semantic knowledge, has been in the attention for some time, aiming to create powerful knowledge-intensive systems capable of proposing appropriate solutions to problems. This entails having collected an appropriate number of previous cases as well as having established a suitable ontology for the application domain. This paper focuses on the integration of CBR and ontologies to support the case representation, case base storage, and semantic similarity estimation. Different alternatives for such integration are explored and the approach has been tested in the creation of a Decision-Support System for the design of predictive maintenance systems. This work opens the window to significantly improve the capabilities of a CBR system by using the knowledge materialization and the reasoning features of a specific domain ontology.

INTRODUCTION

Since a long time ago, man has been fascinated with making a machine that had the same capabilities as human beings (think, speak, move, ...). That is the origin of the vast domain of Artificial Intelligence, which definition for this paper is considered as the following:

Artificial intelligence supplies a collection of techniques to manipulate knowledge in such a manner that new results emerge and new inferences that were not explicitly programmed.

Broadly, two areas can be distinguished in AI: symbolic AI and sub-symbolic AI. The latter comprises neural networks, data-driven techniques, that have had a lot of attention since the last 20 years. The former, symbolic AI, concerns knowledge-based systems and may involve techniques such as rule-based, fuzzy logic, Case-Based Reasoning where the knowledge is contained in well-defined blocks of previous experience, etc (Vingerhoeds et al. 1995).

In this paper, the focus is on Case-Based Reasoning and ontologies. Case-Based Reasoning was developed under the philosophy that human beings think and reason using analogies and examples, rather than rules (Kolodner 1993). The idea is that one may recall previous similar situations when being confronted with a new problem. Starting from this previous experience (knowledge) ideas can be derived for addressing the new situation. Case-Based Reasoning is therefore an approach in which specific knowledge of previously experienced problem situations is being used to solve a new problem. This is being done by finding similar previous cases and reusing previous experiences. Allowing continuously to add new cases, new pieces of knowledge, Case-Based Reasoning supports incremental, sustained learning, where information from new situations is kept for future use.

Ontologies are formal explicit descriptions of concepts in a domain of discourse, properties of each concept describing its features, attributes and restrictions (Noy and McGuinness 2000). One of the most common goals in developing ontologies is "sharing a common understanding of the structure information among people and software agents" (Gruber 1993). However, there are also several other motivations to create ontology models such as enabling reuse and analysis of domain knowledge or making domain assumptions explicit. Ontologies are powerful tools for knowledge representation.

In this paper, the integration of ontologies with Case-Based Reasoning is being addressed. The goal of such an integration is to make use of the advantages of both approaches. For example, ontologies enable the processing and sharing of knowledge that can be used at different tasks of Case-Based Reasoning systems, such as representing the input problem, enhancing similarity assessments, case representation, case abstraction and case adaptation.

The paper is organised as follows. In the next section, the context is presented, including an overview of CBR, ontologies, and their integration. After that, an-

other section is focused on explaining the use case for which the integration of CBR and ontologies has been developed. Then, the Development section shows how the integration was carried out, followed by some results and a discussion. The paper concludes by summarizing the lessons learnt and providing indications for future work

CONTEXT

The trigger for the work presented in this paper can be found in the development of a Decision-Support System for the design of predictive maintenance systems (Montero Jimenez et al. 2021). Ontologies have been used as formal knowledge representations that support a CBR system. This section provides the theoretical background of both technologies.

Case-Based Reasoning (CBR)

Case-Based Reasoning consists of proposing solutions to problems within a certain domain profiting of the knowledge derived from previous cases, representing previous knowledge. Indeed, one of the characteristics of human learning is to use past experiences as a reference for the future. A complete reasoning cycle in CBR is divided into four phases(Aamodt and Plaza 1994): retrieval, reuse, revise and retain.

The development of CBR systems starts by defining the case structure, a set of variables that will be used to describe the problem and its corresponding solution from the past. The problem attributes or features are used to estimate the similarity between the target case and the cases stored in a case base. Once the features have been established, the next step is to define similarity functions. These are algorithms, with diverse levels of complexity, that when applied to a pair of values of the corresponding variable they return a similarity value number, normally between 0 and 1. Each problem attribute has its own local similarity. An aggregation function is used to consolidate all local similarities in a global similarity value. Each local similarity can receive special weights that multiply the local similarity values when calculating the global similarity. Case-based reasoning is a flexible reasoning paradigm. It is capable to compute similarities and retrieve similar cases from a case base even when only partial information is known from the target case.

In this work, the software *myCBR*[1] was used. The choice for this software was due to the availability of the *Java* source code, which allowed for adapting the system to the needs of the study. Different types of case attributes for similarity definition are available in *myCBR*, of which three have been used in the current research: *String*, *Symbol* and *Integer*. Both *String* and *Symbol* attributes have textual values, whereas for *Symbol* the values are limited to a list of allowed values and fixed similarities among them, and *String* similarity is based on free text. By default, the *String* similarity function assigns a binary similarity to each string but this function can also include further analysis by applying the *Levenshtein* string comparison method that can tolerate typing errors in the string and still compute a similarity value. The function applied to *Integer* attributes is very flexible, and the user can control the shape of the function $\mathbb{Z} \rightarrow \mathbb{R}$. Once the case concept is created and the attributes are defined, a case base can be stored as a list of instances of the concept with values assigned to each one of the attributes. Then, the system will be ready for querying and retrieving cases. A deeper explanation of how to model knowledge in *myCBR* can be found in (Bach et al. 2014).

[1]Available online. URL: http://www.mycbr-project.org/. Last visited June 2021.

Ontologies

An ontology defines a set of representational primitives with which to model a domain of knowledge or discourse (Gruber 2009). In a practical sense, ontologies represent a vocabulary of concepts whose basic structure is a hierarchy of classes and subclasses. Instances can be defined as individuals belonging to classes in ontology, so as they will match the common features defining such classes. Important components of ontologies are the *Object Properties* which can establish links between pairs of instances or classes, and *Data Properties*, which can assign information tokens to particular instances. These properties are defined within a certain domain and range, which means that they are restricted, by definition, to particular classes or data types. In general, when referring to ontology entities, that includes classes, individuals and properties. The *Protégé* ontology editor (Stanford University 2016-2020) was used for the development of the ontology for this research.

The initiative of the Semantic Web by the World Wide Web Consortium (OWL Working Group 2012) is to gather as much knowledge as possible into the internet in a format that is readable for computers. For such purposes, the *OWL2* ontology language is used. Standard ontologies like the BFO (Basic Formal Ontology) (International Organization for Standardization 2020) and CCO (Common Core Ontologies) (Rudnicki 2019) are developed and published to be the roots of all the domain-specific ontologies that may be proposed.

Other for the current study relevant aspects of ontologies concern reasoning and queries. A reasoning process can be performed on an ontology to check its logical consistency. This allows verifying, for example, if the instancing of individuals reveals no class contradictions or if the *Object Properties* declarations respect the restrictions of domain and range. Another important aspect of ontology reasoning is the inference of relations; some links could be established between certain entities in the ontology that was not initially explicit but logically deduced from the original ontology

structure. Various reasoners for *OWL* language exist, amongst which the *HermiT* reasoner (Glimm et al. 2014), which was used for this study. Queries allows extracting information from ontologies; they are questions that can be posed to get a specific information out of the ontology. Three different procedures or languages to query *OWL* ontologies exist: *SPARQL*, *Description Logics (DL)* and *SQWRL*. The *SPARQL* language allow extracting table structured information from an ontology by querying for entities that match certain conditions and have some kind of relation between them. This type of query does not need a reasoning process. A plug-in is available for *Protégé* to execute *SPARQL* queries. In this study, for the *Java* implementation, the *Jena API* has been used. *Description Logics* allow executing simple queries by using a reasoner and checking at first the ontology consistency. They are available by default in *Protégé* and also accessible with the *OWL API* implementation for *Java*. The *SQWRL* language is based on the *SWRL* rule language, and it allows to execute very accurate and specific queries using a reasoner. Again, a *Protégé* plug-in is available. In *Java* it may be used the *SWRL API* with a *drools* reasoning engine implementation available.

An ontology for the selection and assessment of predictive maintenance models was developed to support the CBR approach (Montero Jimenez et al. 2021). The *OMSSA* (Ontology model for Maintenance Strategy Selection and Assessment) includes all the necessary concepts of the maintenance domain to describe predictive maintenance systems architecture and application cases. The ontology was built as an extension of the standard BFO and CCO ontologies in order to be as general as possible and to possibly be re-used in the future by other ontology developers.

Related work: integration CBR-ontologies

Ontologies and Case-Based Reasoning have been increasingly used together in the last decade. One of the most important aspects of developing CBR systems is the vocabulary framework. Ontologies have played an important role in providing this vocabulary framework for several CBR applications. For example, (Qin et al. 2018) implemented and ontology supported case-based reasoning approach for computer-aided tolerance specification. In (Amailef and Lu 2013) an ontology was integrated with a CBR system for emergency response services. (Recio-Garía and Díaz-Agudo 2007) presented a system that brought together the CBR *Java* framework *jCOLIBRI* and *Description Logics* (DL) to calculate a concept-based similarity values between terms according to their position in the classification structure within the ontology. The same framework was used in (Kowalski et al. 2013) for the domain of logistics adding some variables that were defined in a specific domain ontology. In (Yin et al. 2010), CBR and ontologies are used to test a criminal investigation system. An ontology is used as a casebase and structure-based similarity values are calculated for case retrieval.

Instantiated ontologies can be used as case bases for CBR implementations as the case structure can be easily represented in the ontology. Ontologies may also help to compute semantic similarity based on ontological similarity, which can be classified into structure-based similarity and feature-based similarity. Structure-based similarity considers the ontology as a graph where concepts are linked by relations (taxonomic or others). Some methods may be proposed to measure the path distance between a pair of concepts to define their similarity value. For example, in (Avdeenko and Makarova 2018) a hierarchical ontology is used to assign similarity values between some pairs of concepts within a specific domain. Feature-based similarity is focused on comparing sets of features of two different terms, established through their property assertions. Feature-based similarity could provide a deeper and more flexible understanding of the domain vocabulary and thus better results in the semantic similarity computation. That is why in this work a feature-based similarity method is used, see also (Sánchez et al. 2012) and (Bai et al. 2008).

The actual integration of ontologies and CBR does not always receive enough attention in application articles. This paper attempts to cover such gap by providing insight on the different possibilities to integrate ontologies and CBR for the development of knowledge-intensive smart systems.

USE CASE: PREDICTIVE MAINTENANCE SYSTEMS DESIGN

Predictive Maintenance (PdM) is a strategy that aims at triggering maintenance actions based on accurate diagnostics or prognostics before an undesired failure occurs. More specifically, predictive maintenance includes monitoring and modelling the system health, estimating the remaining useful life, and detecting and identifying the actual faults. To perform these tasks there exist several types of models that can be used for diagnostics and prognostics purposes (Montero Jiménez et al. 2020). These models analyze the physical variables measured during the operation of the system of interest. For example, in the case of a turbo-machine, some relevant operation variables to consider for predictive maintenance purposes could be the measurements of combustion temperature, pressure and axis vibrations.

A predictive maintenance approach is normally focused on the health state of the system to be maintained or on the incipient failures that may appear during its functioning, sometimes modelling the evolution of these features over time. For example, a predictive maintenance strategy conceived for health assessment may have as main objective to measure the degradation of the system compared to the optimal operation con-

dition that the system had at the beginning. Another possible task in predictive maintenance is to detect or identify automatically failures that are affecting the system and that might perturb its functioning. In addition, the power of predictive maintenance applications increases when considering the capabilities to forecast the evolution of the system health and even predict an estimation of the remaining useful life.

In order to perform all these tasks, three families of models may be considered: data-driven models, knowledge-based models and physics-based models (Montero Jiménez et al. 2020). Data-driven models have an increased popularity during the last years thanks to the current advances in computational power. Statistical models, stochastic models and machine learning are the main model types within data-driven models. In knowledge-based models, previous experiences are used to infer solutions to current problems, in form of for example rules or cases. Physics-based models are very specific for each application case, as they use a physical simulation of the system to perform predictive maintenance functions with the available data. When developing predictive maintenance systems, one of the challenges lies in the selection of the appropriate model for the knowledge representation (a large amount of options) and to position the retained model(s) for the knowledge representation within the systems architecture (Montero Jiménez and Vingerhoeds 2019).

This work is focused on providing a knowledge-intensive system to find the most adequate predictive maintenance solutions for the particular situations. For this purpose, Case-Based Reasoning appears to a suitable technique, as it allows to deal with symbolic information gathered previous predictive maintenance realisations.

INTEGRATION OF ONTOLOGIES AND CBR FOR THE DEVELOPMENT OF A DECISION SUPPORT SYSTEM

In the current study, a domain ontology (OMSSA) (Montero Jimenez et al. 2021), developed with the ontology editor *Protégé* (Stanford University 2016-2020), was integrated with the *myCBR* engine. An instantiated version of OMSSA serves as case base and provides the knowledge for feature-based similarity measures for the *myCBR* engine. The methodology to estimate feature-based similarity was adopted from (Sánchez et al. 2012). The *OWL API* (Horridge and Bechhofer 2011) is used to develop a direct link between the *myCBR* data files *.prj* and the *.owl* ontologies. The *HermiT* reasoner was used to verify ontology database consistency, infer relations and execute "Description Logic" queries for information extracting, necessary for the feature-based similarity functions. Ontology instantiation is carried out by queries that can retrieve data from different sources, such as for example *.csv* files.

Variables definition for the application case

The application case is a Predictive Maintenance Decision-Support System. In this section, some of the implementation details will be presented, so to show how CBR and ontologies can work together. As mentioned before, a first step to implement a CBR application concerns the definition of the variables describing the cases. In the current study, such cases describe previous solutions of Predictive Maintenance (PdM) that have been successfully implemented on different systems of interest. Based on (Montero Jiménez et al. 2020), the main characteristics of each case were described in a systematic set of data fields. Some of the data fields are used as case retrieval variables. Following the logic of *myCBR*, those variables are attributes that must be included in the concept definition:

- *Task (Symbol)*: what is the specific function of the predictive maintenance system (*e.g.* health modelling).
- *Case study type (Symbol)*: what is the type of maintainable system (*e.g.* a rotary machine).
- *Case study (String)*: specific system to be maintained (*e.g.* jet engines).
- *Input type (Symbol)*: list of measurable physical variables (*e.g.* temperature, pressure and power).
- *Online/Off-line (Symbol)*: predictive maintenance analysis to be done online or offline?
- *Input for the model (Symbol)*: data format type provided to the predictive maintenance module (*e.g.* signal data).
- *Publication year (Integer)*: the year in which the study was published.

The attributes of type *Symbol* take textual values among a list of allowed values that must be specified for each of them. In addition to these data fields that are important for the case retrieval, other variables are considered as solution attributes of the case and may be exploited once the relevant cases are retrieved. Some of the solution attributes suggested by the decision support system include: *study title, publication identifier, predictive maintenance model used, performance indicators, etc.*

Instancing cases in the ontology

Originally, *myCBR* stores the case base in *.csv* tabular format. This *.csv* tabular format was replaced by an instantiated version of OMSSA. A *Java* code was developed with specific methods to read all data and to make it fit in an ontology structure: classes, instances and *Object Properties/Data Properties* assertions. An *OWL API* implementation in *Java* was needed for such a purpose. The *CCO* ontologies are taken as a base, especially the *Information Entity Ontology* and the *Artifact Ontology* (Rudnicki 2019). Most of the classes

in the ontology are defined as subclasses of the already existing classes in the previously mentioned ontologies. The relations used to form the case structure are those compiled in the Table 1. Properties written in blue are included in *CCO*, those in *green* are included in *OBO Relations Ontology* from *BFO* (International Organization for Standardization 2020) and those in black have been specifically created in *OMSSA*. Parentheses contain the parent ontology, considering that a property can be a sub-property of a parent.

Table 1: *Object Properties* in OMSSA.

Property	Domain	Range
designates	*Designative Information Content Entity*	No specific range
describes	*Descriptive Information Content Entity*	No specific range
is carrier of	*Independent continuant*	*Generically dependent continuant*
has part	No specific Domain	No specific range
has title (is carrier of)	*Predictive maintenance article*	*Article title*
has identifier (is carrier of)	*Predictive maintenance article*	*Article identifier*
has publication year (is carrier of)	*Predictive maintenance article*	*Publication year*
has synchronization (has quality)	*Predictive maintenance system module*	*Module synchronization*
has predictive maintenance function (has function)	*Predictive maintenance system module*	*Predictive maintenance module function*

The very basic structure of a case starts with an instance of a *Predictive maintenance case* (subclass of *Designative Information Content Entity*), designating an instance of a *Predictive maintenance system module* (subclass of *Information Processing Artifact*). The items to be maintained are instances of the different subclasses in *Maintainable item* (equivalent to *Artifact*). Then, the models are instances of the type subclasses in the *Predictive maintenance model*. The models operate over a set of variables that are recovered from the operation of the item. These variables are instances of *Data variable* (subclass of *Descriptive Information Content Entity*). As the instances of variables are physical concepts (for example, the temperature), they can be reused in more than one case of the database. For each one of the different measurable magnitudes that are mentioned in the database, there exists one single instance of *Data variable*. The functioning of the module and the models used for a certain case are explained in the corresponding *Predictive maintenance article*. The title and the identifier of those articles are assigned to instances *Article title* and *Article identifier* respectively (both are subclasses of *Designative Information Content Entity*). In summary, a maintenance module uses one or more models to perform a predictive maintenance diagnosis or prognostic task on a certain system (*Maintainable item*) based on the data of measured magnitudes recovered during the system operation. This is identified as a predictive maintenance case which is described in a specific research article (see also Figure 1, please note that some elements were omitted for clarity reasons).

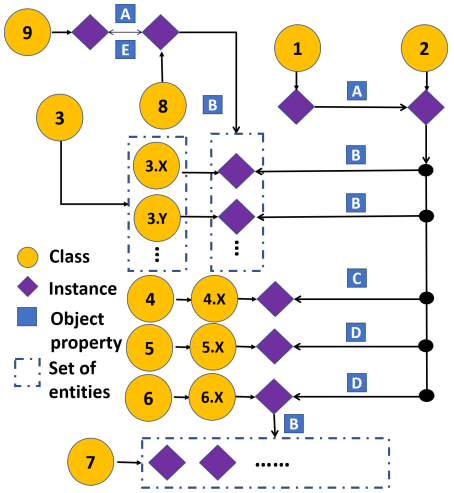


Figure 1: Partial schema of the entities participating in the case representation in the ontology.

- Yellow nodes in Figure 1 represent OMSSA classes:
  1. *Predictive maintenance case*
  2. *Predictive maintenance system module*
  3. *Predictive maintenance model*
  4. *Predictive maintenance function*
  5. *Maintainable item*
  6. *Maintainable item record*
  7. *Data variable*
  8. *Predictive maintenance article*
  9. *Article title*
- The labelled edges in Figure 1 represent OMSSA object properties:
  A. *designates*
  B. *is carrier of*
  C. *has predictive maintenance function*
  D. *has part*
  E. *has title*
- The classes with a tag *N.X* are subclasses of the parent class *N*.

In order to assign the textual values to the article title, the article identifier and the reference tag of the case, which will be materialized as *Designative Information Content Entities* in the ontology, the property *information has text value* is used.

Similarity functions

When defining a similarity function, the type of variable is a major point to consider. The similarity functions of a *Symbol* variable are specified in *myCBR* with a set of pairs of textual values with a similarity value assigned between 0 and 1. These values are estimated according to experimental and physical similarity criteria for the case variables *Case study type*, *Input for the model* and *Online/offline*. One of the objectives of this study is to assess feature-based semantic methods to obtain similarity values, so the procedure proposed (Sánchez et al. 2012) was adapted to the application case and the relations existing in the ontology. The basis of this method is to compare two sets of elements A and B. The following formula is used to obtain a normalized similarity of the sets:

$$Similarity = 1 - \log_2(1 + \frac{A/B + B/A}{A/B + B/A + A \cap B}) \quad (1)$$

In the equation above, A/B means the difference between sets A and B, so the elements from A that do not belong to B. A∩B is the intersection of A and B.

The case feature *Input type* is a list of variables that are monitored during the operation of the system to be maintained. These variables are represented in the ontology with instances of the class *Data variable*. The above-mentioned method is used to compare the set of values in the query case with all cases in the case base. The attribute *Input type* in *myCBR* will store the list of variables as a textual value separated by commas. When executing a query, a list of variables names is proposed, so the algorithm first has to add this list to the allowed values of the attribute. Then, the similarity values are assigned to all the pairs formed by the query string and all the textual values existing in the database. The variable names are separated for each of them and the formula in Equation (1) is applied. Also, a *Levenshtein* method function was implemented to manage slight misspelling in the query string.

Whilst the mathematical method to calculate the ontological similarities for the attribute *Task* is the same as before, the sets of elements to be compared are obtained in a different way. The relation *function uses model* is defined as an inference resulting of the property chain *is predictive maintenance function of* and *is a carrier of* to link instances of functions with instances of models (ontology class *Predictive maintenance model*). This means that the database will be queried automatically for information on the types of models that were used for each type of task concerning predictive maintenance. That allows estimating a semantic similarity between the different subclasses of functions.

The types considered in the ontology are: *Fault detection, Fault feature extraction, Fault identification, Future state forecast, Health assessment, Health modelling*, and *Remaining useful life estimation*. The algorithm will query for each of the mentioned subclasses all the instances of models that are linked to their individuals through the inferred relation *function uses the model*. Each of the instances of models belongs to one of the several subclasses of the *Predictive maintenance model* that were defined in the ontology. The purpose is to list all the model types that have instances linked to any instances of each of the function types.

This is how sets of model types related to functions are compared using the method of Equation (1) to get similarity values. An implementation of the *HermiT* reasoner in *Java* has been used to check the ontology consistency, infer the relations and extract the required information through *Description Logics* queries.

From a similarity point of view, using the variable *Publication year*, referring to the year when a case study was published, the more recent an article is, the more accurate and relevant is supposed to be. A simple *Integer* similarity function has been tested with a similarity of 1 for cases published 5 years ago or less and a constant slope descending to a similarity of 0 at 40 years.

Finally, the default *myCBR* string comparison function with *Levenshtein* method has been used for the *String* attribute *Case study*. To obtain the global similarity value of a case from the similarities of the variables, *myCBR* allows using a simple weighted sum or a euclidean average. The weights may be adjusted, but for this study, they have been set to 1 for all the variables.

Procedure to link ontologies and *myCBR*

The default method for importing databases into *myCBR* is via *.csv* files. As one of the main goals of this project was to create a direct link between ontologies and *myCBR* and use ontologies as database holders, *SPARQL* queries were used to extract the information as organized tabular data and load it automatically into a *myCBR* project file. Using the *Jena API* tool, single string queries with multiple statements were formulated to obtain a query results table with entities names or *Data Properties* content in which the rows correspond to cases and the columns to the variables defining those cases. This table is then stored inside a *myCBR* project file. The procedure may require dividing the query pieces to be executed sequentially when the volume of data is too large, so in that case, the partial results would be simply merged in one data table. The developed application allows loading a case base ready to be used by *myCBR* directly from an ontology .owl file following data translation rules specified in *SPARQL* language.

RESULTS AND DISCUSSION

Keeping in mind the objective to propose an integration procedure between *myCBR* and ontologies,

different kinds of queries were tested in *Java* for *OWL* ontologies. *SPARQL* queries seem to be best placed for such integration, having as major advantages to having a fast execution without the using of a reasoner and that the outputs obtained are immediately organized in tabular data. This makes it easy to introduce the data in the *myCBR* project files.

For the information extraction for semantic similarity values definition, *SQWRL* queries were tested, but the obtained performance was low in terms of calculation speed and memory consumption. For the reasoner implementation tested for *SQWRL* queries, it was found that again a slow process resulted, potentially due to the complexity of the case base. In the end, in this study, *Description Logics* queries were used, only requiring to run the reasoner once for the execution of series of queries.

A feature-based similarity method has been tested in two of the variables describing the study case: *Input type* and *Task*. For the case retrieval variable *Input type*, the feature-based method seemed to be suitable criteria to compare sets of elements, suggested in (Qin et al. 2018). The in this way obtained results have been shown to be relevant and useful, demonstrating that the integration procedure is practical for defining accurate similarity values automatically. As to the similarity values between pairs of predictive maintenance tasks or functions, the corresponding similarity values matrix is shown in Table 2, of which the letters should be read considering:

A) *Fault detection*

B) *Fault feature extraction*

C) *Fault identification*

D) *One step future state forecast*

E) *Multiple steps future state forecast*

F) *Health assessment*

G) *Health modelling*

H) *Remaining useful life estimation*

Table 2: Similarity values matrix for predictive maintenance functions.

	A	**B**	**C**	**D**	**E**	**F**	**G**	**H**
A	1	0.033	0.225	0.02	0.03	0.093	0.079	0.063
B	0.033	1	0.018	0	0	0.02	0.033	0.023
C	0.225	0.018	1	0	0	0.082	0.07	0.073
D	0.02	0	0	1	0.061	0.025	0	0.034
E	0.03	0	0	0.061	1	0.037	0.015	0.03
F	0.093	0.02	0.082	0.025	0.037	1	0.13	0.076
G	0.079	0.033	0.07	0	0.015	0.13	1	0.078
H	0.063	0.023	0.073	0.034	0.03	0.076	0.078	1

As can be seen, the similarity values for non-identical concepts are overall very low meaning that the instances in OMSSA seldom show a model that has been used for two different tasks. Note the existence of one pair (*Fault detection-Fault identification*) that is much higher (over 10% similarity) in comparison to the rest; this is expected as classification models can be used for fault detection and also for fault identification purposes. Another interesting result concerns the pair *One step future step forecast-Multiple steps future state forecast*, basically the latter being an extension of the former, where the similarity could be expected to be close to 1. Both concepts are subclasses of the same predictive maintenance function in the ontology *Future state forecast*. Hence, from a global point of view, the analysis of the results suggest that there is possibly some aspect about the data or the similarity definition that makes it difficult to obtain really meaningful values. A first explanation hypothesis concerns the high degree of specialization of predictive maintenance models in the ontology. The class *Predictive maintenance models* have been automatically filled up with as many subclasses as specific types of models were reported in the literature and therewith declared in the database. Grouping all those types of models into less specific types could have changed the outcome. Another option would be to use a bigger case base, which in itself goes against the generally assumed guidelines for Case-Based Reasoning to work with relatively small case bases. This needs to be investigated in the next steps of the study.

So, the results suggest that some modifications could be needed, specifically in what concerns the similarity between the types of predictive maintenance models considered. However, the value of what has been obtained is not limited to the results in themselves, but also the fact that the integration procedure for ontologies and CBR has been validated so as to be exploited and improved to achieve a superior level of performance for the decision support system.

CONCLUSION AND FUTURE WORK

This paper presents an approach to integrate ontologies and Case-Based Reasoning for a practical application case base for the development of predictive maintenance systems. The automatic transmission of the data in tabular format to ontologies has shown satisfactory results. The implementation of an ontology reasoner and some semantic feature-based similarity methods using *Description Logics* was possible thanks to the *OWL API*. It helped to improve the performances and accuracy of the system for this application and others to be tested in the future. The integration of *myCBR* and *.owl* ontologies through *SPARQL* queries using the *Jena API* is functional and reusable for other applications

This study has especially focused on the retrieval of cases, but the use of ontological knowledge could also be powerful to improve the rest of CBR functions as well. There may be good potential for future work to consider the complete CBR cycle and to automatize all CBR tasks (as suggested in (Mántaras et al. 2005)).

In addition, the study to improve ontological similarity is interesting; more complex and accurate semantic similarity functions could be implemented. In particular, exploiting the ontological reasoner inference could help to make deeper use of the knowledge in the ontology. Another potential improvement may come from taxonomic similarity functions that take better advantage of the hierarchical structure of an ontology. Additional validation work should aim for testing the validity of the recommendations given by the system within the application domain at hand (the design of predictive maintenance system) and to extend with developments on other application domains.

REFERENCES

Aamodt A. and Plaza E., 1994. *Case-Based Reasoning: Foundational Issues, Methodological Variations, and System Approaches. AI Communications IOS Press*, Vol. 7: 1, pp. 39–59.

Amailef K. and Lu J., 2013. *Ontology-supported case-based reasoning approach for intelligent m-Government emergency response services. Decision Support Systems*, 55, 79–97. ISSN 01679236. doi:10.1016/j.dss.2012.12.034.

Avdeenko T.V. and Makarova E.S., 2018. *Knowledge Representation Model Based on Case-Based Reasoning and the Domain Ontology: Application to the IT Consultation. IFAC-PapersOnLine*, 51, no. 11, 1218–1223. ISSN 2405-8963. doi:https://doi.org/10.1016/j.ifacol.2018.08.424. URL `https://www.sciencedirect.com/science/article/pii/S2405896318315519`. 16th IFAC Symposium on Information Control Problems in Manufacturing INCOM 2018.

Bach K.; Sauer C.; Althoff K.D.; and Roth-Berghofer T., 2014. *Knowledge Modeling with the Open Source Tool myCBR.* vol. 1289.

Bai Y.; Yang J.; and Qiu Y., 2008. *OntoCBR: Ontology-based CBR in context-aware applications.* ISBN 978-0-7695-3134-2, 164–169. doi:10.1109/MUE.2008.56.

Glimm B.; Horrocks I.; Motik B.; Stoilos G.; and Wang Z., 2014. *Hermit: An Owl 2 Reasoner. Journal of Automated Reasoning*, 53. doi:10.1007/s10817-014-9305-1.

Gruber T., 2009. *Ontology*, Springer US, Boston, MA. ISBN 978-0-387-39940-9, 1963–1965. doi:10.1007/978-0-387-39940-9_1318.

Gruber T.R., 1993. *A translation approach to portable ontology specifications. Knowledge Acquisition*, 5, no. 2, 199–220. ISSN 10428143. doi:10.1006/knac.1993.1008.

Horridge M. and Bechhofer S., 2011. *The owl api: A java api for owl ontologies. Semantic Web*, 2, 11–21. doi:10.3233/SW-2011-0025.

International Organization for Standardization, 2020. *ISO/IEC FDIS 21838-2.2 Information technology — Top-level ontologies (TLO) — Part 2: Basic Formal Ontology (BFO).* Available online. URL: https://www.iso.org/standard/74572.html.

Kolodner J., 1993. *Case based reasoning.* Morgan Kaufmann. ISBN 978-1558602373.

Kowalski M.; Klüpfel H.; Zelewski S.; and Bergenrodt D., 2013. *Integration of Case-Based and Ontology-Based Reasoning for the Intelligent Reuse of Project-Related Knowledge.* ISBN 978-3642328374, 289–299. doi:10.1007/978-3-642-32838-1_31.

Montero Jimenez J.J.; Vingerhoeds R.; and Grabot B., 2021. *Enhancing predictive maintenance architecture process by using ontology-enabled Case-Based Reasoning.* IEEE ISSE 2021.

Montero Jiménez J.J.; Sebastien S.; Vingerhoeds R.; Grabot B.; and Salaün M., 2020. *Towards multi-model approaches to predictive maintenance: A systematic literature survey on diagnostics and prognostics. Journal of Manufacturing Systems*, 56, 539–557. doi:10.1016/j.jmsy.2020.07.008.

Montero Jiménez J.J. and Vingerhoeds R., 2019. *A System Engineering Approach to Predictive Maintenance Systems: from needs and desires to logical architecture.* 1–8. doi:10.1109/ISSE46696.2019.8984559.

Mántaras R.; Mcsherry D.; Bridge D.; Leake D.; Smyth B.; Craw S.; Faltings B.; Maher M.; Cox M.; Keane M.; Aamodt A.; and Watson I., 2005. *Retrieval, reuse, revision and retention in case-based reasoning. Knowledge Eng Review*, 20, 215–240. doi:10.1017/S0269888906000646.

Noy N.F. and McGuinness D.L., 2000. *Ontology Development 101: A Guide to Creating Your First Ontology.* Stanford University. Available online. URL: `https://protege.stanford.edu/publications/ontology_development/ontology101-noy-mcguinness.html` . Accessed June 2021.

OWL Working Group, 2012. *OWL 2 Web Ontology Language.* World Wide Web Consortium. Available online. URL: http://www.w3.org/TR/2012/REC-owl2-overview-20121211/. Accessed March 2021.

Qin Y.; Lu W.; Qi Q.; Liu X.; Huang M.; Scott P.; and Jiang X., 2018. *Towards an ontology-supported case-based reasoning approach for computer-aided tolerance specification. Knowledge-Based Systems*, 141, 129–147. doi:10.1016/j.knosys.2017.11.013.

Recio-García J. and Díaz-Agudo B., 2007. *Ontology based CBR with jCOLIBRI.* ISBN 978-1-84628-665-0, 149–162. doi:10.1007/978-1-84628-666-7_12.

Rudnicki R., 2019. *An Overview of the Common Core Ontologies.* CUBRC Inc, Buffalo, NY. Available online. URL: `https://www.nist.gov/system/files/documents/2019/05/30/nist-ai-rfi-cubrc_inc_004.pdf`.

Stanford University, 2016-2020. *Protégé official website.* Available online. URL: https://protege.stanford.edu/. Last visited June 2021.

Sánchez D.; Batet M.; Isern D.; and Valls A., 2012. *Ontology-based semantic similarity: A new feature-based approach. Expert Systems with Applications*, 39, no. 9, 7718–7728. ISSN 0957-4174. doi:https://doi.org/10.1016/j.eswa.2012.01.082.

Vingerhoeds R.A.; Janssens P.; Netten B.D.; and Aznar Fernández-Montesinos M., 1995. *Enhancing off-line and on-line condition monitoring and fault diagnosis. Control Engineering Practice*, 3, no. 11, 1515–1528. ISSN 09670661. doi:10.1016/0967-0661(95)00162-N.

Yin Z.; Gao Y.; and Chen B., 2010. *On development of supplementary Criminal analysis system based on CBR and Ontology.* In *2010 International Conference on Computer Application and System Modeling (ICCASM 2010).* vol. 14, V14–653–V14–655. doi:10.1109/ICCASM.2010.5622227.

APPENDIX C

DSS code Guide

The content in this appendix corresponds to a user guide developed in ISAE-SUPAERO for the ontology-enabled CBR engine for predictive maintenance component selection. Hugo Munoz Hernandez, Juan José Montero Jimenez, Rob Vingerhoeds. "Decision Support System Code Guide: Application to Predictive Maintenance Systems Design".

Intentionally left blank

Decision Support System Code Guide

Application to Predictive Maintenance Systems Design

Hugo MUNOZ HERNANDEZ

Juan Jose MONTERO JIMENEZ

Rob VINGERHOEDS

July, 2021

Contents

1 Installation and configuration

For the installation procedure described here it is assumed that the user has installed a version of *Eclipse* that supports at least *Java* 8. Under the previous condition, the following steps may be followed to ensure the correct installation:

1. Download the *.zip* compressed folder of the project and save it in a selected local address in your computer.
2. Make right click over the compressed folder and use *Extract Here*, a decompressed folder with the same denomination has been created. Inside this folder, another folder named *InternshipProject* may be found.
3. Open *Eclipse* and browse to select *InternshipProject* as the working folder.
4. Once *Eclipse* has been initialized, go to *File* → *Import* → *General* → *Existing Projects into Workspace* → *Browse* and select the folder *InternshipProject*. The project will be built in the current workspace of *Eclipse*. The recognition of the folder as an *Eclipse* project is possible because of the file *.project* in the same folder.
5. In the case that some referenced libraries (*.jar*) seem to be missed, right click on *InternshipProject* at the workspace menu on the left → *Build Path* → *Configure Build Path* → *Libraries* → click on *Classpath* in the list below and now the option buttons on the right are available. Delete the paths that are indicated as erroneous and click *Add JARs* on the left to reintroduce the libraries that are missed. Browse to the *external-libs* folder and select the *.jar* files that have to be restored, then just click OK and *Apply and Close*. However, this problem is not likely to occur as the *.classpath* file store the path to all the dependencies of the project. Another possible solution to the problem if this existed would be to open the *.classpath* file with a plain text editor and change the paths that are referencing a local folder in another user computer to a general path starting at the folder *external-libs* of the project.

Now that the code is installed, some configurations are needed to star execution. There is a class named *AppConfiguration* in the *User* package which only contains public static attributes and a couple of methods to build variables. These attributes are read and used by other classes of the project, so they are stored together in the accessory class *AppConfiguration* to be accessed (not modified) by other pieces of code in the application. The very first change that the user must make in the mentioned class before executing the code is setting the *data˙path* attribute to the actual local path of the *data* folder of the project. From that point, the attributes on *AppConfiguration* should be updated to the file names that are wanted to be used inside the *data* folder (*.csv, .owl,.prj*, etc). See the *javadoc* of the project and the comments in the source code of *AppConfiguration* to get known about the meaning of the attributes.

2 Basic usage

The usage of this application will be performed simply by the execution of java classes from Eclipse (or any other IDE). It may also require the manipulation of files (ontology files, *.csv*, *myCBR* project files, etc) in the designated file folders. Unless a modification of the source code is needed for some reason, there are 5 executable classes in the project that concern to the user at this moment: *CSVtoOntologyExec*, *myCBRSetting*, *SPARQL*, *GUI2*, *GUI3*, *OntologytoCSVExec* and *SWRLAPIexec*.

2.1 *SPARQL* queries

A simple executable class has been added to the project to execute *SPARQL* queries on the working ontology in the same way that they are available in *Protégé*. The *Jena* tool (see Section 3 for more information) is used for that purpose. The *SPARQL* queries allows the user to obtain an accurate required information from the ontology. A general reference for the *SPARQL* language can be found at [11]. The user just needs to write the query in a correct syntactical form and execute the class file to get the results of the query on the console. These queries are very quick in their execution as they do not need to run a reasoner and check the ontology consistency. Even if the current implementation in the project is accessory, the *SPARQL* queries could be potentially used to extract information from an ontology for the definition of ontological similarity values.

2.2 *SQWRL* queries

The implementation of the *SWRL API* together with the reasoning engine *drools* allows to add *SWRL* rules to the working ontology execute *SQWRL* queries. See Section 3 for more information. The *SQWRL* language, which is based on the *Semantic Web Rule Language* (*SWRL*), opens the possibility to accurate queries with a relatively simple syntax. There is a paper [21] where the creator of this language give an introduction to the main rules and basic syntax of the queries. An inconvenient has been found for the use the *SQWRL* queries for the application case in the current research project: the reasoner must be run once for each query, so, when working with ontologies having a big number of individuals declared, this process can require an important amount of *RAM* memory in the computer and it can last long if many queries are wanted to be executed. That is why, at this moment, the direct querying through the *OWL API* is used for extracting information from an ontology, as it only requires to run the reasoner (*HermiT*) once before executing as many queries as desired. Indeed, the example class in the project *SWRLAPIexec* allowing to execute *SQWRL* queries may not work correctly due to the compatibility problems of *SWRL API* with the latest version of *OWL API*. See Section 3. However, as it is

potentially useful for the development of the project, the *SWRL API* implementation is still included in the project.

2.3 Load data in a table format (*.csv*) to an ontology file (*.owl*) using *CSVtoOntologyExec*

When executed, the class *CSVtoOntologyExec* will read the content in the specified data base in a *.csv* file and load the information into an ontology file (*.owl*). An important consideration is that the *.owl* files should be written in *RDF/XML* syntax (choose that option when using *Save as* in *Protégé*). In what concerns to *myCBR* retrieval, this class may only be executed when the case base information and similarity values contained in the *.prj* file should be changed or updated to perform CBR queries using new data. The path for working file folder should have been set in the *AppConfiguration* class, together with the corresponding files denominations. In this folder, it is recommended to locate a clean version of the ontology that is wanted to be used (without instances or individual or property assertions) and another copy (of course with a different name) where the data base will be stored. So, when updating the data base the procedure should be as follows:

- Delete the data base ontology file.
- Make a copy of the clean ontology file and change the name as desired . Of course, the same name should be specified in the *AppConfiguration* class (*ont_file_name*).
- Execute *CSVtoOntologyExec*, which will read the clean ontology file (specified in *AppConfiguration* as *base_ont_file_name*) and the data base of the *.csv* file (specified in *AppConfiguration* as *csv*) to merge the information in the ontology data base file.

The translation of the information stored in the table to ontological entities is stated with the appropriate using of the methods of the class *CSVtoOntology* in the package *OntologyTools*. See the *javadoc* of the project for details. By this way, the code is flexible to adapt to the ontological meaning of the content in the different columns and cells forming the table tabular data base.

In this particular case, the executable code in the *CSVtoOntologyExec* class is configured for the Predictive Maintenance data base and the *OPMAD* ontology. Nevertheless, the class could be modified if needed to suit to another different case or to adapt to a restructuring of the current data base and ontology.

2.4 Load data from an ontology file (*.owl*) to a table format *.csv* file using *OntologytoCSVExec*

The class *OntologytoCSVExec* is an executable class allowing to extract data from an ontology file and rewrite it in an organized table format into a *.csv* file. The class *OntologytoTabular* is used to get the data and organize it in a structured tabular *List* object. In the mentioned class, a *Jena API* implementation is used to be able to execute a series of *SPARQL* queries on the working ontology and get a tabular data as a result. The advantages of this type of queries for this application are their execution time (fast execution as they do not need reasoning) and the retrieval results obtained on the shape of organized tables. Now, one important issue should be underlined: the *SPARQL* queries have to be adapted to the particular ontology of the user, as well as the organization criteria for the data when stored as a *.csv* table. Even if it is possible to use one single query for the data extraction, it is recommended to divide the query in various of them when the size of the database is remarkable, as it would be much faster to execute. The list of queries is automatically built by a method in the *AppConfiguration* file. It is required to specify the static *String* parameter *queryHead* in the mentioned class to set the first part of the *SPARQL* queries, that will be common to all the queries in the list. This part of the query aims to define the variables that are going to be retrieved and to initialize the selection block. The closing of the query is also common to all the queries in the list, so it can be specified with the parameter *queryEnd*. The body commands of the queries, which will use the relations existing in the ontology to extract the data, are listed in the parameter *queryBodyList* in the *AppConfiguration* file. Then, the method *queryList* will build the strings with the full queries adding one head and one closing to each one of the body commands blocks. A reference to the *SPARQL* syntax can be found at [11]. The general procedure to determine the query structure is to choose the variables defining the cases, establish which entities in the ontology contain that information and look for properties or ontological relations that build links between the different pieces of data.

The output data of a *SPARQL* query is organized in columns , one for each requested variable. Cells in the table can store one single data value. If a variable value in one of the columns matches more than one value of other variable, then there will be one row in the query results table for each combination of values. That comes to say, using a simplified example, that having a table with only two columns in which each one of the n values of the first variable matches two values of the second variable, then the table will have $2n$ rows, where the values of the first variable variable will appear twice in the column. The same logic extends to several variables with multiple values, that would imply to add as many rows as necessary to include all the combinations of values. An example for the application case of Predictive Maintenance Decision-Support System is shown in Figure 1, where it is observed that there are as many rows concerning the reference index '1' as values of the field *Input type* existing for such case. Note that the indices are ordered using the ontological *Protégé* criteria instead of mathematical order.

Reference	Publication_Year	Task	Case_study	Case_study_type	Input_for_the_model	Input_type
"1"	2019	Health modelling	Simulated_jet-engines_data	Rotary_machines	Time_series	Baypass_ratio
"1"	2019	Health modelling	Simulated_jet-engines_data	Rotary_machines	Time_series	Temperature
"1"	2019	Health modelling	Simulated_jet-engines_data	Rotary_machines	Time_series	Fluid_Pressure
"1"	2019	Health modelling	Simulated_jet-engines_data	Rotary_machines	Time_series	Spinning_speed
"10"	2016	One_step_future_state_forecast	Proton_exchange_membrane_fuel_cell	Energy_cells_and_batte	Time_series	Voltage
"100"	2013	Multiple_steps_future_state_forecast	Lithium-ion_battery	Energy_cells_and_batte	Time_series	Impedance
"100"	2013	Multiple_steps_future_state_forecast	Lithium-ion_battery	Energy_cells_and_batte	Time_series	Temperature
"100"	2013	Multiple_steps_future_state_forecast	Lithium-ion_battery	Energy_cells_and_batte	Time_series	Voltage
"100"	2013	Multiple_steps_future_state_forecast	Lithium-ion_battery	Energy_cells_and_batte	Time_series	Current

Figure 1: Example of SPARQL query results table for the application Predictive Maintenance Decision-Support System.

Additionally, a complete *SPARQL* query for the previously mentioned application case is included in Figure 2 as example to be modified by the user. There is an important detail that must be clarified in what concerns to the variable designations in the *SPARQL* query: as the characters blank space, '-' and '/' are not allowed, they are substituted respectively by '_', double '_' and triple '_'. When rewriting data in a tabular shape using the class *OntologytoTabular*, the headers of the table containing the variables designations are automatically recovered following the inverse transformation rule.

To optimize the execution time of the queries, they can be divided. Individual queries can be executed to obtain partial data tables in which the reference indices column will be extracted together with one or more of the other columns. It is necessary that all the partial queries get the reference indices column as the first column of their results table, so as later all the partial tables can be merged in one. Hence, in most of the cases the columns can be extracted individually, but sometimes it could be required that some columns are extracted through the same query. When two or more columns that store multiple elements in their cells have an order dependency between them, they must be associated to the same query. The order dependency means that the elements listed in the cells of one of the columns are linked to another corresponding element among the ones listed in the cell of another columns. So, the elements in both columns must be listed in the same order to preserve the relation between the elements of data.

Once defined the queries, the results coming from their execution will be rewritten to tables where each row should match only one case, merging the rows coming from the *SPARQL* result to gather in the correspondent cells the set of values for those fields that are multiple-valued. A reference column is chosen (first column) with integer reference indices so as the content of the cells of all the rows with the same index is merged. The general criteria to do so is that no value is repeated in the multiple values cells. However, a set of variables names can be listed using the parameter *Repetition_allowed* for those columns in which repeated values in the same cell should be allowed. As blank spaces are forbidden in the *SPARQL* syntax, when the text values are recovered they are rewritten substituting '_' characters by blank spaces, unless the user specifies no to do so via the parameter *Not_Spaced*. At the end, all the tables will be put together according to the reference indices column, so as the data coming from the list of queries is gathered into one single data table.

Then, here are the features that must be set up in order to define the data transference from the ontology to a *.csv* file:

- *SPARQL* queries list that will determine what information is requested to the ontology and how it is organized. To be set up in the *AppConfiguration* file.
- Parameter list *Not_Spaced* to be specified in the class *AppConfiguration*. In this list it must be included the designation tags of those variables or fields in the table where blank spaces are not desired or expected. For those columns of the table, the text value will be written as it is in origin in the query results, without replacing characters '_' by blank spaces.
- Parameter list *Repetition_allowed* to be specified in the class *AppConfiguration*. In this list the user must include, among those variables (columns) that could contain multiple elements in the same cell, in which of them it is allowed that elements are repeated more than once in the same cell.

For the example below, concerning the Predictive Maintenance Decision-Support System application, it is illustrated the structure of the *SPARQL* query (Figure 2), the results for a case with multiple values (Figure 3) and the final format in the *.csv* table. So, when the results are retrieved from the *SPARQL* query, more than one row may be obtained for each case to get all the values combinations of all the fields that are multiple-valued. In the Figure 3 it is observed how the case with reference index '8' has multiple values for the fields *Input type* and *Models*, while the description tags of the field *Model Type* are matched to particular values of *Models*. In this example, a single query has been used in order to clarify the explanation, but in the real application case the query was divided in parts. Indeed, a query was used for each of the columns in the table except the pair *Model Type* and *Models* and the pair *Performance indicator* and *Performance*, which were respectively included in the same query. The reason for that is that they must keep an order relation for the elements that are listed in the cells. The *Model Type* values must be stored in the same order that the corresponding elements in the column *Models* that they are describing. Same condition is needed for the elements listed in *Performance indicator* and the their value in the column *Performance*. The column *Model Type* is declared in the list *Repetition˙allowed*, so as more than one of the *Models* concerning one case could be described with the same type if they belong to the same family of models.

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
                        PREFIX owl: <http://www.w3.org/2002/07/owl#>
                        PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
                        PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
                        PREFIX def: <http://www.semanticweb.org/j.montero-jimenez/ontologies/2021/2/OPMAD#>
                        PREFIX obo: <http://purl.obolibrary.org/obo/>
                        PREFIX cco: <http://www.ontologyrepository.com/CommonCoreOntologies/>
                        SELECT ?Reference ?Publication_Year ?Task ?Case_study ?Case_study_type ?Input_for_the_model ?Input_type ?Model_Type
?Models ?Online__Off___line ?Performance_indicator ?Performance ?Complementary_notes ?Study_title ?Publication_identifier
                                WHERE {

           ?a rdf:type def:Predictive_maintenance_system_module .
           ?a  obo:RO_0010002 ?d .
           ?b  obo:RO_0010002 ?d .
           ?b rdf:type def:Predictive_Maintenance_Article .
           ?Models rdfs:subClassOf  def:Predictive_maintenance_model .
           ?d rdf:type ?Models .
           ?e rdf:type def:Predictive_maintenance_case .
           ?e cco:designates ?a .
           ?e def:has_text_value ?Reference .
           ?b def:has_publication_year ?Publication_Year .
           ?Task rdfs:subClassOf def:Predictive_maintenance_module_function .
           ?h rdf:type ?Task .
           ?a def:has_predictive_maintenance_function ?h .
           ?a obo:BFO_0000051 ?j .
           ?j rdf:type ?Case_study .
           ?Case_study rdfs:subClassOf def:Maintainable_item .
           ?Case_study_type rdf:type def:item_type .
           ?b obo:RO_0010002 ?Case_study_type .
           ?Input_for_the_model rdfs:subClassOf def:maintainable_item_record .
           ?n rdf:type ?Input_for_the_model .
           ?a obo:BFO_0000051 ?n .
           ?Input_type rdf:type def:Data_variable .
           ?n obo:RO_0010002 ?Input_type .
           ?Model_Type rdf:type def:Model_type .
           ?Model_Type cco:describes ?d .
           ?a def:has_synchronization ?Online__Off___line .
           ?Online__Off___line rdf:type def:Module_synchronization .
           ?b def:has_title ?r .
           ?r def:has_text_value ?Study_title .
           ?b def:has_identifier ?s .
           ?s def:has_text_value ?Publication_identifier .
           ?Performance_indicator rdfs:subClassOf def:Performance_value .
           ?t rdf:type ?Performance_indicator .
           ?t cco:describes ?a .
           ?t def:has_text_value ?Performance .
           ?u rdf:type def:Complementary_notes .
           ?u cco:describes ?b .
           ?u def:has_text_value ?Complementary_notes .

                       }ORDER BY (?Reference)
```

Figure 2: Example of complete SPARQL for the application Predictive Maintenance Decision-Support System.

Reference	Publication_Year	Task	Case_study	Case_stud...	Input_for_the_...	Input_type	Model_Type	Models
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Plastic_strains	Data-driven	Bayes_model
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Mechanical_Pressure	Data-driven	Bayes_model
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Life_cycles	Data-driven	Bayes_model
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Mechanical_stresses	Data-driven	Bayes_model
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Elastic_strains	Data-driven	Bayes_model
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Plastic_strains	Physics-based	Particle_Filter
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Mechanical_Pressure	Physics-based	Particle_Filter
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Life_cycles	Physics-based	Particle_Filter
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Mechanical_stresses	Physics-based	Particle_Filter
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Elastic_strains	Physics-based	Particle_Filter
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Plastic_strains	Physics-based	Physics-based_model_for_track_settlement
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Mechanical_Pressure	Physics-based	Physics-based_model_for_track_settlement
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Life_cycles	Physics-based	Physics-based_model_for_track_settlement
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Mechanical_stresses	Physics-based	Physics-based_model_for_track_settlement
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Elastic_strains	Physics-based	Physics-based_model_for_track_settlement

(a)

Online__Off___line	Performance_indicator	Performance	Complementary_notes	Study_title	Publication_identifier
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa	"A_knowledge-based_prognostics_framework_	"doi.org/10.1016/j.ress.2018.07.004"
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa	"A_knowledge-based_prognostics_framework_	"doi.org/10.1016/j.ress.2018.07.004"
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa	"A_knowledge-based_prognostics_framework_	"doi.org/10.1016/j.ress.2018.07.004"
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa	"A_knowledge-based_prognostics_framework_	"doi.org/10.1016/j.ress.2018.07.004"
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa	"A_knowledge-based_prognostics_framework_	"doi.org/10.1016/j.ress.2018.07.004"
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa	"A_knowledge-based_prognostics_framework_	"doi.org/10.1016/j.ress.2018.07.004"
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa	"A_knowledge-based_prognostics_framework_	"doi.org/10.1016/j.ress.2018.07.004"
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa	"A_knowledge-based_prognostics_framework_	"doi.org/10.1016/j.ress.2018.07.004"
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa	"A_knowledge-based_prognostics_framework_	"doi.org/10.1016/j.ress.2018.07.004"
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa	"A_knowledge-based_prognostics_framework_	"doi.org/10.1016/j.ress.2018.07.004"
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa	"A_knowledge-based_prognostics_framework_	"doi.org/10.1016/j.ress.2018.07.004"
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa	"A_knowledge-based_prognostics_framework_	"doi.org/10.1016/j.ress.2018.07.004"
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa	"A_knowledge-based_prognostics_framework_	"doi.org/10.1016/j.ress.2018.07.004"
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa	"A_knowledge-based_prognostics_framework_	"doi.org/10.1016/j.ress.2018.07.004"
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa	"A_knowledge-based_prognostics_framework_	"doi.org/10.1016/j.ress.2018.07.004"

(b)

Figure 3: SPARQL query results table for one particular case for the application Predictive Maintenance Decision-Support System.

Reference	Publication Year	Task	Case study	Case study type	Input for the model	Input type	Data Pre-proc	Model Approa	Model Type	Models
1	2019	Health modell	Simulated jet-engines data	Rotary machine	Time series	Temperature,	yes	Single model	Data-driven	Logistic regression
2	2019	Health assessi	Simulated jet-engines data	Rotary machine	Time series	Temperature,	yes	Single model	Data-driven	Logistic regression
3	2019	Remaining use	Simulated jet-engines data	Rotary machine	Time series	Health index	yes	Single model	Data-driven	OS-ELM (Online-secuential e
4	2019	Remaining use	Simulated jet-engines data	Rotary machine	Time series	Health index	yes	Multi model	Data-driven	KFOS-ELM (Kalma filter-base
5	2019	Remaining use	Simulated jet-engines data	Rotary machine	Time series	Health index	yes	Multi model	Data-driven	EOS-ELM (Esemble of OS-ELI
6	2019	Remaining use	Simulated jet-engines data	Rotary machine	Time series	Health index	yes	Multi model	Data-driven	AEKFOS-ELM (adaptive-weig
7	2019	Health assessi	Railway track geometry	Structures	Time series	Mechanical st	No	Multi model	Physics-based, Data-driven	Physics-based model for trac
8	2019	Remaining use	Railway track geometry	Structures	Time series	Mechanical st	No	Multi model	Physics-based, Data-driven, Physics-based	Physics-based model for trac

(a)

Online/Off-lin	Performance indicator	Performance	Complementary notes	Study title	Publication identifier
Off-line	Mean absolute percentage error	(<0.05)	1 operational mode, 10(	Aircraft engine	doi.org/10.1016/j.ast.2018.09.044
Off-line	Mean absolute percentage error	(<0.05)	1 operational mode, 10(	Aircraft engine	doi.org/10.1016/j.ast.2018.09.044
Off-line	Score function (the smaller the better), Mean accuracy, Error range	(58.96), (93.26), ([21,33])	1 operational mode, 10(	Aircraft engine	doi.org/10.1016/j.ast.2018.09.044
Off-line	Score function (the smaller the better), Mean accuracy, Error range	(35.78), (95.78), ([15,22])	1 operational mode, 10(	Aircraft engine	doi.org/10.1016/j.ast.2018.09.044
Off-line	Score function (the smaller the better), Mean accuracy, Error range	(34.56), (96.54), ([13,19])	1 operational mode, 10(	Aircraft engine	doi.org/10.1016/j.ast.2018.09.044
Off-line	Score function (the smaller the better), Mean accuracy, Error range	(33.13), (96.76), ([11,18])	1 operational mode, 10(	Aircraft engine	doi.org/10.1016/j.ast.2018.09.044
Off-line	N/A	N/A	No info about operatior	A knowledge-b	doi.org/10.1016/j.ress.2018.07.004
Online	Mean absolute percentage error	(<5%)	No info about operatior	A knowledge-b	doi.org/10.1016/j.ress.2018.07.004

(b)

Figure 4: Final results table as it is loaded in the *.csv* file for the application Predictive Maintenance Decision-Support System.

2.5 Preparing project case base and similarity values for *myCBR* with *myCBRSetting*

The class *myCBRSetting* may be executed when the data base in the *.csv* (file name specified as *csv* in the class *AppConfiguration*) file or the ontology *.owl* file have been changed (normally their modifications are coordinated). In order to perform *myCBR* queries on a new case base, or a case base that has been updated, the information must be written in the *.prj* (file name specified as *projectName* in the class *AppConfiguration*) file, which is the one that contains the project data for *myCBR*. Moreover, the class will run an *OWL* reasoner (more precisely the *HermiT* reasoner, see Section 3) on the ontology that will check its consistency and infer relations for the knowledge contained in the ontology. In consequence, the execution of the class can take a bit of minutes, depending on the ontology size. The main reason to use the reasoner is to be able to perform queries on the ontology to extract information that can be used to calculate ontological similarity values. Moreover, the *HermiT* reasoner allows to make queries about relations that are not initially explicit in the ontology but inferred during the reasoning process. Obviously, if the ontology is not consistent the execution will stop, so to work with an ontological data base it must be consistent. Furthermore, even if the data base can be loaded to the project file either from a *.csv* data file or from an *.owl* ontology, the ontology file is always necessary to obtain the similarity values. The class *myCBRSetting* may be modified to adapt the code to a new application other than the Predictive Maintenance Decision-Support System. This is because the ontology relations and the method for semantic similarity calculation depend obviously on the application.

When executing, at first, the class uses an instance of *CBREngine* to load the current *.prj* file, delete all the existing instances in the case base and introduce the new ones, with their corresponding attribute values, by importing the chosen *.csv* file or the *.owl* ontology file. If the *.owl* database is chosen, the procedure to extract the data is exactly the same as the one followed when executing *OntologytoCSVExec*, see the Subsection 2.4 for more details. The class *OntologytoTabular* will get the data from the ontology following the structure of the *SPARQL* queries list. Then, once the data has been tabulated, it is loaded directly into the *myCBR* project file instead of creating a *.csv* file. So, using the direct importing from the *.owl* file has the same result that creating the *.csv* file form the ontology and loading it with the default *myCBR* importer.

After that, the similarity functions are established with the appropriate values. In particular for the Predictive Maintenance Decision-Support System application, for the field *Task* associated to each one of the cases, the similarity values are calculated using an ontological method. The querying of the ontology uses the classes *DLQueryEngineIRI* and *DLQueryParserIRI* of the package *OnotlogyTools*. See the *javadoc* of the project for more details.

Case variable	Variable type (*myCBR*)	Values
Task	Symbol	Fault feature extraction, Fault detection, Fault identification, Health modelling, Health assessment, Remaining useful life estimation, One step future state forecast, Multiple steps future state forecast.
Case study type	Symbol	Rotary machines, Reciprocating machines, Electrical components, Structures, Energy cells and batteries, Production lines, Others.
Case study	String	The *myCBR* Levenshtein function is used. Similarity is calculated with the quotient $\frac{number\ of\ characters\ reference\ String - Levenshtein\ distance}{number\ of\ characters\ reference\ String}$
Input type	Symbol	A list of variables must be provided separated by ', ' and where all the words should begin with capital letters as it is established in the case base. An additional Levenshtein method in *Java* allows to support misspelling up to 3 erroneous characters.
Online/Off-line	Symbol	Online, Off-line, Both, Unknown synchronization.
Input for the model	Symbol	Signals, Structured text-based, Text based maintenance/ operations logs, Time series.
Publication Year	Integer	This field is not provided by the user, the application will used the current date automatically. The most recent cases in the case base will be prioritized over the older ones.

Table 1: Case variables for querying and retrieval

2.6 Query and retrieval using the *GUI*'s

Once executed *CSVtoOntologyExec* to load the data base into the ontology and *myCBRSetting* to configure the *.prj* file for *myCBR*, only the executable *GUI*'s are required to query and retrieve until the case base is wanted to be modified. The tool *myCBR* uses the case base, the attributes and the similarity functions specified in the *.prj* file to search the most suitable case for the given query. To visualize or to modify manually the project file the *myCBR Workbench* application may be used.

When executing any one of the *GUI's*, the class *Recommender* is used. Most of the similarity functions are defined during the execution of the class *myCBRSetting*, but there is one particular field in the Predictive Maintenance Decision-Support System which similarity values are defined by comparing the current query to all the cases in the data base individually, and that is *Input type*. An analog method is used, which is analog to the one applied to establish the similarity values between the different Predictive Maintenance functions (field *Task*).

Two executable *Graphical User Interfaces* are provided for querying: *GUI2* and *GUI3*.

2.6.1 *GUI2*

The *GUI2* allows the user to perform one query at a time by specifying the following parameters:

- Values of the case fields for the retrieval. Some of them must be typed (*Case Study* and *Input type*) and for the rest of the fields the values are selected form the ones available in a drop down menu.
- Value of the weights assigned to each field.
- Type of amalgamation function that is used to get the global similarity value of each case.
- Number of cases to be retrieved. The resulting list of cases (ordered from higher to lower similarity value) will be shown in the screen after submitting the query.

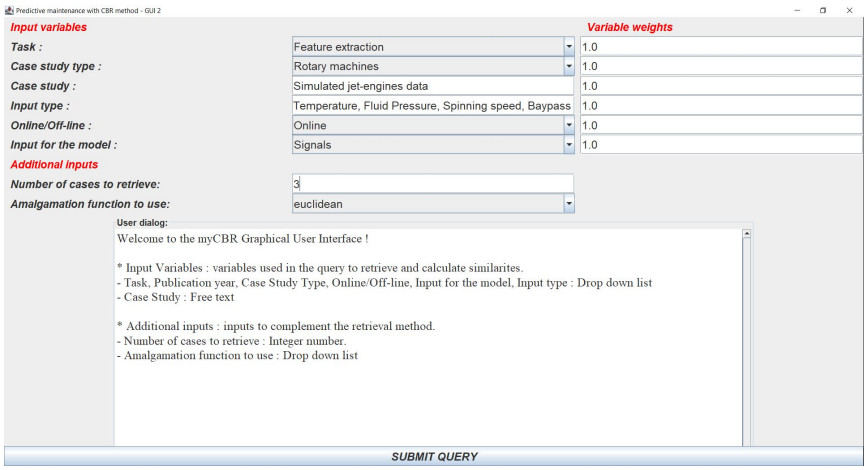


Figure 5: Window of the *GUI2*

If one of the fields is left blank, then its weight in the global similarity value is automatically set to 0. The value of the field *Case study* is expected to be equal to one already existing in the data base, otherwise the similarity value will be 0 for all the cases.

The field *Input type* contains a list of variables (most of them physical variables) that are considered in the Predictive Maintenance model of each case. This list must be typed with the terms separated by ', ' and all the words starting by capital letters, as they appear in the case base. Nevertheless, both fields can manage with possible misspelling errors using the Levenshtein distance method. In particular, the *Case study* field is defined as string type in *myCBR*, and it uses the default Levenshtein comparison function. But, for the *Input type* field, the Levenshtein method is not available in *myCBR* as it is declared as symbolic value. So, an additional method (class *LevenshteinDistanceDP*) to allow up to a distance of three erroneous characters in the spelling is implemented in the similarity function definition (*Recommender*). See the *javadoc* for more details.

2.6.2 *GUI3*

Using this *GUI*, the user is able to execute a list of consecutive queries provided in an input file in *.csv* format and to save their results in separate files (one for each query in the list). It is necessary to prepare an input file with the appropriate structure (see Figure 6). An example of input file is also provided in the actual project folder of the application. For the fields that are stated as symbolic in *myCBR* and for the string variable *Case study*, the user must type in the input file a value which is included among the possible values for each field (see Table 1), otherwise the similarity will be just 0. Symbolic fields, with the exception of *Input type*, do not support misspelling. The field *Input type* contains a list of variables separated by ', ' where the words should begin by capital letters (as they are in the data base). As soon as one of the variables of the list exist in one of the cases in the data base the similarity value will not be 0 for that case.

Task	w1	Case study ty	w2	Case study	w3	Online/Offline	w4	Input for the model	w5	Input type	w6	Number of ca	Amalgamation function
Fault detectio	1	Rotary machi	1	Simulated jet-	1	Online	1	Time series	1	Temperature	1	20	euclidean
Feature extra	1	Rotary machi	1	Simulated jet-	1	Offline	1	Time series	1	Temperature	1	1	euclidean
Fault detectio	1	Rotary machi	1	Simulated jet-	1	Offline	1	Time series	1	Temperature	1	5	euclidean

Figure 6: Example *.csv* input file for the *GUI3*

After having prepared the input file, the *GUI* window will just require to the user to provide the name of the input file and also that of the result files, as shown in Figure 7. For each one of the queries in the list, a result file will be generated with the denomination specified by the user and an index added to the name indicating its position in the list of queries. An example result file is shown in the Figure 8. The content of the result files is another list containing the information about the cases that have been retrieved (as many as demanded by each query).

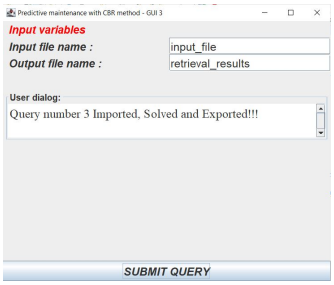


Figure 7: Window of the *GUI3*

Reference	Sim	Task	Case study ty	Case study	Online/Off-lin	Input for the	Model Approx	Models	Input type	Number of in	Performance	Performance	Complementa	Publication identifier
209	0,919	Fault detectio	Rotary machi	Simulated jet-	Online	Time series	Single model	Voting metho	Spinning spee	3	N/A,	N/A,	No info about	doi: 10.12700/APH.15.1.2018.2.10
191	0,901	Fault detectio	Rotary machi	Simulated jet-	Online	Time series	Multi model	Bayes model,	Measurement	2	Robustness ,	N/A,	10 operationa	doi :10.1109/TCST.2011.2177981
166	0,845	Fault detectio	Rotary machi	Wind turbines	Off-line	Time series	Single model	Gaussian proc	Temperature	22	Accuracy,	<0.945,	one case of s	https://doi.org/10.1016/j.renene.2018.10.088
167	0,845	Fault detectio	Rotary machi	Wind turbines	Online	Time series	Single model	Gaussian proc	Wind power	22	N/A,	N/A,	one case of s	https://doi.org/10.1016/j.renene.2018.10.088
210	0,837	Fault identific	Rotary machi	Simulated jet-	Online	Time series	Single model	Expert system	Results of a fa	1	N/A,	N/A,	No info about	doi: 10.12700/APH.15.1.2018.2.10
211	0,818	Fault detectio	Rotary machi	Simulated jet-	Off-line	Time series	Multi model	Hybrid Kalma	Compressor T	8	Visual indicat	N/A,	6 operation m	doi: 10.1109/ACC.2013.6580567
192	0,818	Fault identific	Rotary machi	Simulated jet-	Online	Time series	Multi model	Bayes model,	Measurement	2	Robustness ,	N/A,	10 operationa	doi :10.1109/TCST.2011.2177981
195	0,786	Fault detectio	Rotary machi	Rotor shafts	Online	Time series	Multi model	Gauss-Marko	Imbalance so	2	N/A,	N/A,	No info about	doi.org/10.1117/12.475502
72	0,772	Fault detectio	Structures	Tank reactor	Online	Time series	Multi model	Updated Rule	Temperature,	3	Probability,	From graphics	2 operational	doi:10.1016/j.ejor.2010.03.032
98	0,760	Fault detectio	Rotary machi	Steam Turbin	Online	Signals	Single model	Extreme Grad	Condenser va	7	N/A,	N/A,	No info about	doi.org/10.1016/j.microrel.2013.03.010
59	0,758	Remaining us	Rotary machi	Aircraft bearii	Online	Time series	Multi model	Relevance ve	Health index,	1	Score functio	19,66,	No info about	doi.org/10.1016/j.ress.2017.12.016
58	0,758	Remaining us	Rotary machi	Aircraft bearii	Online	Time series	Multi model	Ensemble lea	Health index,	1	Score functio	7,8,	No info about	doi.org/10.1016/j.ress.2017.12.016
60	0,758	Remaining us	Rotary machi	Aircraft bearii	Online	Time series	Single model	Particle filter,	Health index,	1	Score functio	301,8,	No info about	doi.org/10.1016/j.ress.2017.12.016
204	0,756	Remaining us	Rotary machi	Wind turbines	Online	Time series	Single model	Geolocation p	Distance, Deg	2	Prognostic ho	(65),(0.7),	No info about	doi.org/10.1016/j.renene.2017.05.020
2	0,755	Health assess	Rotary machi	Simulated jet-	Off-line	Time series	Single model	Logistic regre	Temperature,	5	Mean absolut	(<0.05),	1 operational	doi.org/10.1016/j.ast.2018.09.044
1	0,755	Health model	Rotary machi	Simulated jet-	Off-line	Time series	Single model	Logistic regre	Temperature,	5	Mean absolut	(<0.05),	1 operational	doi.org/10.1016/j.ast.2018.09.044
51	0,754	Remaining us	Rotary machi	Simulated jet-	Off-line	Time series	Multi model	LSTM (Long-S	Temperature,	14	Root mean sq	6,9±4,7,	6 CMPASS dat	doi.org/10.1016/j.ast.2019.105423
53	0,754	Remaining us	Rotary machi	Simulated jet-	Off-line	Time series	Single model	Recurrent Ne	Temperature,	14	Root mean sq	10,2±5,8,	6 CMPASS dat	doi.org/10.1016/j.ast.2019.105423
52	0,754	Remaining us	Rotary machi	Simulated jet-	Off-line	Time series	Single model	LSTM (Long-S	Temperature,	14	Root mean sq	8,2±5,7,	6 CMPASS dat	doi.org/10.1016/j.ast.2019.105423
54	0,754	Remaining us	Rotary machi	Simulated jet-	Off-line	Time series	Single model	Gated recurre	Temperature,	14	Root mean sq	10,0±6,0,	6 CMPASS dat	doi.org/10.1016/j.ast.2019.105423

Figure 8: Example *.csv* result file for the *GUI3*

2.7 The *javadoc*

A complete *javadoc* has been generated for this project. It is available in the folder *javadoc* of the project. The easiest way to access to the documentation is opening the *index* file in the mentioned folder. It is an *HTML* file in the standard format for *javadoc* resources available in the web, and it may be opened with a web browser. The documentation contains a detailed descriptions of all the classes and methods organized in linked pages, allowing to navigate through the structure of the code. To update the documentation using *Eclipse*, go to *Project* → *Generate Javadoc*. When the window is open, the user must select the folder where the *javadoc* will be saved. It is recommended to use the folder *javadoc*, after having deleted the previous content to avoid problems. Of course, the generation of *javadoc* depends on the *javadoc* format comments that have been added to the code. See [12] for more details on the *javadoc* comments format.

2.8 Management of the *data* folder

The purpose of the *data* folder in the project is to store all the files that are needed for the execution tasks of the application. It is very important to remember that the denomination of the files that are wanted to be used inside this folder must be in coordination with the names introduced in *AppConfiguration*, so as the code is able to find such files. Some of the main types of files and their function are listed:

- Files *.owl* : the ontology files. One important consideration is that these files must be written in the *RDF/XML* syntax (choose that option when using *Save as* in *Protégé*). They can be opened with *Protégé* to visualize and edit the ontology or with a simple plain text editor, which allows to modify manually the statements. Two files may be used, one file should contain a clean ontology (without the declaration of the individuals in the data base) and the other one with all the data base loaded (and a different name, of course). An ontological data base could be directly used for the *myCBR* project build-up by selecting the appropriate option (user text input) when executing the class *myCBRSetting*. In any case, an ontology structure is required, either formed from a *.csv* table or directly provided by the user.

- Files *.csv* : files with a table format. A data base in *.csv* format may be needed to be loaded in the working ontology. The input and result files used by the *GUI3* are also *.csv* format. These files can be opened with *Excel*. Using a plane text editor to open *.csv* files may be recommended to ensure that no weird characters have been introduced by *Excel* at the beginning of the data (an unusual bug in *Excel* that may alter the name of the first column of data).

- Files *.prj* : the project files of *myCBR*, where all the information concerning the query and retrieval must be stored. This type of file can be opened with the *myCBR Workbench* application.

- Files *.ttl* : these files contain the ontology dependencies (BFO, CCO ontologies, etc). They are necessary to read the ontologies when working with local imports, so as the ontology is independent of the online servers. They are available at the GitHub repository [22], and may be updated with newer versions from time to time. For the execution of the code, some mappers are set using *OWL API* to link the *.ttl* files with the *URI* that are used by the ontologies to invoke the corresponding dependencies. For the *OPMAD* ontology, the following files are required: *ArtifactOntology.ttl, EventOntology.ttl, ExtendedRelationalOntology.ttl, GeospatialOntology.ttl, InformationEntityOntology.ttl, ro-import.ttl, TimeOntology.ttl.*

- File catalog-v001.xml : this file stores the paths to allow *Protégé* to import the local ontology dependencies in *.ttl* format when opening an ontology file that needs those dependencies.

3 Dependencies

Here are listed the main *APIs* and libraries which are needed for the application, which are included in the *external-libs* folder of the project:

- *OWL API* : it is an *API* that allows to read, modify, manipulate and create *.owl* ontologies. The version currently used in the application is 5.1.9, but future updates could be possible. The *API* is implemented by including the necessary *.jar* files in the *external-libs* folder of the application. The last version of *OWL API* is available at the *Maven* repository [2] (artifact owlapi-distribution), but it has been directly obtained as *.jar* files from [1] with all dependencies. Previous versions are also available. A complete documentation can be accessed at [3] and there is an introduction tutorial written by the creators at [6]. The licenses concerning the *API* are the Apache License [4] and the GNU license [5].
- *HermiT* reasoner: it is a reasoner that works in *Protégé*, the most used software for ontology manipulation. There is and implementation for *Java* based on the *OWL API*. The version used is the latest one (1.4.5.519), and it can be found at the *Maven* repository [7] or downloaded from [8] with all dependencies. The reasoner is needed to perform queries on ontologies through *Java*. The documentation for a previous equivalent version (1.3.8.4) is available at [9]. The GNU license es applicable [5].
- *myCBR*: it is an open source tool for case-based reasoning applications. The *Software Development Kit* of *myCBR* project has been implemented in the application with the appropriate *.jar* file. The *myCBR Workbench* application may be very useful for visualizing *.prj* files. All the information concerning *myCBR* project (source code, installation guide, tutorials, javdoc, etc) is available at the website [10].
- *Jena*: it is a tool for ontology manipulation. In paritcular, it is used in this application for adding the capability to perform *SPARQL* queries on ontologies through *Java* with the *SPARQL* executable class in the project, that uses the *Jena* methods. The latest version is used (4.0.0), and it can be found at the *Maven* repository [13] (artifact jena-arq) or downloaded from [14] with all dependencies. A complete documentation of the *Jena Core* is available at [15]. The Apache license [4] is applicable.
- *SWRL API* with *drools* rule engine: the *SWRL API* may be used to implement *SWRL* rules in an ontology through a *Java* application. Moreover, if the *drools* reasoner is added, the application is able to execute *SQWRL* queries on the working ontology. In this case, the latest version of *SWRL API* (2.0.9) is used, which is available at the *Maven* repository [16]and at [17] for direct *.jar* download with all dependencies. A *javadoc* is available at [18]. Nevertheless, the compatibility of the *SWRL API* with *OWL API* is only guaranteed up to version 4.5.9, even if some later versions could be supported. Moreover, to query an ontology with the *SWRL API* and *SQWRL* language, an additional implementation of a reasoner is necessary: the implementation of *drools* engine is available for this purpose. Once again, it may be

found at the *Maven* repository [19] or for direct *.jar* download with all dependencies at [20].

References

[1] JAR download (April 2021): https://jar-download.com/artifacts/net.sourceforge.owlapi/owlapi-distribution

[2] MVN repository (April 2021): https://mvnrepository.com/artifact/net.sourceforge.owlapi/owlapi-distribution/5.1.17

[3] OWL API javadoc (April 2021): https://javadoc.io/doc/net.sourceforge.owlapi/owlapi-distribution/latest/index.html

[4] Apache License version 2.0: https://www.apache.org/licenses/LICENSE-2.0

[5] GNU LESSER GENERAL PUBLIC LICENSE: http://www.gnu.org/licenses/lgpl-3.0.txt

[6] MATENTZOGLU Nicolas., PALMISANO Ignazio. *An introduction to OWL API*. University of Manchester, 2016.

http://syllabus.cs.manchester.ac.uk/pgt/2020/COMP62342/introduction-owl-api-msc.pdf

[7] MVN repository (April 2021): https://mvnrepository.com/artifact/net.sourceforge.owlapi/org.semanticv

[8] JAR download (April 2021): https://jar-download.com/?search˙box=org.semanticweb.hermit

[9] HermiT 1.3.8.4 javadoc (April 2021): http://javadox.com/com.hermit-reasoner/org.semanticweb.hermit/1.3.8.4/overview-summary.html

[10] myCBR website (April 2021): http://mycbr-project.org/index.

[11] SPARQL syntax reference (April 2021): https://www.w3.org/TR/sparql11-query/

[12] *Javadoc* reference (April 2021): https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html

[13] MVN repository (April 2021): https://mvnrepository.com/artifact/org.apache.jena/jena-arq/4.0.0

[14] JAR download (April 2021): https://jar-download.com/artifacts/org.apache.jena/jena-arq

[15] Jena Core 4.0.0 (April 2021): https://jena.apache.org/documentation/javadoc/jena/

[16] MVN repository (April 2021): https://mvnrepository.com/artifact/edu.stanford.swrl/swrlapi/2.0.9

[17] JAR download (April 2021): https://jar-download.com/artifacts/edu.stanford.swrl/swrlapi

[18] SWRL API Javadoc (April 2021): http://soft.vub.ac.be/svn-pub/PlatformKit/platformkit-kb-owlapi3-doc/doc/owlapi3/javadoc/overview-summary.html

[19] MVN repository (April 2021): https://mvnrepository.com/artifact/org.apache.jena/jena-arq/4.0.0

[20] JAR download (April 2021): https://jar-download.com/artifacts/org.apache.jena/jena-arq

[21] O'CONNOR Martin Joseph, DAS Amar. *SQWRL: A Query Language for OWL.* Stanford Center for Biomedical Informatics Research, 2009.

[22] GitHub repository (April 2021): https://github.com/CommonCoreOntology/CommonCoreOntologies

APPENDIX D

Summary in French / Résumé en français

Sommaire

D.1 Introduction

D.1.1 Maintenance Prédictive

La maintenance prédictive est une stratégie de maintenance qui vise à déterminer le moment précis pour effectuer des actions de maintenance. Pas trop tôt, car les actions de maintenance peuvent entraîner le changement de pièces qui ont encore une durée de vie restante significative ce qui représente un coût pour les entreprises ; mais pas trop tard non plus, car une panne inattendue peut à son tour donner lieu à une série de conséquences négatives. La maintenance prédictive est une alternative aux autres stratégies de maintenance traditionnelles, telles que la maintenance corrective et la maintenance préventive. Au lieu d'attendre qu'une panne se produise ou d'effectuer une maintenance basée sur des intervalles de fonctionnement fixes, la maintenance prédictive est basée sur le diagnostic de l'état actuel de l'équipement et sur la prédiction du moment où une éventuelle panne pourrait survenir.

La maintenance prédictive est fortement liée à la maintenance basée sur l'état (Condition-Based Monitoring, CBM) et à la gestion de la santé basée sur les prévisions (Prognostics and Health Management, PHM). La maintenance prédictive et ces deux disciplines englobent le diagnostic et le pronostic de maintenance ; parfois ces termes sont même utilisés comme synonymes. Les deux termes maintenance prédictive et CBM sont apparus dans les années 1940 et font référence à la stratégie de maintenance qui vise à anticiper les pannes des machines en fonction de leur état. Cependant, ce n'est qu'au début des années 1990 que cette stratégie prend de l'importance grâce à la mise en place de systèmes de surveillance et d'outils de calcul capables de surveiller les tâches de diagnostic. Quant au pronostic de durée de vie restante, faisant partie de la maintenance prédictive et de la CBM, cela restait une discipline imprécise. Dès le début des années 2000, la discipline PHM a émergé avec l'objectif de couvrir des problèmes de pronostic. Depuis lors, les diagnostics de maintenance et la recherche de pronostics ont suscité beaucoup d'attention de la part des universités et de l'industrie ; grâce à l'augmentation continue de la puissance de calcul et compte tenu des avantages de sa mise en œuvre. Au cours de la dernière décennie, différentes contributions ont été apportées sous des termes différents (maintenance prédictive, CBM et PHM) ; elles se réfèrent au même domaine de recherche.

D.1.2 Idée conceptuelle et architecture d'un système

Selon le manuel INCOSE [INC15], le cycle de vie d'un système peut être divisé en six étapes génériques (voir Figure D.1). Le cycle de vie d'un système commence par l'étape conceptuelle, qui vise à explorer des solutions possibles pour répondre aux besoins et souhaits initiaux des parties prenantes. La phase conceptuelle est suivie de la phase de développement, au cours de laquelle une conception détaillée des composants du système et de leurs interfaces est réalisée. Après la conception détaillée, commence l'étape de production qui est dédiée à la mise en œuvre du système, c'est-à-dire la fabrication, le codage, la création des différents composants et leur intégration finale dans le système. La vérification et la validation du système font partie de la phase de production. Une fois le système validé, il est livré au client pour sa phase d'utilisation. Au stade de l'utilisation, le système remplit la fonction pour laquelle il a été créé. La phase de maintenance se déroule parallèlement à la phase d'utilisation pendant le fonctionnement du système. Cette étape de maintenance est destinée à assurer l'état de fonctionnement optimal du système. En fin de cycle de vie du système, la phase de retrait vise à bien gérer le retrait du système, sa mise hors service, et son démantèlement.

Le processus d'architecture du système se positionne au cœur de la phase conceptuelle. Cette phase commence par rassembler tous les besoins et souhaits des parties prenantes et les formaliser dans les exigences des different acteurs. Ces exigences sont ensuite utilisées pour créer l'architecture du système

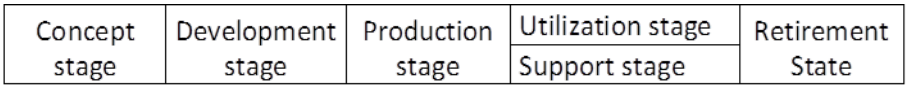


Figure. D.1: Étapes génériques du cycle de vie d'un système (traduit de l'anglais) [INC15]

qui servira de base à la conception détaillée et à la mise en œuvre du système. Le développement de l'architecture peut être divisé en trois niveaux [Roq18]; [INC15] : architecture fonctionnelle, architecture logique et architecture physique. L'architecture fonctionnelle décrit comment les différentes (sous-)fonctions d'un système interagissent les unes avec les autres pour atteindre un objectif spécifique, mais ne fournit aucun détail sur les composants qui remplissent chaque (sous-)fonction. L'architecture logique fournit autant de détails que possible sur les composants de l'architecture et leurs interfaces, mais n'implique pas à ce stade les choix quant aux réalisations et/ou aux technologies spécifiques, ce qui signifie que l'architecture logique montre des composants génériques. L'architecture physique fournit les détails des technologies à affecter à chaque composant logique. L'union de ces trois niveaux d'architecture peut être appelée l'architecture du système et peut alors être définie comme les « concepts ou propriétés fondamentaux d'un système dans son environnement incorporés dans ses éléments, ses relations et dans les principes de sa conception et évolution » [ISO11]. En termes simples, l'architecture traite de la manière dont un ensemble d'éléments, qui peuvent être physiques ou informationnels, sont organisés pour répondre à un objectif spécifique.

D.1.3 Questions à la base de ce projet de recherche

La maintenance prédictive est réalisée à l'aide d'approches spécialisées pour effectuer des tâches de diagnostic et de pronostic, afin de déterminer le bon moment pour déclencher les actions de maintenance. En ce moment, la conception et le développement de ces systèmes spécialisés continuent d'être basés sur des essais et des erreurs. Il existe plusieurs architectures génériques dans les normes et standards, comme [MIM01], mais en réalité la conception d'un nouveau système de maintenance prédictive commence beaucoup plus tôt, au stade conceptuel, moment auquel ces architecture génériques n'aident pas encore l'architecte. En effet, une architecture générique ne fournit pas de liens avec les besoins et souhaits initiaux d'un nouveau système de maintenance prédictive, et ces besoins et souhaits peuvent bien ne pas être couverts par une architecture générique. De plus, une architecture générique ne fournit aucune indication pour sélectionner les technologies appropriées pouvant réaliser les composants génériques [MV19], par exemple pour la détection d'un défaut ou pour l'estimation d'une durée de vie restante à partir d'une série de mesures.

Même si la maintenance prédictive traite en général des tâches de diagnostic et de pronostic, tous les systèmes prédictifs ne couvrent pas exactement les mêmes fonctions. Par exemple, un nouveau système de maintenance prédictive peut être destiné à effectuer des diagnostics sur différents composants d'un système ; plusieurs modules de diagnostic du même type seront nécessaires et les modules de prédiction ne seront pas inclus. Les architectures génériques ne fournissent pas une approche systématique pour les conceptions qui n'ont besoin que d'un sous-ensemble des composants génériques proposés ou lorsque plusieurs composants du même type sont nécessaires.

Il existe un nombre important d'options pour réaliser les fonctions de diagnostic et de pronostic dans un système de maintenance prédictive, et il n'y a pas de directives pour aider l'architecte à sélectionner les modèles, techniques ou algorithmes appropriés qui peuvent exécuter ces fonctions. L'exploration de l'espace de solution pour déterminer les composants appropriés peut alors être complexe et prendre du temps. Cette thèse vise à faciliter le processus architectural des systèmes de maintenance prédictive en permettant une manière plus efficace d'explorer l'espace des solutions et de proposer les composants les plus adaptés à l'architecture du système.

Avant d'aborder la conception de systèmes de maintenance prédictive, il est important de comprendre le domaine de la recherche en maintenance prédictive lui-même et les différentes options disponibles pour effectuer des diagnostics et des pronostics. Les questions de recherche suivantes sont proposées pour guider l'étude de l'état de l'art dans ce domaine :

1. Quelles sont les tendances actuelles du diagnostic et du pronostic en maintenance prédictive ?
2. Quels types de modèles, techniques ou méthodes sont utilisés pour traiter le diagnostic et le pronostic en maintenance prédictive ?
3. Quels sont les principaux défis rencontrés par la maintenance prédictive dans le diagnostic et le pronostic ?

Après l'étude de l'état de l'art, les questions de recherche sont adaptées en fonction des résultats de l'étude et de la motivation initiale de cette recherche liée à la conception de systèmes de maintenance prédictive (voir section D.2).

D.1.4 Organisation du résumé

La section D.2 aborde l'état de l'art de la maintenance prédictive sur la base des questions de recherche initiales introduites ci-dessus. Cette section est liée au chapitre 2 de la thèse, qui est composé d'un article publié qui comprend les tendances actuelles des modèles de diagnostic et de pronostic dans le domaine de la maintenance. La section se termine en affinant les questions de recherche qui ont motivé le reste de la recherche.

La section D.3 propose une approche d'ingénierie des systèmes pour la conception de systèmes de maintenance prédictive, en particulier au stade conceptuel de l'analyse des besoins et souhaits initiaux des parties prenantes jusqu'à la proposition d'une architecture logique. Cette section est liée au chapitre 3 de la thèse qui est composé d'un article publié et présenté lors d'une conférence et se termine en présentant la sélection de la composante maintenance prédictive comme le défi principal qui sera abordé dans les chapitres suivants. Un système d'aide à la décision (Decision Support System, DSS) qui combine des ontologies et un raisonnement basé aux cas est proposé comme une solution possible pour aborder la sélection de composants dans l'approche systématique.

La section D.4 explique l'un des éléments de base du système d'aide à la décision : les ontologies. Des recherches complémentaires ont été spécifiquement menées sur les ontologies qui ont abouti à un article de revue, accepté pour publication au moment de la rédaction de ce manuscrit, et qui présente un modèle ontologique pour la sélection et l'évaluation des stratégies de maintenance (Ontology for Maintenance Strategy Selection and Architecture, OMSSA). Le contexte théorique des ontologies est présenté en soulignant leur importance dans la communauté des chercheurs en raison de leurs capacités à modéliser formellement le vocabulaire d'un domaine spécifique et à le raisonner. La section D.4 explique le développement de l'ontologie qui sera ensuite utilisée dans le DSS pour la sélection des composants.

La section D.5 présente le deuxième pilier du DSS : le raisonnement basé aux cas (Case-Based Reasoning, CBR). Les principes du paradigme de la CBR ont été présentés, y compris les phases de son cycle générique. Dans une première approche pour adresser les questions à la base de cette thèse, le DSS se concentre sur la phase de récupération de la CBR. Une explication de la mise en œuvre du moteur de récupération CBR est fournie à l'aide de code source logicielle.

La section D.6 explique le cadre général d'intégration des composants de base du DSS et comment il s'intègre dans l'approche systématique de la conception des systèmes de maintenance prédictive. Cette

section est liée au chapitre 6 de la thèse qui est composé d'un article de conférence publié et qui est la suite de l'article présenté dans la section D.2. Une validation croisée est effectuée pour démontrer les capacités du DSS.

La section D.7 est destinée à étendre la validation du DSS. Une étude de cas est présentée pour développer l'approche complète proposée dans la recherche actuelle et un exemple de modèle de maintenance prédictive est mis en œuvre pour l'étude de cas sur la base des suggestions du DSS. Les résultats de cette mise en œuvre sont expliqués et analysés.

La section D.8 tire des conclusions de travaux présentés et propose des perspectives pour une suite dans ces activités de recherche.

D.2 Vers une approche multimodèle de la maintenance prédictive : une revue littéraire systématique sur le diagnostic et le pronostic

Une revue systématique de la littérature montre que la maintenance prédictive gagne de plus en plus en importance dans la communauté universitaire, en particulier au cours des 25 dernières années. La Figure D.2 montre le nombre de publications mentionnant les termes « maintenance prédictive », « maintenance conditionnelle » et « pronostic et prise en charge de l'état de la santé » au cours des 25 dernières années dans les sources de recherche consultées.

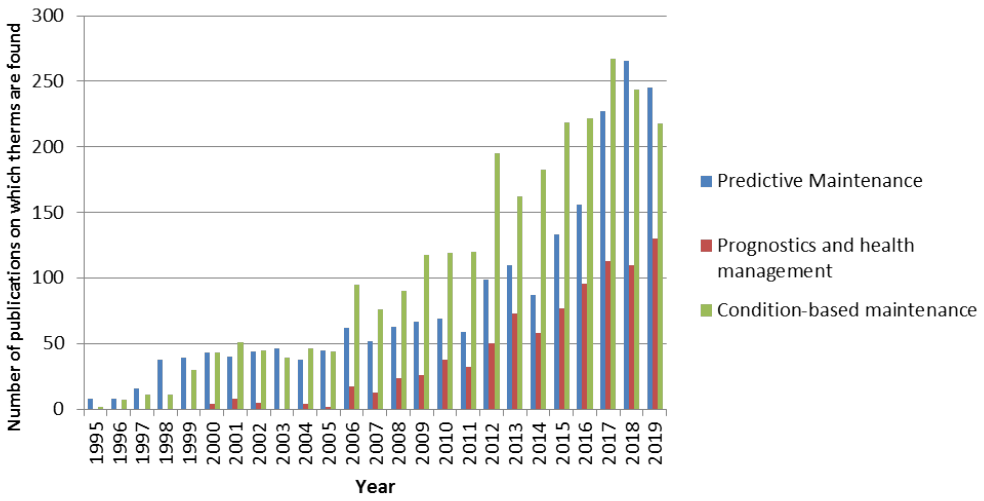


Figure. D.2: Nombre de publications liées à la maintenance prédictive au cours des 25 dernières années

La revue de la littérature se compose de deux parties. La première portait sur les précédentes revues bibliographiques sur la maintenance prédictive, qui ont permis d'identifier les modèles utilisés pour la maintenance prédictive et l'évolution des tendances au cours des années. Les taxonomies utilisées pour classer les différents modèles prédictifs montrent de légères variations terminologiques d'une étude à l'autre dans les études diagnostiques et pronostiques en maintenance. On peut distinguer deux approches principales : les approches mono-modèles et les approches multi-modèles. Pour les approches mono-modèles, on peut identifier trois familles de modèles : modèles basés sur la connaissance, modèles basés sur des données et modèles basés sur la physique. Les approches multi-modèles combinent au moins deux modèles des trois familles de modèles mentionnées. Les approches multi-modèles peuvent avoir différentes configurations et sont parfois appelées modèles hybrides.

La seconde partie de la revue de la littérature a été consacrée à l'étude des tendances actuelles des différentes approches des modèles. La revue a démontré l'existance d'une tendance à la mise en œuvre d'approches multi-modèles puisqu'un seul modèle souvent ne satisfait pas à toutes les fonctions d'un système de maintenance prédictive. La figure D.3 montre toutes les possibilités de combinaison de modèles dans des approches multi-modèles.

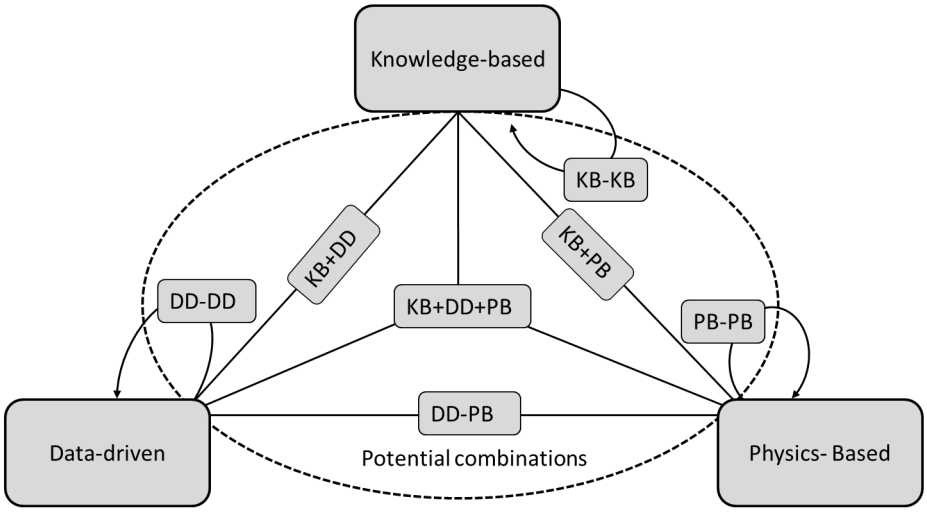


Figure. D.3: Combinaisons possibles de modèles de maintenance prédictive

La revue bibliographique a permis d'identifier les enjeux actuels de la maintenance prédictive :

- Extrapolation des solutions de maintenance prédictive existantes dans des applications complexes, incluant plusieurs composants et leurs défaillances associées. La plupart des applications identifiées se concentraient sur un seul composant avec un nombre limité de défauts possibles. Cependant, les applications réelles sont souvent des systèmes complexes constitués de nombreux composants et de nombreux défauts associés à chaque composant et au système lui-même. Les approches multi-modèles offrent une solution potentielle pour surmonter la complexité des systèmes de maintenance prédictive.
- L'absence d'une approche systématique pour concevoir et développer des systèmes de maintenance prédictive. Il existe des standards, des normes et des architectures génériques pour développer de nouveaux systèmes de maintenance prédictive, tels que OSA-CBM. Cependant, ils se concentrent uniquement sur les composants fonctionnels de base du système et ne couvrent pas les aspects importants des indicateurs de performance ou des contraintes du contexte du système. De plus, ils n'offrent pas d'explication cohérente sur les modèles à utiliser en fonction des besoins initiaux du système de maintenance prédictive. L'absence d'approche systématique limite la mise en œuvre de systèmes de maintenance prédictive dans les applications industrielles à grande échelle.
- La fusion de diverses sources de données de contrôle de l'état des systèmes. Ce défi est lié à l'extrapolation des modèles actuels de maintenance prédictive aux systèmes complexes. Les systèmes techniques peuvent avoir différents types de sources de données, par exemple des mesures de capteurs, des enregistrements de maintenance, des enregistrements opérationnels, des documents de conception, etc. Des informations importantes pourraient être recueillies auprès de toutes ces sources pour mettre en œuvre de nouveaux systèmes de maintenance prédictive.

- Intégration des données d'influence externe à l'exploitation des systèmes. Le fonctionnement des systèmes peut varier en fonction du contexte d'exploitation. Des changements dans ce contexte opérationnel peuvent affecter directement les performances du système et, par conséquent, les analyses de surveillance de l'état de santé du système. Il peut ainsi déclencher de fausses alarmes suggérant l'existence de défauts, ou il peut à l'inverse empêcher l'identification de défauts existants. Des modèles complémentaires capables d'intégrer une influence externe à des fins de maintenance prédictive pourraient contribuer à une solution.
- • Gestion de l'incertitude numérique des mesures. L'incertitude de mesures et de calculs affecte directement l'exactitude du diagnostic et du pronostic. Cela peut être dû aux données collectées ou aux imperfections du modèle utilisé pour l'analyse. Cela peut affecter la fiabilité des résultats. La gestion de l'incertitude est vitale pour les systèmes critiques soumis à la réglementation des autorités. C'est le cas des systèmes critiques tels que les centrales nucléaires et les avions où les réglementations sont restrictives pour maintenir les normes de sécurité et éviter les événements catastrophiques.

L'étude de l'état de l'art a motivé l'amélioration des questions de recherche. Le reste de la recherche a été motivé par les questions suivantes :

1. Comment aborder l'architecture et la conception des systèmes de maintenance prédictive?
2. Comment sélectionner un modèle ou une combinaison de modèles appropriés compte tenu d'un nouveau problème de maintenance prédictive à résoudre?
3. Comment proposer une approche adaptée pour une solution de maintenance prédictive?
4. Comment un concepteur peut-il bénéficier de l'expérience des systèmes existants pour développer de nouvelles solutions de maintenance prédictive?

D.3 Des besoins et souhaits pour une architecture logique de systèmes de maintenance prédictive

La question de recherche No. 1 obtenue à l'issue de l'étude de l'état de l'art porte sur la conception de nouveaux systèmes de maintenance prédictive. Dans l'énoncé de recherche présenté dans la section D.1, il a été expliqué que la conception et le développement de systèmes de maintenance prédictive sont toujours basés sur des essais et des erreurs. Malgré l'existence de plusieurs architectures génériques dans les normes et standards, telles que [MIM01], l'étape conceptuelle de tels systèmes n'est pas entièrement couverte. Il existe un écart dans le développement de tels systèmes, depuis la collecte des besoins et des souhaits pour le nouveau système jusqu'à la création de l'architecture qui répond aux besoins et souhaits initiaux.

La création d'un nouveau système doit toujours commencer par une analyse des besoins et souhaits exprimés (ou pas) par les parties prenantes. Ces besoins et souhaits permettent d'établir une liste des exigences des parties prenantes pour le nouveau système. Les recherches actuelles proposent une approche d'ingénierie des systèmes pour couvrir l'étape conceptuelle des systèmes de maintenance prédictive.

La Figure D.4 montre les différentes étapes abordées dans l'approche d'ingénierie des systèmes proposée dans cette thèse. Il commence par recueillir les besoins et les souhaits initiaux des parties prenantes, qui sont par la suite traduits dans une liste formelle des exigences des parties prenantes. Les exigences sont hiérarchisées et classées en exigences fonctionnelles, de performance, structurelles et expérientielles. Cette classification aidera à la création de l'architecture du système. Les exigences fonctionnelles sont utilisées

pour démarrer le processus d'architecture, en particulier pour effectuer l'analyse fonctionnelle et créer le fonctionnel. L'architecture fonctionnelle est ensuite utilisée pour développer l'architecture logique qui reste générique. Les exigences de performance, structurelles et expérientielles sont utilisées pour faire la sélection des composants pour répondre à l'architecture logique et créer l'architecture physique.

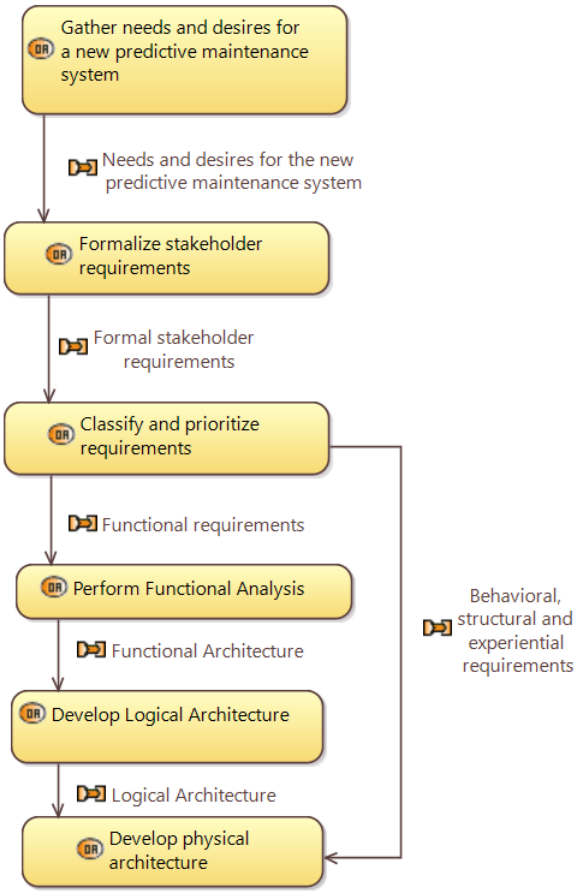


Figure. D.4: Étapes abordées dans l'approche d'ingénierie des systèmes pour la conception de systèmes de maintenance prédictive

Cette section est liée au chapitre 3 de la thèse dans lequel une approche systématique a été proposée pour aborder l'étape conceptuelle des systèmes de maintenance prédictive afin de répondre à la première question de recherche affinée. Les différentes étapes depuis la satisfaction des besoins et souhaits initiaux des parties prenantes jusqu'à la définition de l'architecture logique ont été couvertes et différentes méthodes ont été proposées pour y répondre. Cependant, le reste des questions de recherche affinées n'a pas encore trouvé de réponse. En maintenance prédictive, l'espace des solutions est vaste et complexe comme le montre le chapitre 2 de la thèse. Il n'y a pas de règles spécifiques qu'un architecte peut suivre pour sélectionner les bons modèles et approches pour résoudre de nouveaux problèmes de maintenance prédictive. Dans cette recherche, une hypothèse est proposée pour surmonter ce problème : la mise en œuvre d'un système d'aide à la décision (DSS) basé sur le raisonnement basé au cas (CBR) et soutenu par des ontologies pourrait aider l'architecte à sélectionner des composants appropriés sur la base d'expériences passées à partir d'expériences réussies.

D.4 Développement d'une ontologie pour le système de raisonnement par cas

Le DSS est composé de deux blocs principaux : le raisonnement basé aux cas (CBR) et les ontologies. La CBR est un paradigme de raisonnement qui cherche à résoudre de nouveaux problèmes sur la base des expériences de problèmes similaires résolus dans le passé. La CBR est abordée dans le chapitre 5 de la thèse et dans la section D.5 de ce résumé, mais une brève introduction est nécessaire pour comprendre le rôle que jouent les ontologies dans le DSS développé dans les recherches actuelles. Les systèmes CBR sont développés sur un vocabulaire de base [Alt+12]; [Sán+12]. Ce vocabulaire est nécessaire pour structurer les cas de problèmes résolus stockés dans un « case base », pour les mesures de similarité qui comparent le nouveau problème avec ceux du « case base », et pour les connaissances nécessaires pour adapter la solution récupérée au nouveau problème (voir Figure D.5). Pour les besoins de cette thèse, une ontologie est choisie pour servir de cadre terminologique (vocabulaire de base). Un modèle d'ontologie fournit les termes, les définitions et les relations entre les termes qui sont utilisés pour construire la structure de cas, les similitudes et les connaissances adaptatives dans le DSS. Ce chapitre est consacré au contexte théorique et au développement du modèle d'ontologie pour le DSS proposé.

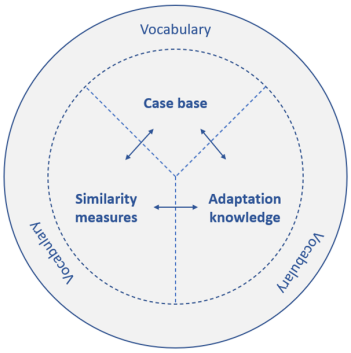


Figure. D.5: Raisonnement basé aux cas, développé sur un cadre de vocabulaire [Alt+12]

En sciences de l'information, une ontologie est une description explicite formelle de concepts dans un domaine de discours, propriétés de chaque concept qui décrivent ses caractéristiques, attributs et restrictions [NM01]. L'un des objectifs les plus courants dans le développement d'ontologies est de « partager une compréhension commune de la structure des informations entre les personnes et les agents logiciels » [Gru93]; [Mus92]. Cela signifie que le vocabulaire utilisé par les personnes dans un domaine de connaissance spécifique peut être « lisible par machine ». Tous les concepts d'une ontologie sont représentés par des classes liées par des propriétés (également appelées relations). Les ontologies sont développées avec des langages formels. L'un des plus reconnus est le langage d'ontologie Web dans sa deuxième version (OWL2) qui est pris en charge par le World Wide Web Consortium (W3C) [Wor12].

D.4.1 Ontologie pour la sélection et l'évaluation des stratégies de maintenance (OMSSA)

Une enquête complémentaire au fil conducteur de cette thèse a permis la création d'un modèle ontologique pour la sélection et l'évaluation des stratégies de maintenance (OMSSA). La création d'OMSSA suit les normes les plus élevées et les dernières tendances en matière de création d'ontologies à l'aide d'ontologies de référence de haut et de moyen niveau. Cela facilitera la réutilisation et l'intégration d'OMSSA avec d'autres ontologies connexes. La contribution de l'OMSSA a été consolidée dans un article scientifique qui

a été accepté pour publication au moment où cette thèse a été écrite. OMSSA sert de base à la création du modèle ontologique utilisé pour le système d'aide à la décision (DSS) proposé dans cette thèse.

D.4.2 Ontologie pour la conception de maintenance prédictive (OPMAD)

OPMAD est une extension de l'OMSSA et les mêmes normes et méthodologies ont été suivies pour son développement. Dans ce qui suit dans ce résumé, les noms de classe d'OPMAD seront conversés en anglais afin de maintenir la cohérence avec le modèle et les graphiques. La Figure D.6 présente les classes et relations les plus importantes dans OPMAD. En raison des limitations d'espace, les sous-classes ne sont pas représentées sur la figure. Comme l'objectif d'OPMAD est d'aider le DSS à identifier des modèles appropriés pour les systèmes de maintenance prédictive, l'explication des classes d'ontologies et de leurs relations commence à partir de la classe Modèle de maintenance prédictive (`Predictive Maintenance Model`) et est basée sur les termes présentés dans la Figure D.6. Un modèle de maintenance prédictive est transporté dans un module de maintenance prédictive (`Predictive Maintenance Module`) qui est un composant d'un système de maintenance prédictive (`Predictive Maintenance System`). Chaque module de maintenance prédictive a une fonction de maintenance prédictive (`Predictive Maintenance Function`), cette recherche se concentre sur les fonctions de diagnostic et de pronostic.

Le modèle de maintenance prédictive intégré dans le module est directement lié à un élément maintenable (`Maintainable item`), qui est la classe qui décrit le système maintenable pour lequel le système de maintenance prédictive est développé. L'élément maintenable est classé par la classe de type d'élément maintenable (`Maintainable item type`) qui a été ajoutée à des fins de calcul de similarité dans le DSS ; ces types de classes aident à comparer le nouveau problème à traiter avec le problème résolu dans un « case base » d'un système CBR. Les éléments maintenables appartenant au même type partagent des caractéristiques de dégradation importantes et donc les mêmes modèles de maintenance prédictive peuvent être proposés pour résoudre les mêmes fonctions de maintenance prédictive. L'élément maintenable a sa propre fonction (`Function`) qui est affectée par une défaillance (`Failure`) se manifestant par un mode de défaillance (`Failure Mode`). Un élément maintenable a un ou plusieurs modes de défaillance qui font également l'objet du modèle de maintenance prédictive. Dans le modèle de maintenance prédictive sont entrées les données d'état (`Condition`) qui sont utilisées pour effectuer des diagnostics et des pronostics. Les deux qualités importantes pour la mise en œuvre d'un modèle de maintenance prédictive sont son type et sa configuration. Le type de modèle de maintenance prédictive (`Predictive Maintenance Model Type`) classe le modèle en familles de modèles basés sur les connaissances, sur les données et sur la physique. Ce type de classification est utile aux fins de la similitude et des préférences dans le DSS.

La configuration du modèle de maintenance prédictive (`Predictive Maintenance Model Configuration`) indique si un modèle doit être complété par d'autres modèles pour remplir sa fonction ; il peut améliorer les performances, mais augmente la complexité du développement. Le module de maintenance prédictive est noté par indicateur de synchronisation (`Module Synchronization`) et de performance (`Performance indicator`). Ces deux qualités fournissent des informations utiles sur la façon dont les modèles de maintenance prédictive intégrés dans les modules doivent être testés et synchronisés avec l'élément maintenable. Toutes ces informations sur les modèles de maintenance prédictive, les éléments maintenables, leurs modèles de défaillance, entre autres, sont collectées dans le cas de la maintenance prédictive (`Predictive Maintenance Case`) qui est documentée et publiée via un article de maintenance prédictive (`Predictive Maintenance Article`), un type spécial de l'article qui se concentre sur la maintenance prédictive. A partir de ces articles de maintenance prédictive, certains indicateurs bibliométriques sont également inclus dans l'ontologie qui sera utilisée dans le système CBR. Ces indicateurs bibliométriques sont fournis en tant qu'attributs de solution afin que l'ingénieur puisse facilement trouver la source d'information pour un cas spécifique. Le titre, l'identifiant et l'année de publication de l'article peuvent être utilisés à des fins de similarité et/ou comme source d'information pour plus de détails sur les modèles de maintenance

prédictive et leur mise en œuvre.

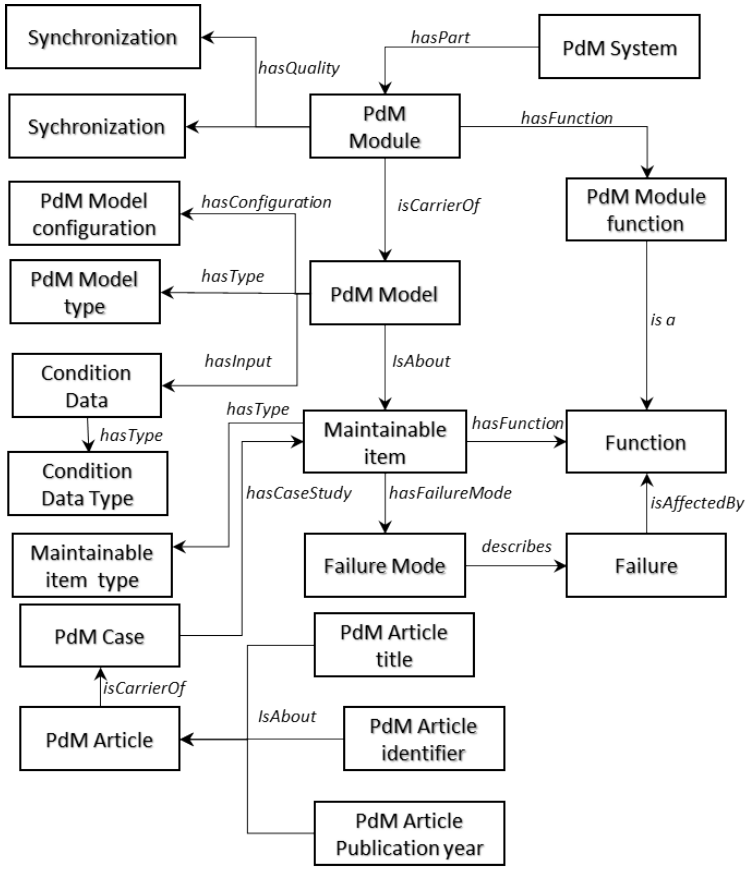


Figure. D.6: Classes et relations dans OPMAD

D.5 Développement de systèmes de raisonnement par cas

Le raisonnement basé sur les cas (CBR) est une méthodologie de résolution de problèmes basée sur la récupération de solutions précédentes pour des problèmes similaires [De +05]. Le raisonnement basé sur les cas a été développé sous la philosophie selon laquelle les êtres humains pensent et raisonnent en utilisant des analogies et des exemples, plutôt que des structures SI-ALORS, ces dernières formant la base du raisonnement basé sur des règles. La résolution du problème s'effectue dans un processus cyclique de plusieurs étapes : le cycle CBR [AP94]. Ce cycle est composé de quatre phases : récupération, réutilisation, révision et conservation (voir Figure D.7).

Le cycle CBR démarre lorsqu'un nouveau problème est présenté. La première phase vise à récupérer les cas les plus similaires à partir d'une base de connaissances qui stocke tous les cas précédents. Le cas cible (nouveau) est comparé aux cas existants dans la base de connaissances à l'aide de différentes mesures de similarité. Le cas récupéré le plus proche est proposé comme solution possible en phase de réutilisation. Une certaine adaptation peut être nécessaire pour appliquer la solution dans le cas prévu. Une fois la solution suggérée et mise en œuvre, la phase d'examen est effectuée. Si la solution suggérée parvient à résoudre le problème, elle est confirmée et dans la dernière phase, elle est conservée dans la base de connaissances afin qu'elle puisse être réutilisée dans de futurs problèmes similaires. Plus de détails sur chaque phase de la

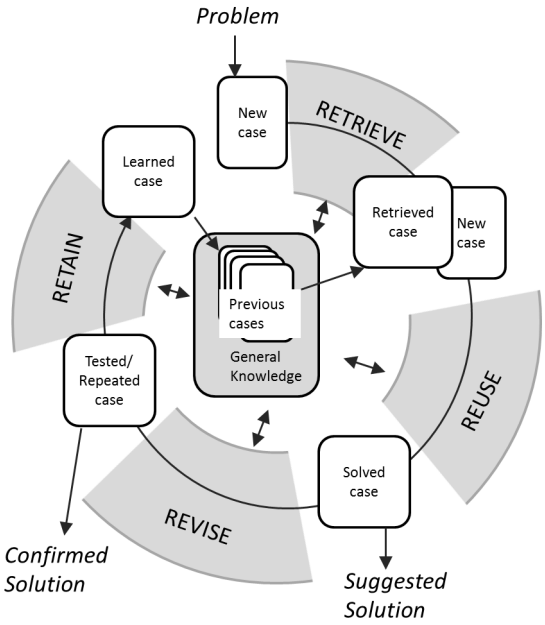


Figure. D.7: Cycle de raisonnement basé aux cas, image inspirée par [AP94]

CBR sont fournis dans les sous-sections suivantes.

D.5.1 Plateforme MyCBR

Le module CBR du DSS dans ce projet de recherche a été créé à l'aide de myCBR, un outil de recherche pour le raisonnement basé aux cas « open source » [Alt+12]. Lors du développement d'un système de récupération CBR à l'aide de myCBR, la première étape consiste à déterminer les attributs des cas. Les cas sont représentés par un vecteur d'attributs charactérisant les problèmes et les solutions :

Case = [Attributs de problème, Attributs de solution]

Les attributs de problème sont utilisés pour mesurer la similarité d'un cas cible et des cas d'un « case base »". Les attributs de la solution, comme son nom l'indique, fournissent des informations pertinentes relatives à la solution du problème dans chaque cas stocké dans la base de cas.

La deuxième étape consiste à attribuer une similarité locale à chaque attribut du problème. MyCBR fournit plusieurs options pour calculer la similarité pour chaque attribut du problème. Parmi les options disponibles, trois fonctions de similarité ont été utilisées pour les recherches actuelles :

1. Similarité nombre entier/flottant : cette fonction de similarité est utilisée pour les attributs numériques. La similarité est obtenue par une différence entre une valeur de référence et les valeurs d'entrée de la fonction. Une fonction mathématique est nécessaire pour définir comment la similarité diminue à mesure que les valeurs d'entrée s'éloignent de la valeur de référence. Cette fonction mathématique peut être linéaire, exponentielle ou déterminée par des points discrets sur le plan cartésien.

2. Similitude de symbole : Cette mesure de similarité est recommandée pour les variables avec un ensemble fixe d'options. Ces options sont organisées dans une matrice de similarité et certaines valeurs numériques sont données pour établir la similarité entre les options. Cette fonction de similarité a été modifiée afin que certains attributs de la recherche actuelle interagissent avec le modèle ontologique. Les valeurs de la matrice de similarité sont obtenues automatiquement à partir des classes et relations d'une ontologie en utilisant l'approche de similarité basée sur les caractéristiques proposées par [Sán+12].

3. Similitude de chaîne de caractères : la similarité d'attribut est obtenue sur la base de chaînes de texte ouvertes. Contrairement à la similarité de symboles qui a un nombre limité d'options pour définir la similarité, la similarité de chaîne n'a que la restriction d'avoir des phrases ou des mots en entrée. MyCBR propose trois options pour calculer la similarité basée sur les chaînes : Equality, Ngram et Levenshtein. Pour l'étude de cas actuelle, la fonction Levenshtein a été sélectionnée pour calculer les attributs en fonction des chaînes de similarité. La fonction Levenshtein offre un moyen flexible de calculer la similarité en fonction de chaque caractère de la chaîne [Lev66]. Ceci est particulièrement utile lorsqu'il existe un grand nombre d'options inconnues lors de la création de similitudes, et il tolère également les fautes d'orthographe mineures.

Après le calcul de similitude pour chacun des attributs du problème, ces similitudes individuelles sont combinées pour en dériver une similitude globale. Chaque attribut a un poids et à travers une fonction d'agrégation la similarité globale est calculée. L'objectif des poids est de donner une importance particulière aux attributs du problème. Aux fins de cette recherche, tous les attributs reçoivent un poids de 1, ce qui signifie qu'ils sont d'égale importance. MyCBR propose deux options différentes pour calculer la similarité globale : somme pondérée et distance euclidienne.

- Somme pondérée : comme son nom l'indique, c'est la somme des similitudes compte tenu du poids de chacun.
- Distance euclidienne : la distance euclidienne entre deux points dans l'espace euclidien est un nombre, la longueur d'un segment de droite entre les deux points.

Dans cette thèse, les deux fonctions de fusion peuvent être choisies par l'utilisateur. Il est important de noter que seuls les attributs retenus dans la description du problème sont pris en compte pour calculer la similarité globale. Cela signifie que si l'architecte n'a que deux ou trois attributs sur les sept qui décrivent le problème, la similarité globale sera calculée en fonction de ces deux ou trois attributs.

D.5.2 Représentation des cas

« Un cas est une connaissance dans un contexte particulier qui représente une expérience qui enseigne une leçon essentielle pour atteindre le but du raisonneur » [Kol93]. Les cas sont souvent représentés sous une forme couplée [problème, solution], dans laquelle les similitudes sont appliquées à la partie "problème" afin que la partie "solution" soit récupérée. La représentation du cas est composée de trois parties principales [BKP05] :

1. Définissez les attributs du cas.
2. Définissez la structure du contenu du cas.
3. Organisez le « case base ».

La représentation et la structure du cas se font à l'aide d'OPMAD. La Figure D.6 montre les différents attributs (classes en ontologie) d'un cas représenté dans OPMAD. Ces classes d'ontologies peuvent être divisées en deux groupes différents : les attributs de problème et les attributs de solution :

- **Attributs du problème** = [PdM Function, Maintainable Item, Maintainable Item Type, Condition Data Type, Module synchronization, PdM Article Publication year]
- **Attributs du problème** = [PdM Model, PdM Model Configuration, PdM Model Type, Module Performance Indicator, PdM Article Identifier, PdM Article Title]

D.5.3 Développement du moteur de récupération de composants pour les systèmes de maintenance prédictive

Le « case base » dans le DSS est une version instanciée de l'ontologie OPMAD qui a été renseignée de cas réussis d'implémentations de maintenance prédictive à partir d'une revue bibliographique approfondie. Le processus d'instanciation d'OPMAD comprend la définition des différentes variables de recherche et les options possibles pour chaque variable ; cela permet une meilleure structure du « case base » et facilite la récupération des cas pendant le raisonnement. Pour chaque attribut de problème, une similarité locale est sélectionnée parmi les options possibles : entier, symbole, basé sur une ontologie, texte ouvert. Le Tableau D.1 montre les fonctions de similarité attribuées à chacun des attributs du problème. Deux fonctions d'agrégation différentes ont été ajoutées pour calculer la similarité globale entre un cas cible et les cas stockés dans le « case base ». Une interface graphique a été développée pour faciliter la vérification et la validation du moteur de récupération. Le moteur de récupération développé est le cœur du DSS pour la sélection des composants de maintenance prédictive. Le chapitre suivant est orienté pour montrer le cadre complet du système CBR avec une ontologie pour la sélection des composants de maintenance prédictive. Les détails du développement du moteur de récupération sont fournis dans le guide de code à l'annexe C.

Tableau D.1: Affectation des fonctions de similarité

Attibute	**Similarity function**
PdM Function	Symbole (Ontologie)
Maintainable item	Chaîne (Levenshtein)
Maintainable item type	Symbole (égalité)
Condition Data	Symbole (Ontologie)
Condition Data Type	Symbole (égalité)
Module synchronization	Symbole (égalité)
PdM Article Publication Year	Entier (fonction définie par des points)

D.6 Développer un cadre pour la sélection de modèles de maintenance prédictive

Une étape commune dans le développement de l'architecture des systèmes concerne l'architecture logique. L'architecture logique fournit autant de détails que possible tout en conservant les composants génériques. Ces composants génériques doivent être remplacés par des composants spécifiques créant ainsi l'architecture physique du système. Pour la sélection des composants, il est possible d'appliquer la créativité structurée. La créativité structurée dans l'architecture logique est basée sur l'analyse des combinaisons possibles des différents composants physiques/informatifs qui peuvent être sélectionnés pour répondre à chaque

composante logique. Cela permet d'explorer l'espace des solutions et peut aider l'architecte à identifier des solutions innovantes pour un nouveau système. Si plusieurs solutions possibles sont identifiées, une analyse de compromis peut être nécessaire pour sélectionner la plus appropriée. L'architecture choisie sert de base à la conception détaillée des systèmes. Les connaissances acquises grâce au nouveau système mis en œuvre peuvent être utilisées par l'architecte pour développer de futurs systèmes. Il existe une analogie entre le travail de créativité structuré effectué par l'architecte pour sélectionner les composants appropriés pour répondre à l'architecture logique et les quatre phases du raisonnement basé aux cas : récupérer, réutiliser, réviser et conserver. Une représentation graphique de cette analogie est présentée à la Figure D.8 (en notation Capella [Roq18]). L'analogie proposée relie la phase de récupération CBR à la recherche de l'architecte dans les systèmes précédents connexes qui peuvent servir d'inspiration pour le développement du nouveau système.

- La phase de réutilisation du CBR serait l'affectation des composants identifiés pour se conformer à la nouvelle architecture du système.
- La phase d'examen CBR commencerait par l'analyse de compensation effectuée par l'architecte pour identifier les composants les plus appropriés. Cette phase se poursuit jusqu'à la vérification et la validation du système mis en œuvre, qui sont généralement réalisées par des acteurs autres que l'architecte.
- Après la validation du nouveau système, l'architecte peut conserver les enregistrements qui seront utilisés à l'avenir ; ce serait la phase de rétention CBR.

Cette analogie est générique en termes du système que l'architecte va développer. Toutes les activités sont effectuées « manuellement » par l'architecte et les enregistrements des expériences précédentes peuvent ne pas être structurés pour faciliter leur réutilisation. Cette approche manuelle peut fonctionner lorsque le nombre d'expériences précédentes est limité afin que l'espace de solution à explorer par l'architecte reste gérable. Lorsque le nombre d'expériences précédentes est élevé ou que les exigences sont complexes, la sélection des composants les plus adaptés peut ne pas être facile. Reprendre connaissance d'un nombre important d'architectures plus anciennes peut prendre du temps et des options importantes peuvent être perdues.

Compte tenu de cette analogie entre la CBR et le travail de créativité structuré, différents algorithmes peuvent être proposés pour accompagner l'architecte dans les différentes phases du travail architectural. Dans une première étape, la recherche actuelle s'est concentrée sur le développement d'un système d'aide à la décision (DSS) capable de rechercher et de recommander des composants physiques / informationnels appropriés. Cela peut aider l'architecte à gagner du temps dans la phase de conception, et permet une analyse plus large effectuée par des machines, analyses qui peuvent difficilement être approchées par l'homme.

La Figure D.9 présente un concept de DSS et comment il s'intègre dans l'analogie présentée à la Figure D.8. Ce concept est composé de trois parties principales : la base de données «le case base », le moteur de recherche et l'ontologie spécifique au domaine. La base de données de cas stocke les cas passés dans un format structuré pour une réutilisation facile. L'architecte présentera les informations du nouveau système en cours de développement au moteur de récupération et récupérera, dans le case base, un ensemble de cas les plus similaires qui servira d'inspiration lors de la sélection des composants appropriés pour l'architecture logique.

Au sein de cette structure, l'ontologie joue un rôle essentiel. Les cas, les attributs de ces cas et les similitudes entre les différentes variables textuelles sont souvent décrits en langage naturel. Modéliser ce langage naturel et le rendre lisible par machine est nécessaire pour automatiser la recherche de cas. L'intégration de l'ontologie proposée et du module de récupération CBR est expliquée plus en détail dans

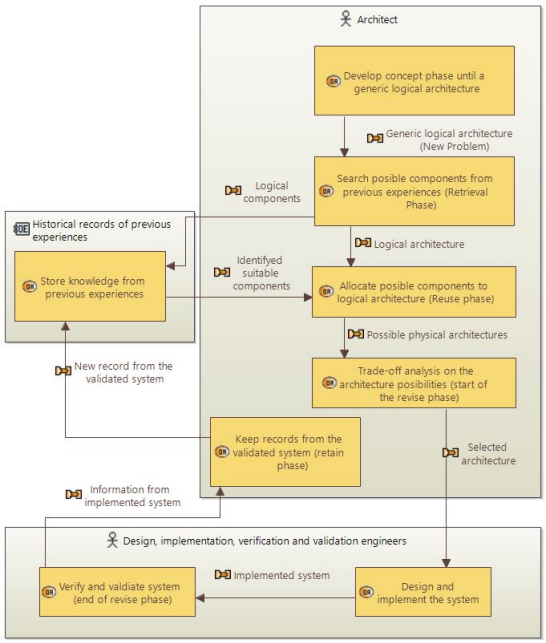


Figure. D.8: Analogie entre le raisonnement basé au cas et les tâches effectuées par un concepteur de systèmes, en notation Capella [Roq18]

une enquête complémentaire présentée à l'annexe B. La section suivante traite de la validation du DSS proposé et de la discussion des résultats obtenus.

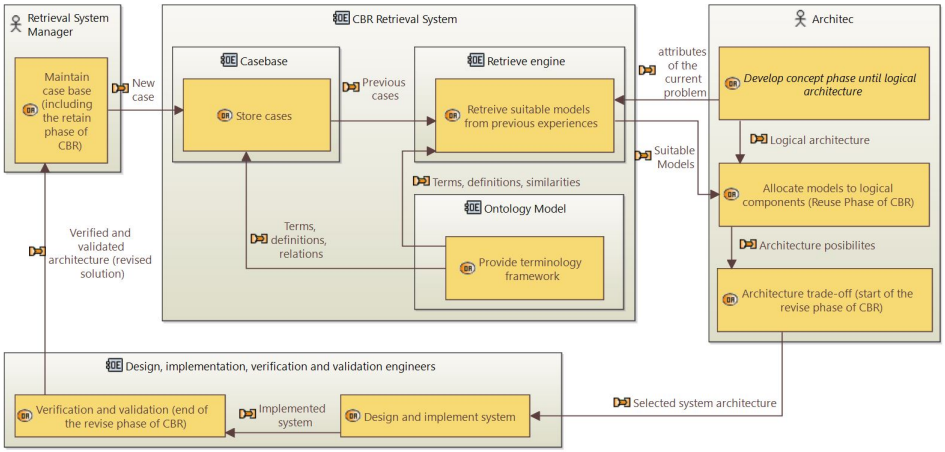


Figure. D.9: Idée conceptuelle d'incorporation de DSS pour la sélection des composants du système

D.7 Validation et discussion des résultats

La validation du DSS est divisée en deux parties : une validation croisée pour confirmer la cohérence du DSS et une mise en œuvre d'un modèle suggéré dans une étude de cas pour tester les recommandations du DSS dans un exemple pratique.

D.7.1 Validation croisée

La validation croisée est une technique qui peut être utilisée pour tester l'efficacité de modèles d'intelligence artificielle entraînés. Pour l'approche actuelle, une approche fractionnée de données de test et d'apprentissage est retenue pour effectuer une validation croisée. Pour l'ensemble de données, 63 des 263 cas dans le « case base » ont été tirés au sort pour les tests, laissant le reste comme ensemble d'apprentissage. Pour chacun des 63 cas tiré au sort, les attributs du problème ont été présentés au DSS à l'aide de l'interface graphique. Puis, l'attention s'est portée sur les 10 cas les plus similaires lors de la récupération. La Figure D.10 montre l'exemple d'un test de récupération dans lequel tous les attributs de problème dans le cas cible correspondent aux mêmes attributs dans le cas 35. Cette correspondance complète entre les attributs entraîne une similarité globale de 1.

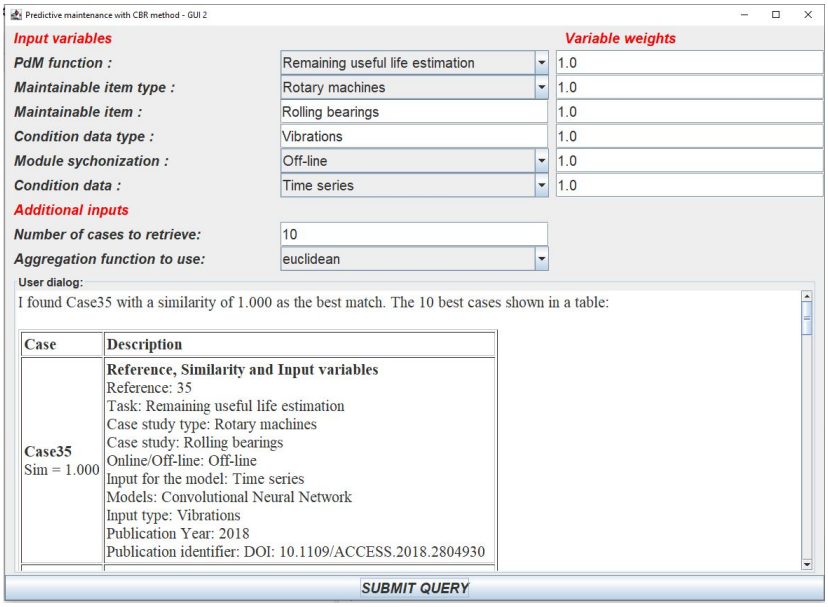


Figure. D.10: Exemple de reprise de dossier à l'aide de l'interface DSS

L'espace de solution des modèles de maintenance prédictive peut être divisé en deux groupes comme expliqué dans [Mon+20]. Le premier groupe est composé d'approches à modèle unique, divisées en trois catégories principales : les modèles basés sur la connaissance, les modèles basés sur les données et les modèles basés sur la physique. Le deuxième groupe est constitué d'approches multi-modèles, dans lesquelles au moins deux modèles de l'une des trois catégories mentionnées sont combinés pour réaliser une fonction spécifique du système de maintenance prédictive. La maintenance prédictive est composée de tâches de diagnostic et de pronostic. Selon la revue de la littérature [Mon+20], les tâches de diagnostic telles que la détection des défauts, l'identification des défauts et la modélisation de l'état de santé peuvent être abordées par des modèles des trois catégories d'approches à modèle unique ou d'approches multi-modèles. En revanche, pour les tâches de prédiction, il est difficile de trouver des modèles basés sur les connaissances. Par exemple, les fonctions de prédiction, telles que l'estimation de la durée de vie utile restante et les fonctions de prévision de l'état suivant, sont généralement réalisées par des modèles basés sur la physique, des modèles basés sur les données et des approches multi-modèles qui combinent des modèles basés sur la physique et des données. La validation croisée du DSS a confirmé ce comportement. Les récupérations pour les tâches de diagnostic ont proposé des solutions à modèle unique et multi-modèles en tenant compte des modèles des trois catégories. Les récupérations pour les prévisions proposaient également des solutions à un ou plusieurs modèles, mais les modèles proposés ne provenaient que de catégories physiques et axées sur les données. Cela permet de confirmer que l'espace de solution est bien couvert.

Pour chaque cas de test, les cas récupérés sont classés en fonction de la similitude des attributs du problème. Pour certains des tests, plusieurs cas récupérés avaient la même valeur de similarité maximale et/ou proposaient le même modèle pour réaliser une fonction PdM spécifique. Du point de vue de l'ingénierie des systèmes, cela représente deux limitations. La première fait référence à au moins deux cas avec la même similarité maximale car aucune information supplémentaire n'est fournie pour effectuer l'analyse de compromis pour sélectionné le composant le plus approprié. La seconde limitation est liée à la diversité des cas récupérés, en particulier lorsque deux cas ou plus suggèrent que le même modèle réalise une fonction spécifique. Un architecte en quête d'inspiration pour développer des solutions innovantes aura besoin que le DSS propose un ensemble diversifié de modèles. Ce sont des points d'amélioration à considérer à l'avenir pour le DSS. Une analyse plus détaillée peut également être effectuée qui inclut l'effort d'adapter la solution d'un cas récupéré au cas cible. Cela peut aider à effectuer une analyse de compromis sur les options rappelées et par conséquent à améliorer les performances du DSS.

D.7.2 Implementation d'un cas d'étude pratique

Dans le cadre de la validation, un exemple de mise en œuvre a été développé à partir des suggestions DSS dans l'étude de cas. L'étude de cas consiste en un ensemble de données de moteurs d'avion [Cha+21]. L'ensemble de données contient les enregistrements de dysfonctionnements de 128 moteurs d'avion dans des conditions de vol réelles et a été généré avec le modèle Commercial Modular Aero-Propulsion System Simulation (CMAPSS) développé par la NASA. Ces données sont appelées l'ensemble de données N-CMAPSS. Le but n'est pas seulement de tester les capacités du système CBR activé par l'ontologie (le DSS), mais aussi d'identifier les points d'amélioration pour le DSS. L'objectif de cette validation est de mettre en œuvre l'un des modèles proposés par le DSS pour remplir une fonction de maintenance prédictive pour la base de données N-CMAPSS. Cette implémentation permet de démontrer que DSS est capable de suggérer des composants appropriés pour les systèmes de maintenance prédictive, en complément de la validation croisée.

Afin de mieux valider les recommandations du DSS, l'un des modèles proposés a été pris en exemple et appliqué pour remplir la fonction de maintenance prédictive correspondante. Le DSS recommande la SOM et la régression logistique pour la modélisation de la santé comme les modèles les plus appropriés (similarité égale à 0,879) pour l'étude de cas N-CMAPSS. Construisant sur l'expérience de l'équipe de recherche en cartes d'auto-organisation (SOM), une implémentation du volet modélisation de la santé des moteurs est réalisée à l'aide d'un SOM. La section suivante fournit des explications supplémentaires sur l'implémentation de l'échantillon SOM et ses résultats préliminaires.

La méthodologie pour mettre en œuvre les cartes d'auto-organisation (SOM) pour la modélisation de la santé a été adoptée à partir de [Sch+20], (voir également l'annexe A). Les cartes auto-organisées sont des réseaux de neurones artificiels avec un entraînement non-supervisé qui sont capables de regrouper des instances de données en fonction des attributs d'instance. Un SOM est normalement constitué d'une couche carrée de neurones et les différents groupes après l'entraînement SOM peuvent être représentés graphiquement sur des cartes comme des régions bien définies. Il a été utilisé avec succès pour modéliser le processus de dégradation de différentes machines telles que les moteurs à réaction [MV18]. Dans ces cas, les neurones représentent la santé ou la dégradation de la machine dans un mode de fonctionnement spécifique. Le SOM entraîné aura une seule région, mais une transition du blanc (état optimal) au noir (état d'échec). Lors de l'évaluation de la santé ou de la dégradation d'une machine à l'aide du SOM entraîné, un neurone s'affichera sur la carte indiquant à quel point la dégradation est avancée ou à quel point la santé a diminué. C'est le véritable objectif de l'implémentation actuelle : obtenir un SOM entraîné capable de montrer une transition de l'état optimal à l'état défaillant. La Figure D.11 montre la carte auto-organisatrice entraînée avec le N-CMAPSS. Le résultat correspond au comportement attendu. Les différents modèles de dégradation ont été disposés sur la carte.

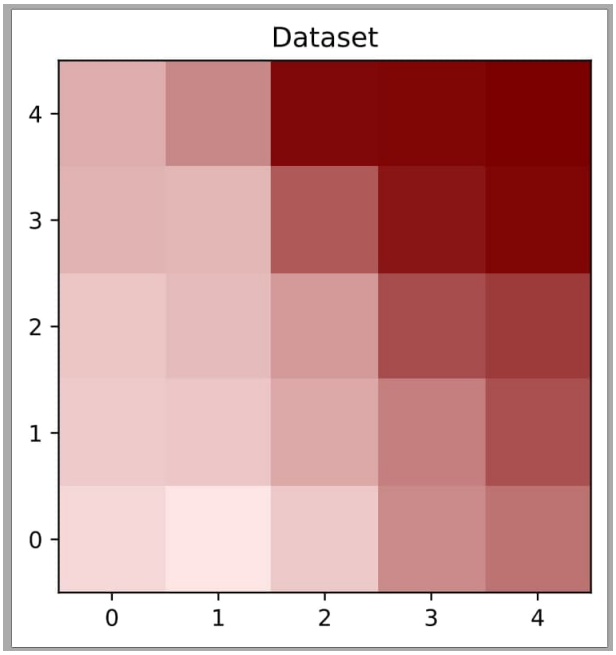


Figure. D.11: Carte auto-organisatrice formée avec l'étude de cas N-CMAPSS

Il est important de se rappeler que la validation d'un système d'aide à la décision (DSS) est une tâche difficile. La validation du SOM pour modéliser la santé du N-CMAPSS peut être utilisée pour valider indirectement le DSS ; cependant, il est important de souligner que d'autres implémentations seraient nécessaires pour confirmer les capacités et la précision du DSS proposé pour la sélection des modèles de maintenance prédictive. Dans l'implémentation actuelle, le N-CMPASS représente le cas cible pour lequel un nouveau système de maintenance prédictive doit être développé. Le système de recommandation CBR avec ontologie (également appelé DSS dans ce manuscrit) a proposé différents modèles pour remplir chaque fonction de maintenance prédictive. L'un des modèles proposés par le DSS pour modéliser la fonction de santé était la carte d'auto-organisation (SOM). La mise en œuvre du SOM a montré avec succès la tendance à la dégradation des moteurs à réaction, du fonctionnement nominal à l'état de défaillance en utilisant un sous-ensemble du N-CMPASS. Le SOM entraîné peut également être utilisé pour évaluer la santé/la dégradation d'un autre moteur du même type et dans les mêmes conditions de fonctionnement. Il est important de préciser que cette validation vise à démontrer la pertinence du modèle proposé par le DSS, mais davantage de comparaisons entre les modèles proposés sont nécessaires pour déterminer le meilleur modèle.

D.8 Conclusion et perspectives de travaux futurs

D.8.1 Résumé des contributions

Cette thèse comprend quatre articles qui ont été publiés ou acceptés pour publication dans des 2 revues scientifiques avec comité de relecture et 2 conférences internationales avec comité de relecture. Les principales contributions ont été regroupées dans les articles comme suit :

1. L'article de revue bibliographique résumait l'état de l'art en matière de diagnostics et de pronostics. Une proposition a été avancée pour différencier les modèles hybrides des approches multi-modèles.

Des tendances vers des approches multi-modèles ont été notées. Au moment de la rédaction de ce manuscrit, l'article de synthèse a été cité plus de vingt fois.

2. L'approche d'ingénierie des systèmes pour la conception de systèmes de maintenance prédictive propose, contrairement aux architectures génériques existantes, une approche systématique passant par une étape conceptuelle des systèmes de maintenance prédictive à partir des besoins et souhaits initiaux et ceci jusqu'à l'architecture logique. Cela permet d'aider l'architecte à déterminer les composants du système et à maintenir la traçabilité des besoins et souhaits initiaux des parties prenantes. Ces résultats ont été publiés dans une contribution de conférence à une conférence internationale.

3. Même si les ontologies n'étaient pas le domaine initial de ce projet de recherche, leur étude s'avérait nécessaire et ce travail de recherche a produit ses propres résultats. La création d'OMSSA, un modèle d'ontologie pour la sélection et l'évaluation des stratégies de maintenance, fournit un cadre terminologique qui peut être utilisé par des agents intelligents pour automatiser des tâches complexes de gestion de stratégies de maintenance. Une extension à OMSSA (OPMAD) est l'ontologie utilisée pour créer le « case base » et les mesures de similarité DSS pour la sélection des composants de maintenance prédictive. Le travail de recherche effectué dans la création de l'OMSSA a été consolidé dans un article de revue, actuellement accepté pour publication.

4. Un système d'aide à la décision (DSS) capable de consulter des cas réussis de mise en œuvre d'un système de maintenance prédictive peut aider un architecte système à sélectionner des modèles de maintenance prédictive appropriés pour effectuer des tâches de diagnostic et de prévision. Les premiers résultats de cette recherche ont été publiés dans une contribution de conférence à une conférence internationale.

D.8.2 Perspectives de travaux futurs

Pendant ce projet de recherche, il n'a pas été possible d'expliquer et de prouver tout. Il est donc important de mettre les choses en perspective pour repérer les axes d'amélioration et les lister comme perspectives de travaux futurs. Les perspectives de travaux futurs sont organisées en trois grands groupes :

1. Perspectives liées à l'approche d'ingénierie systèmes pour la conception de systèmes de maintenance prédictive
2. Perspectives pour OMSSA et OPMAD
3. Perspectives pour le DSS basé sur le raisonnement à base des cas

D.8.2.1 Perspectives liées à l'approche d'ingénierie systèmes pour la conception de systèmes de maintenance prédictive

- Affiner la liste des besoins, des souhaits et des exigences du système de maintenance prédictive : après la publication de l'article relatif à l'étape conceptuelle des systèmes de maintenance prédictive, plusieurs besoins, souhaits et exigences ont été identifiés qui n'avaient pas été initialement pris en compte. La liste des besoins, des souhaits et des exigences possibles pour les nouveaux systèmes de maintenance prédictive peut être améliorée. Ces listes peuvent être stockées dans des bases de données qui peuvent être utilisées par des agents intelligents automatisés pour aider les ingénieurs à évaluer les besoins et les souhaits initiaux d'un système de maintenance prédictive et à suggérer les exigences correspondantes des parties intéressées. Un DSS pourrait être mis au point pour définir les besoins des parties prenantes en fonction des besoins initiaux et des souhaits du nouveau système de maintenance prédictive.

- Inclure la technique phase des « trade-offs » dans le cadre systématique proposé : Les analyses de trade-off sont présentes à divers moments de la phase conceptuelle. L'intégration d'un outil d'analyse de trade-off (compromis) peut aider à améliorer le DSS et faciliter la sélection des composants de l'architecture.

- Inclure la formalisation des exigences système : un aspect important d'une approche d'ingénierie système consiste à obtenir les exigences système. Même lorsque la création de ces exigences est en théorie avant le processus d'architecture, en pratique, ces exigences sont généralement établies en parallèle avec le développement de l'architecture du système ou même à la fin, lorsque toutes les capacités des différents composants logiques sont connues. Ces exigences sont importantes pour la phase de conception détaillée. L'obtention de ces exigences sortait du cadre de la recherche actuelle, mais constitue un sujet complémentaire intéressant pour les recherches futures.

D.8.2.2 Perspectives pour OMSSA et OPMAD

- Affiner les classes et les relations : le développement d'ontologies est un sujet en en constante évolution. Il existe plusieurs initiatives pour fournir des ontologies de niveau supérieur et intermédiaire dans différents domaines. Le domaine industriel ne fait pas exception. Des travaux supplémentaires seront nécessaires pour aligner OMSSA et OPMAD sur ces ontologies normalisées afin de favoriser leur intégration et leur réutilisation dans d'autres applications. Ce travail comprend l'affinement des classes et les relations entre elles.

- Extension du raisonnement en ontologie : OMSSA et OPMAD ont été utiles dans la recherche actuelle, mais en tant qu'ontologies, ils ont été sous-utilisés. Les ontologies ont diverses capacités qui n'ont pas été exploitées. L'une d'elles est la mise en œuvre de règles sémantiques. Cela élargit les options de raisonnement et peut être utilisé comme moyen complémentaire au système CBR.

D.8.2.3 Perspectives pour le DSS basé sur le raisonnement à base des cas

- Étendre l'utilisation de l'ontologie pour des fonctions de similarité plus locales : l'ontologie a été utile mais sous-utilisée. Certaines autres mesures de similarité qui ont été attribuées à une mesure de similarité de symboles binaires peuvent être réorganisées pour utiliser des similarités basées sur des ontologies.

- Utiliser l'ontologie pour estimer les poids pour le calcul de similarité globale : l'ontologie peut également être utilisée pour calculer les poids de chaque similarité locale lors du calcul de la similarité globale. Dans la première tentative de calcul de similitude globale effectuée dans cette enquête, toutes les similitudes locales ont reçu le même poids. Une amélioration du calcul de la similarité globale pourrait être de donner un rang d'importance à chaque similarité locale. Les poids peuvent être calculés à l'aide de l'ontologie peuplée.

- Étendre le système d'aide à la décision en développant d'autres phases du cycle de CBR, telles que la phase d'adaptation : la portée du DSS était dans la phase de reprise du cycle de CBR. Le système peut être étendu pour faciliter la phase d'adaptation des composants proposés pour le nouveau système de maintenance prédictive. Cette extension peut également inclure l'analyse de la compensation des différentes composantes suggérées par le DSS. Les résultats de la recherche actuelle ont montré plusieurs opportunités d'amélioration qui devraient être abordées à l'avenir avant la mise en œuvre éventuelle du DSS.

- Affiner le processus d'instanciation pour éviter les problèmes de diversité mais en gardant tout l'espace de solution couvert : l'instanciation du “case base" doit être affinée. Une opportunité d'amélioration

peut être de diviser l'espace de solution par les différentes fonctions de maintenance prédictive et de s'assurer que l'espace de solution est entièrement couvert par chacune d'entre elles. Des cas généralisés pour chaque fonction de maintenance prédictive peuvent résoudre le problème de diversité dans les cas de récupération.

- Analyse de compromis pour mieux sélectionner la technique parmi les propositions : comme pour le DSS, l'analyse des trade-offs peut aider l'architecte à discriminer entre les composants proposés par le DSS. Lors de la validation, il a été mentionné que parfois le DSS suggérait deux composants différents avec la même similitude. L'ajout d'attributs supplémentaires en tant qu'indicateurs de performance peut aider l'architecte à prendre la bonne décision pour son problème.

APPENDIX E

Summary in Spanish / Resumen en Español

Contenido

E.1 Introducción

E.1.1 Mantenimiento predictivo

Mantenimiento predictivo es una estrategia de mantenimiento que tiene como objetivo determinar el momento preciso para realizar las acciones de mantenimiento. No muy antes porque las acciones de mantenimiento podrían provocar el cambio de piezas que aún tienen una vida útil importante, lo que representa costos para las compañías; pero no muy tarde porque una falla inesperada puede ocurrir trayendo consigo una serie de consecuencias negativas. Mantenimiento predictivo es una alternativa a otras estrategias tradicionales de mantenimiento, como lo son mantenimiento correctivo y preventivo. En lugar de esperar a que una falla ocurra o realizar mantenimientos basados en intervalos fijos de operación, mantenimiento predictivo se basa en el diagnóstico del estado actual de los equipos y en el pronóstico de cuando una eventual falla podría ocurrir.

Mantenimiento predictivo está fuertemente relacionado con Mantenimiento Basado en Condición (CBM por sus siglas en inglés) y Administración de la Salud basado en Pronósticos (PHM por sus siglas en inglés). Tanto mantenimiento predictivo como estas dos disciplinas abarcan los diagnósticos y pronósticos para mantenimiento; en algunas ocasiones estos términos son incluso utilizados como sinónimos. Aparentemente, los términos de mantenimiento predictivo y CBM aparecieron en la década de 1940. Ambos términos se refieren a la estrategia de mantenimiento que pretende anticipar los fallos de las máquinas en función de su condición. Sin embargo, no es hasta principios de los años noventa cuando esta estrategia adquiere importancia gracias a la implantación de sistemas de monitorización y herramientas computacionales capaces de continuar con las tareas de diagnóstico. En cuanto al pronóstico, incluso cuando se menciona en el mantenimiento predictivo y CBM, siempre fue una disciplina inexacta. A principios de la década del 2000, la disciplina de PHM surgió con el objetivo de cubrir la brecha en investigación en pronósticos para mantenimiento predictivo. Desde entonces, la investigación en diagnóstico y pronóstico para el mantenimiento ha ganado mucha atención de la academia y la industria; gracias a los continuos incrementos en poder computacional y dados los beneficios de su implementación. En la última década, se han hecho diferentes contribuciones bajo los diferentes términos (mantenimiento predictivo, CBM y PHM) y se refieren al mismo campo de investigación.

E.1.2 Idea conceptual y arquitectura de un sistema

Según el manual de INCOSE [INC15], el ciclo de vida de un sistema puede dividirse en seis etapas genéricas (véase la Figura E.1). El ciclo de vida del sistema comienza con la etapa conceptual, cuyo objetivo es explorar posibles soluciones para satisfacer las necesidades iniciales de las partes interesadas. La fase conceptual va seguida de la fase de desarrollo, en la que se realiza un diseño detallado de los componentes del sistema y sus interfaces. Después del diseño detallado, comienza la etapa de producción que está dedicada a la implementación del sistema, lo que significa la fabricación, codificación, creación de los diferentes componentes y su integración final en el sistema. La verificación y validación del sistema forman parte de la fase de producción. Una vez validado el sistema, se entrega al cliente para su etapa de utilización. En la etapa de utilización, el sistema cumple la función para la que fue creado. La etapa de mantenimiento va en paralelo a la etapa de utilización durante el funcionamiento del sistema. Esta etapa de mantenimiento tiene como objetivo asegurar el estado de funcionamiento óptimo del sistema. Al final del ciclo de vida del sistema, la fase de eliminación tiene como objetivo gestionar adecuadamente los residuos del sistema cuando se retira de la operación.

El proceso de arquitectura del sistema tiene lugar dentro de la etapa conceptual. La etapa conceptual comienza por reunir todas las necesidades y deseos de las partes interesadas y formalizarlos en los requisitos

Concept stage	Development stage	Production stage	Utilization stage	Retirement State
			Support stage	

Figura. E.1: Etapas genéricas del ciclo de vida de un Sistema (en inglés) [INC15]

de las partes interesadas. Estos requisitos se utilizan luego para la creación de la arquitectura del sistema que servirá de base para el diseño detallado y la implementación del sistema. El desarrollo de la arquitectura se puede dividir en tres niveles [Roq18]; [INC15]: arquitectura funcional, arquitectura lógica y arquitectura física. La arquitectura funcional describe cómo las diferentes subfunciones de un sistema interactúan entre sí para cumplir un objetivo específico, pero no proporciona ningún detalle sobre los componentes que cumplen cada subfunción. La arquitectura lógica proporciona tanto detalle como sea posible de los componentes de la arquitectura y sus interfaces, pero no involucra a ninguna tecnología específica, lo que significa que la arquitectura lógica muestra componentes genéricos. La arquitectura física proporciona los detalles de las tecnologías que se asignarán para cada componente lógico. La unión de estos tres niveles de la arquitectura puede ser llamada como la arquitectura del sistema y puede entonces ser definida como los "conceptos fundamentales o propiedades de un sistema en su entorno encarnado en sus elementos, relaciones, y en los principios de su diseño y evolución" [ISO11]. En palabras simples, la arquitectura de un sistema muestra el cómo un conjunto de elementos, que podrían ser físicos o informativos, se organizan para cumplir un objetivo específico.

E.1.3 Declaración de investigación

El mantenimiento predictivo se lleva a cabo mediante sistemas especializados para realizar tareas de diagnóstico y pronóstico, para determinar el momento adecuado para desencadenar acciones de mantenimiento. El diseño y desarrollo de estos sistemas especializados sigue basándose en prueba y error. Existen varias arquitecturas genéricas en normas y estándares, como por ejemplo [MIM01], pero el diseño de un nuevo sistema de mantenimiento predictivo comienza mucho antes, en la etapa conceptual. Una arquitectura genérica no proporciona un vínculo con las necesidades y deseos iniciales de un nuevo sistema de mantenimiento predictivo y, a menudo, estas necesidades y deseos no se satisfacen con una arquitectura genérica. Además, una arquitectura genérica no proporciona ninguna guía para seleccionar las tecnologías adecuadas que puedan cumplir los componentes genéricos que propone [MV19].

El mantenimiento predictivo aborda tareas de diagnóstico y pronóstico, pero no todos los sistemas predictivos cubren las mismas funciones. Por ejemplo, un nuevo sistema de mantenimiento predictivo puede estar destinado a realizar diagnósticos en diferentes componentes de un sistema y se necesitarán varios módulos de diagnóstico del mismo tipo y no se incluirán módulos de pronóstico. Las arquitecturas genéricas no proporcionan un enfoque sistemático para diseños que sólo necesitan un subconjunto de los componentes genéricos propuestos o cuando se necesitan varios componentes del mismo tipo.

Existe un número importante de opciones para cumplir con las funciones de diagnóstico y pronóstico en un sistema de mantenimiento predictivo, y no hay directrices para ayudar al arquitecto a seleccionar los modelos, técnicas o algoritmos adecuados que pueden llevar a cabo estas funciones. La exploración del espacio de solución para determinar los componentes adecuados puede entonces ser compleja y de larga duración. Esta tesis tiene como objetivo facilitar el proceso de arquitectura de los sistemas de mantenimiento predictivo al permitir una manera más eficiente de explorar el espacio de solución y proponer los componentes más adecuados para la arquitectura del sistema. Antes de profundizar en el diseño de sistemas de mantenimiento predictivo, es importante entender el campo de investigación del mantenimiento predictivo en sí y las diferentes opciones disponibles para realizar diagnósticos y pronósticos. Se proponen

las siguientes preguntas de investigación para orientar el estudio del estado del arte en este tema:

1. ¿Cuáles son las tendencias actuales en diagnóstico y pronóstico en mantenimiento predictivo?
2. ¿Qué tipo de modelos, técnicas o métodos se utilizan para abordar el diagnóstico y el pronóstico en el mantenimiento predictivo?
3. ¿Cuáles son los principales desafíos que enfrenta el mantenimiento predictivo en el diagnóstico y el pronóstico?

Después del estudio de vanguardia, las preguntas de investigación se adaptan de acuerdo con los resultados del estudio y la motivación inicial de esta investigación relacionada con el diseño de sistemas de mantenimiento predictivo (ver sección E.2)

E.1.4 Organización del resumen

La sección E.2 aborda el estado del arte del mantenimiento predictivo basado en las preguntas iniciales de investigación introducidas en este capítulo. Esta sección está relacionada con el capítulo 2 de la tesis que se compone de un artículo publicado que incluye las tendencias actuales en modelos para diagnósticos y pronósticos en el campo del mantenimiento. La sección termina refinando las preguntas de investigación que motivaron el resto de la investigación.

La sección E.3 propone un enfoque de ingeniería de sistemas para el diseño de mantenimiento predictivo, específicamente en la etapa conceptual desde la reunión de las necesidades y deseos iniciales de las partes interesadas hasta la propuesta de una arquitectura lógica. Esta sección está relacionada con el capítulo 3 de la tesis que se compone de un artículo publicado en la conferencia y termina presentando la selección del componente de mantenimiento predictivo como el principal desafío que se abordará en los siguientes capítulos. Se propone un Sistema de Apoyo a las Decisiones (DSS por sus siglas en inglés) que combina ontologías y razonamiento basado en casos como posible solución para abordar la selección de componentes en el enfoque sistemático.

La sección E.4 explica uno de los elementos básicos del Sistema de Apoyo a las Decisiones: las ontologías. La investigación complementaria se ha realizado específicamente en ontologías que han resultado en un artículo de una revista, aceptado para su publicación en el momento en que este manuscrito fue escrito. El fondo teórico de las ontologías se presenta destacando su importancia en la comunidad de investigación debido a sus capacidades para modelar formalmente el vocabulario de un dominio en específico y realizar razonamiento con él. La sección E.4 explica el desarrollo de la ontología que más tarde se utiliza en el DSS para la selección de componentes. . Esta sección está relacionada con el capítulo 4 de la tesis que incluye un artículo aceptado para publicación que trata de un modelo ontológico para la selección y evaluación de estrategias de mantenimiento (OMSSA por sus siglas en inglés).

La sección E.5 explica el segundo pilar del DSS: Razonamiento basado en casos (CBR). Los principios del paradigma CBR se introdujeron incluyendo las fases del ciclo CBR. En un primer intento, el DSS se centra en la fase de recuperación de CBR. Se proporciona una explicación de la implementación del motor de recuperación de CBR utilizando código de fuente abierta.

La sección E.6 explica el marco general de la integración de los componentes básicos del DSS y cómo encaja en el enfoque sistemático para diseñar sistemas de mantenimiento predictivo. Esta sección está relacionada con el capítulo 6 de la tesis que se compone de un artículo de conferencia publicado que es la continuación del artículo presentado en la sección E.2. La validación cruzada se realiza para demostrar las capacidades del DSS.

La sección E.7 tiene por objeto extender la validación del DSS. Se selecciona un caso de estudio para desarrollar el enfoque completo propuesto en la investigación actual y se implementa un ejemplo de modelo de mantenimiento predictivo para el caso de estudio basado en las sugerencias del DSS. Se explican y analizan los resultados de esta implementación.

En la sección E.8 se presentan las conclusiones de los trabajos realizados y se proponen perspectivas para una continuación de esas actividades de investigación en el futuro.

E.2 Hacia un enfoque multimodelo en mantenimiento predictivo: una revisión literaria sistemática en diagnósticos y pronósticos.

La revisión sistemática de la literatura muestra que el mantenimiento predictivo está ganando importancia en la comunidad académica, especialmente en los últimos 25 años. La Figura E.2 muestra el número de publicaciones que mencionan los términos "mantenimiento predictivo", "mantenimiento basado en condición" y "pronóstico y gestión de la salud" en los últimos 25 años en las fuentes de búsqueda consultadas.

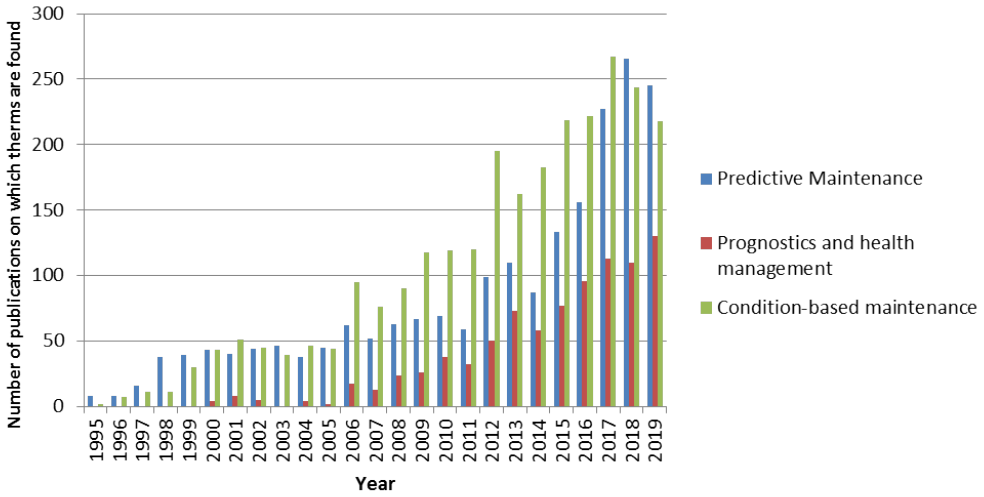


Figura. E.2: Número de publicaciones relacionadas con mantenimiento predictivo en los últimos 25 años.

La revisión bibliográfica se dividió en dos partes. En la primera se abordaron revisiones bibliográficas previas sobre el mantenimiento predictivo lo que ayudó a identificar los modelos utilizados para el mantenimiento predictivo y la evolución de las tendencias a lo largo de los años. Las taxonomías utilizadas para clasificar los diferentes modelos de diagnóstico y pronóstico en mantenimiento predictivo muestran ligeras variaciones en la terminología de un estudio a otro. Pueden extraerse dos enfoques principales: los enfoques de un solo modelo y los enfoques de varios modelos. Para los enfoques de un solo modelo se pueden identificar tres familias de modelos; para este estudio estas familias de modelos se denominarán de la siguiente manera: modelos basados en el conocimiento, modelos basados en datos y modelos basados en la física. Los enfoques multimodelo combinan al menos dos modelos de las tres familias de modelos mencionadas. Los enfoques multimodelo pueden tener diferentes configuraciones y a veces son nombrados como modelos híbridos.

La segunda parte de la revisión literaria se dedicó al estudio de las tendencias actuales en los diferentes enfoques de los modelos. La revisión literaria ha mostrado que existe una tendencia en la implementación de enfoques multimodelo ya que un solo modelo difícilmente satisface todas las funciones en un sistema de

mantenimiento predictivo. La Figura E.3 muestra todas las posibilidades de combinar modelos en enfoques multimodelo.

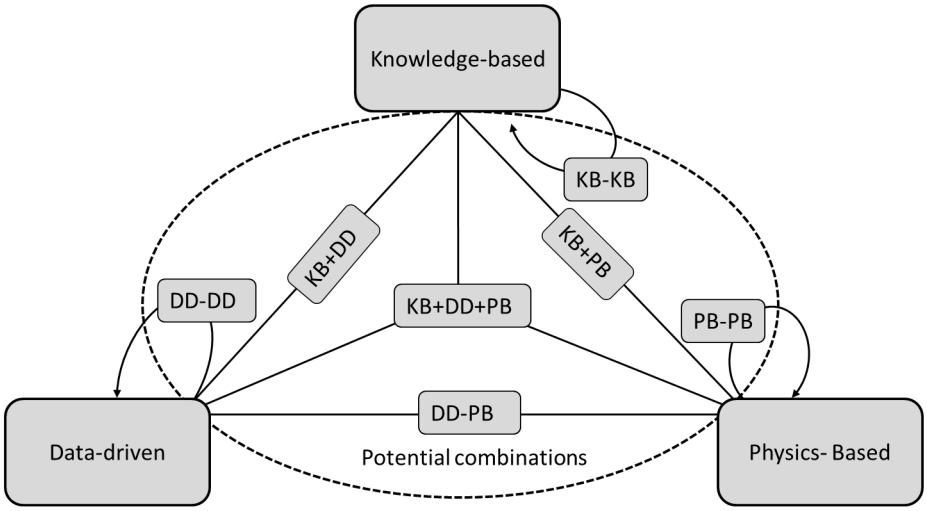


Figura. E.3: Posibles combinaciones de modelos de mantenimiento predictivo.

La revisión bibliográfica permitió identificar los desafíos actuales para el mantenimiento predictivo:

- La extrapolación de soluciones existentes de mantenimiento predictivo en aplicaciones complejas, incluyendo múltiples componentes y sus fallas asociadas. La mayoría de las aplicaciones identificadas se centraron en un solo componente con un número limitado de fallos. Sin embargo, las aplicaciones de la vida real son frecuentemente sistemas complejos compuestos de muchos componentes y muchas fallas asociadas a cada componente y al propio sistema.
- La falta de un enfoque sistemático para diseñar y desarrollar sistemas de mantenimiento predictivo. Existen estándares, normas y arquitecturas genéricas para desarrollar nuevos sistemas de mante nimiento predictivo, como OSA-CBM. Sin embargo, sólo se centran en los componentes funcionales básicos del sistema y no abarcan aspectos importantes relativos a los indicadores de rendimiento o las limitaciones de contexto del sistema. Además, todavía no ofrecen una explicación consistente sobre qué modelos utilizar en función de las necesidades iniciales del sistema de mantenimiento predictivo. La falta de un enfoque sistemático limita la implementación de sistemas de mantenimiento predictivo en aplicaciones industriales a escala real.
- La fusión de diversas fuentes de datos de monitoreo de condición. Este desafío está relacionado con la extrapolación de los modelos actuales en mantenimiento predictivo a sistemas complejos. Los sistemas técnicos pueden tener diferentes tipos de fuentes de datos, por ejemplo mediciones de sensores, registros de mantenimiento, registros operacionales, documentos de diseño, etc. Se podrían reunir información importante de todas estas fuentes para implementar nuevos sistemas de mantenimiento predictivo.
- La incorporación de datos de influencia externa. El funcionamiento de los sistemas puede variar dependiendo de su contexto operativo. Los cambios en el contexto operativo pueden afectar directamente al rendimiento del sistema y, por consiguiente, a las lecturas del seguimiento sanitario. Puede activar

falsas alarmas sugiriendo la existencia de fallas, o puede impedir la identificación de fallas existentes. Esto podría abordarse mediante modelos complementarios capaces de incorporar la influencia externa con fines de mantenimiento predictivo.

- Manejo de la incertidumbre. La incertidumbre afecta directamente la precisión del diagnóstico y el pronóstico. Puede deberse a los datos recopilados o a las imperfecciones del modelo utilizado para el análisis. Puede afectar la fiabilidad de los resultados. La gestión de la incertidumbre es vital para los sistemas críticos sujetos a las regulaciones de las autoridades. Este es el caso de sistemas críticos como las centrales nucleares y las aeronaves en las que las regulaciones son restrictivas para mantener los estándares de seguridad y evitar eventos catastróficos.

El estudio del estado del arte motivó la mejora de las preguntas de investigación. El resto de la investigación fue impulsado por las siguientes preguntas:

1. ¿Cómo abordar la arquitectura y diseño de sistemas de mantenimiento predictivo?
2. ¿Cómo seleccionar un modelo adecuado o combinación de modelos dado un nuevo problema de mantenimiento predictivo para resolver?
3. ¿Cómo sugerir un enfoque adecuado para una solución de mantenimiento predictivo?
4. ¿Cómo puede un diseñador beneficiarse de la experiencia de los sistemas existentes para desarrollar nuevas soluciones de mantenimiento predictivo?

E.3 De necesidades y deseos hasta una arquitectura lógica de sistemas de mantenimiento predictivo

La primera pregunta de investigación obtenida al final del estudio del estado del arte es sobre el diseño de nuevos sistemas de mantenimiento predictivo. En la declaración de investigación presentada en la sección D.1 se explicó que el diseño y desarrollo de sistemas de mantenimiento predictivo todavía se basa en prueba y error. A pesar de la existencia de varias arquitecturas genéricas en normas y estándares, como por ejemplo [MIM01], la etapa conceptual de tales sistemas no está totalmente cubierta. Hay una brecha en el desarrollo de tales sistemas desde la recolección de necesidades y deseos para el nuevo sistema hasta la creación de la arquitectura que satisfaga las necesidades y deseos iniciales.

La creación de un nuevo sistema debe comenzar siempre escuchando a aquellos que están relacionados o interesados en el proyecto, conocidos como partes interesadas. Ellos proveen todas las necesidades y deseos para ser cumplidos por un nuevo sistema. Estas necesidades y deseos son la fuente de información para establecer la lista de requisitos de las partes interesadas del nuevo sistema. La investigación actual propone un enfoque de ingeniería de sistemas para cubrir la etapa conceptual de los sistemas de mantenimiento predictivo.

En el gráfico D.4 se muestran las diferentes etapas abordadas en el enfoque propuesto de ingeniería de sistemas. Comienza por reunir las necesidades y deseos iniciales de las partes interesadas. Estas necesidades y deseos se traducen en una lista formal de requisitos de las partes interesadas. Los requerimientos son priorizados y clasificados en requerimientos funcionales, de desempeño, estructurales y experienciales. Esta clasificación ayudará en la creación de la arquitectura del sistema. Los requisitos funcionales se utilizan para iniciar el proceso de arquitectura, específicamente para realizar el análisis funcional y crear el funcional. La arquitectura funcional se utiliza entonces para desarrollar la arquitectura lógica que sigue

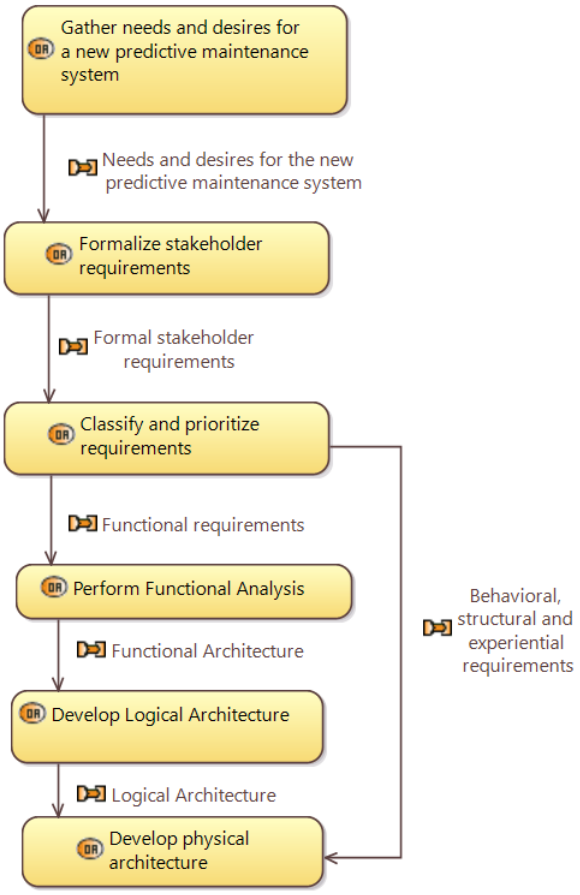


Figura. E.4: Pasos abordados en el enfoque de ingeniería de sistemas para el diseño de sistemas de mantenimiento predictivo

siendo genérica. Los requisitos de desempeño, estructurales y experienciales se utilizan para realizar la selección de componentes para cumplir con la arquitectura lógica y crear la arquitectura física.

Esta sección está relacionada con capítulo 3 de la tesis en el que se propuso un enfoque sistemático para abordar la etapa conceptual de los sistemas de mantenimiento predictivo a fin de dar respuesta a la primera pregunta de investigación perfeccionada. Los diferentes pasos desde la reunión de las necesidades y deseos iniciales de las partes interesadas hasta la definición de la arquitectura lógica se han cubierto y diferentes métodos se han propuesto para abordarlos. Sin embargo, el resto de las preguntas de investigación refinadas están aún por responder. En el mantenimiento predictivo, el espacio de solución es vasto y complejo como se muestra en el Capítulo 2 de la tesis. No hay reglas específicas que un arquitecto pueda seguir para seleccionar los modelos y enfoques adecuados para resolver nuevos problemas de mantenimiento predictivo. En esta investigación se propone una hipótesis para superar este problema: la implementación de un Sistema de Apoyo a la Decisión (DSS, por sus siglas en inglés) basado en el Razonamiento Basado en Casos (CBR, por sus siglas en inglés) y apoyado por ontologías podría ayudar al arquitecto a seleccionar componentes adecuados basados en experiencias pasadas extraídas de implementaciones exitosas de sistemas de mantenimiento predictivo.

E.4 Desarrollo de una ontología para el sistema de razonamiento basado en casos

El DSS se compone de dos bloques de construcción principales: Razonamiento basado en casos (CBR) y ontologías. CBR es un paradigma de razonamiento que busca resolver nuevos problemas basados en las experiencias de problemas similares resueltos en el pasado. CBR se aborda en el capítulo 5 de la tesis y en la sección D.5 del presente resumen, pero es necesaria una breve introducción para comprender el papel que desempeñan las ontologías en el DSS desarrollado en la investigación actual. Los sistemas CBR se desarrollan sobre un vocabulario base [Alt+12]; [Sán+12]. Este vocabulario es necesario para dar estructura a los casos almacenados de problemas resueltos en un caso basado, a las medidas de similitud que comparan el nuevo problema con los de la base de casos, y a los conocimientos necesarios para adaptar la solución recuperada para el nuevo problema (véase Figura E.5). Para efectos de esta tesis se elige una ontología para que sirva de marco terminológico (vocabulario base). Un modelo de ontología proporciona los términos, definiciones y relaciones entre los términos que se utilizan para construir la estructura de casos, las similitudes y el conocimiento de adaptación en el DSS. Este capítulo está dedicado a los antecedentes teóricos y el desarrollo del modelo de ontología para el DSS propuesto.

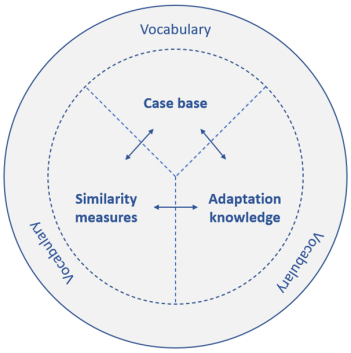


Figura. E.5: Razonamiento basado en casos desarrollado sobre un marco de vocabulario [Alt+12]

En ciencias de la información, una ontología es una descripción explícita formal de conceptos en un dominio del discurso, propiedades de cada concepto que describen sus características, atributos y restricciones [NM01]. Uno de los objetivos más comunes en el desarrollo de ontologías es "compartir un entendimiento común de la información de la estructura entre personas y agentes de software" [Gru93]; [Mus92]. Esto significa que el vocabulario utilizado por las personas en un dominio específico de conocimiento está habilitado para ser "legible por máquinas". Todos los conceptos en una ontología están representados por clases que están vinculadas por propiedades (también llamadas relaciones). Las ontologías se desarrollan con lenguajes formales. Uno de los más reconocidos es el Lenguaje de Ontología Web en su segunda versión (OWL2) que es apoyado por el World Wide Web Consortium (W3C) [Wor12].

E.4.1 Ontología para la selección y evaluación de estrategias de mantenimiento (OMSSA por sus siglas en inglés)

Una investigación complementaria al hilo principal de esta tesis permitió la creación de un modelo ontológico para la selección y evaluación de estrategias de mantenimiento (OMSSA por sus siglas en inglés). La creación de OMSSA sigue los mayores estándares y últimas tendencias en la creación de ontologías utilizando ontologías de alto nivel y medio nivel de referencia. Esto facilitará la reutilización e integración de OMSSA con otras ontologías relacionadas. La contribución de OMSSA fue consolidada en un artículo

científico que fue aceptado para su publicación en el momento que esta tesis fue escrita. OMSSA sirve de base para la creación del modelo ontológico que se usa para el sistema de ayuda a la decisión (DSS) propuesto en esta investigación.

E.4.2 Ontología para la concepción de mantenimiento predictivo (OPMAD por sus siglas en inglés)

Para esta sección se mantendrán los nombres de las clases de OPMAD en inglés para mantener congruencia con el modelo y los gráficos. OPMAD es una extensión de OMSSA y se han seguido los mismos estándares y metodologías para su desarrollo. La Figura E.6 presenta las clases y relaciones más importantes en OPMAD. Por limitación de espacio, las subclases no se muestran en la figura. Como el objetivo de OPMAD es apoyar el DSS para identificar modelos adecuados para los sistemas de mantenimiento predictivo, la explicación de las clases de ontología y sus relaciones comienza a partir de la clase Modelo de Mantenimiento Predictivo (`Predictive Maintenance Model`), y se basa en los términos presentados en la Figura E.6. Un modelo de mantenimiento predictivo se lleva en un módulo de mantenimiento predictivo (`Predictive Maintenance Module`) que es un componente de un sistema de mantenimiento predictivo (`Predictive Maintenance System`). Cada módulo de mantenimiento predictivo tiene una Función de Mantenimiento Predictivo (Predictive Maintenance Function), esta investigación se centra en funciones de diagnóstico y pronóstico.

El modelo de mantenimiento predictivo integrado en el módulo está directamente vinculado a un elemento mantenible (`Maintainable item`), que es la clase que describe el sistema mantenible para el que se desarrolla el sistema de mantenimiento predictivo. El elemento mantenible se clasifica por la clase tipo de elemento mantenible (`Maintainable item Type`) que se agregó con fines de cálculo de similitud en el DSS; este tipo de clases ayudan a comparar el nuevo problema a tratar con el problema resuelto en una base de casos de un sistema CBR. Los elementos mantenibles pertenecientes al mismo tipo comparten importantes características de degradación y por lo tanto se pueden proponer los mismos modelos de mantenimiento predictivo para resolver las mismas funciones de mantenimiento predictivo.

El elemento mantenible tiene su propia función (`Function`) que se ve afectada por una falla (`Failure`) que se manifiesta a través de un modo de falla (`Failure Mode`). Un elemento mantenible tiene uno o varios modos de fallo que también son objeto del modelo de mantenimiento predictivo. El modelo de mantenimiento predictivo tiene como entrada los datos de condición (`Condition`) que se utilizan para realizar diagnósticos y pronósticos. Dos cualidades importantes para la implementación de un modelo de mantenimiento predictivo es su tipo y configuración. El tipo de modelo de mantenimiento predictivo (`Predictive Maintenance Model Type`) clasifica el modelo dentro de las familias de modelos basados en conocimiento, basados en datos y basados en la física. Este tipo de clasificación ayuda a efectos de similitud y preferencias en el DSS. La configuración del modelo de mantenimiento predictivo (`Predictive Maintenance Model Configuration`) muestra si un modelo debe complementarse con otros modelos para cumplir su función; puede mejorar el rendimiento, pero aumenta la complejidad del desarrollo. El módulo de mantenimiento predictivo está calificado por la sincronización (`Module Synchronization`) y el indicador de rendimiento (`Module Performance Indicator`). Estas dos cualidades proporcionan información útil sobre cómo se deben probar y sincronizar los modelos de mantenimiento predictivos incorporados en los módulos con el elemento mantenible.

Toda esta información de los modelos de mantenimiento predictivo, elementos mantenibles, sus modelos de fallo, entre otras, se colecta en el caso de mantenimiento predictivo (`Predictive Maintenance Case`) que se documenta y publica a través de un artículo de mantenimiento predictivo (`Predictive Maintenance Article`), un tipo especial de artículo que se centra en el mantenimiento predictivo. A partir de estos artículos de mantenimiento predictivo también se incluyen algunos indicadores bibliométricos en la ontología que se utilizará en el sistema CBR. Estos indicadores bibliométricos se proporcionan como atributos de

solución para que el ingeniero pueda encontrar fácilmente la fuente de información de un caso específico. El título, el identificador y el año de publicación del artículo pueden utilizarse con fines de similitud y/o como fuente de información para más detalles de los modelos de mantenimiento predictivo y su implementación.

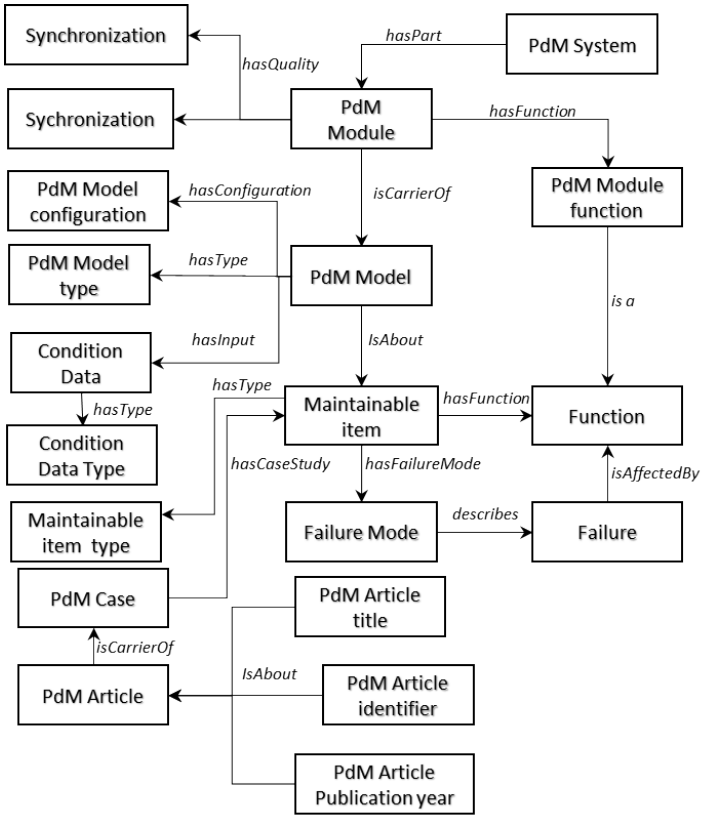


Figura. E.6: Clases and relaciones en OPMAD

E.5 Desarrollo de sistemas de razonamiento basado en casos

El razonamiento basado en casos (CBR) es una metodología de resolución de problemas basada en recuperación de soluciones previas para problemas similares [De +05]. El razonamiento basado en el caso se desarrolló bajo la filosofía de que los seres humanos piensan y razonan usando analogías y ejemplos, en lugar de estructuras SI-ENTONCES, estas últimas formando la base para el razonamiento basado en reglas. La solución del problema se realiza en un proceso cíclico de varios pasos: el ciclo CBR [AP94]. Este ciclo se compone de cuatro fases: recuperar, reutilizar, revisar y retener (ver Figura E.7).

El ciclo CBR se activa cuando se encuentra un nuevo problema. Esta primera fase tiene como objetivo recuperar los casos más similares de una base de conocimientos que almacena todos los casos anteriores. El caso objetivo (nuevo) se compara con los casos existentes en la base de conocimientos utilizando diferentes mediciones de similitud. El caso más cercano recuperado se propone como una posible solución en la fase de reutilización. Tal vez se necesite alguna adaptación para aplicar la solución en el caso previsto. Después de sugerir e implementar la solución se lleva a cabo la fase de revisión. Si la solución sugerida logra resolver el problema, se confirma y en la última fase, se retiene en la base de conocimiento para que pueda ser reutilizada en futuros problemas similares. En las subsecciones siguientes se proporcionan más detalles sobre cada fase de la RBC.

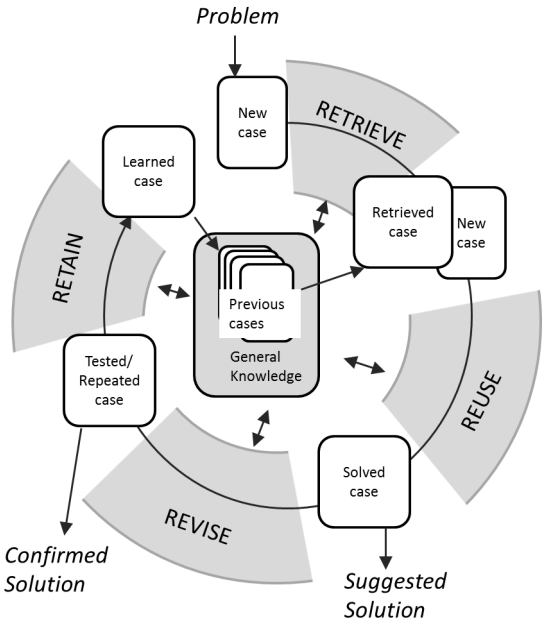


Figura. E.7: Ciclo de razonamiento basado en casos, ilustración inspirada en [AP94]

E.5.1 Plataforma MyCBR

El módulo CBR del DSS para la investigación actual se ha creado utilizando myCBR, una herramienta de búsqueda basada en similitudes de código abierto para el razonamiento basado en casos (CBR) [Alt+12]. Al desarrollar un sistema de recuperación de CBR usando myCBR el primer paso es determinar los atributos del caso. Los casos están representados por un vector acoplado de atributos:

Caso = [Atributos del problema, Atributos de la solución].

Los atributos de problema se utilizan para medir la similitud de un caso objetivo y los casos en una base de casos. Los atributos de la solución, como su nombre lo dice proveen información relevante relacionada con la solución del problema en cada caso almacenado en la base de casos.

Una vez los atributos han sido determinados, el segundo paso es la asignación de una medida de similitud local a cada atributo del problema. MyCBR proporciona varias opciones para calcular la similitud para cada atributo del problema. Dentro de las opciones disponibles, se han utilizado tres funciones de similitud para la investigación actual:

1. Similitud entero/flotante: esta función de similitud se utiliza para los atributos numéricos. La similitud se obtiene por una diferencia entre un valor de referencia y los valores de entrada de la función. Se necesita una función matemática para definir cómo disminuye la similitud a medida que los valores de entrada se alejan del valor de referencia. Esta función matemática puede ser lineal, exponencial o determinada por puntos discretos en el plano cartesiano.

2. Similitud de símbolo: esta medida de similitud es aconsejable para variables con un conjunto fijo de opciones. Estas opciones se organizan en una matriz de similitud y se dan algunos valores numéricos

para establecer la similitud entre las opciones. Esta función de similitud ha sido modificada para que algunos atributos de la investigación actual interactúen con el modelo ontológico. Los valores de la matriz de similitud se obtienen automáticamente a partir de las clases y relaciones de una ontología utilizando el enfoque de similitud basado en características propuesto por [Sán+12].

3. Similitud de cadena de caracteres: la similitud de atributos se obtiene con base en cadenas de texto abiertas. A diferencia de la similitud de símbolo que tiene un número limitado de opciones para definir la similitud, la similitud de cadena de caracteres solo tiene la restricción de tener oraciones o palabras como entrada. MyCBR ofrece tres opciones para calcular la similitud basada en cadenas: Igualdad, Ngram y Levenshtein. Para el estudio de caso actual, se seleccionó la función Levenshtein para calcular los atributos basados en cadenas de caracteres de similitud. La función Levenshtein ofrece un medio flexible para calcular la similitud basada en cada carácter de la cadena [Lev66]. Esto es especialmente útil cuando hay un amplio conjunto de opciones que son desconocidas al crear similitudes y también tolera pequeños errores ortográficos.

Después del cálculo de similitud para cada uno de los atributos del problema, se combinan estas similitudes individuales en una similitud global. Por medio de una función de agregación se calcula la similitud global en la que cada atributo tiene un peso específico. El objetivo de los pesos es dar especial importancia a los atributos de problemas. Para efectos de esta investigación, todos los atributos reciben el peso de 1, lo que significa que tienen la misma importancia. MyCBR ofrece dos opciones diferentes para calcular la similitud global: suma ponderada y distancia euclidiana.

- Suma ponderada: como dice el nombre, es la suma de similitudes considerando el peso de cada una.
- Distancia euclidiana: la distancia euclidiana entre dos puntos en el espacio euclidiano es un número, la longitud de un segmento de línea entre los dos puntos.

En esta tesis, ambas funciones de amalgamación pueden ser utilizadas por el usuario. Es importante señalar que para calcular la similitud global sólo se consideran los atributos disponibles. Esto significa que, si el arquitecto cuenta sólo con dos o tres atributos de los siete que describen el problema, la similitud global se calculará sobre la base de esos dos o tres atributos.

E.5.2 Representación de los casos

"Un caso es una pieza de conocimiento en un contexto particular que representa una experiencia que enseña una lección esencial para alcanzar la meta del razonador" [Kol93]. Los casos se representan a menudo en una forma acoplada [problema, solución], en la que se aplican similitudes en la parte del "problema" para que la parte de la "solución" se recupera. La representación del caso se compone de tres partes principales [BKP05]:

1. Definir los atributos del caso.
2. Definir la estructura del contenido del caso.
3. Organizar la base de casos.

La representación y estructura del caso se hace usando OPMAD. La Figura E.6 mostró los diferentes atributos (clases en ontología) del caso representado en OPMAD. Estas clases de ontología se pueden dividir en dos grupos diferentes: los atributos del problema y los atributos de la solución (mantienen sus nombres originales en inglés):

- **Atributos del problema** = [PdM Function, Maintainable Item, Maintainable Item Type, Condition Data Type, Module synchronization, PdM Article Publication year]
- **Atributos de l solución** = [PdM Model, PdM Model Configuration, PdM Model Type, Module Performance Indicator, PdM Article Identifier, PdM Article Title]

E.5.3 Desarrollo del motor de recuperación de componentes para sistemas de mantenimiento predictivo

La base de casos del DSS es una versión instanciada de OPMAD que ha sido poblada con casos exitosos de implementaciones de mantenimiento predictivo. El proceso para instanciar OPMAD a partir de una extensa revisión bibliográfica incluye la definición de las diferentes variables a buscar y las posibles opciones para cada variable; esto permite una mejor estructura de base de casos y facilita la recuperación. Para cada atributo de problema, se selecciona una similitud local entre las opciones posibles: entero, símbolo, basado en ontología, texto abierto. Tabla E.1 muestra las funciones de similitud asignadas a cada uno de los atributos del problema. Se han añadido dos funciones de agregación diferentes para calcular la similitud global entre un caso objetivo y los casos almacenados en la base de casos. Se ha desarrollado un GUI para facilitar la verificación y validación del motor de recuperación. El motor de recuperación desarrollado es el núcleo del DSS para la selección de componentes de mantenimiento predictivo. El siguiente capítulo está orientado a mostrar el marco completo del sistema CBR con ontología para la selección de componentes de mantenimiento predictivo. Los detalles de desarrollo del motor de recuperación se proporcionan en la guía de códigos del apéndice C.

Tabla E.1: Asignación de funciones de similitud

Attibute	**Similarity function**
PdM Function	Símbolo (Ontología)
Maintainable item	Cadena (Levenshtein)
Maintainable item type	Símbolo (igualdad)
Condition Data	Símbolo (Ontología)
Condition Data Type	Símbolo (igualdad)
Module synchronization	Símbolo (igualdad)
PdM Article Publication Year	Entero (función definida por puntos)

E.6 Desarrollando un marco de trabajo para la selección de modelos de mantenimiento predictivo

Un hito común en el desarrollo de la arquitectura de sistemas es la arquitectura lógica. En la arquitectura lógica se provee la mayor cantidad de detalle posible manteniendo los componentes genéricos. Estos componentes genéricos deben ser sustituidos por componentes específicos creando así la arquitectura física del sistema. Para la selección de los componentes se puede aplicar la creatividad estructurada. La creatividad estructurada en la arquitectura lógica se basa en analizar las posibles combinaciones de los diferentes componentes físicos/informativos que pueden ser seleccionados para satisfacer cada componente lógico. Esto permite la exploración del espacio de solución y puede ayudar al arquitecto a identificar soluciones innovadoras para un nuevo sistema. Si se identifican varias soluciones posibles, puede ser necesario un análisis de compensación para seleccionar la más adecuada. La arquitectura seleccionada sirve de base para el diseño detallado de los sistemas. Los conocimientos obtenidos del nuevo sistema implementado pueden ser utilizados por el arquitecto para desarrollar futuros sistemas. Existe una analogía entre el trabajo de

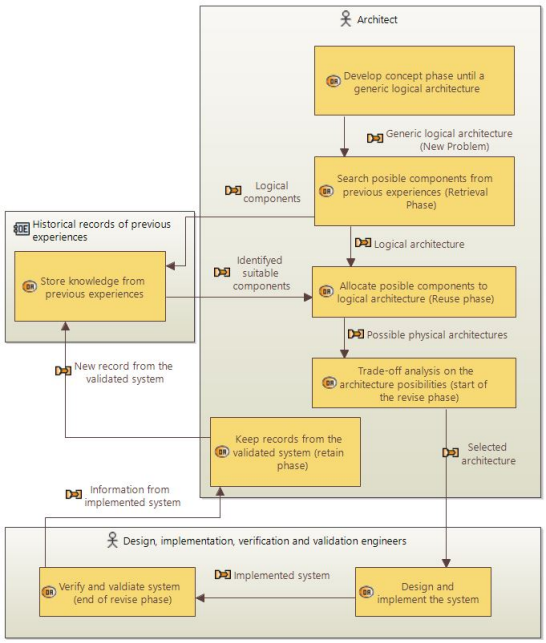


Figura. E.8: Analogía entre razonamiento basado en casos y las tareas que realiza un diseñador de sistemas, en notación Capella [Roq18]

creatividad estructurada realizado por el arquitecto para seleccionar componentes adecuados para cumplir con la arquitectura lógica y las cuatro fases del Razonamiento Basado en Casos: recuperar, reutilizar, revisar y retener. Una representación gráfica de esta analogía se presenta en Figura E.8 (en notación de Capella [Roq18]).

La analogía propuesta relaciona la fase de recuperación de CBR con la búsqueda realizada por el arquitecto en sistemas anteriores relacionados que pueden servir de inspiración para el desarrollo del nuevo sistema.

- La fase de reutilización de CBR sería la asignación de los componentes identificados para cumplir con la nueva arquitectura del sistema.
- La fase de revisión del CBR comenzaría por el análisis de compensación realizado por el arquitecto para identificar los componentes más adecuados. Esta fase continúa hasta la verificación y validación del sistema implementado que generalmente son realizados por otros actores distintos del arquitecto.
- Después de la validación del nuevo sistema, el arquitecto puede mantener los registros que se utilizarán en el futuro; esta sería la fase de retención de CBR.

Esta analogía sigue siendo genérica en cuanto al sistema que el arquitecto va a desarrollar. Todas las actividades son realizadas "manualmente" por el arquitecto y los registros de experiencias previas pueden no estar estructurados para facilitar su reutilización. Este enfoque manual puede funcionar cuando la cantidad de experiencias previas es limitada para que el espacio de solución a ser explorado por el arquitecto siga siendo manejable. Cuando el número de experiencias previas es alto o los requisitos complejos, la selección de los componentes más adecuados puede no ser fácil. Recuperar conocimiento de un número importante de arquitecturas anteriores puede consumir demasiado tiempo y se pueden perder opciones importantes.

Considerando esta analogía entre CBR y el trabajo de creatividad estructurada, se pueden proponer diferentes algoritmos para apoyar al arquitecto en las diferentes fases del trabajo de arquitectura. En un

primer paso, la investigación actual se centró en el desarrollo de un Sistema de Apoyo a la Decisión (DSS) capaz de realizar la búsqueda y recomendación de componentes físicos/informativos adecuados. Esto puede ayudar al arquitecto a ahorrar tiempo en la fase de concepto, y permite un análisis más amplio realizado por máquinas, análisis que difícilmente pueden ser abordados por los seres humanos.

La Figura E.9 presenta un concepto del DSS y cómo encaja en la analogía presentada en la Figura E.8. Este concepto se compone de tres partes principales: la base de casos, el motor de recuperación y la ontología específica del dominio. La base de casos almacena los casos del pasado en un formato estructurado para facilitar su reutilización. El arquitecto presentará al motor de recuperación la información del nuevo sistema en desarrollo y obtendrá un conjunto de los casos más similares recuperados de la base de casos que servirá de inspiración al seleccionar componentes adecuados para la arquitectura lógica.

Dentro de esta estructura la ontología juega un papel vital. Los casos, los atributos de estos casos y las similitudes entre las diferentes variables basadas en texto se describen a menudo en lenguaje natural. Modelar este lenguaje natural y hacerlo legible por máquina es necesario para automatizar la recuperación del caso. La integración de la ontología propuesta y el módulo de recuperación de CBR se explica con más detalle en una investigación complementaria presentada en el Apéndice B. La siguiente sección aborda la validación del DSS propuesto y la discusión de los resultados obtenidos.

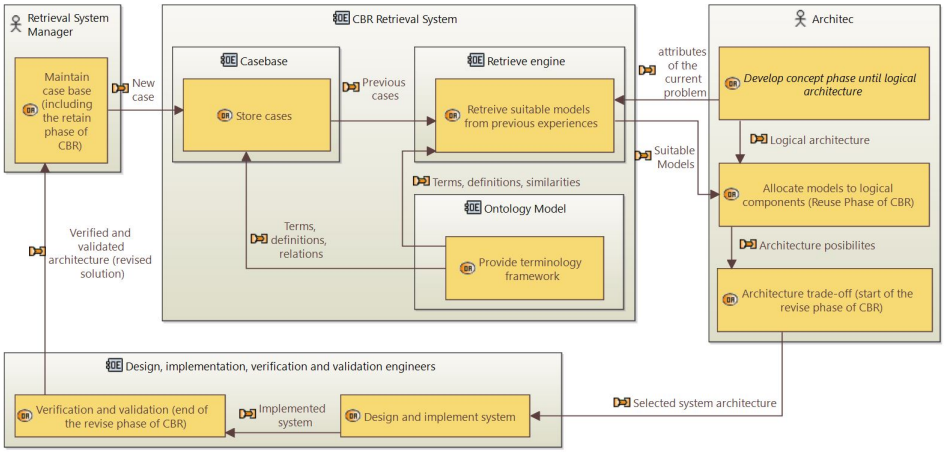


Figura. E.9: Idea conceptual de la incorporación del DSS para la selección de componentes de sistemas

E.7 Validación y discusión de resultados

La validación del DSS se divide en dos partes: una validación cruzada para confirmar la coherencia del DSS y una implementación de un modelo sugerido en un estudio de caso para probar las recomendaciones del DSS en un ejemplo práctico.

E.7.1 Validación cruzada

La validación cruzada es una técnica que se puede utilizar para probar la eficacia de los modelos de inteligencia artificial entrenados. Para el abordaje actual, se selecciona un enfoque de división de la prueba de tren para realizar la validación cruzada. Para el conjunto de pruebas, 63 de los 263 casos de la base de casos se extrajeron al azar y el resto se ha dejado en la base de casos como conjunto de entrenamiento. Para cada uno de los 63 casos extraídos, los atributos del problema fueron presentados al DSS usando GUI.

La atención se centró en los 10 casos más similares al realizar la recuperación. La Figura E.10 muestra un ejemplo de una prueba de recuperación en la que todos los atributos de problema del caso objetivo se corresponden con los atributos de solución correspondientes del caso 35. Esta coincidencia completa entre los atributos resulta en una similitud global de 1.

El espacio de solución de los modelos de mantenimiento predictivo se puede dividir en dos grupos como se explica en [Mon+20]. El primer grupo está compuesto por enfoques de modelo único, divididos en tres categorías principales: modelos basados en el conocimiento, modelos basados en datos y modelos basados en la física. El segundo grupo está compuesto por enfoques multimodelo, en los que al menos dos modelos de cualquiera de las tres categorías mencionadas se combinan para cumplir una función específica del sistema de mantenimiento predictivo. El mantenimiento predictivo está compuesto por tareas de diagnóstico y pronóstico. Según la revisión de la literatura [Mon+20], las tareas de diagnóstico como la detección de fallas, la identificación de fallas y la modelización del estado de salud pueden ser abordadas por modelos de las tres categorías de enfoques de modelo único o enfoques multimodelo. En cambio, para las tareas de pronóstico es difícil encontrar modelos basados en el conocimiento. Por ejemplo, las funciones de pronóstico, como la estimación de la vida útil restante y las funciones de previsión del estado siguiente, se cumplen normalmente mediante modelos basados en la física, modelos basados en datos y enfoques multimodelo que combinan modelos basados en la física y en datos. La validación cruzada del DSS permitió confirmar este comportamiento. Las recuperaciones para tareas de diagnóstico propusieron soluciones de modelo único y multimodelo considerando modelos de las tres categorías de modelos. Las recuperaciones para pronósticos también propusieron soluciones de modelo único y multimodelo, pero los modelos propuestos fueron sólo de categorías basadas en datos y en la física. Esto ayuda a confirmar que el espacio de solución está bien cubierto.

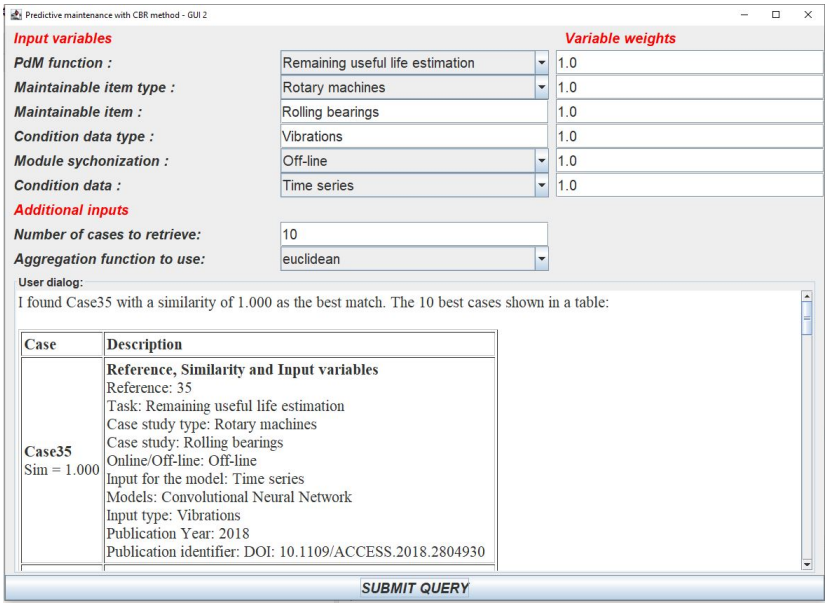


Figura. E.10: Ejemplo de recuperación de caso usando la interfaz del DSS

Para cada caso de prueba, los casos recuperados se clasifican en función de la similitud de los atributos del problema. Para algunas de las pruebas, el remolque o más casos recuperados tenían el mismo valor de similitud máximo y/o proponían el mismo modelo para cumplir una función PdM específica. Desde una perspectiva de Ingeniería de Sistemas, esto representa dos limitaciones. El primero se refiere a dos o más casos recuperados con la misma similitud máxima porque no se proporciona más información para realizar el análisis de compensación y se selecciona el más adecuado. La segunda limitación está relacionada con la diversidad en los casos recuperados, especialmente cuando dos o más casos recuperados sugieren

que el mismo modelo cumple una función específica. Un arquitecto que busca inspiración para desarrollar soluciones innovadoras necesitará del DSS para proponer un conjunto diverso de modelos. Estos son puntos de mejora para que el DSS sea considerado en el futuro. También se puede realizar un análisis más detallado que incluya el esfuerzo de adaptar la solución de un caso recuperado para el caso objetivo. Esto puede ayudar a realizar el análisis de compensación en las opciones recuperadas y, en consecuencia, mejorar el rendimiento del DSS.

E.7.2 Implementación del caso de estudio práctico

Como parte de la validación, se ha desarrollado un ejemplo de implementación a partir de las sugerencias del DSS en caso de estudio. El caso de estudio consiste en un conjunto de datos de motor de avión [Cha+21]. El conjunto de datos contiene los registros de fallos de funcionamiento de 128 motores de avión en condiciones reales de vuelo y se ha generado con el modelo Commercial Modular Aero-Propulsion System Simulation (CMAPSS) desarrollado por la NASA. Estos datos se llaman el conjunto de datos N-CMAPSS. El propósito no es sólo comprobar las capacidades del sistema CBR habilitado por ontología (el DSS) sino también identificar los puntos de mejora para el DSS. El objetivo de esta validación es implementar uno de los modelos propuestos por el DSS para cumplir una función de mantenimiento predictivo para la base de datos N-CMAPSS. Esta implementación ayuda a demostrar que el DSS es capaz de sugerir componentes adecuados para sistemas de mantenimiento predictivo, complementando la validación cruzada.

A fin de validar mejor las recomendaciones del Departamento de Seguridad, se ha tomado como ejemplo uno de los modelos propuestos y se ha aplicado para cumplir la función de mantenimiento predictivo correspondiente. Aprovechando la experiencia del equipo de investigación en Mapas de Auto Organización (SOM), se realiza una implementación del componente de modelado de salud utilizando el SOM. El DSS recomienda el SOM y la regresión logística para el modelado de salud como los modelos más adecuados (similitud igual a 0.879) para el estudio de caso N-CMAPSS. La siguiente sección proporciona explicaciones adicionales sobre la implementación del ejemplo SOM y sus resultados preliminares.

La metodología para implementar los Mapas de Auto-Organización (SOM) para la modelización de la salud ha sido adoptada de [Sch+20], (véase también el Apéndice A). Los mapas auto-organizados son redes neuronales artificiales con entrenamiento no supervisado que son capaces de agrupar instancias de datos dependiendo de los atributos de instancia. SOM se compone normalmente de una capa cuadrada de neuronas y los diferentes grupos después de la formación SOM se puede representar gráficamente en los mapas como regiones bien definidas. Se ha utilizado con éxito para modelar el proceso de degradación de diferentes máquinas como los motores a reacción [MV18]. En estos casos, las neuronas representan la salud o la degradación de la máquina en un modo operativo específico. El SOM entrenado tendrá una sola región, pero una transición de blanco (estado óptimo) a negro (estado fallido). Al evaluar la salud o la degradación de una máquina usando el SOM entrenado, una neurona saldrá en el mapa mostrando qué tan avanzada está la degradación o cuánto ha disminuido la salud. Este es el verdadero objetivo de la implementación actual: obtener un SOM capacitado capaz de mostrar una transición del estado óptimo al estado fallido. La figura E.11 muestra el mapa auto-organizativo entrenado con el N-CMAPSS. El resultado corresponde con el comportamiento esperado. Los diferentes modelos de degradación han sido ordenados en el mapa.

Es importante recordar que la validación de un Sistema de Apoyo a las Decisiones (DSS) es una tarea difícil. La validación del SOM para modelar la salud del N-CMAPSS puede utilizarse para validar indirectamente el DSS; sin embargo, es importante resaltar que se requerirían otras implementaciones que confirmen las capacidades y la precisión del DSS propuesto para la selección de modelos de mantenimiento predictivo. En la implementación actual, el N-CMPASS representa el caso objetivo para el que se tiene que desarrollar un nuevo sistema de mantenimiento predictivo. El sistema de recomendación CBR con ontología (también llamado DSS en este manuscrito) propuso diferentes modelos para cumplir cada función

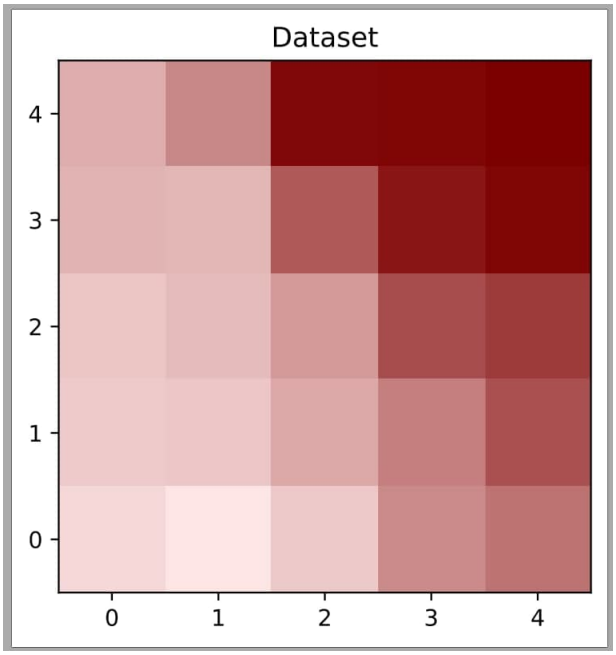


Figura. E.11: Mapa auto-organizativo entrenado con el caso de estudio N-CMAPSS

de mantenimiento predictivo. Uno de los modelos propuestos por el DSS para el modelado de la función de salud fue el Mapa de Auto-Organización (SOM). La implementación de SOM mostró con éxito la tendencia de degradación de los motores a reacción desde la operación nominal a la condición fallida utilizando un subconjunto del N-CMPASS. El SOM entrenado también se puede utilizar para evaluar la salud/ degradación de otro motor del mismo tipo y bajo las mismas condiciones de operación. Es importante aclarar que esta validación tiene por objeto demostrar la idoneidad del modelo propuesto por el DSS, pero se necesitan más comparaciones entre los modelos propuestos para determinar el mejor modelo.

E.8 Conclusión y perspectivas de trabajo futuro

E.8.1 Resumen de contribuciones

La tesis está compuesta por cuatro artículos que han sido publicados o aceptados para su publicación en revistas o conferencias internacionales. Las principales contribuciones de la investigación doctoral se han consolidado en los artículos de la siguiente manera:

1. El artículo de revisión bibliográfica resumió el estado del arte en diagnósticos y pronósticos. Se ha presentado una propuesta para diferenciar los modelos híbridos de los enfoques multimodelo. Se han señalado las tendencias hacia la aplicación de enfoques multimodelo. En el momento en que este manuscrito fue terminado, el artículo de revisión fue citado más de veinte veces. La revisión de la literatura puede ser vista como una contribución científica ya que proporciona un punto de partida para aquellos investigadores interesados en el mantenimiento predictivo.

2. El enfoque de ingeniería de sistemas para el diseño de sistemas de mantenimiento predictivo ha sido la primera contribución científica publicada en una conferencia internacional durante el desarrollo del doctorado. A diferencia de las arquitecturas genéricas existentes para el diseño de sistemas de

mantenimiento predictivo, el enfoque sistemático propuesto aborda la etapa conceptual de los sistemas de mantenimiento predictivo desde las necesidades y deseos iniciales hasta la arquitectura lógica. Ayuda al arquitecto a determinar con precisión los componentes del sistema y mantener la trazabilidad con las necesidades y deseos iniciales obtenidos de las partes interesadas.

3. Incluso cuando las ontologías no eran el ámbito principal de la investigación actual, una buena cantidad de trabajo de investigación se ha realizado en el dominio y ha producido sus propios resultados de investigación. La creación de OMSSA, un modelo de ontología para la selección y evaluación de estrategias de mantenimiento, proporciona un marco terminológico que puede ser utilizado por agentes inteligentes para automatizar las complejas tareas de gestión de estrategias de mantenimiento. Una extensión a OMSSA (OPMAD) es la ontología utilizada para crear la base de casos y las medidas de similitud del DSS para la selección de componentes de mantenimiento predictivo. El trabajo de investigación realizado en la creación de OMSSA se ha consolidado en un artículo de la revista que estaba bajo una segunda revisión después de una revisión importante en el momento en que este documento fue escrito.

4. El alcance principal de la investigación actual ha producido una contribución. Un sistema de apoyo a la toma de decisiones (DSS) capaz de consultar casos exitosos de implementación de sistemas de mantenimiento predictivo puede ayudar a un arquitecto de sistemas a seleccionar modelos de mantenimiento predictivo adecuados para realizar tareas de diagnóstico y pronóstico. Los primeros resultados de esta investigación se han publicado en un documento de conferencia que se ha publicado en una conferencia internacional.

E.8.2 Perspectivas de trabajo futuro

Mientras se trabaja en la investigación no es factible explicar y probar todo. Para lograr los objetivos de la investigación en un plazo fijo, algunos de los pasos intermedios de la investigación tienen que ser simplificados y a veces se toman decisiones pragmáticas. Al lograr los resultados, es importante poner las cosas en perspectiva para detectar los puntos de mejora y enumerarlos como perspectivas de trabajo futuro. La investigación nunca termina, y una de las partes principales de cualquier manuscrito de doctorado es indicar lo que faltaba y se espera que se haga con el fin de continuar la línea de investigación. Las perspectivas de la labor futura se organizan en tres grupos principales:

1. Perspectivas relacionadas con el enfoque de ingeniería de sistemas para el diseño de sistemas de mantenimiento predictivo.
2. Perspectivas relacionadas con las ontologías desarrolladas en la investigación actual.
3. Perspectiva relacionada con el sistema de recomendación basado en casos, habilitado para la ontología, para la selección predictiva de componentes.

E.8.2.1 Perspectivas relacionadas con el enfoque de ingeniería de sistemas para el diseño de sistemas de mantenimiento predictivo.

- Mejorar la lista de necesidades, deseos y requerimientos del sistema de mantenimiento predictivo: después de publicar el artículo relacionado con la etapa conceptual de los requerimientos que no fueron considerados originalmente. Se puede actualizar y refinar la lista de posibles necesidades, deseos y requisitos para nuevos sistemas de mantenimiento predictivo. Estas listas pueden almacenarse en bases de datos que pueden ser utilizadas por agentes inteligentes automatizados para ayudar a los ingenieros a evaluar las necesidades y deseos iniciales de un sistema de mantenimiento predictivo y

sugerir los requisitos correspondientes de las partes interesadas. Se podría desarrollar un DSS para definir los requisitos correctos de las partes interesadas basados en las necesidades iniciales y los deseos para el nuevo sistema de mantenimiento predictivo.

- Incluir la técnica de análisis de compensación en el marco sistemático: Los análisis de compensación están presentes en varios puntos durante la etapa conceptual. Por falta de tiempo, aún no se ha abordado. La integración de una herramienta de análisis de compensación puede ayudar a mejorar el DSS y facilitar la selección de los componentes de la arquitectura.
- Incluir la formalización de los requisitos del sistema: un aspecto importante de un enfoque de ingeniería de sistemas es la obtención de los requisitos del sistema. Incluso cuando la creación de estos requisitos es en teoría antes del proceso de arquitectura, en la práctica, estos requisitos se establecen normalmente paralelamente al desarrollo de la arquitectura del sistema o incluso al final, cuando se conocen todas las prestaciones de los diferentes componentes lógicos. Estos requisitos son importantes para la fase de diseño detallado. La obtención de estos requisitos estaba fuera del alcance de la investigación actual, pero es un tema complementario interesante para futuras investigaciones.

E.8.2.2 Perspectivas relacionadas con las ontologías desarrolladas en la presente investigación.

- Refinar clases y relaciones: el desarrollo de ontologías es un tema en rápida evolución. Hay varias iniciativas para proporcionar ontologías de nivel superior y medio en diferentes dominios. El dominio industrial no es la excepción. Será necesario seguir trabajando para alinear OMSSA y OPMAD a estas ontologías estandarizadas, para impulsar su integración y reutilización en otras aplicaciones industriales. Este trabajo incluye el refinamiento de las clases y las relaciones entre ellas.
- Extender el razonamiento en la ontología: OMSSA y OPMAD han sido útiles en la investigación actual, pero como ontologías, han sido infrautilizados. Las ontologías tienen varias capacidades que no fueron explotadas. Una de ellas es la implementación de reglas semánticas. Esto amplía las opciones de razonamiento y se puede utilizar como un medio complementario al sistema CBR.

E.8.2.3 Perspectiva relacionada con el sistema de recomendación basado en casos, habilitado para la ontología, para la selección predictiva de componentes.

- Ampliar el uso de la ontología para funciones de similitud más locales: como ya se ha mencionado, la ontología ha sido útil pero infrautilizada. Algunas otras medidas de similitud que han sido asignadas a una medida de similitud de símbolos binarios pueden ser reorganizadas para usar similitudes basadas en ontología.
- Utilizar la ontología para estimar los pesos para el cálculo de similitud global: relacionado con la observación anterior. la ontología también se puede utilizar para calcular los pesos de cada similitud local al calcular la similitud global. En el primer intento de cálculo de similitud global realizado en esta investigación, todas las similitudes locales recibieron el mismo peso. Una mejora en el cálculo de la similitud global podría ser dando un rango de importancia a cada similitud local. Los pesos se pueden calcular usando la ontología poblada.
- Ampliar el Sistema de Apoyo a las Decisiones desarrollando otras fases del ciclo CBR, como la fase de adaptación: El alcance del DSS estaba en la fase de recuperación del ciclo CBR. El sistema puede ampliarse para facilitar la fase de adaptación de los componentes propuestos para el nuevo sistema de mantenimiento predictivo. Esta ampliación también puede incluir el análisis de compensación de los diferentes componentes sugerido por el DSS. Los resultados de la investigación actual mostraron varias oportunidades de mejora que deberían abordarse en el futuro antes de la eventual implantación del DSS.

- Refinar el proceso de instanciación para evitar problemas de diversidad pero manteniendo todo el espacio de solución cubierto: Como se mencionó en el capítulo de validación, la instanciación de la base de casos debe ser refinada. Una oportunidad de mejora puede ser dividir el espacio de solución por las diferentes funciones de mantenimiento predictivo y asegurarse de que el espacio de solución está completamente cubierto para cada uno de ellos. Casos generalizados para cada función de mantenimiento predictivo pueden resolver el problema de la diversidad en los casos de recuperación.
- Análisis de compensación para seleccionar mejor la técnica entre las propuestas: ya se mencionó como la perspectiva de trabajo futuro para el enfoque sistemático para el diseño de sistemas de mantenimiento predictivo. En cuanto al DSS, el análisis de compensación puede ayudar al arquitecto a discriminar entre los componentes sugeridos por el DSS. En la validación se mencionó que a veces el DSS sugería dos componentes diferentes con la misma similitud. Agregar algunos atributos adicionales como indicadores de rendimiento puede ayudar al arquitecto a tomar la decisión correcta para su problema.

Résumé — La maintenance prédictive vise à déterminer le bon moment pour déclencher des actions de maintenance en fonction de l'état de santé d'un système. La maintenance prédictive est effectuée par des systèmes spécialisés dont la conception est encore basée sur des essais et des erreurs. Deux des principaux défis dans le développement des systèmes de maintenance prédictive sont l'absence d'une approche systématique pour aborder leur état conceptuel, et la sélection des composants appropriés pour traiter les tâches de diagnostic et de pronostic. Cette recherche vise à proposer des solutions possibles pour relever ces défis. Une approche d'ingénierie des systèmes est proposée pour aborder l'étape du concept, et un système d'aide à la décision (DSS) est proposé pour aider l'architecte à sélectionner les composants appropriés en fonction des mises en œuvre de maintenance prédictives réussies précédentes. Pour le développement du DSS deux technologies sont intégrées : Case-Based Reasoning et ontologies. La validation du DSS comprend la mise en œuvre de l'un des composants suggérés par le DSS pour accomplir une tâche de diagnostic dans une étude de cas de données de moteur à réaction simulée.

Mots clés : maintenance prédictive, architecture des systèmes

Abstract — Predictive maintenance aims at determining the right moment to trigger maintenance actions based on the health state of a system. Predictive maintenance is carried out by specialized systems whose design is still based on trial and error. Two of the main challenges in the development of predictive maintenance systems is the lack of a systematic approach to address their concept state, and the selection of suitable components to address the diagnostics and prognostics tasks. This research aims at proposing possible solutions to address these challenges. A systems engineering approach is proposed to address the concept stage, and a Decision Support System (DSS) is proposed to help the architect select suitable components based on previous successful predictive maintenance implementations. For the development of the DSS two technologies are integrated: Case-Based Reasoning and ontologies. The validation of the DSS includes the implementation of one of the suggested components by the DSS to fulfil a diagnostics tasks in a simulated jet-engine data case study.

Keywords: Predictive maintenance, systems architecture, knowledge reuse

ISAE-SUPAERO, 10 Avenue Edouard Belin
Toulouse